

Reglamento y Apuntes de Sistemas de Inteligencia Artificial

La búsqueda por artificializar la inteligencia

SIA



Copyright © 2024 Catedra de SIA

PUBLICADO POR ITBA

Las siguientes personas participaron activamente del desarrollo de este documento: Paula Oseroff, Santiago Reyes, Marco Scilipoti, Eugenia Sol Piñeiro, Marina Fuster, Alan Pierri, Rodrigo Ramele, Luciano Bianchi, Juan Miguel Santos y Juliana Gamibini.

Comentarios, sugerencias, y correcciones (probablemente muchas): rramele@gmail.com

Typesetting with LaTeXon template by Mathias Legrand and Vel.
First printing, July 2024, version 0.18

Índice general

1 Course Regulations 11
1.1 Artificial Intelligence Systems 11
1.2 Classes 11
1.3 Course Assessment 11
1.4 Practical Assignments 12
1.4.1 Development 12
1.4.2 Presentation 12
1.4.3 Defense 12
1.5 Course Passing Requirements 13
1.6 Communication 13
1.7 A Note on Bibliography 13
1.8 Disclaimer for International Students 14

I The Beginnings

2 Introduction 17
2.1 What is intelligence? 17
2.2 The Origins 17
2.2.1 Symbolism 18
2.2.2 See You in Dartmouth 18

2.2.3 Language Processing . . . 19
2.2.4 GPS – General Problem Solver . . . 20
2.2.5 Computing in Action . . . 20
2.2.6 The Evolutionary Approach . . . 20
2.2.7 Expert Systems . . . 21
2.2.8 The Connectionist Approach . . . 21
2.2.9 Machine Learning . . . 21
2.2.10 Deep Learning . . . 22
2.2.11 AI in Argentina . . . 23
2.3 Definition of Intelligence 23

3 Preliminary Concepts . . . 27
3.1 Nomenclature 27
3.1.1 Data Matrix . . . 27
3.1.2 Features . . . 27
3.1.3 Learning and Generalization . . . 28
3.1.4 Nomenclature . . . 28
3.1.5 Learning Methods . . . 28
3.1.6 The Four Tasks . . . 28

II Search Algorithms

4 Classical Artificial Intelligence . . . 31
4.1 Agent 31
4.1.1 Environments . . . 32
4.2 Problem 32
4.3 State Space and Search Tree 33
4.4 Uninformed Search Algorithms 33
4.4.1 Breadth-First Search (BFS) . . . 33
4.4.2 Depth-First Search (DFS) . . . 33
4.4.3 Depth-Limited Search . . . 34
4.4.4 Iterative Deepening Search (IDS) . . . 34
4.4.5 Uniform-Cost Search . . . 34
4.5 Informed (Heuristic) Search 34
4.5.1 Greedy Best-First Search . . . 34
4.5.2 A* Search . . . 34
4.6 Heuristic Design 35
4.6.1 Relaxed Problems . . . 35
4.6.2 Combining Heuristics . . . 35
4.7 Local Search and Optimization Algorithms 35
4.7.1 Hill Climbing . . . 35
4.7.2 Variants of Hill Climbing . . . 36
4.7.3 Simulated Annealing . . . 36

4.7.4 Genetic Algorithms . . . 36
4.7.5 Local Beam Search . . . 36
4.7.6 Genetic Algorithms vs. Local Beam Search . . . 37
4.8 Constraint Satisfaction Problems (CSPs) 37
4.8.1 Examples of CSPs . . . 37
4.8.2 Types of Constraints . . . 37
4.8.3 CSP Solving Techniques . . . 37
4.8.4 Summary . . . 38

III Search Algorithms

5 Classical Artificial Intelligence . . . 41
5.1 Agent 41
5.1.1 Environments . . . 42
5.2 The Importance of Search 43
5.3 Search Methods 43
5.3.1 Efficiency . . . 43
5.3.2 Search Strategies . . . 44
5.3.3 Problem Representation . . . 44
5.4 Uninformed Search Methods 46
5.4.1 Breadth First Search . . . 46
5.4.2 Uniform Cost Search . . . 46
5.4.3 Depth First Search . . . 46
5.4.4 Repeated States and Backtracking . . . 46
5.5 Informed Search Methods 47
5.5.1 Local Greedy Search . . . 47
5.5.2 A* . . . 48
5.5.3 Iterative Deepening A* . . . 49
5.5.4 Heuristic Path Planning . . . 50
5.5.5 Game Example: Sokoban . . . 50
5.6 Iterative Improvement Algorithms 50
5.6.1 Hill Climbing . . . 52
5.6.2 Simulated Annealing . . . 53
5.6.3 Beam Search . . . 54

6 Genetic Algorithms . . . 55
6.1 Evolutionary Definitions 55
6.2 Crossover 59
6.2.1 One-Point Crossover . . . 59
6.2.2 Two-Point Crossover . . . 60
6.2.3 Ring Crossover . . . 60
6.2.4 Uniform Crossover . . . 60

6.3 Mutation **60**
6.3.1 Generating Generations 60

IV **Perceptron**

7 Mathematical Optimization 65
7.1 The Coke Can **65**
7.2 The Optimization Problem **67**
7.3 The Simplex Method **67**
7.4 Unconstrained Nonlinear Optimization **68**
7.4.1 Preliminary Definitions 69
7.4.2 Gradient Descent 70
7.4.3 Newton's Method 72
7.4.4 Quasi-Newton Methods 72
7.4.5 Conjugate Direction Methods 72
7.4.6 Conjugate Gradient Methods, 1952 73
7.4.7 Powell's Method, 1964 73
7.4.8 Stochastic Gradient Descent 73

8 The Neuron Model 75
8.1 The Bio-Inspiration **76**
8.2 Linear Separability **76**
8.2.1 Hebbian Learning 77
8.2.2 The Simple Perceptron Algorithm 79
8.2.3 Linear Perceptron 80
8.2.4 Learning in the Nonlinear Simple Perceptron 82

9 The Era of Connectionism 85
9.1 Hierarchical Agglomerative Structure **85**
9.1.1 Feedforward 85
9.1.2 Backward Backpropagation 86
9.1.3 Incremental Learning 92
9.1.4 Batch Learning 92
9.1.5 Multilayer Perceptron 92
9.2 Architecture **93**
9.3 Optimization in Neural Networks **94**
9.3.1 Momentum 94
9.3.2 Adaptive η 94
9.3.3 RMSProp 94
9.3.4 Adam 95

10 Evaluation Metrics . . . 97
10.1 Identifying the Pattern Correctly 97
10.2 Standard Metrics 97
10.2.1 Confusion Matrix . . . 98
10.2.2 Metrics . . . 98
10.2.3 Fitting, Underfitting, and Overfitting . . . 99
10.2.4 Cross-Validation . . . 100

V Automating Learning

11 Machine Learning . . . 105
11.1 Redefining Learning 105
11.2 Learning Methods 106
11.3 Machine Learning Pipeline 106
11.3.1 The Inverse Problem . . . 107

12 Data Transformations . . . 111
12.1 Internal Structure of the Data 111
12.1.1 Clustering and Grouping . . . 112

13 Principal Component Analysis . . . 115
13.1 Preliminaries 115
13.1.1 PCA Transformation . . . 116

14 Kohonen Network . . . 121
14.1 Self-Organizing Map 121
14.1.1 Competitive Learning . . . 121
14.1.2 Architecture . . . 122
14.1.3 Algorithm . . . 123

15 Hopfield Network . . . 125
15.1 Hopfield 1982 125
15.1.1 Associative Memory . . . 127
15.1.2 Stability . . . 128
15.1.3 Convergence . . . 129
15.1.4 Limitations . . . 130

16 Oja and Sanger Model . . . 131
16.1 The Search for the Power of Synthesis 131
16.2 Oja Network 131
16.2.1 Implementation . . . 133

16.3 Sanger Network 134

VI Deep Learning

17 Deep Brain 137
17.1 Neural Biomimesis 137
17.1.1 Intelligent Fat 138
17.1.2 A Neuro-Computer 139
17.2 The Upgraded Neuron 140
17.2.1 How Does a Neuron Fire? 140
17.2.2 A Nanotechnological Computer 142
17.2.3 Saltatory Conduction 143
17.2.4 Synapse 144

18 Deep Learning 145
18.1 A Guide for the Uncle and Aunt Who Ask 145
18.2 Depth 147
18.2.1 The Problems of Shallow Learning 147
18.2.2 The Fundamental Problem of Neural Networks 148
18.3 Representation Learning 150
18.4 Datasets 150
18.5 GPUs 151

19 Autoencoders 153
19.1 Autoassociators 153
19.1.1 Architecture 153
19.1.2 Linear Autoencoder 154
19.1.3 Outlier Detection 156
19.1.4 SAE - Sparse Autoencoders 156
19.2 Generative Autoencoder 156
19.2.1 The Structure of the Latent Space 157
19.3 VAE - Variational Autoencoder 159
19.3.1 Stochastic Neural Networks 159
19.3.2 Variational Inference 160
19.3.3 Optimizing the Variational Autoencoder 161
19.4 Appendix 166

20 Convolutional Neural Networks 173
20.1 The Convolution Operation 174
20.1.1 Moving Average 174
20.1.2 Padding 175

20.2 Convolution on Images 175
20.3 Neocognitron 176
20.4 The Convolutional Layer 178
20.5 The Pooling Layer 180
20.6 Full Layer and Softmax 181
20.7 Backpropagation 181
20.8 Why Do They Work? 182
20.9 Intelligibility 182
20.10 Reusable Architectures 183
20.10.1 ResNet . . . 184
20.10.2 UNet . . . 184
20.11 Beyond Images 185
20.12 I Want More 185

21 Generative Adversarial Networks . . . 187
21.1 Generating Kittens 187
21.2 The MinMax Game 188
21.3 Mode Collapse 190
21.4 The Great Simulator 190

22 Transformers . . . 193
22.1 Recurrent Networks 193
22.1.1 Long Short Term Memory . . . 194
22.2 Transformer Architecture 195
22.3 Attention is All You Need 196
22.3.1 A Continuous Indexed Lookup Table . . . 196
22.3.2 Kernel Smoothing . . . 197
22.3.3 Cognitive Attention . . . 198
22.4 Embedding 198
22.5 Training 199

VII Professional Pragmatism

23 The Science of Data . . . 203
23.1 Quantitative Analysis 203
23.2 EDAs and ETLs 205
23.3 Data Science Fallacies 205
23.4 Artificial Intelligence and the Digital Frontier 207

24 Guide for the Impatient 209
24.1 Presentations 209
24.2 Ingredients of a Neural Network 209
24.2.1 Data Transformation 210
24.2.2 Class Balancing 210
24.2.3 Cost Functions 211
24.2.4 Activation Functions 211
24.3 Software Infrastructure 211
24.4 I Want More 214

Index 227



1. Course Regulations

1.1 Artificial Intelligence Systems

The course aims to introduce students to the techniques and challenges of the artificial and computational development of what we call intelligence. It begins by including philosophical aspects of the concept. It then delves into the so-called classical tools. The core focuses on the approach to artificial neural networks viewed as an optimization problem. Finally, Deep Learning techniques are studied. The approach is to offer a theoretical framework that facilitates practical development to understand the concepts in depth.

1.2 Classes

The course is hybrid. This means that classes are mostly remote, but some relevant classes are in person. Classes are composed of related but fairly independent modules. All presentations will be available on the campus platform, as well as extra material from books, papers, and online articles. All classes are already available as a resource on YouTube and most classes are recorded. **Please note that some classes will not be recorded**. Attendance for virtual classes will be taken directly through the Campus platform. **Students who do not reach** 70 % **attendance will have 1 (one) point deducted from their final average.**

1.3 Course Assessment

The course is oriented toward *learning by doing*. This means that assessment is carried out through 5 Practical Assignments that students must complete and defend. The assessment is based on three parts: the first concerns the presentation and defense. Part of the objectives of this course is to introduce students to data curation, evaluation, analysis, and presentation. The second part concerns the defense of the implementation code: similar to a **code review**, we will individually evaluate the code produced,

which must be known by the entire team, emphasizing accountability for what is submitted. Finally, the last point is the completeness and timely delivery of the work. The grade for each assignment is individual for each student and derives from the work presented by the team, and the individual role during the presentation, explanation, and code defense.

Additionally, there is a set of mandatory exercises throughout the course that are not graded but must be submitted. For example: the final essay (Essay), Kahoot, Transformers, and GAN.

1.4 Practical Assignments

The practical assignments are central to this course. There are 5, each oriented toward one of the important topics studied: Classical Artificial Intelligence, Genetic Algorithms, Neural Networks, Unsupervised Learning, Deep Learning.

1.4.1 Development

The development of each assignment involves several parts. The first is studying and reviewing the concepts, then researching how to implement them. The next step is the implementation itself, followed by running the experiments that yield data to be analyzed. Finally, the elaboration of the report with the results.

Code, References, Libraries, and Tools

Regarding code, prior to the defense date of each assignment, students must submit a repository referencing a commit or tag with the implementation source files.

We invite students to research and incorporate varied sources and references to solve the problem. We will make the official course literature available, but you may supplement it with any resource you find convenient and useful.

Due to the nature of the practical assignments — which have a didactic learning objective — for each of them there is a set of libraries that may NOT be used, precisely because those are the ones students must implement themselves. Think of yourselves as creating seeds that could be alternatives to those libraries. Not everything has been built yet.

We also invite you to use all the tools you need to make the work more efficient. But it is mandatory that:

- ALL sources used to obtain information are cited precisely.
- The tools used are referenced and disclosed.
- You rely on construction tools while understanding what you are implementing and doing.

1.4.2 Presentation

In the first classes, instructions are given on how to develop these presentations — what topics to emphasize, how to create graphs, and how to show results.

1.4.3 Defense

The defense must be concise and must not exceed 30 minutes. The presentation date is the day agreed upon according to the schedule. Students who require a particular time adjustment for the defense must present an exceptional, justified reason. Presenting directly on the make-up date implies starting from a score of 6 (six) that must be defended during the defense.

1.5 Course Passing Requirements

Practical Assignments, Oral Evaluations, and the Final Exam — each and every one — must be passed with a grade greater than or equal to **6 (six)**. Passing the course depends on the grade obtained from the 10 scores, calculated as the arithmetic mean of all grades. The **presentation** of the PCA analysis offers an additional 10 (ten). Failing to meet the 70 % virtual class attendance requirement deducts a full point from the average, which can result in NOT passing the course.

The final exam format is an examination based primarily on the content of these course notes. The exam may be written or oral depending on the number of students sitting for it, and the general University rules apply.

1.6 Communication

Several communication channels are available:

- **Campus**: This is the **official** communication channel via Campus messages. Everything concerning class format (in-person/virtual), submission deadlines, and class information will be communicated through this channel.
- **Email**: You have access to all the email addresses of the professors that make up the teaching team for any course-related matter.
- **Discord**: A course space is available with a general channel and group channels where you can ask questions, especially regarding the development of the practical assignments.
- **In person**: This course is run from the Neurotronics Lab at ITBA, physically located in room S206 of the Rectorate Building. You are welcome to drop by at any time with questions and inquiries.

1.7 A Note on Bibliography

The field is currently at the crest of the wave (circa 2025) of innovation, and rarely has there been such a massive surge of new work and advances in the area, at such a dizzying pace, to the point that it is humanly impossible to keep up [1].

The classic Artificial Intelligence book is Russell and Norvig [2]. This book is useful for understanding above all the vision more associated with logical inference in this field — the more classical view. At one time, studying AI meant studying this book. Two other books, also classics, that were mandatory as an introduction to neural networks are Hertz [3] and Haykin [4]. These books evidence the shared roots between the study of neural networks, computational physics, dynamical systems, and complex systems (they are very *physics-flavored* books!).

For genetic algorithms, the course material is generally sufficient for what needs to be solved in the practical assignment. However, for those who want to truly understand and delve into the evolutionary foundations of what we cover in GA, the book by Melanie Mitchell, *An Introduction to Genetic Algorithms* [5], chapters 1 and 2, are excellent.

Moving into Deep Learning, an excellent book is Goodfellow [6]. On the other hand, the book by Aggarwal [7] is easy to read and very comprehensive.

For mathematics, an excellent book with incredible pedagogy is *Linear Algebra Done Right* [8]. It will be useful for those with little time who want a pragmatic review of linear algebra. On the Calculus side, Jean Gallier's 2000-page compendium is exhaustive to the point of excess, but contains every conceivable topic you might need for this course; use it to look up and review a specific topic [9].

In Optimization, an excellent book is Boyd's *Convex Optimization* [10]. Boyd is a Stanford researcher and one of the first to begin uploading his lecture videos to the Internet.

Throughout the course we will also share many papers that we recommend you read — or at least know that they exist and understand what they are about.

1.8 Disclaimer for International Students

First of all, welcome to Argentina, to Buenos Aires, to ITBA and to this SIA lecture!

We wanted first to let you know a few things about this lecture to calibrate better with what you may be looking for in this class.

This lecture requires a lot of work developing and writing algorithms to implement the material covered in each lecture. Hence, the requirements are:

1. Programming: we do a lot of programming. If you feel that you do not have, and you do not want to learn the programming knowledge to sit down and work hard on your own version of a neural network, this may not be the lecture for you.
2. Teamwork at groups: this class also requires the availability of students to work together in groups. So if you think you are going to be travelling heavily around and that you will not have the time to be with teammates (remotely or physically) it may be better to join another lecture.
3. Be present at the presentation: you have to be remotely present to defend the work done on each assignment with your team.
4. This class is what is called *English Friendly*. This means that the lectures and core material is mostly in Spanish, but please do not feel that to be a problem in itself. There is a handout available in Spanish (this) that you can easily translate and all of the references are books and papers in English. Additionally, you can ask all your questions, write all your presentation, code, etc entirely in English.

Keep in mind that this University offers an alternative course, with the same name, with code 71.22, which is more theoretical with less programming requirements. Additionally, Machine Learning 72.75 covers related materials with a little bit less programming. You can switch to those alternative courses if you prefer.

Regarding the groups, you can join any group that you want. We encourage you to reach out to ITBA students and join them.

The Beginnings

2 Introduction 17
2.1 What is intelligence?
2.2 The Origins
2.3 Definition of Intelligence

3 Preliminary Concepts 27
3.1 Nomenclature



2. Introduction

2.1 What is intelligence?

Artificial Intelligence exists and arises with the very birth of the idea of computability and the machines to automate it. Its current boom dates back to the post-war period, where the first physical implementations emerged from the convergence of diverse earlier ideas.

Probably one of the first obvious truths to appear when studying this discipline is that we don't really know what intelligence is. Homo sapiens sapiens live without deeply questioning what intelligence is, and although we lack a clear definition, we trust we have a rough idea of what it means; we recognize it when we see it.

The approach proposed here is to address this problem by asking: what is it that we are trying to artificialize? A first strategy for answering that is to take a historical look at the topic to understand how we got here.

2.2 The Origins

Artificial Intelligence (aka AI)[1] arises directly from the conception of computation theory as proposed by Alan Turing [11], and from the development of digital computers: at the end of WWII and into the post-war period.

Figure 2.1 shows a timeline with the major milestones in the history of Artificial Intelligence [12]. Developments in cryptanalysis at Bletchley Park to help crack the Enigma machine, or the MAGIC project in the USA, as well as remote control of torpedoes or radar signal analysis in the Battle of Britain, were all computational needs that drove the creation and growth of computers and, with them, the early development of AI.

[1] In this text, we'll refer to AI as the discipline. With its popularization, AI is often used to refer to any practical implementation of an agent, program, or algorithm that fits this category. For instance, it's common to hear phrases like: "I have several *Artificial Intelligences* implemented." We are aware that this battle is clearly lost.

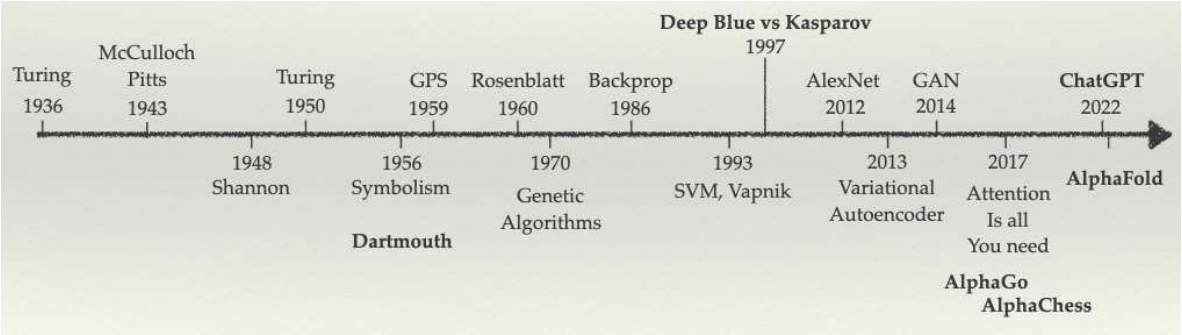


Figura 2.1: Timeline of major milestones in the development of Artificial Intelligence.

In 1943, Warren McCulloch and Walter Pitts [13] presented the neuron model. This simple organized model could solve any computable function. Then in 1949, Donald Hebb [14] proposed a conjecture on how the influence of one neuron on another changes based on the activity of both, and presented his **Hebb's Rule**. This was used by Rosenblatt to develop his **Perceptron** (Chapter 8) for the US Navy.

In 1950, Turing wrote *Computing Machinery and Intelligence* [15, 16], where he identified the complexity of defining intelligence from a computability perspective and introduced his famous **Turing Test**, which has recently become somewhat obsolete [17].



Figura 2.2: McCulloch, calmly reflecting on his model.

2.2.1 Symbolism

In 1956, Newell and Simon [18] created a software program called the **Logic Theorist** that implemented an early version of a decision tree. The hypothesis was placed at the root of the tree, and each branch generated a deduction from the previous statement. Eventually, one of the branches would yield the actual proof of the hypothesis. The program simply searched the branches until it found an answer.

The rise of symbolism showed how the use of **logical inference** could produce high-level results previously thought unimaginable (this phenomenon recurs, and it's one of the reasons for both the hype and subsequent AI winters). Logic Theorist successfully proved several theorems from *Principia Mathematica*.

2.2.2 See You in Dartmouth

In 1956, the prevailing mood was that the AI revolution had arrived and they were on the brink of automating intelligence[2]. At Dartmouth College in Hanover, New Hampshire, USA, the leading thinkers of the time organized a workshop—a summer retreat—to gather and discuss *the details* of how to finally artificialize intelligence. John McCarthy convinced other researchers including Minsky, Shannon, and Rochester to organize the *Dartmouth Summer Research Project on Artificial Intelligence* [19].

"We propose that a study be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire, with **10 people over 2 months**. The goal is to explore how to make **machines use language**, form **abstractions and concepts**, solve problems currently reserved for humans, and **improve themselves**. We believe that **a significant advance** can be made in one or more of these problems **if a carefully selected group of scientists works together for a summer**."

[2]Sounds familiar, doesn't it?



Figura 2.3: Oliver Selfridge, Nathaniel Rochester, Marvin Minsky, John McCarthy, Ray Solomonoff, Unknown Person (Peter Milner or someone from the future), Claude Shannon.

The term Artificial Intelligence gained prominence at this event, largely due to personal and political reasons. John McCarthy explicitly wanted to distance himself from Automata Theory [20], the theoretical basis of programming languages, which he considered esoteric and limited in scope. He also wanted to dissociate from Cybernetics [21] due to personal friction with its founder Norbert Wiener [22, 23, 24]. This also marked the schism between AI and control theory, which—despite addressing similar problems related to automation—went on separate paths [25].

Did they solve the artificialization or automation of intelligence? No, but they started it all: Artificial Intelligence, Cybernetics, Machine Learning, Pattern Recognition, etc.

2.2.3 Language Processing

A deeply human trait strongly associated with intelligence is the ability to process language. We create, produce, and understand different languages, and with them express thoughts and abstract concepts. Naturally, this led to attempts to artificialize intelligence through the synthesis of language processing, or NLP: Natural Language Processing.

In 1950, John McCarthy developed LISP, a programming language specifically designed to naturally capture automatable behaviors. The construction of programming languages led to the theoretical development of grammar theory, computability, and automatons such as finite state machines, which inherently carried the idea of intelligence [26]. In the early days, grammars and compilers were considered part of artificial intelligence. This thread culminates in 1965 when Joseph Weizenbaum presented Eliza [27]. Eliza simulated a psychotherapist who skillfully paraphrased input text and used simple rule-based strategies to produce open-ended responses and encourage the user to continue.

Despite being a very simple system, it led many people to believe they were truly conversing with a real person.

2.2.4 GPS – General Problem Solver

In 1959, Herbert Simon, Cliff Shaw, and Allen Newell created GPS as an evolution of the work done with the Logic Theorist. The strategy was to symbolically represent more general problems in a way that allowed Logic Theorist's problem-solving methods to be applied to them, and then translate the solutions back to the original domain.

In this way, symbolic representation could be used to solve any kind of problem—as long as it could be formally represented using logical expressions. The intuitive idea that humans solve problems by breaking down goals into subgoals, which are achieved through the execution of tasks, could now be directly represented in a computer.

At that time, Herbert Simon enthusiastically declared:

> It is not my aim to surprise or shock you—but the simplest way I can summarize the current state of the art (in the world) is to say that **today there are machines that think, that learn, and that create**. Moreover, their abilities to do these things are increasing rapidly, and **in the near future**, the range of problems they can handle **will coexist with those the human mind has tackled**.

Is logical inference then a tool for formally and consciously managing solutions? Is it the natural mechanism of thought?

The problem with GPS was that it worked only for very simple problems—toy examples that weren't too demanding. When the problem size increased, GPS often failed to find a solution.

2.2.5 Computing in Action

All these advances led many to believe that if a computer could prove a theorem from *Principia Mathematica*, then implementing it in a physical mechanism—a **robot** or **automaton**—would be trivial. Much of Asimov's science fiction and the futurism of the era are rooted in this belief.

However, Moravec's Paradox soon emerged:

Definición 2.2.1 — Moravec's Paradox. The real world—partially observable, stochastic, with infinite states—poses a far more complex challenge than any logical problem solved in the early days of AI. This was difficult for researchers to perceive at the time because, for humans, such activities are very automatic and subconscious. We are not aware of the complexity they involve.

When these algorithms were deployed in the real world, the enormous challenge of dealing with a physical, stochastic, and partially observable environment with infinite states became apparent. One of the first major successes in robotics was *Shakey*, a mobile robot developed in 1959 that could autonomously navigate the hallways of the *Stanford Research Institute*.

2.2.6 The Evolutionary Approach

In the early 1970s, John Holland introduced genetic algorithms (Chapter 6). These were based on a model of Darwin's theory of evolution, adapted to perform optimizations: searching for parameter sets that minimize a cost function using a stochastic heuristic.

The evolutionary approach used individuals that represented different combinations of parameter values. Based on their success or failure in optimizing the cost function, some individuals were selected to replicate their genomes.

Like many other developments in AI, this approach generated a massive hype wave, which was abruptly interrupted when it couldn't be applied to real-world problems (which, in fact, were the ones



Figura 2.4: Yan LeCun with his gaming PC demonstrating the capabilities of his convolutional neural network to tackle the handwritten digit recognition problem (i.e., MNIST).

funding the research).

2.2.7 Expert Systems

Expert systems originated with a project called DENDRAL, developed by Ed Feigenbaum (a student of Simon), B. Buchanan (a philosopher working in computer science), and J. Lederberg (a geneticist). The idea was to extend GPS to a specific, narrow, well-defined domain.

They applied this directly to infer the structure of a molecule from mass spectrometry data gathered by electron beam bombardment.

As the era of personal computers and the digital revolution of the late 20th century arrived, this tactic expanded to extract specific knowledge from experts in a particular domain. These concepts were formalized as logical propositions, from which inferences could be drawn to reach conclusions, for example, in medical diagnostics.

Examples include the Mycin system or IBM's expert medical systems under Watson Health [28].

2.2.8 The Connectionist Approach

In 1986, Rumelhart, Hinton, and Williams [29] proposed the backpropagation algorithm, revitalizing Rosenblatt's old perceptron, McCulloch and Pitts' neuron models, and Hebb's learning rule. They introduced one of the core ideas of neural networks: hierarchical agglomerative structures.

This proposal directly addressed the limitations pointed out by Minsky and Papert [30] regarding the XOR problem, sparking what is known as the second wave of AI. The multilayer perceptron (Chapter 9) gained momentum, and connectionism spread to other neural network learning models such as Hopfield (Chapter 15), Kohonen (Chapter 14), and Goldberg, among others.

Artificial neural networks produced many solutions to problems involving noisy data, but they struggled to scale up to more complex problems.

2.2.9 Machine Learning

The machine learning approach, also known as **inductive learning**, proposes a different strategy from the symbolic one: instead of providing a structure of logical rules, the idea is to present data—input-

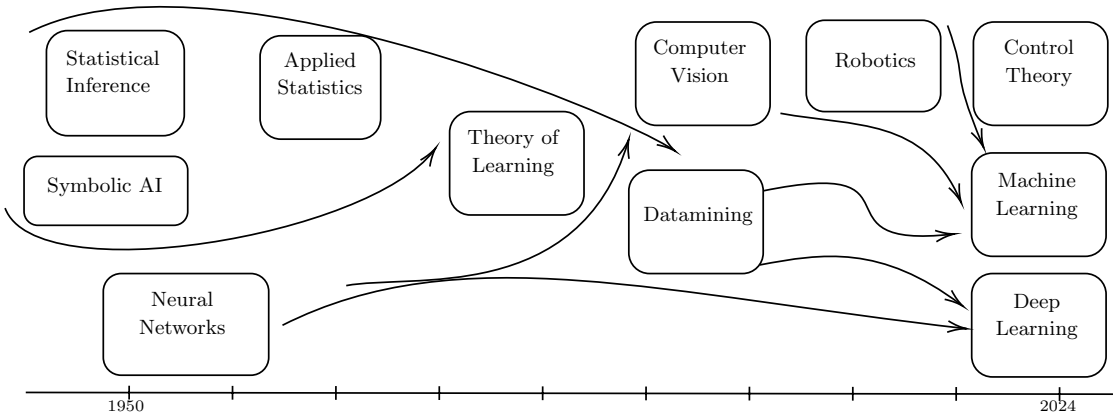


Figura 2.5: Different disciplines associated with Artificial Intelligence and their evolution.

output examples—so that the algorithm can automatically learn the rules and relationships between them. This movement became stronger in the late 1990s and early 2000s, alongside the availability of large datasets on the internet, such as ImageNet.

Learning problems were now framed as **optimization problems**, often based on an empirical risk function (or *loss function*) constructed from the available data. Under this approach, the world is viewed as probabilistic, and it is accepted that some conclusions might be incorrect. Hence, the interest lies in maximizing generalization capacity, rather than being infallible.

This approach is essentially a response to the failure of expert systems to generalize beyond their original domain, as well as to the issues with symbolic systems when facing an uncertain world.

The traditional machine learning approach relies on a **pipeline**, where data is first preprocessed to extract relevant features (a stage known as **feature extraction**), which are then fed into an algorithm. Depending on the availability of output labels for the data, this algorithm can be:

- **Supervised learning:** where each example is a pair (x, y), and the algorithm aims to approximate a function f such that $f(x) \approx y$.
- **Unsupervised learning:** where only the input x is known, and the algorithm attempts to find regularities or groupings in the data.

The typical machine learning pipeline includes a data preprocessing stage, feature engineering, algorithm training, and model evaluation.

2.2.10 Deep Learning

The deep learning revolution brought a fundamental change: it proposed to unify the process of feature extraction and learning into a single algorithm.

Deep learning is based on multilayer perceptrons with many layers (hence the term *deep*). These are essentially the same models studied in the 1980s and 1990s but now applied using large computing resources (especially GPUs), big datasets, and advanced optimization techniques. New ideas also emerged, such as:

- **Convolutional networks**, popularized by LeCun [**?**], which introduced architectures capable of

exploiting spatial patterns in the data, especially images.
- **Recurrent networks**, capable of handling time series and sequential data.
- **Transformers**, which replaced recurrence with attention mechanisms [**?**], and became the standard in natural language processing.

With the arrival of tools like TensorFlow and PyTorch, the development of deep learning systems was greatly democratized, allowing many people to implement these complex models.

2.2.11 AI in Argentina

The development of AI in Argentina has had its historical milestones, often driven by the individual initiative of researchers, with little support or coordination from the State.

One of the first efforts came in the 1960s from the Faculty of Exact and Natural Sciences of the University of Buenos Aires. After the Night of the Long Batons and the exile of many intellectuals, AI development continued in different parts of the country.

During the 1980s, with the return of democracy, several AI research groups were formed. The Universidad Nacional del Sur (UNS) in Bahía Blanca was a notable center. Other important institutions were the University of Buenos Aires, UNICEN in Tandil, and the Balseiro Institute.

In the 2000s, AI research diversified and began to find applications in industry and public services. CONICET began to finance research projects in the area, and international collaboration increased.

Since 2010, with the expansion of the internet and the technology industry, companies and startups oriented toward AI services began to emerge. Open-source communities and local meetups appeared.

Today, the development of AI in Argentina continues to grow and has the potential to contribute globally, as long as it is supported by stable and sustained investment in education, research, and technology transfer.

2.3 Definition of Intelligence

The concept of the technological singularity hypothesizes that in the development of artificial intelligence, at the point when it becomes truly artificialized, there is a change in the growth rate, a discontinuity, since that artificialized intelligence will in turn be capable of creating another entity with even better artificialized intelligence, at a speed that will be inherently different — exponential. This point is called the **singularity** [31].

In line with this, then, different authors speak of three types of artificial intelligence systems, according to their scope:

Let us consider that we can describe artificial intelligence systems as:

- Artificial Narrow Intelligence (ANI)
- Artificial General Intelligence (AGI [32])
- Artificial Super Intelligence (ASI)

They also establish that the evolution of artificial intelligence would follow the order: ANI, AGI, and finally ASI.

Here we propose an analogy that we believe is more comprehensive: to see intelligence as a mountain with its summit hidden behind clouds (or behind other peaks). It is a *moving goalpost* where the very identification of what an intelligent entity must do shifts as we get closer. Moreover, there are many trails that can traverse it, with different levels of advancement. Is our intelligence better or more advanced than that displayed by dolphins or octopuses? Is human intelligence more or less advanced than that manifested by an ant colony? We cannot be sure whether the intelligence we know and exhibit



Figura 2.6: Intelligence is a complex mountain, full of paths and variations, where the summit, from each perspective, appears behind peaks or is hidden behind clouds.

is **The Intelligence**. In this analogy, intelligence is traces and paths on that mountain, in a continuum, with different stages.

Shane Legg and Marcus Hutter [33, 34] compiled various definitions of intelligence. That article summarizes about 70 drawn from an extensive literature review. Below, five definitions are chosen arbitrarily and a new one is proposed.

Definición 2.3.1 — Intelligence - American Psychological Association. Individuals differ from each other in their ability to understand complex ideas, adapt effectively to the environment, learn from experience, engage in various forms of reasoning, and overcome obstacles by thinking.

Definición 2.3.2 — Intelligence - Statement by 52 Experts. Intelligence is a very general mental capacity that, among other things, involves the ability to reason, plan, solve problems, think abstractly, comprehend complex ideas, learn quickly, and learn from experience.

Definición 2.3.3 — Intelligence - A. Anastasi. Intelligence is not a single ability but a composition of various functions. The term denotes that combination of skills required to survive and progress in a particular culture.

Definición 2.3.4 — Intelligence - E. Boring. Intelligence is what is measured by intelligence tests.

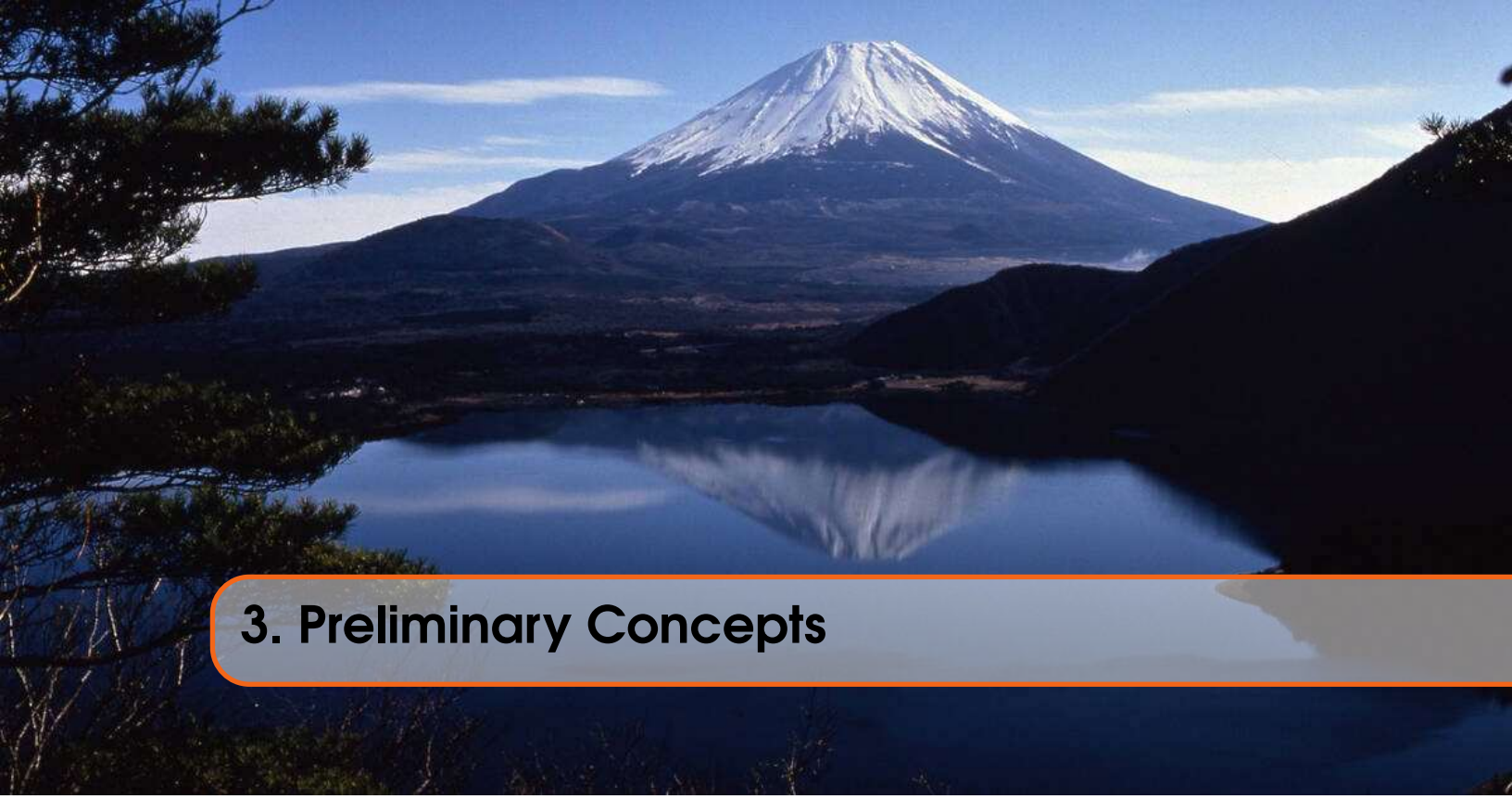
Definición 2.3.5 — Intelligence - J. S. Albus. It is the ability of a system to act appropriately in an eventually unknown environment, where the appropriate action is the one that increases the probability of success, with success being the completion of behavioral subgoals that support the system's major goal.

Definición 2.3.6 — Intelligence - Artificializing Intelligence.

- A system that senses, processes, and acts in a way that is appropriate in a particular physical and temporal context.

- It requires embodiment to manifest clearly.
- It is a continuum, a *moving goalpost*.
- It exhibits **Autonomy** as an emergent behavior.
- It manifests a **Consciousness** [35], materialized as behaviors that represent knowledge of its own existence, in relation to the physical and temporal context, and a differentiation in relation to others.

Definition 2.3.6 is the one we are proposing here, strongly influenced by the conception of intelligence that comes from robotics.



3. Preliminary Concepts

3.1 Nomenclature

This section includes general definitions of common terms in the field, which may have different meanings, so it is necessary to clarify them.

3.1.1 Data Matrix

Several of the methods presented in this text are based on the use of data to adjust algorithm parameters. The data matrix, which we will call X, is composed as follows:

$$X = \begin{pmatrix} & var1 & var2 & var3 \\ sample_1 & \dots & \dots & \dots \\ sample_2 & \dots & \dots & \dots \\ sample_3 & \dots & \dots & \dots \end{pmatrix}$$

where the columns are the variables and the rows are each of the samples, patterns. This matrix is of size $X^{P\times N}$ where N represents the number of variables, the data dimension, and P the number of available patterns. Each sample X_i may have an associated *label* y. This is often a binary value, categorical, a scalar, or possibly an output vector of a particular dimension.

3.1.2 Features

Raw input data can be processed and transformed into a format that is more suitable for subsequent processing. This transformation can range from simple rescaling of the information to a much more complex multi-stage process. After that processing, the input data are transformed into **features**, predictors, that structure the information in a better way to capture potential underlying patterns.

These features are synthesized into a vector or *feature vector* of a particular dimension. This is what is used as input for the mapping function $f(X) = y$ which carries the discriminative power.

3.1.3 Learning and Generalization

To Learn: Artificial learning has a precise definition (11.1.1) that differs from the human concept of learning. It involves the algorithmic process by which we find or adjust the free parameters of an algorithm to optimize a cost function, based on known data obtained from the real world.

To Generalize: It means applying what has been learned — that is, using the free parameters, or *weights*, to run the algorithm and generate an output for a new data point that was NOT used during the learning process or parameter adjustment.

3.1.4 Nomenclature

- **Training set**: dataset used to compute the weights.
- **Test set**: dataset NOT used to compute the weights.
- **Parameters**: the free knobs, the weights, the independent variables of the cost function, which optimization seeks to find. The values x^*.
- **Hyperparameters**: general algorithm parameters that are not adjusted by the optimization process itself. They are tuned **manually**, by intuition, trial-and-error, or some *hyperoptimization* method. They are not part of the cost function.
- **To learn well**: given an input sample, obtain the expected output.
- **To converge**: to learn the entire training set well. That is, to adjust the free parameters so that all outputs match their expected values.
- **To generalize**: to use the learned weights or free parameters to, given a sample that was not used during training, generate an output, a prediction, a mapping.

3.1.5 Learning Methods

Given the mapping function we want to learn $f(X) = Y$.

Supervised Learning: Both X and Y are known, obtained from the real world.
Unsupervised Learning: Only X is known, the input data, but the output is not known.
Reinforcement Learning: Neither X nor Y is known.

3.1.6 The Four Tasks

Typically, a program that falls into the category of AI will be able to perform four different tasks.

To Classify: Given a dataset with labels, the **Classifier** learns to predict the label of each sample. Classification can be binary or multiclass. Also called **Discrimination**.

To Regress: In this case, the **regressor** learns to interpolate and extrapolate samples, being able to estimate intermediate values. It is a classifier with continuous labels. When aiming to infer future values of a time series, it is often called a **Predictor** or **Forecaster**.

To Optimize: Most AI algorithms inherently involve optimization, so they can be directly adapted to solve typical optimization problems (e.g., Traveling Salesman Problem, Knapsack Problem, Integer Partition).

To Generate: Now, algorithms can directly generate new samples, estimate valid X_i values. This is the foundation of generative algorithms.

II Search Algorithms

4 Classical Artificial Intelligence 31
4.1 Agent
4.2 Problem
4.3 State Space and Search Tree
4.4 Uninformed Search Algorithms
4.5 Informed (Heuristic) Search
4.6 Heuristic Design
4.7 Local Search and Optimization Algorithms
4.8 Constraint Satisfaction Problems (CSPs)



4. Classical Artificial Intelligence

In this text, we refer to Classical Artificial Intelligence as the traditional methods of modeling and guided solution search. Faced with a problem, it can be modeled as a set of concrete states and actions to be executed in order to transition from one state to another. If such a modeling is possible, the solution to the problem consists in finding a transition, a path of successive valid actions under the problem's rules, that allows, starting from an initial state, to reach the final state which is the solution. And among all the possible ways to achieve this, we aim to find those specific paths that are more optimal[1] in the sense of minimizing/maximizing the use of resources, or optimizing a particular objective function.

4.1 Agent

The concept of agent is key to the idea of artificial intelligence and aligns with the notion that artificial intelligence truly manifests when it is embodied [36].

An agent is defined as anything that can be considered to perceive and respond or act in an environment, with a certain degree of independence or autonomy. Ethologically, it is assumed that the agent has a goal, a reason for being, becoming a **rational agent** or **utility-based agent**. This is materialized through its **performance function**. This determines the agent's success, and in turn, can measure it in the short or long term—that is, having some sense of the temporal horizon of that success (i.e., strategic decisions). The agent, rationally, then seeks to perform actions that maximize its performance measure.

On the other hand, it is possible to identify **omniscient agents**, which always know the actual outcome their actions will produce.

Agents can also be classified based on whether they have an internal state or not. Those without an internal state are usually called reactive agents $a = f(P)$ with P being the perception and a the

[1] If something is optimal, it cannot be *more optimal*, but you get the idea.

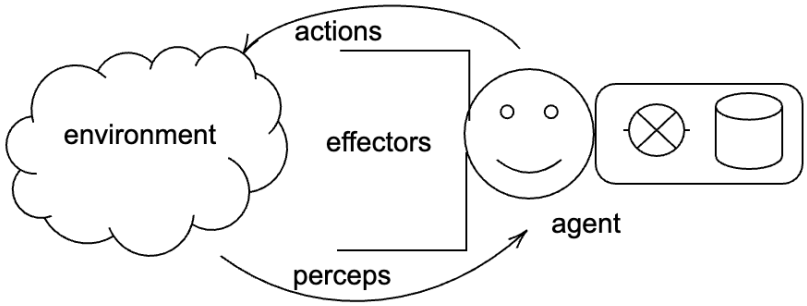


Figura 4.1: The agent and its environment

action. In contrast, those with internal state—also called model-based agents—are characterized by $a = f(P,M)$, where M is the internal model, something the agent uses to understand its environment.

4.1.1 Environments

The environment is the world or context with which the agent interacts. The agent perceives the environment and affects it through its actions. From the point of view of decision-making, the properties of the environment are fundamental:

- **Observable vs. Partially Observable:** In an observable environment, the agent has access to the complete state of the world. If the agent has no access or only partial access to that state, the environment is partially observable. For example, in chess, the environment is observable, while in a card game like poker, it is not.
- **Deterministic vs. Stochastic:** If the outcome of each action is uniquely determined, the environment is deterministic. Otherwise, it is stochastic.
- **Episodic vs. Sequential:** In an episodic environment, the experience is divided into episodes, and each episode is independent of the others. In a sequential environment, current decisions can affect future states.
- **Static vs. Dynamic:** In a static environment, the world does not change while the agent deliberates. In a dynamic one, it does.
- **Discrete vs. Continuous:** Environments may be discrete (finite states and actions) or continuous.

4.2 Problem

To solve a problem by search, it must be possible to describe it formally through:

- An initial state s_0,
- A set of actions A applicable to each state,
- A transition function $\delta(s,a)$ that returns the new state resulting from applying action $a \in A$ to state s,
- A goal test $\Gamma(s)$ that determines whether a given state s is a goal state,
- A cost function $c(s,a)$ that returns the cost of applying action a in state s.

The agent interacts with the environment through its sensors and actuators. From its point of view, this interaction is perceived as a succession of discrete steps, and at each step, it is in a state $s \in S$. From

each state, it can perform a set of actions $A(s) \subseteq A$, and each action leads to a new state (possibly more than one, in nondeterministic environments).

A **solution** is a sequence of actions $\langle a_1, a_2, ..., a_n \rangle$ that, when applied to the initial state s_0, leads to a goal state s_n satisfying $\Gamma(s_n)$.

An **optimal solution** is one that minimizes (or maximizes) the accumulated cost of the actions along the path.

4.3 State Space and Search Tree

Solving a problem requires searching in the space of possible states reachable from the initial state. This search space can be represented as a tree, where:

- Each node represents a state of the world,
- The root is the initial state s_0,
- Each branch corresponds to an action,
- Each child node is the result of applying an action to its parent node.

This structure is called a **search tree**, and it is a conceptual tool for organizing the process of exploring possible solutions. The tree can grow very large, even infinite, so different strategies are needed to explore it efficiently.

Each node n in the tree stores:

- The state $s(n)$ it represents,
- The parent node,
- The action $a(n)$ that led to this node,
- The path cost $g(n)$ from the root to this node.

The solution to the problem is then found by finding a path in the tree from the root to a node whose state satisfies the goal condition Γ.

4.4 Uninformed Search Algorithms

Uninformed (or blind) search algorithms explore the search space without any additional information about the goal other than the definition of the problem itself. These algorithms do not use heuristics or domain-specific knowledge.

4.4.1 Breadth-First Search (BFS)

Breadth-First Search explores the search tree level by level. It uses a queue (FIFO) to keep track of the frontier nodes.

- Always expands the shallowest unexpanded node.
- Complete if the branching factor b is finite.
- Optimal if all actions have the same cost.
- Time and space complexity: $O(b^d)$, where d is the depth of the shallowest solution.

4.4.2 Depth-First Search (DFS)

Depth-First Search expands the deepest node in the current path first. It uses a stack (LIFO) to manage the frontier.

- Not complete in infinite-depth spaces or if it gets stuck in loops.
- Not optimal.
- Time complexity: $O(b^m)$ where m is the maximum depth.

- Space complexity: $O(bm)$.

4.4.3 Depth-Limited Search

It is a variation of DFS where a limit is set on the maximum depth to explore.

- Complete if the solution is within the depth limit.
- Not optimal.
- Useful to prevent infinite descent.

4.4.4 Iterative Deepening Search (IDS)

Combines the benefits of BFS and DFS. Repeatedly applies DFS with increasing depth limits.

- Complete and optimal (for unit cost).
- Time complexity: $O(b^d)$.
- Space complexity: $O(bd)$.

4.4.5 Uniform-Cost Search

Expands the node with the lowest total path cost $g(n)$. Uses a priority queue ordered by $g(n)$.

- Complete and optimal (even with variable costs).
- Time and space complexity: $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$, where C^* is the cost of the optimal solution and ε is the smallest step cost.

4.5 Informed (Heuristic) Search

Informed search algorithms use additional knowledge about the problem domain to find solutions more efficiently. This knowledge is provided through a heuristic function $h(n)$ that estimates the cost from node n to a goal state.

A good heuristic can dramatically reduce the search time by guiding the algorithm toward promising paths.

4.5.1 Greedy Best-First Search

This algorithm expands the node that appears to be closest to the goal, according to the heuristic function $h(n)$.

- Not complete (can get stuck in loops).
- Not optimal.
- Time and space complexity: can be exponential.
- Fast but not guaranteed to find the best path.

4.5.2 A* Search

A* search evaluates nodes using $f(n) = g(n) + h(n)$, where:

- $g(n)$ is the cost from the initial state to node n,
- $h(n)$ is the estimated cost from n to a goal.
- Complete and optimal if $h(n)$ is admissible (never overestimates the cost to reach the goal).
- Time and space complexity: exponential in the worst case.
- If $h(n)$ is also consistent (or monotonic), A* avoids revisiting nodes and is more efficient.

Admissibility:

A heuristic $h(n)$ is admissible if $\forall n$, $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost from n to the nearest goal.

Consistency (Monotonicity):

A heuristic is consistent if for every node n and every successor n' of n generated by action a, it holds that:

$$h(n) \leq c(n,a,n') + h(n')$$

This property ensures that the f values along any path are non-decreasing.

4.6 Heuristic Design

Designing effective heuristics is crucial to improving the performance of informed search algorithms. Good heuristics are typically derived from relaxed versions of the original problem or from domain-specific knowledge.

4.6.1 Relaxed Problems

A common method for designing admissible heuristics is to simplify the problem by removing constraints, which makes it easier to solve. The cost of the optimal solution in the relaxed problem is a valid heuristic for the original problem because it will never overestimate the true cost.

Example: In the 8-puzzle, if we remove the constraint that only adjacent tiles can move, we can define heuristics like:

- $h_1(n)$: Number of misplaced tiles.
- $h_2(n)$: Sum of the Manhattan distances of each tile from its goal position.

Both are admissible. The second (h_2) is generally more accurate and guides the search better.

4.6.2 Combining Heuristics

If h_1 and h_2 are admissible heuristics, then:

- $h(n) = \max(h_1(n), h_2(n))$ is also admissible and often better than either alone.
- $h(n) = w \cdot h_1(n) + (1-w) \cdot h_2(n)$ may or may not be admissible, depending on weights and properties.

The choice of heuristic has a major impact on performance—fewer nodes expanded usually means a better heuristic.

4.7 Local Search and Optimization Algorithms

Local search algorithms operate using a single current state and attempt to find better states by exploring its neighbors. These methods are useful for optimization problems where the goal is to find a state that maximizes or minimizes a cost or objective function.

4.7.1 Hill Climbing

Hill climbing is a simple local search algorithm that iteratively moves to a neighbor state with a better value until no improvement is possible.

- **Basic algorithm:**
  1. Start with an initial state s.

2. Evaluate the neighbors of s.
3. Move to the neighbor with the highest value (for maximization).
4. Repeat until no neighbor is better.

- **Advantages:** Simple, fast.
- **Disadvantages:** Can get stuck in local maxima, plateaus, or ridges.

4.7.2 Variants of Hill Climbing

- **Stochastic hill climbing:** Select a random neighbor that improves the current state.
- **First-choice hill climbing:** Randomly generates neighbors until a better one is found.
- **Random-restart hill climbing:** Runs multiple hill climbing searches from random initial states to avoid local maxima.

4.7.3 Simulated Annealing

Simulated annealing tries to overcome local maxima by sometimes accepting worse states with a probability that decreases over time, controlled by a temperature parameter.

- **Algorithm idea:** At each step, select a random neighbor. If it is better, move there. If worse, move there with probability $e^{-\Delta E/T}$ where ΔE is the cost increase and T is temperature.
- **Cooling schedule:** The temperature T decreases gradually to zero, reducing the chance of accepting worse moves.
- **Advantages:** Can escape local maxima and find near-global optima.

4.7.4 Genetic Algorithms

Genetic algorithms (GAs) are inspired by biological evolution. They maintain a population of candidate solutions and use selection, crossover, and mutation to evolve better solutions over generations.

- **Basic components:**
  - *Population:* A set of candidate solutions (individuals).
  - *Fitness function:* Evaluates how good each individual is.
  - *Selection:* Chooses individuals based on fitness to reproduce.
  - *Crossover:* Combines two parents to produce offspring.
  - *Mutation:* Randomly alters offspring to maintain diversity.
- **Algorithm outline:**
  1. Initialize a random population.
  2. Evaluate fitness of all individuals.
  3. Repeat until stopping condition:
     - Select parents based on fitness.
     - Apply crossover and mutation to create new offspring.
     - Replace some or all of the population with offspring.
     - Evaluate new individuals.
- **Advantages:** Effective for large and complex search spaces.
- **Disadvantages:** Requires careful tuning of parameters; can be computationally expensive.

4.7.5 Local Beam Search

Local beam search keeps track of k states rather than a single one. At each iteration, it generates all neighbors of these states and selects the k best among them.

- **Algorithm steps:**

1. Start with k random states.
2. Generate all successors of these states.
3. Select the k best successors.
4. Repeat until goal found or no improvement.

- **Advantages:** Less likely to get stuck in local maxima than hill climbing.
- **Disadvantages:** Requires more memory and computation.

4.7.6 Genetic Algorithms vs. Local Beam Search

- Genetic algorithms use crossover and mutation, focusing on population diversity.
- Local beam search focuses on the best states, discarding less promising states.
- Both try to avoid local maxima but have different exploration strategies.

4.8 Constraint Satisfaction Problems (CSPs)

A **Constraint Satisfaction Problem** is a problem composed of:

- A set of **variables** $X = \{X_1, X_2, \ldots, X_n\}$,
- A set of **domains** $D = \{D_1, D_2, \ldots, D_n\}$ where each D_i is the set of possible values for variable X_i,
- A set of **constraints** specifying allowable combinations of values.

The goal is to find an assignment of values to all variables such that all constraints are satisfied.

4.8.1 Examples of CSPs

- Map coloring: Variables are regions, domains are colors, constraints prevent adjacent regions from sharing the same color.
- Sudoku: Variables are cells, domains are digits 1–9, constraints enforce unique numbers per row, column, and box.

4.8.2 Types of Constraints

- **Unary constraints:** Restrictions on a single variable.
- **Binary constraints:** Restrictions involving pairs of variables.
- **Higher-order constraints:** Involving three or more variables.

4.8.3 CSP Solving Techniques

Backtracking Search

A depth-first search that assigns values to variables one at a time and backtracks when a constraint is violated.

- **Algorithm outline:**
  1. Select an unassigned variable.
  2. Assign a value consistent with constraints.
  3. Recursively proceed to next variable.
  4. If no valid assignment, backtrack.

Improving Backtracking: Heuristics

- **Minimum Remaining Values (MRV):** Choose the variable with the fewest legal values left.
- **Degree heuristic:** Choose the variable involved in the most constraints on unassigned variables.

- **Least Constraining Value:** When assigning a value, pick the one that rules out the fewest choices for neighbors.

Constraint Propagation

Techniques that reduce domains of variables by enforcing local consistency before or during search.

- **Arc consistency:** For every value of a variable, there is some consistent value in the connected variable's domain.
- **Forward checking:** After assigning a variable, eliminate inconsistent values in neighbors.

Local Search for CSPs

Algorithms like min-conflicts can start with a complete assignment and iteratively reduce the number of conflicts.

4.8.4 Summary

CSPs offer a powerful framework for many combinatorial problems, where exploiting problem structure with heuristics and constraint propagation greatly improves efficiency.

III Search Algorithms

5 Classical Artificial Intelligence 41
5.1 Agent
5.2 The Importance of Search
5.3 Search Methods
5.4 Uninformed Search Methods
5.5 Informed Search Methods
5.6 Iterative Improvement Algorithms

6 Genetic Algorithms 55
6.1 Evolutionary Definitions
6.2 Crossover
6.3 Mutation



5. Classical Artificial Intelligence

In this text we refer to Classical Artificial Intelligence as the traditional methods of modeling and guided solution search. Given a problem, it can be modeled as a set of concrete states and actions to execute in order to transition from one state to another. If this modeling is possible, solving the problem amounts to finding some transition — a path of successive valid actions under the rules of the problem — that allows reaching the final state, which is the solution, starting from an initial state. And among all the possible ways that may exist to achieve this, to search for those specific paths that are most optimal[1] in the sense of minimizing/maximizing the use of resources, or a particular objective function.

5.1 Agent

The concept of an agent is key to the idea of artificial intelligence, and aligns with the conceptualization that artificial intelligence truly manifests itself when a body hosts it [36].

It is defined as anything that can be considered to perceive and respond or act in an environment, with a certain degree of independence or autonomy. Ethologically, it is assumed that the agent has an objective, a reason for being, constituting a **rational agent** or **utility-based agent**. This is materialized through its **performance function**. This determines the success of the agent, and can in turn measure it in the short or long term, that is, having some measure of the time horizon of that success (i.e., strategic decisions). The agent thus, rationally, seeks to perform actions that maximize its performance measure.

On the other hand, it is possible to identify **omniscient agents**. They know at all times the actual result their actions will produce.

Agents can also be classified in relation to whether or not they possess an internal state. Depending on whether or not they maintain an internal state, those that do not are usually called reactive $a = f(P)$, with P being perception and a being action. On the other hand, those that possess an internal state, also

[1] If something is optimal, it cannot be *more optimal*, but you understand the idea.

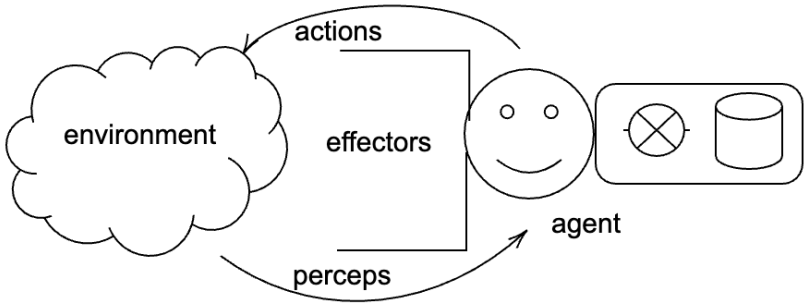


Figura 5.1: The agent and its environment

called those with an internal model, are characterized by $a = f(P,M)$, where M is the internal model, something the agent uses to understand the environment.

5.1.1 Environments

Environment is the term given to the components that surround the agent [2]. It is what provides the source of perception as well as the recipient of the agent's actions. Environments can be **discrete** or **continuous**, depending on whether the number of states and actions is finite or forms a continuum.

They can be **deterministic** or **stochastic**, in relation to whether or not the resulting state of an action applied to the environment from a known prior state is predictable with probability one. In the stochastic case, the transition depends on a probabilistic stochastic process (i.e., it is not known precisely and is estimated probabilistically). In turn, the environment can be **fully observable**, when it is possible for the agent to determine any characteristic of it, or **partially observable**, when precisely this situation cannot be determined. A partially observable environment may appear stochastic to the agent. This idea can be extended to the agent's own learning process. The environment is **known** if the agent knows a priori everything it can know about the environment, while it is **unknown** when it requires an exploration process in order to acquire that knowledge.

The concept of agent is also central in Game Theory [37], where interactions between multiple agents are modeled. Thus, when there is a single agent, the environment is **single-agent**, while when there are several simultaneous agents the environment is **multi-agent**. In this case the agents can be **collaborative** or **adversarial**. A **strategic deterministic** environment can arise precisely when we do not know the actions of the other agents (but do know how the environment operates).

Agents have an experience in the environment given by the sequence of actions they execute and the states they transition through. The scenario is **episodic** when it is possible to determine finite sequences of actions that have a start state and an end state, and above all each episode is completely independent of previous and future episodes. When this independence is not present, the process is said to be **sequential**.

Static environments are those where the state does not change while the agent is not executing its actions, and is **dynamic** when this does occur. On the other hand, if what changes is the score, the evaluation of the objective function, the environment is **semi-dynamic**.

5.2 The Importance of Search

Given an adequate model, in a digital system with finite precision, any artificial intelligence problem could be solved, optimized within that architecture, by brute force, searching all possible options and evaluating the cost function, running it on the system and verifying which one is the best. For example, in a game of Chess, or any other board game. The critical point is that one does not have infinite computing power available, and we want to perform a search that is computationally feasible.

5.3 Search Methods

The resolution of search-based problems starts from the basis of the agent model, where there are different components that pose a well-formed problem, in which there exist:

- **States**: different states s that model the different configurations in which the problem can be.
- **Initial state**: initial configuration s_0 where the problem is not yet solved.
- **Actions**: actions a.
- **Transition Model T**: describes the effect of applying action a while in state s, which leads to a state s', called the successor.
- **Cost function**: Determines the cost $g(T(s,a)) = g(s,a)$ for the transition model T.
- **Solution condition**: Determines whether a particular state s is or is not a solution to the problem.

Using states, actions, and transitions, and above all the evaluation we can make of the solution at each state, it is possible to define a search tree that, starting from an initial state s_0, transitions through states by applying actions and evaluating each of the paths in an attempt to find the optimum: the one with the lowest accumulated cost. Each node n_i of the tree corresponds to a specific sequence of transitions. Each node can then have an associated cost $G(n) = \sum g(s_i, a_i)$ obtained as the sum of the costs of all transitions, such that starting from the initial state it is possible to reach the final state s'_{i+1}. This tree can be viewed as a graph if more than one root node is considered.

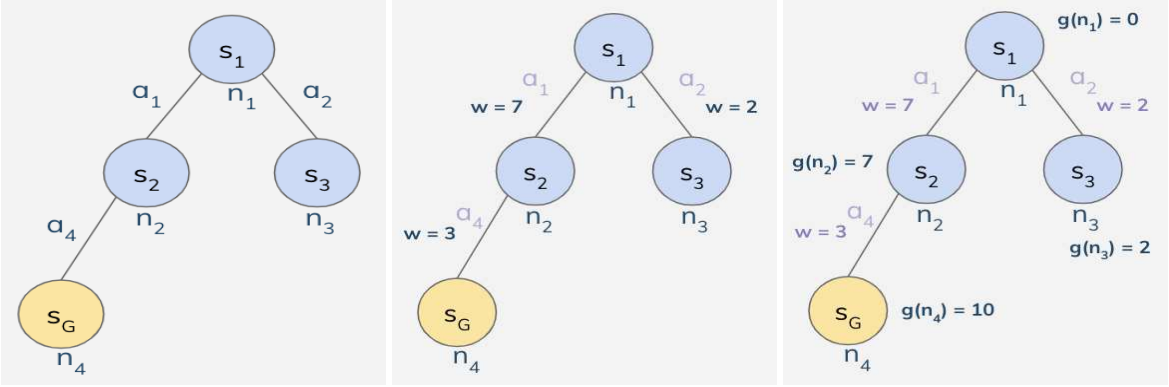
This can be seen in Figure 5.2. Each state transition $T(s,a) = s'$ has an associated cost $g(s,a) = w$. This cost accumulates as the state tree is traversed. Thus, for example, at node n_2 the cost is $G(n_2) = 7$ and at n_3 it is $G(n_3) = 10$.

The most complex aspect lies precisely in finding an adequate representation of the problem, based on defining a state architecture that represents all possible options, keeping the actions bounded and useful, and seeking the possibility of finding computational optimizations in symmetries.

The definition of actions also involves defining meta-actions that can be an aggregate of other smaller actions and that provide direct information toward the objective of the search method (i.e., it is not always necessary to maintain a correspondence between the actions established in the model and concrete actions of the problem in question).

5.3.1 Efficiency

When performing a search it is necessary to have a measure of the efficiency of the procedure. The most basic is pure pragmatism: **finding a solution**. The second is based on **optimizing the cost** of the solution, which can be based on a maximum, minimum, or average cost. It can be measured based on the **number of actions to be performed**, or even based on the actual **computational cost** in terms of memory usage or processing time.



(a) State Tree (b) State Tree with weights (c) State Tree with accumulated costs

Figura 5.2: Base state model.

δ	a_1	a_2	a_3	a_4
s_1	s_2	s_3	Ø	Ø
s_2	s_1	Ø	Ø	s_G
s_3	Ø	Ø	Ø	s_1
s_G	Ø	Ø	Ø	Ø

(a)

Figura 5.3: Table of possible states. State s_G is the goal state.

The search can in turn be **forward**, starting from the initial state of the problem itself, or it can be **backward**, where the root is the solution itself (and from there towards the initial state or states).

5.3.2 Search Strategies

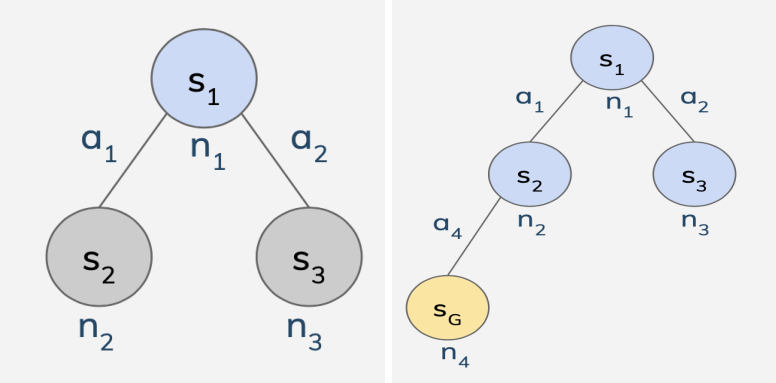
A search is **complete** when it reaches and finds the solution (if it exists and is reachable). It is **optimal** when it finds the reachable solution that has the lowest cost according to some criterion.

5.3.3 Problem Representation

In the state tree *Tr*, given a node n, the child nodes obtained by expanding node n are represented as $s(n_i) = \{n_{i+1}, n_{i+2}, ..., n_{i+3}\}$ (successors). The set formed by all possible expanded nodes is the frontier set *Fr*.

A state is **expanded** when all applicable actions have been applied to it within the search method. The expanded states form the group of explored states *Exp*.

A node is not a state in itself, but is always associated with a particular one. Therefore, the same state can be linked to different nodes. Being $G(n)$ the accumulated cost of the node (the sum of the arc



(a) Unexplored Nodes (b) Explored Nodes

Figura 5.4: Representation Trees.

from n_0 to n), we can find two nodes n_1 and n_2 that can be associated with the same state, such that

$$G(n_1) \neq G(n_2).$$

The different variants of the methods, both uninformed and informed, explore different ways of applying and evaluating algorithm 1. This presented variant maintains a list of expanded nodes to avoid cycles in the graph.

Algoritmo 1 General Search Algorithm (GRAPH-SEARCH)

Search tree Tr
Frontier set Fr
Set of explored nodes Exp
Create Tr,Fr,Exp empty.
Insert initial node $n_0 \rightarrow Tr, Fr$
while $Fr \neq \emptyset$ **do**
    Extract first node from $Fr \rightarrow n$
    if n Goal **then**
        Solution formed by the arc between root $n_0 \rightarrow n \in Tr$. Terminate
    end if
    Expand node n generating successors $\{n_{+1}\}$
    $\{n_{+1}\} \rightarrow Fr$
    $\{n_{+1}\} \rightarrow Tr$, as successors of n
    $n \rightarrow Exp$
    Reorder Fr according to criterion (method-dependent)
end while
if Solution empty **then**
    No reachable solution exists
end if

5.4 Uninformed Search Methods

Uninformed search methods only have the information arising from the problem model itself, distinguishing states by nothing other than whether or not they are **goal** states, i.e., the target states that the search process itself attempts to find. They are inefficient in problems where estimations could be made.

The most common uninformed search methods are:

- BFS
- DFS
- IDDFS
- Uniform Cost

5.4.1 Breadth First Search

BFS expands the shallowest nodes first. That is, it selects the first node to expand and recursively expands until reaching the maximum depth. For this to make sense, the check of whether a state is a goal state or not is performed when the node is added to the frontier. It is complete if the branching factor is finite, finding the solution with the lowest depth. When all actions have the same cost, the solution is optimal. The downside of BFS is that it has complexity $\mathcal{O}(b^d)$, where b is the number of nodes at the same level and d is the depth. This does not scale.

5.4.2 Uniform Cost Search

UCS, or Uniform Search, expands the node with the lowest accumulated cost. Of course, when the cost is uniform, it is exactly equal to BFS. The search is optimal when $G(s(n)) \geq G(n)\ \forall n$ holds, or equivalently that $g(s,a) \geq 0\ \forall a,s$.

5.4.3 Depth First Search

DFS expands the deepest node. It is not optimal, but usually has much lower spatial complexity, especially in its version without repeated states, since with repeated states it can get stuck in infinite loops. When many solutions exist, it is usually faster than BFS. From an implementation standpoint, BFS uses a FIFO queue for *Fr* while DFS uses LIFO.

To avoid the problem of infinite state search, the search depth can be limited with an arbitrary hyperparameter value called **Depth Limited Search**, generating a search that is neither complete nor optimal. To avoid having to arbitrarily set this parameter, IDDFS **Iterative Deepening Depth First Search** is DLS where the depth can vary according to a certain criterion. It can be implemented by maintaining the leaf nodes that were not expanded due to the depth limit between each iteration. With an infinite depth value it is precisely vanilla DFS.

5.4.4 Repeated States and Backtracking

The motivation behind the use of repeated states is to avoid returning to ancestor states, and to avoid creating paths that have cycles that break the tree structure. Nor to re-expand states that have already been expanded previously. It is a trade-off between the cost of storing and checking states.

Backtracking refers to the ability to generate successors and add them to the frontier one at a time, and after verifying that the chosen path does not lead to the optimum, to be able to undo all changes made to the supporting data structures of the algorithm, and choose another individual node, and evaluate it.

5.5 Informed Search Methods

Informed methods, on the contrary, have more information to compare between different states. With this they can make an estimate to calculate how much remains to solve the problem, by computing a **heuristic**. With that information they can also calculate the cost of the problem. This more refined model allows addressing problems that are intrinsically more complex and implementing search mechanisms that are in turn more efficient. The two main representatives are:

- Greedy
- A*

These methods are based on the definition of a **heuristic** (def 5.5.1), which is a *heuristic* estimate of the cost associated with each state, that is easy to compute [2].

Definición 5.5.1 — Heuristic. Heuristic function $h(s)$ represents the estimated cost of the cheapest route from state s to a goal state.
If s Goal $\rightarrow h(s) = 0$
If s not Goal $g(s,a) > 0 \; \forall a$: Action $\rightarrow h(s) > 0$
If s not Goal $g(s,a) \geq 0 \; \forall a$: Action $\rightarrow h(s) \geq 0$

One can abuse the notation by writing

$$h(n) = h(s),$$

indicating that s is the state represented in node n. The value of the heuristic depends on the state.

Proposición 5.5.1 — Admissible Heuristic. A heuristic is admissible if it never overestimates the actual cost. It is perfect if it estimates exactly the actual cost $h^*(n)$.

Heuristics can be found by dividing the problem into sub-problems, relaxing the constraints or rules of the problem. The heuristic can also arise as the maximum of other metrics, or can emerge from combining heuristics, considering the evaluation of each one, especially when they address different sub-problems.

5.5.1 Local Greedy Search

Being **Greedy** refers to the fact that the algorithm searches for the best option at hand, without looking further ahead in relation to the following steps. This informed variant starts from the root node and expands it, and on the expanded nodes estimates h. It then takes the node with the lowest h; this is the greedy step. If the newly expanded set is empty, it needs to perform **backtracking**, going back in the algorithm and exploring another option.

This solution is neither optimal nor complete for the case where it does not evaluate state repetition. As in all informed algorithms, the key usually lies in the selection of a very good heuristic function, which will determine the temporal and spatial complexity of the algorithm.

If the algorithm used is 1 but using $h(n)$ to establish the ordering of Fr at step 15, the algorithm is called **Global Greedy Search**. In this case, despite having a higher cost than LGS, backtracking is more effective as it can evaluate more options.

[2] From physics, there is a similar method called the *Fermi method*, which is a quick technique that, given a problem, allows one to understand the order of magnitude of the sought answer, especially when data is not at hand or a quick response is needed [38]

5.5.2 A*

It is the **star** algorithm of search methods and one of the most widely used algorithms in the world. It is based on the function

$$f(n) = G(n) + h(n),$$

and orders the nodes in Fr based on this function at step 15, breaking ties based on $h(n)$ when $f(n)$ values are equal. It is identical to uniform cost, except that it uses $f(n)$ to estimate the lowest cost.

The algorithm guarantees that if $h(n)$ is admissible, then $f(n)$ never overestimates the actual cost of the best solution passing through node n. The search is complete if there is finite branching and the cost is greater than some $\varepsilon > 0$. On the other hand, it is optimal when the following holds:

Teorema 5.5.2 A* is optimal (the optimal path to the solution is found if it exists) when:

- Each node of the graph must have a finite number of successors.
- The cost of each arc must be greater than some $\varepsilon > 0$.
- The heuristic must be admissible $h(n) \leq h^*(n)$

We define

$$f^*(n) = G(n) + h^*(n)$$

.

Of course it requires memory and processing for the computation of the heuristics.

Proposición 5.5.3 — Lemma 1: Admissible Heuristic. At any given moment in the A* search, there exists a node $n \in Fr$ that satisfies

1. n is on the optimal path to the goal
2. $f(n) \leq f^*(n)$

Demostración. Proof by Induction.

Base Case

For the initial node n_0. All paths begin with n_0 and since a solution exists, n_0 is on the optimal path to the goal.

$$f(n_0) = G(n_0) + h(n_0) = h(n_0) \leq h^*(n_0) = f^*(n_o) \tag{5.1}$$

Inductive Step

The frontier at step k contained a node that was on the optimal path. Then node n^e is chosen for expansion.

If node n^e is not a node on the optimal path, the frontier will continue to contain said node, so (1) holds and n' is that node.

If node n^e is such a node, A* expands it, and some of its children should be on the optimal path, so (1) holds and $n' = n^e$

$$f(n') = G(n') + h(n') \leq G(n') + h^*(n') = f^*(n') \tag{5.2}$$

■

Teorema 5.5.4 — Termination of A*. Assuming the branching is finite, and the cost of the rules/actions is always greater than or equal to a number greater than 0. Then, if A* may not terminate, it will eventually reach a point where

$$f(n) > f*(n),\ \forall n \in Fr$$

.

Definición 5.5.2 — Dominance of Search Methods. We say that M_1 dominates M_2 if every node expanded by M_1 is also expanded by M_2. We say that M_1 **strictly** dominates M_2 if

- M_1 dominates M_2
- M_2 does **not** dominate M_1

Heuristic Efficiency

Assuming h_1 and h_2 are admissible heuristics, if h_2 dominates $h_1 \rightarrow A_2^*$ will expand fewer nodes than A_1^*

This is because there will be nodes that A_1^* expanded (and do not lead to the optimal path) that A_2^* did not, so for those nodes:

$$G(n) + h_1(n) \leq f^*(n) \leq G(n) + h_2(n)$$

In this way, as a general rule it is better to use admissible heuristics that give the highest possible value.

Combination of Heuristics

Given a set of admissible heuristics $h_1, \ldots, h_m$, a new heuristic h' can be defined as their combination, defined as follows:

$$h' = \text{máx}(h_1, \ldots, h_m)$$

This new heuristic has the following properties:

1. It is admissible
2. It dominates all the heuristics that make it up

Consistency of Heuristics

If a heuristic is consistent, every time A* expands a node, it will have found an optimal path to it.
If a heuristic is admissible, this is not necessarily the case.

Completeness of A*

A* might never terminate.
Some of the situations where this can occur are:

- The branching is infinite for at least one state.
- There exist arcs with costs ≤ 0.
- The arcs have costs > 0, but are asymptotically decreasing (Zeno's Paradox)

5.5.3 Iterative Deepening A*

A concept inspired by IDDFS, an iterative cutoff is performed with a limit. DFS is also used here. This search is complete and optimal under the same conditions as A*. It requires less memory than A*,

but may expand more nodes and even expand the same node many times (the same problem as IDDFS).

The limit is a threshold imposed on $f(n)$.

1. Initially, $Lim = f(n_0)$ is taken
2. While no solution is found, DFS is performed up to Lim.
3. If no solution is found, n' is taken as the frontier node with the lowest value of f, and this value is taken as the new limit.

5.5.4 Heuristic Path Planning

Best-First with

$$f(n) = (1-\omega) * G(n) + \omega * h(n)$$

$$f(n) = \begin{cases} \omega = 0 & \text{Uniform Cost Search} \\ \omega = \frac{1}{2} & \text{A* Search} \\ \omega = 1 & \text{Global Greedy Search} \end{cases}$$

5.5.5 Game Example: Sokoban

Sokoban [39] is a classic game, characterized by a warehouse worker who must, in two dimensions, bring a set of boxes to their available destinations. *Soko* means warehouse and *ban* is a suffix representing the person in charge or the worker. It was designed in 1982 by Hiroyuki Imabayashi for Spectrum and Commodore computers. The game consists of different boards with available spaces where the worker can move, impassable walls, the locations where the boxes are, and the destinations where they must be deposited. To move each box, the worker stands behind it and can push it toward one of the four cardinal directions. This game is excellent as a platform for implementing and testing search algorithms. The reason for this is that the problem is NP-Hard and at the same time PSPACE-complete. Simple boards require a large amount of processing capacity and memory to be solved. It also has deadlock situations that cannot be resolved, board configurations where there is no way to move one of the boxes or to bring it to any of the destinations.

5.6 Iterative Improvement Algorithms

Search algorithms with iterative improvement are based on starting from a solution and improving it, optimizing the functional, through perturbations to that solution.

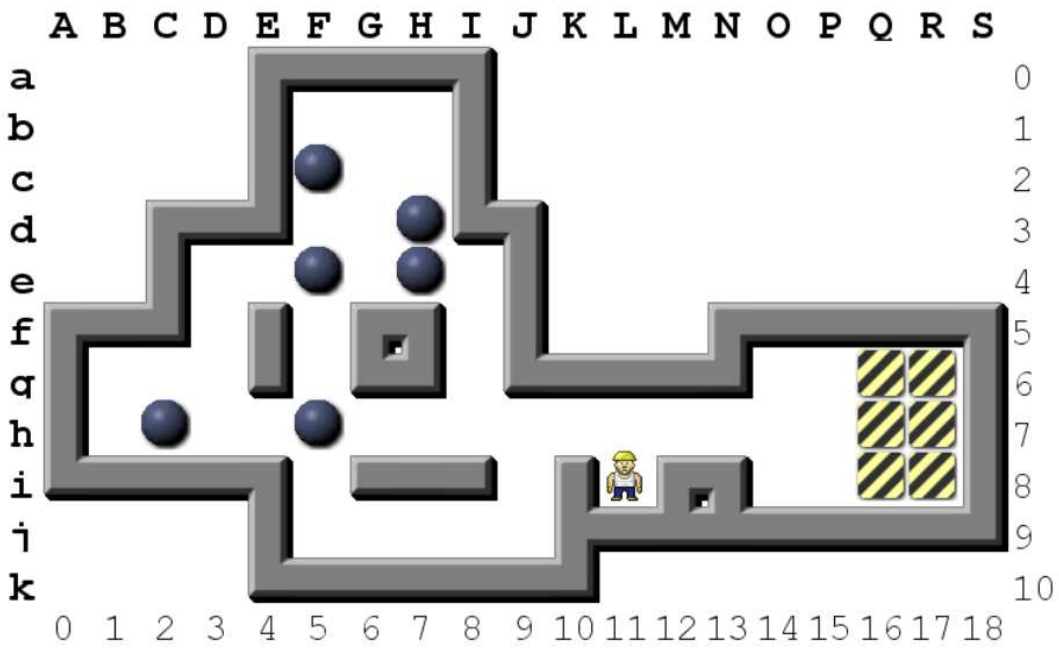
Definición 5.6.1 — Iterative Improvement. Let $f(S)$ be an optimization function that evaluates how good a particular solution S is. Iterative algorithms are based on finding $S*$ such that

$$S* / \forall S \;\; f(S*) \geq f(S)$$

The different options revolve around the idea of randomly varying the found solution, moving in the possible directions of growth or decrease (see Optimizations in chapter 7).

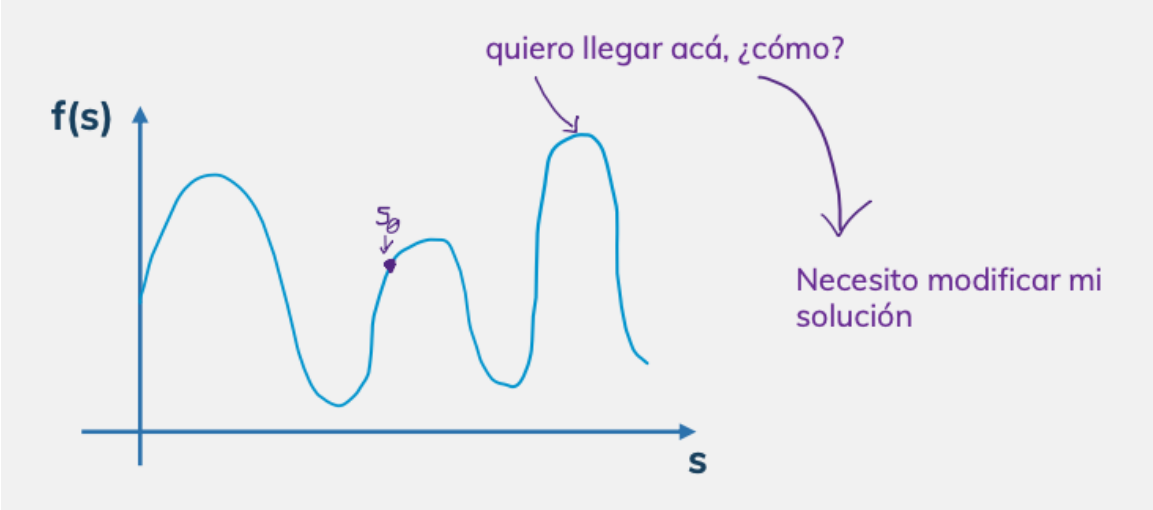
Travelling Salesman Problem

The travelling salesman problem (TSP) is a classic NP-Hard problem. It is a graph traversal problem where there are n cities and the distance between each of them. The problem formulation involves finding the shortest route, visiting each city exactly once and at the end returning to the city from which one started at the beginning.



(a)

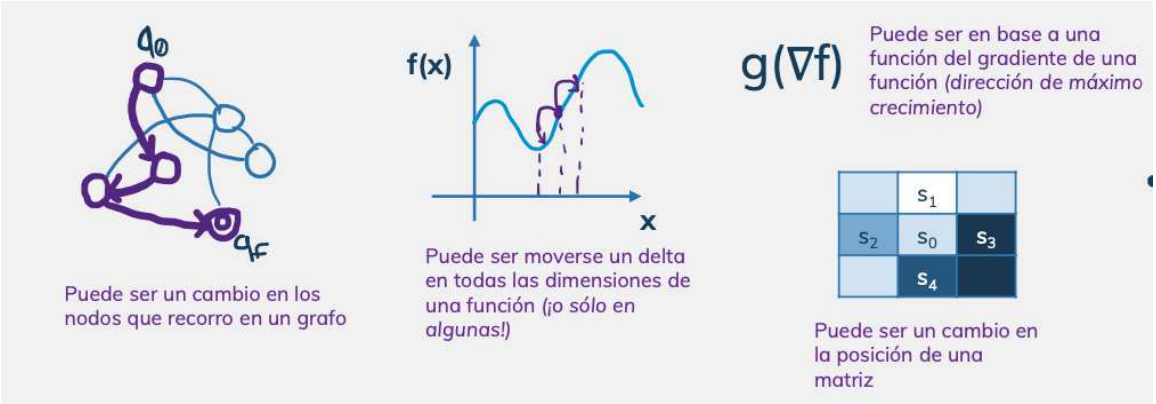
Figura 5.5: Image of a typical Sokoban board, taken from [39].



(a)

Figura 5.6: Optimization function and iterative improvement.

The input to the problem is the number of cities n and the n ordered pairs with the coordinates of each city (x,y). The output of the problem must be the list of cities by their index that must be visited, or eventually the waypoints corresponding to the succession of points that will reflect where the salesman must go (touching and passing through all cities and completing the circuit).

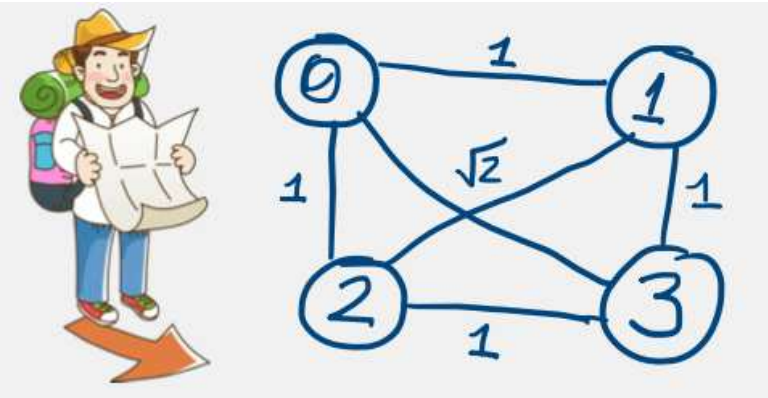


(a)

Figura 5.7: Alternatives for implementing iterative improvement.

Solution by iterative methods

Let us consider the TSP problem in Figure 5.8, with the distances given by the labels on the edges.



(a)

Figura 5.8: Basic Travelling Salesman Problem

To solve this problem iteratively, one can proceed as follows:

1. Take a random solution, for example: $sol = \{0-3-1-2(-0)\}$
2. Evaluate the solution in the objective function: $f(sol) = 4.83$.
3. Random changes are made to the solution sol. For example, a city is chosen and its position is permuted: $sol_{a1} = \{3-0-1-2\}, sol_{a2} = \{0-1-3-2\}, sol_{a3} = \{0-1-2-3\}$
4. One is selected based on its evaluation: $f(sol_{a2}) = 4$

5.6.1 Hill Climbing

In this case the random variations are made in the direction of growth of the function, indicated by the gradient, so as to climb to the summit (hence the name). It is a local heuristic algorithm, where at each step a new frontier Fr is generated (which is not the same frontier used in search methods to identify nodes to expand). The problem with this algorithm is that it tends to get stuck very easily in local maxima.

Algoritmo 2 Hill Climbing Algorithm

1: Frontier set Fr
2: **while** $Fr \neq \emptyset$ **do**
3: Move one arbitrary step in both directions of each dimension.
4: Take each solution point $s_i \rightarrow Fr$ (add it to the frontier).
5: Evaluate the frontier points $f(s_i)\ \forall s_i \in Fr$
6: Select the maximum value of the frontier as the new solution $argmax_{s_i} f(s_i)$
7: **end while**

Simple Hill Climbing: Chooses the first solution in the frontier with a better evaluation than the current state.

Steepest Hill Climbing: Always chooses the solution in the frontier with the best evaluation compared to the current state.

Stochastic Hill Climbing: Randomly chooses a solution from the frontier and, based on how good it is, decides whether to move to it or choose another.

5.6.2 Simulated Annealing

The simulated annealing algorithm, a classic of computational physics. This algorithm arises from modeling the behavior of ferromagnetic metals subjected to a magnetic field, initially at high temperature, where the temperature is gradually reduced, which leads to the material becoming ferromagnetic, since the initial thermal motion gives the molecules the opportunity to settle, and the tendency in which they settle as the temperature decreases is influenced by the magnetic field [4].

The algorithm is similar to Hill Climbing, but involves a parameter that regulates stochasticity, the exploration/exploitation balance, and which in this algorithm represents the **temperature**. As the iterations go by, this temperature is progressively decreased according to a progression profile. The algorithm proceeds similarly to Hill Climbing, establishing a frontier set Fr, but ignores the evaluation of each solution deposited there with a probability relative to the temperature (and optionally, proportional to its evaluation). That is, the algorithm selects the best available option and moves to that solution according to the exponential profile itself (choosing an option where $d < 0$, without improvement, more so at the beginning, and sticking with the better options as the temperature decreases). The temperature can be reduced iteration by iteration or in blocks depending on a fixed value or some condition.

Algoritmo 3 Simulated Annealing

```
1: Take s = s_0 ; t = t_0
2: while Pis False do
3:     while Repeat N_rep do
4:         Select s' ∈ E(s) (neighborhood of s)
5:         d ← f(s') − f(s)
6:         if d > 0 then
7:             Take s' (it is a better solution)
8:         else
9:             u ← rand(0,1)
10:            if u < exp(−d/t) then
11:                s = s'
12:            end if
13:        end if
14:    end while
15:    t = α(t) Temperature reduction
16: end while
```

5.6.3 Beam Search

Similar to Hill Climbing but starts with k initial nodes[3]. At each step, the successors of the k nodes are generated, which will form the frontier. Of all the successors, the best k nodes from the frontier are taken and the process is repeated. It is different from k hill climbings in parallel, since it compares all the best ones across all paths against each other, linking the information among all of them, and can therefore quickly eliminate bad initial nodes in order to take better paths. It can converge heavily among the k nodes, generating so little diversity that it becomes a much more costly hill climbing.

Stochastic Beam Search

Similar to Beam Search, but chooses the k nodes with a probability proportional to their evaluation. In this way, **bad** nodes can also be chosen (with lower probability). Similar to the natural selection process (see chapter 6).

[3]The value of k is known as the beam width



6. Genetic Algorithms

Biotic nature plays an optimization game embodied in evolution and natural selection. The struggle for survival and perpetuity exerts a pressure on living beings so that only those who manage to **find** the right combination of genes for each specific environment survive and perpetuate that combination.

This idea was captured by several researchers who embodied it in an optimization algorithm — in fact a suite of **Genetic Algorithms** that offer a very powerful framework for guiding a search [5].

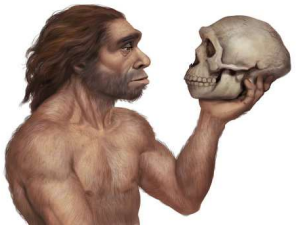


Figura 6.1: Neanderthals and us, a complicated mix.

6.1 Evolutionary Definitions

The approach offered in this text is that based on Holland [40]. It is a probabilistic optimization/adaptation algorithm structured according to a simplified model of the mechanics of natural selection and the theory of evolution. It has the enormous advantage of being highly parallelizable[1] and has multiple hyperparameters and variables.

The general idea of genetic algorithms (GA) consists of generating a population of individuals, where the free parameters to be optimized are encoded in a kind of gene. Thus each individual represents a complete combination of all the parameters. In turn, those parameters are precisely the variables of an optimization function, which in GA is called *fitness*, and which produce a numerical result. Thus the maximization/minimization of that cost function depends on the combination of parameter values embodied in the genetic code of each individual. Through a simulation of the evolution of the population of individuals, the algorithm attempts to find those that precisely reach the optimum being sought.

[1] Why?

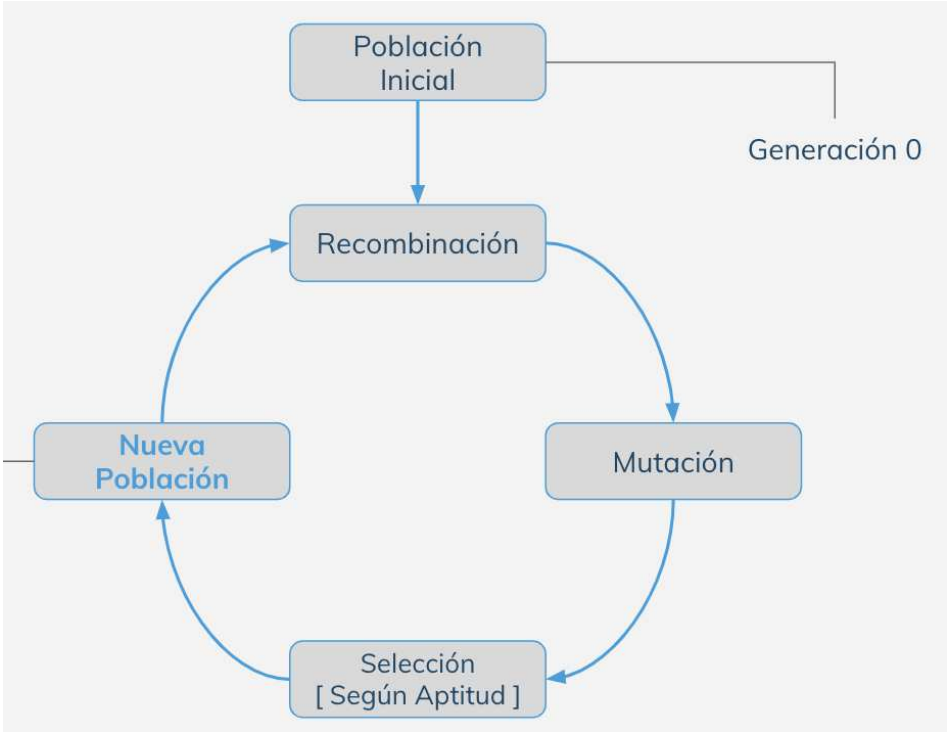


Figura 6.2: Block diagram of the basic pattern of a genetic algorithm

The basic components are:

- Genotype: structure/architecture
- Initial population
- Adaptability function = fitness = aptitude
- Parent selection method
- Crossover method
- Mutation method
- New generation selection method
- Stopping condition.

Definición 6.1.1 — Genotype. Establishes the particular form of the gene, the way in which it will be encoded at the level of bits, bytes, or data structure.

Definición 6.1.2 — Locus. It is the position of a particular gene within the genome itself.

Definición 6.1.3 — Chromosome. The chromosome is the complete set of genes. For example, according to the example in Table 6.1 it would be $[1][0][1][0][1]$.

Definición 6.1.4 — Phenotype. These are the observable characteristics affected by a given gene.

Amarillo Patito	Triangulo
[1][0]	[1][0][1]

Cuadro 6.1: Table caption

Definición 6.1.5 — Alleles. The possible variants of a gene, represented by the encoding established for the gene. The variety of alleles represented in the population is called diversity.

Definición 6.1.6 — Generation 0. Generation 0 is the initial population of individuals of size N.

Definición 6.1.7 — Fitness. It is the cost function $f(i)$ of the optimization that the algorithm attempts to solve. It is evaluated for each individual i.

Definición 6.1.8 — Relative Fitness. Normalized to the set of individuals

$$p(i) = \frac{f(i)}{\sum_{j=1}^{N} f(j)} \tag{6.1}$$

Selection Methods

The different selection methods determine alternatives for selecting individuals from one generation to the next.

⊕ **Direct Sampling: Elite**

Chooses a set K, an elite, from the set of individuals. To do so, they are ordered by fitness and each one is chosen $n(i)$ times according to 6.2.

$$n(i) = \lceil \frac{(K-i)}{N} \rceil \tag{6.2}$$

⊕ **Stochastic: Roulette**

From the relative fitnesses $p(j)$, the cumulative fitnesses for each individual are derived.

$$q(i) = \sum_{j=0}^{i} p(j) \tag{6.3}$$

Then K random numbers are generated from $r_j \leftarrow U[0,1)$ such that $q_{i-1} < r_j < q_i$. That is, the sample space is distributed among all individuals according to their relative fitness (the higher the fitness, the greater the chance of being selected in the roulette).

⊕ **Universal**

Similar to Roulette but changing the equation for r_j based on $r \leftarrow U[0,1)$.

$$r_j = \frac{r+j}{K}, j\varepsilon[0,(K-1)] \tag{6.4}$$

⊕ **Ranking**

Ranking selection derives an alternative pseudo-fitness function $f'(i)$ that is computed based on a ranking. The $rank(i)\varepsilon[1,N]$ is a permutation of assignments to the N individuals given by the actual fitness function $p(i)$. With this, the following is calculated:

$$f'(i) = \frac{N - rank(i)}{N} \tag{6.5}$$

From this value, the computation is performed according to roulette using the relative pseudo-fitness.

⊕ **Boltzmann**

For this method, a pseudo-fitness function is also defined with the following formula:

$$f'(i,g,T) = \frac{e^{\frac{f(i)}{T}}}{< e^{\frac{f(i)}{T}} >_g} \tag{6.6}$$

where i corresponds to the individual, T corresponds to the temperature of the system, $f(i)$ is the fitness function, and $<>_g$ is the computation of the population fitness average. This pseudo-fitness function is then used as the fitness applying selection via the Roulette method.

Temperature-based methods, just as with iterative improvement methods, apply an exponential temperature decrease function that has the effect of balancing exploration/exploitation. This favors the search for variants at the beginning of the algorithm, and the identification of the most efficient ones as the algorithm progresses.

⊕ **Probabilistic and Deterministic Tournaments** Tournament-based selection methods are based

on forcing a competition, where a subset of individuals smaller than the total population is selected. The advantage is that it has a higher execution speed, since not the entire population is processed, nor does it require intermediate variables, using the fitness function directly. It also has ease of implementation. The disadvantage, on the other hand, is that it imposes a very high selection pressure, with all that this entails.

Deterministic selection methods are based on:

- From the original population of size N, M individuals are chosen at random.
- From the M individuals, the best one is chosen.
- The process is repeated until the K required individuals are obtained.

On the other hand, the probabilistic tournament variant introduces an extra stochastic component for the selection of individuals:

- A threshold from $[0.5, 1]$ is chosen.
- From the original population of size N, only 2 individuals are chosen at random.
- A value r is taken at random uniformly from $r \leftarrow U[0, 1)$.
- If $r < Threshold$, the fittest individual is selected.
- Otherwise, the **least** fit individual is selected.
- The process is repeated until the K required individuals are obtained.

Probabilistic tournaments give more opportunities to those individuals to maintain variability in the genetic population.

Premature Convergence

Premature convergence occurs when the population converges to a subset with very little variety without having reached an acceptable fitness. That is, the necessary and sufficient variability to maintain exploration in the space of possibilities is reduced.

This causes the optimization method given by the genetic algorithm to yield a suboptimum, unless that point of convergence is the global maximum or the population escapes from said local maximum through mutation.

The main causes are a **very high** selection pressure, a **very low** mutation probability, or a very small population size[2].

6.2 Crossover

Once the individuals are selected, they are paired and gene recombination takes place. A parameter is usually set that regulates the probability of recombination, P_c, which defines whether or not recombination occurs in the generation of offspring. In the case where there is no recombination, only the impact of mutation applies and the genes are simply copied from parents to offspring.

Gene Encoding

It might seem natural to use one bit per gene in the encoding of a gene, but this poetic idea is usually not a good one.

There are different alternatives for recombination.

6.2.1 One-Point Crossover

Straightforward: a locus is chosen at random and the alleles are exchanged from that locus onward. That is,

$$P = [0, S]$$

with S being the number of genes.

[2] It is believed that 900,000 years ago *Homo sapiens sapiens* almost went extinct, generating premature convergence, with only 1,300 living human individuals existing on Earth at that time, leaving a mark on our genes from that enormous reduction in genetic diversity[41].

6.2.2 Two-Point Crossover

In this case, two loci are chosen at random and the alleles between them are exchanged.

$$P_1 = [0, S]$$

$$P_2 = [0, S]$$

with $P_1 <= P_2$.

6.2.3 Ring Crossover

First, the gene is considered as a ring, where the last connects with the first in a circular manner. Then a random locus P and a length L are chosen. Finally, the segment of length L starting from P is exchanged, taking into account the ring structure. In this case $P = [0, S-1]$ with $L = [0, \lceil \frac{S}{2} \rceil$.

6.2.4 Uniform Crossover

An exchange of alleles occurs at each gene with probability P, which is normally 0.5.

6.3 Mutation

Mutation is defined as a variation in the genetic information that is stored and encoded in the chromosome. For mutation to be more efficient, a correct architecture and separation of genes is necessary. The algorithmic objective of mutation is to enrich genetic diversity (explore the solution space) and avoid local maxima (and escape from them).

The variants are:

- **Gene mutation:** a single gene is altered with a probability P_m.
- **Limited multi-gene mutation:** a random number $[1, M]$ of genes to mutate is selected, with probability P_m.
- **Uniform multi-gene mutation:** each gene has a probability P_m of being mutated.
- **Complete mutation:** with a probability P_m, all genes of the individual are mutated, according to the mutation function defined for each gene.

The mutation or change of a gene may not only imply changing a value (1 to 0) but, depending on the encoding, requires a precise definition. For example, it may involve applying a delta to the gene, in some direction and with some distribution.

6.3.1 Generating Generations

From a pragmatic perspective, there are also different alternatives for building a new generation.

Fill-All

Generate K offspring from K parents. In this case the new generation will be formed by selecting N individuals from the set $[N+K]$, where N are the individuals from the current generation and K are the offspring.

Fill-Parent

Generate K offspring from K parents.

- If $K > N$, the new generation is created by selecting N from the K offspring only.
- If $K <= N$, the new generation will be made up of the K generated offspring $+(N-K)$ individuals selected from the current generation.

Generational Gap

Determines the number of individuals that survive from one generation to the next. This is captured in an indicator $G\varepsilon[0,1]$.

- $G = 1$: the entire population is replaced.
- $G = 0$: no individual is replaced.
- G determines the convex combination between retention and replacement, with $(1-G)*N$ individuals from the current generation and $G*N$ the new replacement individuals.

Stopping Criteria

The last point to clarify is the identification of possible stopping criteria: when to stop.

- Time
- Number of generations
- Acceptable solution (bound on the fitness function).
- Structure: when a relevant portion of the population does not change for a number of generations (population stagnation).
- Content: when the best fitness does not change for a number of generations (optimal values are reached in the function).

Schemas

Schemas serve to group different genes using wildcards. For example, the schema $*01001*01$ represents $001001001, 001001101, 101001001, 101001101$.

Teorema 6.3.1 — Schema Theorem. Those schemas with an average fitness above the population mean, of small length and low order, will increase their presence exponentially in successive generations.

The order of a schema determines the number of information-bearing genes it preserves and is calculated as

$$o(S) = Len - wildcards$$

where *Len* is the length of the gene, and *wildcards* represents the number of wildcards in the schema. For example, $o(*01*) = 2$, $o(**0*1***1) = 3$. The higher the order of the schema, the greater the probability of losing it in a mutation.

The use of schemas allows the composition of **building blocks**. Short schemas of low order but with high fitness can be selected, recombined, and re-selected again to progressively form chromosomes

with higher fitness. This allows reducing the complexity of the problem by dividing it into subproblems: schemas are optimized, new schemas are found, or are eventually grouped together.

IV

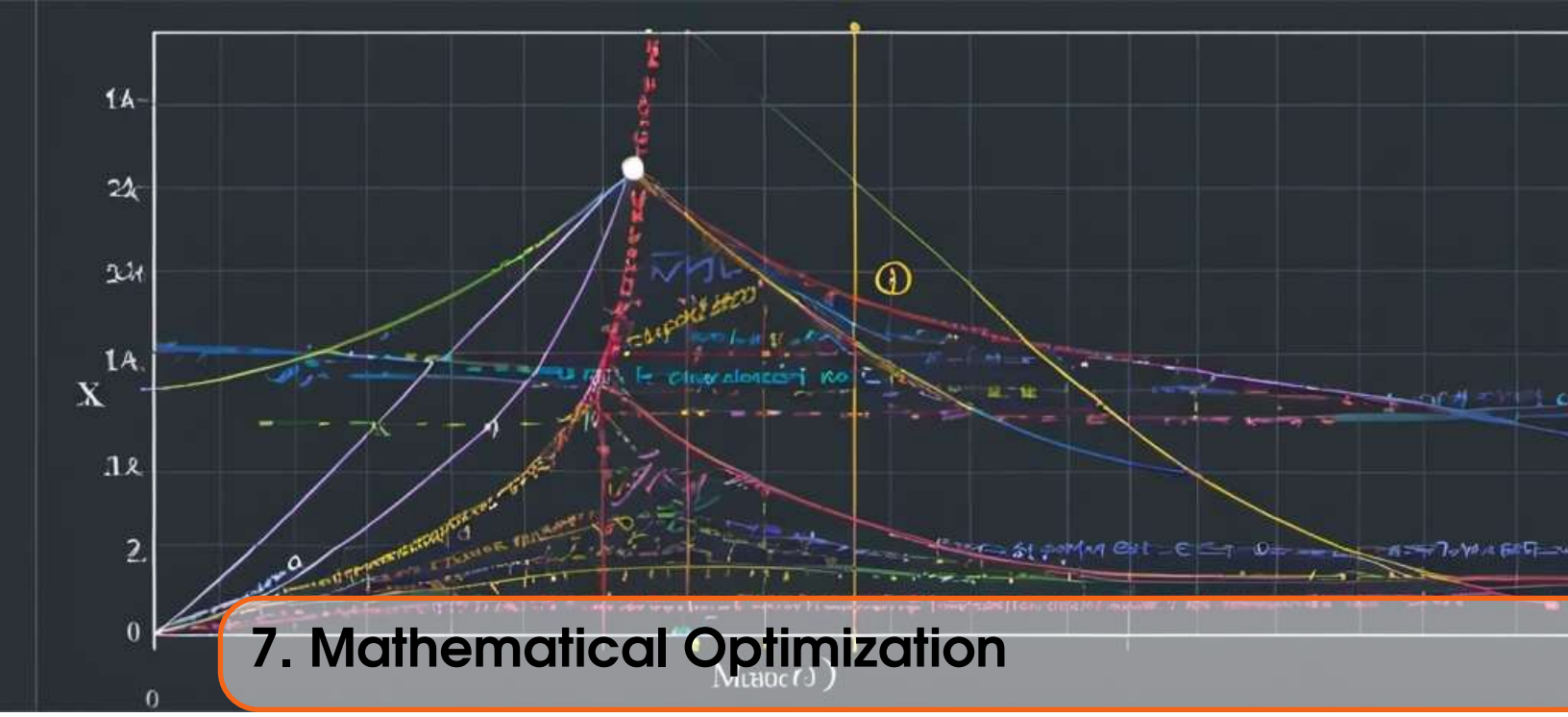
Perceptron

7 Mathematical Optimization 65
7.1 The Coke Can
7.2 The Optimization Problem
7.3 The Simplex Method
7.4 Unconstrained Nonlinear Optimization

8 The Neuron Model 75
8.1 The Bio-Inspiration
8.2 Linear Separability

9 The Era of Connectionism 85
9.1 Hierarchical Agglomerative Structure
9.2 Architecture
9.3 Optimization in Neural Networks

10 Evaluation Metrics 97
10.1 Identifying the Pattern Correctly
10.2 Standard Metrics



7. Mathematical Optimization

The area of mathematical optimization [10] can be considered "half mathematics"[1]. It arises from the early work of Cauchy in 1847 and the proposal of the gradient descent method. Then, in the post-war era, it experiences an important surge driven by the quantitative development of mass industrialization, with linear programming algorithms and the stochastic variant of the gradient descent algorithm. In the 1980s, convex theory and the general optimization framework emerge. Finally, in the first decade of the 21st century, optimization ideas begin to be applied massively to machine learning problems. From 2012 onward, a new push begins that finds a direct application in neural networks. Many professors and students devoted their lives to this subject, sacrificing their happiness with hours of study in a dark and dusty classroom. Now it is our turn [42].

7.1 The Coke Can

Imagine a can of Coca-Cola. An interesting characteristic one might look for is to find what the dimensions of a cylindrical can containing one liter must be in order to minimize its surface area and thus the amount of aluminum required.

This is a typical problem studied in Calculus or Mathematical Analysis courses around the world and it can be solved exactly as an optimization problem. The idea is to start from the basis of defining the problem according to the geometry of the can:

$$V = \pi r^2 h \tag{7.1}$$



Figura 7.1: You're welcome, Coca-Cola Corporation, for solving your problem.

[1] It can be shown that every area of mathematics is half mathematics, but we have no room here to do so.

where r is the radius of the cylinder and h its height. Then the surface area of the can, which will use its aluminum, assuming negligible thickness, is given by

$$S = 2\,\pi\,r^2 + 2\,\pi\,r\,h \tag{7.2}$$

From this the formulation as an optimization problem arises

$$S(r,h) = 2\,\pi\,r^2 + 2\,\pi\,r\,h \qquad\qquad g(r,h) = \pi\,r^2\,h = 1000\,\text{cm}^3 \tag{7.3}$$

That is, we seek to optimize $S(r,h)$ (find the r and h that make it as small as possible) subject to the constraint given in $g(r,h)$ at a fixed volume. The solution involves first solving for h from the second equation, substituting it into the first, and solving directly using the property that the derivative of a function is zero at its extrema.

$$g(r,h) = \pi\,r^2\,h = 1000\,\text{cm}^3$$
$$h = \frac{1000\,\text{cm}^3}{\pi\,r^2}$$

Substituting into 7.2

$$S(r) = 2\,\pi\,r^2 + 2\,\pi\,r\,(\frac{1000\,\text{cm}^3}{\pi\,r^2})$$
$$S(r) = 2\,\pi\,r^2 + \frac{2000\,\text{cm}^3}{r}$$

The extreme point occurs when the derivative of $S(r)$ with respect to r is zero. This is

$$S'(r) = 4\,\pi\,r - \frac{2000\,\text{cm}^3}{r^2} = 0$$
$$4\,\pi\,r = \frac{2000\,\text{cm}^3}{r^2}$$
$$4\,\pi\,r^3 = 2000\,\text{cm}^3$$
$$r^3 = \frac{500\,\text{cm}^3}{\pi}$$
$$r = \sqrt[3]{\frac{500\,\text{cm}^3}{\pi}}$$

Since the point we wish to find is the minimum, this occurs when the derivative goes from negative to positive, and that is precisely at the positive solution $r = 5.41\ cm$, which yields a value of $h = 58\ cm$ for the height[2]. This is a typical nonlinear optimization problem with constraints and, as can be seen, computing the derivative plays a crucial role in its solution.

[2]Note that this is a half-meter-tall can, but it holds one liter!

7.2 The Optimization Problem

Optimization can serve to minimize a company's production costs, solve transportation dynamics problems, be applied in game theory, find the optimal parameters of a distribution function, etc. There exist numerous algorithms and areas of optimization, among which we can find:

- Discrete Optimization: Integer and combinatorial programming
- Discrete Optimization: Diophantine equations
- Linear Optimization: Linear Programming
- Unconstrained convex nonlinear optimization: guarantee of global extrema
- Non-convex optimization: first-order methods: GD, SGD, Adam
- Non-convex optimization: second-order methods: Newton
- Non-convex optimization: zeroth-order: Bayesian, Powell, Simultaneous Optimistic Optimization

To outline the optimization process, the Simplex method is presented first, and then we will focus specifically on unconstrained nonlinear optimization, given its relevance for training neural networks.

Teorema 7.2.1 — Minimum of a monotonic function. The basic optimization problem can be summarized in the following equation

$$\min_{x \in \Re^n} f(x),\ f : \Re^n \to \Re, \vec{x} = [x_1, x_2, ..., x_n] \tag{7.4}$$

$$g_i(\vec{x}) <= 0 \tag{7.5}$$

$$h_j(\vec{x}) = 0 \tag{7.6}$$

where $\vec{x}$ are the design variables, $f(\vec{x})$ is the objective/cost function, while g_i and h_j are constraints. The objective is to find $\vec{x}^*$ that solves the problem for $f(\vec{x}^*)$.

7.3 The Simplex Method

This is a traditional linear programming method, very common in the area of Operations Research (**??**). It was first developed during World War II by George Dantzig, and was very important for the Allies in solving logistics and production problems. It was later independently reinvented in Soviet Russia by Leonid Khachian, who believed it was revolutionary but which in the end turned out to be the same method. An interesting aspect is that it shows how the optimization process inherently involves a search for a solution, and this is seen directly in this method where one identifies the region of validity of the solution, the intersections, and then tests each intersection one by one, choosing the one that will best impact the cost function, until the optimal one is found (or it is determined that no solution is possible) [43, 44].

This method attempts to find the solution to the optimization problem given by a vector product **cx** subject to the constraints given by the solution of a system of equations, according to definition 7.3.1.

Definición 7.3.1 — Simplex. The following linear system is solved
$\min f(\vec{x}) = \mathbf{cx}$
subject to $Ax = b,\ \mathbf{x} > 0$

The method guarantees that any linear optimization problem can be brought to standard form through three rules.

1. $\max(\mathbf{cx} : A\mathbf{x} = \mathbf{b},\ \mathbf{x}) = -\min(-\mathbf{cx} : A\mathbf{x} = \mathbf{b},\ \mathbf{x})$
2. $x \geq 0 \rightarrow x = x_1^+ - x_2^-$
3. Slack: $z = \mathbf{b} - A\mathbf{x} > 0$

The first rule establishes the equivalence that maximizing something is equivalent to finding the additive inverse of the minimization of the additive inverse of the cost function.

The second rule allows, by adding a variable, the representation of variables that can take negative values, as the sum of two variables where both are positive but the second subtracts from the first.

The third rule is for establishing slack variables[3]. Given an inequality, a dependent slack variable can be added that represents the difference between the value of one side versus the value of the other, replacing the inequality with an equality.

7.4 Unconstrained Nonlinear Optimization

Unconstrained nonlinear optimization allows finding the values that optimize non-linear functions, without any additional constraints [45, 46].

$$\min_{x \in \Re^n} f(x),\ f : \Re^n \rightarrow \Re, \tag{7.7}$$

Where f is a nonlinear function, for example:
1. $f(x_1, x_2) = x_1^2 + sen(x_1 * x_2) - x_1 * x_2$
2. $f(x_1, \ldots, x_n) = \sum_{i=1}^{n} (x_i - cos(x_i))^2$
3. $f(x_1, \ldots, x_n) = \sum_{i=1}^{n} (x_i - g(x_i, \xi_i))^2$

This is particularly interesting in the case of neural networks, when the function to be optimized is directly a function called the error function, $E(w)$, which depends on the synaptic weights of the network.

$$\min_{w \in \Re^n} E(w),\ E : \Re^n \rightarrow \Re, \tag{7.8}$$

where

- $E(w_1, \ldots, w_n) = \frac{1}{2} \sum_{\mu=1}^{p} (\zeta^\mu - g(\sum_{i=1}^{n} w_i x_i^\mu))^2$ is the error function.
- $w = (w_1, \ldots, w_n)$ is the vector of synaptic weights we want to adjust.
- n is the number of synaptic weights or free parameters of the network.

[3] Very famous in this method.

- $g(\cdot)$ is a nonlinear activation function.
- $\mathbf{x}$ are the μ samples of the dataset, while ζ are the output labels associated with each sample.

Calling this function an error (or cost) function alludes to the fact that $g(\sum_{i=1}^{n} w_i x_i^{\mu})$ can be considered a mechanism, a transformation of an input x_i^{μ} (an input data point) into an output ζ^{μ}, mediated by free parameters w_i, which are the ones to be adjusted so that each input produces the correct output. The given function measures the error between what the mechanism returns and what **should** come out (the desired value).

There are different alternatives for solving these problems, and much of optimization theory covers these alternatives.

- Exact methods: Compute a closed-form formula for the solution.
- Solution approximation methods:
  - Methods based on first derivatives, or on the gradient.
  - Methods based on second derivatives, or on the Hessian.
  - Derivative-free methods.
  - Stochastic methods (estimate the direction).

7.4.1 Preliminary Definitions

The following definitions and theorems offer the necessary and sufficient conditions for establishing optimality.

Definición 7.4.1 — Positive semi-definite matrix. A matrix A is positive semi-definite if

- If $x^t Ax > 0$, $x \neq 0$.
- If all its eigenvalues are positive.
- There are other characteristics that define positive definite matrices, for example the diagonal values must be greater than the sum of the other elements in the row.
- If $x^t Ax \geq 0$, $x \neq 0$, then the matrix is said to be **positive semi-definite**.

Teorema 7.4.1 — Existence of local and global minima. Let the function f be differentiable with continuous first and second derivatives, then x^*...

- Is a global minimum if $\forall x,\ f(x^*) \leq f(x)$
- Is a local minimum if $f(x^*) \leq f(x)\ \forall x,\ \|x - x^*\| < \varepsilon$

Teorema 7.4.2 — Necessary conditions for optimality. The conditions are:

- **First-order necessary condition**
  If x^* is a local minimum of f then $\nabla f(x^*) = 0$.
- **Second-order necessary condition**
  If x^* is a local minimum of f then $\nabla f(x^*) = 0$ and $H_f(x^*)$ (the Hessian) is a positive semi-definite matrix.

Teorema 7.4.3 — Sufficient condition for optimality. If x^* is such that $\nabla f(x^*) = 0$ and $H_f(x^*)$ is positive definite, then it is a local minimum of f.

The Hessian determines whether the function is concave or convex:

- Convex function → Hessian is a positive definite matrix.
- Concave function → Hessian is a negative definite matrix.

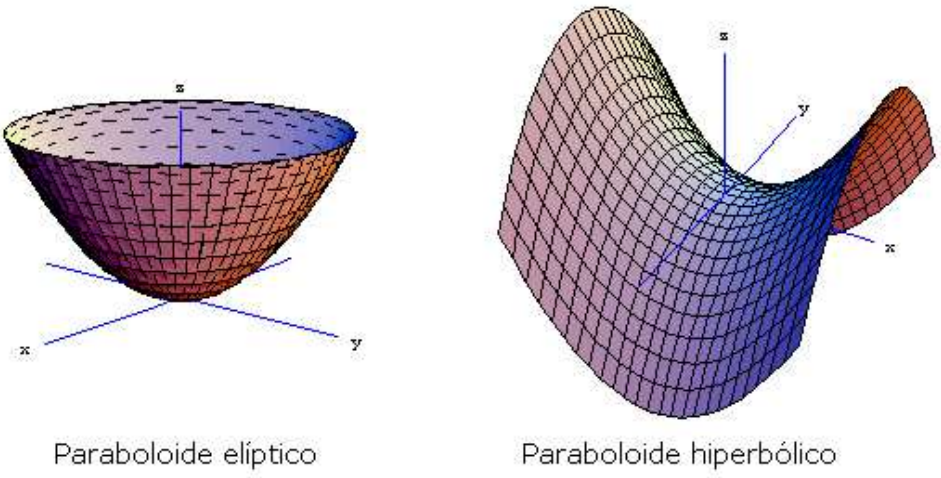


Figura 7.2: Positive definite Hessian Non-positive definite Hessian

Behold, dear readers, that behind any type of algorithm that dares call itself AI, you will always find the following underlying procedure for training its free parameters, in one way or another.

Algoritmo 4 General optimization procedure

1: Initial point x_k, input.
2: Iteratively for each step k.
3: **while** not converged **do**
4: Search for a direction of movement d_k.
5: Compute or decide the step length α_k.
6: Update to the new point $x_{k+1} = x_k + \alpha_k d_k$.
7: **end while**
8: Output: Points x^* ideally the global optimum.

The idea behind Algorithm 4 is to find two things. One is the length of the iteration step and the second is the direction for finding points that satisfy $f(x_{k+1}) < f(x_k)$. The direction of movement must be descending, that is $d_k^t \nabla f(x_k) < 0$, knowing that:

- The gradient is the direction of maximum increase of a function.
- Any direction opposite to that of the gradient is a direction of decrease of the function.

and the step must be such that it allows the algorithm to converge in a bounded time and that it also allows, in a finite number of steps, to always get closer to the solution.

7.4.2 Gradient Descent

This famous method is based precisely on using the direction of maximum decrease of the function to be optimized, which corresponds to the gradient.

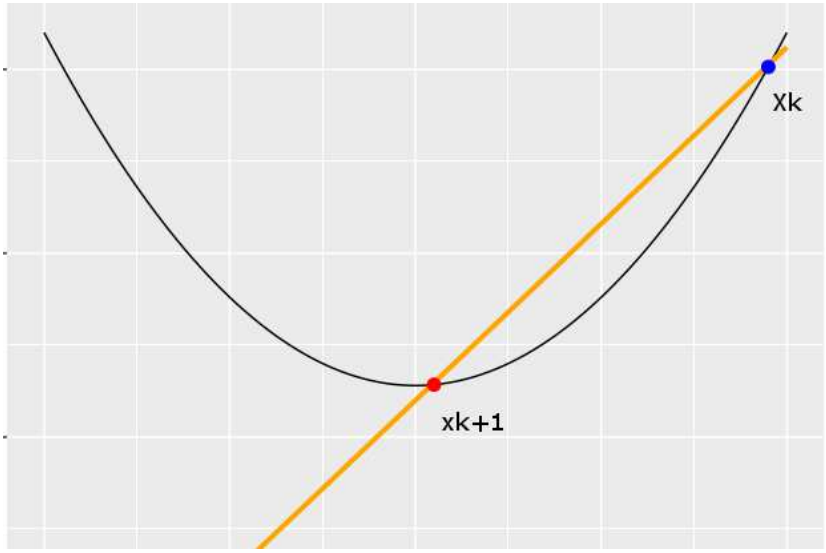


Figura 7.3: The objective of the general optimization procedure is to find the direction and step to move from X_k to an X_{k+1} that produces a smaller value in the cost function.

$$d_k = -\nabla f(x_k) \tag{7.9}$$

Although it has slow convergence, it has the advantage of not requiring information about the second derivative.



Figura 7.4: Gradient descent advances in a zig-zag pattern.

Figure 7.4 shows the level curves of the scalar error function; the center would be the minimum value. The gradient follows the indicated direction, which immediately reveals a problem when approaching the solution in a zig-zag fashion, which is not a good balance between speed of approach and precision. This can be smoothed out somewhat using the concept of **Momentum**, which as we will see later is based on the idea of taking the descent direction as a linear combination of descent directions computed in previous steps.

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k) - \beta \alpha_{k-1} \nabla f(x_{k-1}) \tag{7.10}$$

This weighted average between directions smooths out the zig-zagging of the gradient method with $0 < \beta < 1$. This is because the new direction is a weighted average by beta of the two previous directions.

7.4.3 Newton's Method

As seen in the Coke can example, the second derivative provides information. What we can then do is approximate the gradient itself by the second-order Taylor polynomial,

$$\nabla f(x_{k+1}) = \nabla f(x_k + \alpha_k d_k) = \nabla f(x_k) + \alpha_k H(x_k) d_k \tag{7.11}$$

Then, if x_{k+1} were the minimum, $\nabla f(x_{k+1}) = 0$ and therefore

$$\nabla f(x_k) + \alpha_k H(x_k) d_k = 0$$

from which we get

$$d_k = -H^{-1}(x_k) \nabla f(x_k)$$

This case would seem to be the definitive solution, since one can find the exact point where the minimum occurs (without having to iterate). However, the problem arises in having to estimate the Hessian and/or its inverse for any function that needs to be optimized.

7.4.4 Quasi-Newton Methods

These methods are based on attempting to reduce the computational cost associated with computing the Hessian and particularly its inverse. They rely on computing an approximation of $H^{-1}(x_k)$ and replacing it with a positive definite approximation matrix B_k. The variants of these methods differ precisely in how to perform the approximation of this matrix.

However, even with the improvement in performance, quasi-Newton methods have the disadvantage of having a high computational cost for neural network training applications.

Instead, the **L-BFGS (limited memory Broyden Fletcher Goldfarb Shanno (BFGS))** method can be used [47]. This algorithm, continuing with the chicken-and-egg idea, implements an iterative search to estimate the Hessian $B_{k+1} \sim B_k$.

7.4.5 Conjugate Direction Methods

These methods search for the directions along which to find the solution, exploiting the space of directions established through the concept of directions conjugate with respect to a matrix. They have the advantage of offering convergence guarantees and a bound on the number of steps when the form of the cost function is specific.

Teorema 7.4.4 — A-conjugate vectors. Let the set of directions (vectors) $d_1, \ldots, d_n$ and A be a symmetric positive definite matrix, then:

- If $d_i^t A d_j = 0$, $\forall i \neq j$, then $d_1, \ldots, d_n$ are said to be A-conjugate directions.
- If a set of vectors is A-conjugate, with A symmetric and positive definite, then it is also a linearly independent set.

Teorema 7.4.5 — Convergence bound for Conjugate Direction Methods. Given a quadratic function $f : \Re^n \to \Re$ $f(x) = x^t Hx + b^t x$, if a nonlinear minimization method performs one-dimensional searches along H-conjugate directions, then the method converges in n steps.

Of course it is not always possible for the cost function to have the form required by the theorem, but they serve as a starting point for other methods that attempt to approximate the idea with much more complex functions.

7.4.6 Conjugate Gradient Methods, 1952

This method is based on computing, at each step k, a new direction d_{k+1} that is $H(x_k)$-conjugate with all previous directions $d_1, \ldots, d_k$, based on $\nabla f(x_k)$.

It only requires identifying the gradient at each step.

7.4.7 Powell's Method, 1964

This method is based on determining conjugate directions without requiring the use of partial derivatives (zeroth-order). The method generates at each step k a set of conjugate directions

$$d_1^k, \ldots d_n^k.$$

It begins with a set of linearly independent directions $d_1^0, \ldots d_n^0$ and an initial point x_0. Then at step k, it removes the direction d_k^k from the set and adds a direction conjugate to $d_1^{k-1}, \ldots d_{k-1}^{k-1}$ (those from previous steps). This method is, at its core, a linear search.

7.4.8 Stochastic Gradient Descent

The most widely used method in neural networks is **SGD**, Stochastic Gradient Descent, and it is based on an extension of the gradient descent method designed especially for error functions.

We have p observations $\mathbf{x}^\mu = \{x_1, \ldots x_n\}$, $\mu = 1, \ldots, p$ of dimension n from the training set (X) together with their p outputs ζ^μ of dimension 1 and $\mathbf{w} = (w_1, \ldots, w_n)$ the vector of synaptic weights (free parameters). We then want to find $\mathbf{w}$ that minimizes $E(\mathbf{w})$:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{\mu=1}^{p} (\zeta^\mu - \sum_{i=1}^{n} w_i x_i^\mu) \tag{7.12}$$

$$E(\mathbf{w}) = \frac{1}{p} \sum_{\mu=1}^{p} E^\mu(x^\mu, \mathbf{w}) \tag{7.13}$$

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta_t \frac{1}{p} \sum_{\mu=1}^{p} \nabla_{\mathbf{w}} E^\mu(x^\mu, \mathbf{w}^t)$$

Considering that $\frac{1}{p} \sum_{\mu=1}^{p} \nabla_w E(x^\mu, \mathbf{w}^t)$ is an estimator of the expected value of the gradient, given a training set,... **Why not use any other estimator?**.

For example: a

$$\nabla_w E(x^\nu, \mathbf{w}^t),$$

for some arbitrary ν, or a random subset $\sum_{\mu=1}^{k} \nabla_w E(x^\mu, \mathbf{w}^t)$, $k << p$.

This is the essence of the stochastic gradient descent method. Where only a portion of the data is used to compute the descent direction at each step. The original name of this method was ADALINE, or Adaptive Linear Element.

The authors of the method demonstrate in their book [48] that the method converges, but does not always decrease monotonically.

ADAGrad, 2011 (Adaptive Gradient)

It is a method that improves upon ADALINE, whose main objective is to adapt the value of the learning rate at each step. The update of vector $\mathbf{w}$ is performed coordinate by coordinate. Let $g_t = \nabla E(\mathbf{w}_{t-1})$, then $(g_t)_i$ is the i-th coordinate; $g_{1:t} = \{g_1, \ldots, g_t\}$ are all the previous gradients up to step t.

While in SGD the update is

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta_t \nabla_{\mathbf{w}} E^\mu(x^\mu, \mathbf{w}^t)$$

In ADAGRAD the update is

$$\mathbf{w}_i^{t+1} = \mathbf{w}_i^t - \frac{\eta_t}{\sqrt{G_{ii}^t + \varepsilon}} \nabla_{\mathbf{w_i}} E^\mu(x^\mu, \mathbf{w}^t)$$

with $G^t = \sum_{\tau=1}^{t} g_\tau * g_\tau^t$.

The advantage of this method is that each coordinate has its own update.



8. The Neuron Model

Frank Rosenblatt (1928–1971), trained as an experimental psychologist, developed a mechanism called the Perceptron at Cornell University, as part of a project for the United States Navy[1]. He conceived the algorithm and its hardware implementation, the *Mark I*. It had 400 light sensors that together acted as a retina, providing information to 1000 neurons in a single layer that performed the processing and produced a single output.

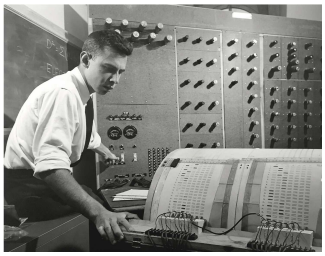


Figura 8.1: Rosenblatt with his Perceptron, working his magic.

In 1958, the New York Times declared that *the machine would be the first device to think as the human brain does* [32, 49]. Rosenblatt's motivation and inspiration lay in his background and inclination toward interdisciplinarity, an essential characteristic of the field of neural networks. Frank sought to *understand the minimum number of elements that a brain must physically have in order to capture the wonderful things it does.* The algorithm is based on a linear combination of inputs, subsequently thresholded to generate a final binary response, similar to the electrical impulses transmitted by neurons in the brain. Rosenblatt was able to prove that, with a single layer and a simple weight update rule, the weights could converge to achieve the computational task required; in short, he established a proven scheme for *learning*. The Perceptron algorithm is a cornerstone of the discipline that seeks to artificialize intelligence, and was the first work that fueled the **hype** surrounding this field — a constant characteristic we can still experience today.

[1] A large portion of the research lines in artificial intelligence have roots that trace back to World War II or the post-war period in the context of the arms race between the Soviet Union and the USA. Interesting accounts on the subject can be found in "A History of Warfare" by Field Marshal Montgomery, "Engineers of Victory: The Problem Solvers Who Turned the Tide in the Second World War" by Paul Kennedy, or "Accessory to War" by Neil deGrasse Tyson.

8.1 The Bio-Inspiration

The idea of the simple Perceptron has its origins in the neuron model created by McCulloch and Pitts in 1943 [13]. It is a simplified model, based on the knowledge that existed at the time regarding the functioning of neurons and the ability to find a simple, comprehensive model that explains the main characteristics. It establishes that a neuron has a set of inputs that arrive at the soma (or cell body), where an integration of the information takes place (summation). If the value of this integration exceeds a certain threshold, a binarized output of the neuron is produced, corresponding to an active output. The simplified model can be seen in Figure 8.2.

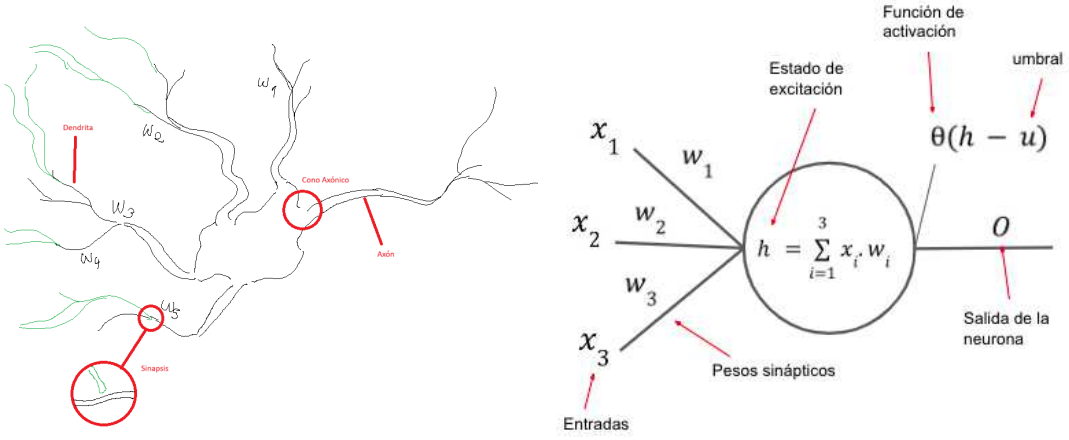


Figura 8.2: Diagram and simplified model of a neuron.

Mathematically this model corresponds to:

$$O = \phi\left(\sum_{i=1}^{n} w_i x_i - u\right) \tag{8.1}$$

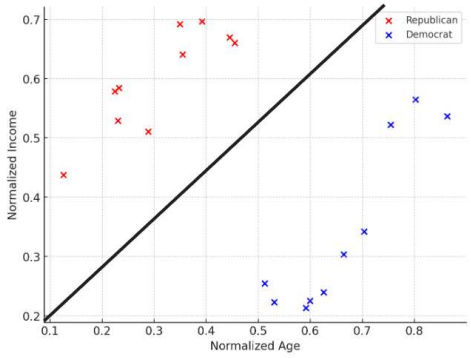
where O is the output, x_i are the inputs, w_i are the weights (free parameters), and u is the threshold. On the other hand, ϕ is the activation function, which in the earliest versions of the Perceptron was restricted to the step or sign function:

$$\phi(x) = \begin{cases} 1 & \text{if } x >= 0, \\ 0 & \text{if } x < 0. \end{cases} \tag{8.2}$$

8.2 Linear Separability

Consider a dataset in two dimensions, where each axis corresponds to a certain feature and the points can be classified into two classes. In this case, individuals are classified according to their political orientation, based on the features of normalized age (values between 0 and 1) and normalized economic income.

Is it possible to find a separating line that perfectly divides the points into two groups according to their class? From Figure 8.3 it can be seen that in this case such a line exists and can be modeled according to the classic equation $y = mx + b$.



Age x_1	Income x_2	Party ζ
0.445	0.669	1.0
0.349	0.692	1.0
0.232	0.585	1.0
0.125	0.438	1.0
0.224	0.579	1.0
0.23	0.529	1.0
0.392	0.696	1.0
0.355	0.641	1.0
0.455	0.66	1.0
0.289	0.51	1.0
0.513	0.255	-1.0
0.755	0.522	-1.0
0.626	0.24	-1.0
0.703	0.342	-1.0
0.863	0.536	-1.0
0.6	0.225	-1.0
0.664	0.303	-1.0
0.802	0.565	-1.0
0.592	0.213	-1.0
0.531	0.223	-1.0

Figura 8.3: Set of points in two dimensions, each belonging to a different class.

The existence of this line allows us to sketch a generalization: given a new point, I can determine which class it belongs to depending on its position with respect to the separating line. To do this, it is possible to project the point onto the component orthogonal to the line and evaluate the sign of that function.

If we recall the equation of a plane, characterized by its normal, we see that its equation is given by

$$(X - N).P = 0$$

with N being the normal to the plane and P a point on the plane. The projection onto this line is then determined by:

$$Proy_N X_\mu = \frac{X_\mu . N}{||N||}\mathbf{N}.$$

The actual magnitude of the projection of each of the points onto the normal ultimately reduces mainly to the inner product given by

$$P = \sum_{i=1}^{n} w_i x_i. \tag{8.3}$$

where n is the dimension of the space. If P is positive, we identify that point as belonging to class 1, while if it is negative, we identify it as belonging to class 2. That is, we have a basic classifier capable of providing a generalization.

$$\phi(P) = \begin{cases} 1 & \text{if } x >= 0, \\ 0 & \text{if } x < 0. \end{cases} \tag{8.4}$$

8.2.1 Hebbian Learning

The McCulloch-Pitts model allows solving a simple binary classification model.



Figura 8.4: Donald Hebb thinking using his own neurons.

In what was explained previously, we have managed to map concepts related to the biological neuron — such as soma, axons, and dendrites — onto a mathematical simplification to obtain this classifier. The missing piece in this neuron model lies in the concept of the synapse, which we have not yet mentioned. The concept of synapse explains the strength of the connection between neurons in the brain and, according to this strength, neurons are capable of exciting or inhibiting other neurons. The synapse becomes stronger through learning processes. So, how can this *learning* process be generated in our system, emulating what occurs in real biological systems? Donald Hebb [14] enters the scene, with the formulation of his eponymous conjecture: the connections (synapses) involving neurons whose states of activity are correlated over time are strengthened, and this reinforcement depends on the intensity and period of time during which this has occurred. This formulation is known by his famous phrase:

"Neurons that fire together, wire together"

This conjecture provided an explanation for different phenomena observed in behavioral psychology. For example, Pavlov's conditioned learning [50].

So, each time the neuron receives a new stimulus, we can apply a learning rule that allows varying each of the synaptic weights. This variation will be made according to the information provided by the stimulus.

$$w_i^{new} = w_i^{old} + \Delta w_i \tag{8.5}$$

where the value Δw_i will depend on both the activation of the input and the output simultaneously.

$$\Delta w_i = 2\eta x^\mu \zeta^\mu \tag{8.6}$$

with η acting as a proportionality constant that regulates learning.

With this, a simple update rule can be derived. When the input x^μ is presented and the activation state O^μ coincides with the desired output ζ^μ, nothing needs to be done. But if they do not coincide, w_i must be updated. This is equivalent to:

$$\Delta w_i = \begin{cases} 2\eta x^\mu \zeta^\mu & \text{if } O^\mu \neq \zeta^\mu, \\ 0 & \text{otherwise} \end{cases} \tag{8.7}$$

Assuming that O^μ and ζ^μ take values 1 and -1, the piecewise function can be written as:

$$\Delta w_i = \eta(\zeta^\mu - O^\mu)x^\mu \tag{8.8}$$

This also allows an interpretation from the perspective of the geometric representation of the linear separability problem. When $\zeta^\mu > O^\mu$, equation 8.8 represents moving the weight vector slightly toward the input pattern x^μ. With this, the next time we compute the inner product $w_i x^\mu$ it will be larger and it will be more likely that ζ^μ and O^μ will match.

From the geometric point of view, the Hebbian update rule is altering the separation plane in order to correctly divide the elements into their classes. This connection, which sounds trivial, is very powerful, because the basis of the rule comes from an experimental mechanistic model of the behavior of the synaptic structure of neurons.

8.2.2 The Simple Perceptron Algorithm

Ladies and gentlemen, nerds and otakus, hold on to your seats, keep your mate and thermos close at hand, because we are in a position to unveil the most important algorithm in this volume: the simple Perceptron algorithm.

Definición 8.2.1 — Epoch. Refers to one pass of the training dataset through equation 8.8.

Definición 8.2.2 — Hyperparameters. These are values that can be specified in the algorithm and that affect the learning process. Although they can be changed, they are NOT updated by the learning process itself.

Definición 8.2.3 — Synaptic weights. These are the elements of the vector $\mathbf{w}$ that determine the separation plane. According to the neuron model, they represent the strength of the synaptic connection. These are the free parameters to be adjusted through learning (including the threshold).

Definición 8.2.4 — Training set. The training set $x[\cdot]$, where each input has dimension N (taking into account that one additional coordinate is added for the threshold). There are p inputs.

Definición 8.2.5 — Expected outputs. The expected outputs are captured in $y[\cdot]$.

Definición 8.2.6 — Activation function. The activation function determines the output of the neuron and restricts the possible values. In the case of the simple perceptron, this function is given by $sign(\cdot)$.

Algoritmo 5 Simple Perceptron Algorithm

Initialize $i = 0$
Initialize $w = zeros(N, 1)$
Initialize $error = 1$
Initialize $error_{min} = p * 2$
Specify hyperparameter $BOUND$.
while $error > 0 \wedge i < BOUND$ **do**
    $i_x = ceil(rand(1,1) * p)$
    $excitation = x(i_x, :) * w$
    $activation = sign(excitation)$
    Compute $\Delta w = \alpha * (y(1, i_x) - activation) * x(i_x, :)$
    $w = w + \Delta w$
    $error = ComputeError(x, y, w, p)$;
    if $error < error_{min}$ **then**
        $error_{min} = error$
        $w_{min} = w$
    end if
    $i = i + 1$
end while
w Result of the trained weights (if possible) for dataset x.

Definición 8.2.7 — Synaptic weight initialization. The weight initialization scheme is one more hyperparameter to specify. A conservative strategy is to initialize them randomly with $w = rand(N, 1) * 2 - 1$.

8.2.3 Linear Perceptron

If the step activation function is replaced by the identity function, the Perceptron becomes linear, where the output depends only on the inner product between the input patterns x and the weights w.

$$O^{\mu} = \sum_{i=1}^{N} w_i x_i^{\mu} \tag{8.9}$$

The values are now continuous and are no longer thresholded between $\{-1, 1\}$ or $\{0, 1\}$ as was the case with step units. The values belong to the reals. From this interpretation, a differentiable cost function can be assumed, which will adopt an absolute minimum when the value of the weight vector w causes the output $O^{\mu} = \zeta^{\mu}$, i.e., it produces the exact output. This function has an arbitrary factor of $\frac{1}{2}$ whose purpose we will see shortly, and is based on SSE, Sum of Squared Errors, which is used in linear regression.

$$\begin{aligned} E(w) &= \frac{1}{2} \sum_{\mu=1}^{p} (\zeta^{\mu} - O^{\mu})^2 \\ &= \frac{1}{2} \sum_{\mu=1}^{p} (\zeta^{\mu} - \sum_{i=1}^{N} w_i x_i^{\mu})^2 \end{aligned}$$

It is important to note that $E(\cdot)$ depends on the weights w_i. Therefore, if we manage to move w in the right direction we could approach the minimum value of $E(\cdot)$ which, being an error function, should ideally be zero. One possibility, pulling out our optimization manual, is to use the gradient, since it represents the direction of greatest increase of the function we are trying to minimize. With this in mind, we will move the synaptic weights in the direction **opposite to that of greatest increase**:

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i} \tag{8.10}$$

for which we compute

$$\frac{\partial E}{\partial w_i} = \frac{\partial E(\frac{1}{2}\sum_{\mu=1}^{p}(\zeta^\mu - O^\mu(w_i))^2)}{\partial w_i} = (2)\,(-1)\frac{1}{2}\sum_{\mu=1}^{p}(\zeta^\mu - \sum_{i=1}^{N} w_i x_i^\mu)\,(x_i^\mu),$$

and substituting this into 8.10:

$$\begin{aligned}\Delta w_i &= -\eta \frac{\partial E}{\partial w_i} \\ &= \eta \sum_{\mu=1}^{p}(\zeta^\mu - O^\mu)x_i^\mu\end{aligned}$$

With this, each individual weight is modified by evaluating all patterns. If we want to perform the adjustment by evaluating only one pattern we get

$$\Delta w_i = \eta(\zeta_i^\mu - O_i^\mu)\xi_i^\mu \tag{8.11}$$

Oops...this is exactly the same update rule as the Perceptron with the step activation function. **Something we formulated inspired by an experimental biological model coincides, from a mathematical point of view, with the formulation of minimizing the value of a function!** This rule, called the **delta rule**, is also known as the Widrow-Hoff rule (1960), and its authors named it ADALINE, or Adaptive Linear Element. [2].

Learning in the Linear Perceptron

The simple step Perceptron solves linearly separable problems. On the other hand, the simple linear Perceptron solves independent linear systems[3]. That is, it finds an approximation of the w that solves

$$\mathbf{xw} = \zeta,$$

that is

$$\begin{aligned} w_1 x_1^1 + w_2 x_2^1 + \ldots + w_N x_N^1 &= \zeta_1^1 \\ w_1 x_1^\mu + w_2 x_2^\mu + \ldots + w_N x_N^\mu &= \zeta_1^\mu \\ w_1 x_1^p + w_2 x_2^p + \ldots + w_N x_N^p &= \zeta_1^p \end{aligned}$$

[2]Ted Hoff was a student of Bernard Widrow and later, in 1968, would become the co-inventor of the predecessor of the microprocessor (a set of 12 different circuits on a single chip that changed its functionality according to the requirement) [51]

[3]The proof is of course obvious and trivial and is left as an exercise for the reader ;).

The convergence of this learning depends strongly on the value of η, recalling that we must update $w_i^{new} = w_i^{old} + \Delta w_i$ based on $\Delta w_i = \eta(\zeta^\mu - O^\mu)x^\mu$, so that

- If η is not small enough, the method may diverge.
- If η is too small, convergence may be slow.

8.2.4 Learning in the Nonlinear Simple Perceptron

The idea of gradient-optimization-based learning can easily be extended to a nonlinear situation. The activation state of the Perceptron is

$$O^\mu = g(\sum_{i=1}^{N} w_i x_i^\mu) \tag{8.12}$$

where $g(\cdot)$ can be a sigmoidal function such as $g(x) = tanh(\beta x)$ or $g(x) = \frac{1}{1+exp^{2\beta x}}$. A more exhaustive list of activation function variants can be found in 24.2.

$g(h)$	$g'(h)$
$tanh(\beta h)$	$\beta(1-g(h)^2)$
$\frac{1}{1+exp^{2\beta x}}$	$2\,\beta g(h)(1-g(h))$

Cuadro 8.1: Simple activation functions and their derivatives (1/2)

The cost function is defined exactly as with the simple linear Perceptron

$$\begin{aligned} E(w) &= \frac{1}{2}\sum_{\mu=1}^{p}(\zeta^\mu - O^\mu)^2 \\ &= \frac{1}{2}\sum_{\mu=1}^{p}(\zeta^\mu - \sum_{i=1}^{N} w_i x_i^\mu)^2. \end{aligned}$$

Now to update the weights we must use gradient descent with

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

but we use **the chain rule**[4] to propagate the derivative of g within the computation as well

$$\Delta w_i = \eta \sum_{\mu=1}^{p}(\zeta^\mu - g(h^\mu))g'(h^\mu)x_i^\mu$$

where we use $h^\mu = \sum_{i=1}^{N} w_i x_i^\mu$.

Again, considering taking only one particular input at a time.

[4]We will see that this is fundamental for the next chapter, where we extend this to multi-layer schemes.

$$\Delta w_i = \eta(\zeta^\mu - O^\mu)g'(h^\mu)x_i^\mu \tag{8.13}$$

The advantage of the logistic function or the hyperbolic tangent is that their derivatives can be expressed as functions of themselves.

All of this is very beautiful. But it has a major problem. In 1969, an influential paper by Minsky and Papert was published, arguing that this mechanism (which promised to decode nothing less than the human brain) failed to solve a trivial Boolean function such as XOR. This was the beginning of the first **winter** of artificial intelligence.



9. The Era of Connectionism

The XOR problem from the paper by Minsky and Papert [30] had a simple resolution that was even stated directly and explicitly in that very work. This was the use of a *hierarchical agglomerative* structure, in successive layers. This idea has a correlate in neuroscience, where it has been verified that such a structure is present in the nervous systems of many living organisms [52].

9.1 Hierarchical Agglomerative Structure

The outputs of the neurons can be used as inputs to neurons located in the next layer. This scheme solves the XOR problem, because it allows adding an additional separation plane, and so on. It is a simple solution that requires a change in the establishment of a neural **network** and its architecture.

This network has a hierarchical structure, divided into layers at different levels: an input layer, intermediate hidden layers, and an output layer. It is also agglomerative, because the information from one layer is used jointly — it is agglomerated — to become the input to the subsequent layer. This is the essence of the Multilayer Perceptron, or MLP.

9.1.1 Feedforward

So how does the evolution of inputs proceed forward through our network? Consider the network shown in Figure 9.2. The synaptic weights w_{jk} correspond to the weights of the hidden layer, while W_{ij} corresponds to the weights of the output layer. We call the activations of the hidden layer V_j.

- k, j, i are the indices for referencing the inputs, the neurons of the hidden layer, and the output neurons.
- The superscript μ indicates the index of the input sample, with p being the number of examples.
- The subscript k indicates the index of the dimension of the example.

When presenting sample μ to the network, the hidden layer computes

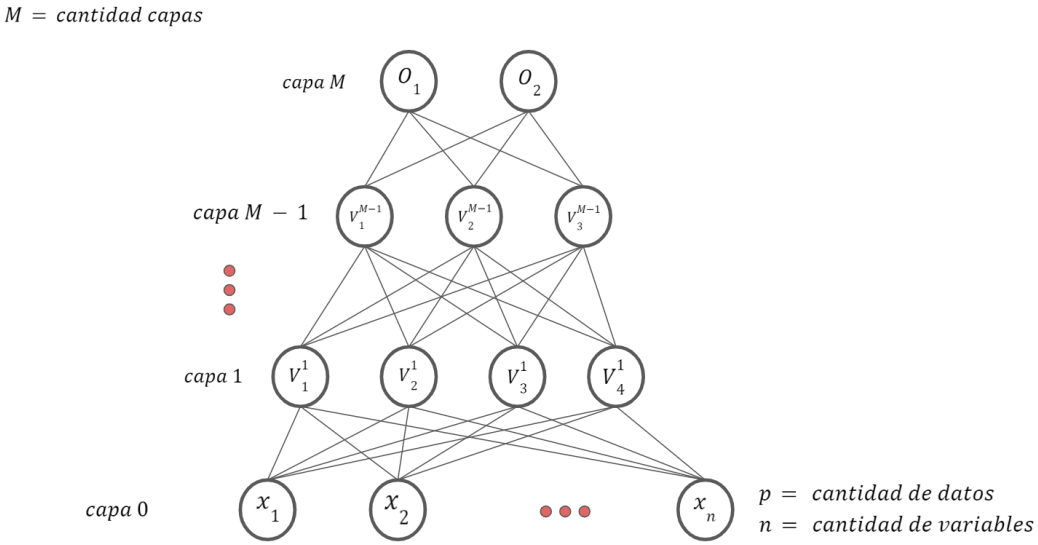


Figura 9.1: Multilayer Perceptron

$$h_j^\mu = \sum_k^N w_{jk} x_k^\mu \tag{9.1}$$

which allows generating an activation in the hidden layer as

$$V_j^\mu = g(h_j^\mu) = g(\sum_k^N w_{jk} x_k^\mu) \tag{9.2}$$

While at the outputs of the hidden layer we have,

$$h_i^\mu = \sum_k^N W_{ij} V_j^\mu = \sum_k^N W_{ij} g(\sum_k^N w_{jk} x_k^\mu) \tag{9.3}$$

$$O_i^\mu = g(h_i^\mu) = g(\sum_i^N W_{ij} V_j^\mu) = g(\sum_i^N W_{ij} g(\sum_k^N w_{jk} x_k^\mu)) \tag{9.4}$$

This determines all the forward outputs of the network.

9.1.2 Backward Backpropagation

In the Perceptron, learning is given by the process of updating the synaptic weights when the output differs from the expected value. This is straightforward, since the contribution of the output to the weight computation is immediate. A proportion of the error directly corrects the weights of the connections that directly link the inputs to the outputs. Now, how do we do this in the case of an

architecture with hidden layers? How do we propagate that error backwards to the hidden weights? What desired activation state must the neurons of the intermediate layer have in order to establish an error and, accordingly, correct the weights of the connections linking the inputs to those intermediate hidden neurons?

In 1986, Rumelhart, Hinton, and Williams presented their proposal to solve this with the Backpropagation algorithm, in the context of their work on PDP (Parallel Distributed Processing) [29].

Returning to learning in the Perceptron.

$$h = \sum_{i=1}^{N} w_i x_i \tag{9.5}$$

$$O^{\mu} = g(h) \tag{9.6}$$

And so the error function is:

$$\begin{aligned} E &= \frac{1}{2}\sum_{\mu=1}^{N}(\zeta^{\mu} - O^{\mu})^2 \\ &= \frac{1}{2}\sum_{\mu=1}^{N}(\zeta^{\mu} - g(\sum_{i=1}^{N} w_i x_i))^2 \end{aligned}$$

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i} \tag{9.7}$$

$$\begin{aligned} \frac{\partial E}{\partial w_i} &= \frac{\partial \frac{1}{2}\sum_{\mu}(\zeta^{\mu} - O^{\mu})^2}{\partial w_i} \\ &= \frac{\partial \frac{1}{2}\sum_{\mu}(\zeta^{\mu} - g(\sum_{i=1}^{n} w_i x_i^{\mu}))^2}{\partial w_i} \\ &= \frac{\frac{1}{2}2\partial \sum_{\mu}(\zeta^{\mu} - g(\sum_{i=1}^{N} w_i x_i^{\mu}))}{\partial w_i} \\ &= \frac{\sum_{\mu}\partial(\zeta^{\mu} - g(\sum_{i=1}^{n} w_i x_i^{\mu}))}{\partial w_i} \\ &= \frac{\sum_{\mu}(\zeta^{\mu} - O^{\mu})g'(h)(\partial \sum_{i=1}^{N} w_i x_i^{\mu})}{\partial w_i} \\ &= \sum_{\mu}(\zeta^{\mu} - O^{\mu})g'(h)(x_i^{\mu}) \end{aligned}$$

$$\Delta w_i = -\eta \sum_{\mu}(\zeta^{\mu} - O^{\mu})g'(h)(x_i^{\mu})$$

The required weight update for a particular pattern:

$$\Delta w_i = -\eta(\zeta^\mu - O^\mu)g'(h)(x_i^\mu)$$

Definición 9.1.1 — Backpropagation. Refers to the method or algorithm by which a hierarchical agglomerative network performs the required adjustments to the free parameters. This algorithm involves a backward propagation through a network by means of the chain rule for first- and second-order optimization methods.

Returning then to Figure 9.2, the activation output in the hidden layer is:

$$V_j = g(h_j) = g(\sum_k w_{jk}x_k)$$

$$O_i = g(h_i) = g(\sum_j W_{ij}V_j)$$

Índice	Descripción
i	índice de la neurona de la capa de salida (o siguiente)
j	índice de la neurona de la capa intermedia
k	índice de la neurona de entrada o de la capa anterior
m	índice de la capa intermedia
p	cantidad datos
μ	dato en particular

w_{jk}^m = *pesos sinápticos*

V_j^m = *neurona de capa intermedia*

W_{ij} = *pesos sináticos de la última capa*

O_i = *neurona de capa de salida*

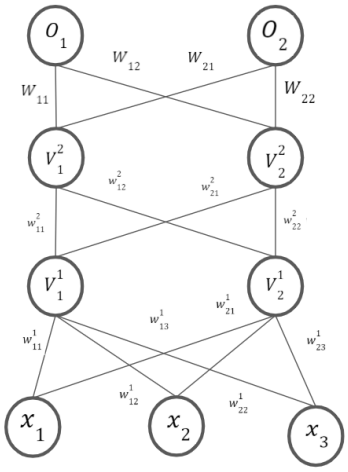


Figura 9.2: Multilayer Perceptron

The error produced by the entire multilayer network is

$$
\begin{aligned}
E(w) &= \frac{1}{2}\sum_{\mu,i}(\zeta_i^\mu - O_i^\mu)^2 \\
&= \frac{1}{2}\sum_{\mu,i}(\zeta_i^\mu - g(\sum_j W_{ij}V_j^\mu))^2 \\
&= \frac{1}{2}\sum_{\mu,i}(\zeta_i^\mu - g(\sum_j W_{ij}\, g(\sum_k w_{jk}x_k^\mu)))^2 \\
&= \frac{1}{2}\sum_{\mu=1}^{p}\sum_{i=1}^{2}(\zeta_i^\mu - g(\sum_j W_{ij}\, g(\sum_k w_{jk}x_k^\mu)))^2
\end{aligned}
$$

To compute the error, we start from the update for the last layer and backpropagate the error $(\zeta_i^\mu - O_i^\mu)$ toward the previous layers.

The update of the weights ΔW_{ij} in the last layer is straightforward:

$$
\begin{aligned}
\Delta W_{ij} &= -\eta\frac{\partial E}{\partial W_{ij}} \\
\Delta W_{ij} &= -\eta(\zeta_i^\mu - O_i^\mu)g'(h_i^\mu)V_j^\mu \\
\Delta W_{ij} &= -\eta\,\delta_i^\mu V_j^\mu
\end{aligned}
$$

with $\delta_i^\mu = (\zeta_i^\mu - O_i^\mu)g'(h_i^\mu)$. From here we can use this information to backpropagate it to the previous layers:

$$
\Delta w_{jk} = -\eta\frac{\partial E}{\partial w_{jk}} \tag{9.8}
$$

$$
\begin{aligned}
\frac{\partial E}{\partial w_{jk}} &= \frac{\partial \frac{1}{2}\sum_{\mu,i}(\zeta_i^\mu - O_i^\mu)^2}{\partial w_{jk}} \\
&= \frac{\partial \sum_{\mu,i}(\zeta_i^\mu - O_i^\mu)}{\partial w_{jk}} \\
&= \sum_{\mu,i}(\zeta_i^\mu - O_i^\mu)(-\frac{\partial O_i}{\partial w_{jk}}) \\
&= -\sum_{\mu,i}(\zeta_i^\mu - O_i^\mu)g'(h_i)\frac{\partial h_i}{\partial w_{jk}} \\
&= -\sum_{\mu,i}(\zeta_i^\mu - O_i^\mu)g'(h_i)\frac{\partial(\sum_j W_{ij}V_j)}{\partial w_{jk}} && \text{Zero } \forall \text{ terms except } ij = jk \\
&= -\sum_{\mu,i}(\zeta_i^\mu - O_i^\mu)g'(h_i^\mu)W_{ij}\frac{\partial V_j}{\partial w_{jk}} \\
&= -\sum_{\mu,i}(\zeta_i^\mu - O_i^\mu)g'(h_i^\mu)W_{ij}\frac{\partial\ (g(\sum_k w_{jk}x_k^\mu))}{\partial w_{jk}} \\
&= -\sum_{\mu,i}(\zeta_i^\mu - O_i^\mu)g'(h_i^\mu)W_{ij}\ g'(h_j^\mu)x_k^\mu
\end{aligned}
$$

Substituting back into 9.8:

$$\Delta w_{jk} = \eta \sum_{\mu,i}(\zeta_i^\mu - O_i^\mu)g'(h_i^\mu)W_{ij}\ g'(h_j^\mu)x_k^\mu$$

Taking into account the definition of δ for the last layer

$$\delta_i^\mu = (\zeta_i^\mu - O_i^\mu).1.g'(h_i) \tag{9.9}$$

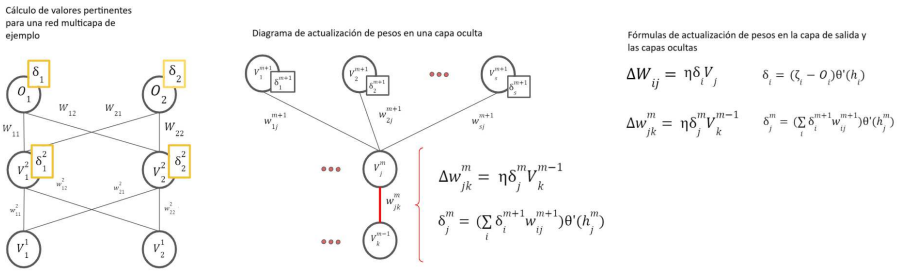
we get:

$$\Delta w_{jk} = \eta \sum_{\mu i}\delta_i^\mu W_{ij}\ g'(h_j^\mu)x_k^\mu$$

We can then extend the definition of δ for a layer one step back as well:

$$\delta_j^\mu = g'(h_j^\mu).\sum_i W_{ij}\delta_i^\mu$$

and we return to rule 9.8, which gives us



$$\Delta w_{jk} = \eta \sum_{\mu i} \delta_i^\mu W_{ij}\, g'(h_j^\mu) x_k^\mu$$

$$\Delta w_{jk} = \eta \sum_{\mu} \sum_{i} \delta_i^\mu W_{ij}\, g'(h_j^\mu) x_k^\mu$$

$$\Delta w_{jk} = \eta \sum_{\mu} g'(h_j^\mu) \sum_{i} W_{ij} \delta_i^\mu\, x_k^\mu \qquad g'(h_j^\mu) \text{ does not depend on } i$$

$$\Delta w_{jk} = \eta \sum_{\mu} (g'(h_j^\mu) \sum_{i} W_{ij} \delta_i^\mu)\, x_k^\mu$$

$$\Delta w_{jk} = \eta \sum_{\mu} \delta_j^\mu\, x_k^\mu$$

Returning then (phew), the formula for the last layer equation is

$$\Delta W_{ij} = -\eta\, \delta_i^\mu V_j^\mu$$

while the formula for the hidden layer is

$$\Delta w_{jk} = \eta \sum_{\mu} \delta_j^\mu\, x_k^\mu .$$

But the most important thing is what we generated in the middle:

$$\delta_j^\mu = g'(h_j^\mu) . \sum_i W_{ij} \delta_i^\mu$$

This formula is very important because it allows propagating the error in any given layer back to the immediately preceding layer. Hence the name **backpropagation**. The architecture of connections is replicated backwards through the errors. The general weight update formula becomes:

$$\Delta W_{ij} = \eta \sum_{\mu} \delta_i^\mu V_j^\mu \tag{9.10}$$

and if a single pattern is updated at a time

$$\Delta W_{ij} = \eta \delta_i^{\mu} V_j^{\mu} \tag{9.11}$$

A key aspect for the computational efficiency of this algorithm is that the weight update depends on the input information V_j^{μ} and δ_i^{μ} that is local to that layer.

9.1.3 Incremental Learning

In the incremental form, each time an input is presented to the network, that input is propagated forward until the activation states of the output layer are obtained. Then, the error is backpropagated, obtaining all the corresponding δ values for each unit in each layer. With all that information, an update (in parallel) of all the weights can be triggered. This is repeated for each input μ. When all inputs in the training set have been processed, one **epoch** has been completed. The sampling of stimuli to present to the network must be done at random.

9.1.4 Batch Learning

In *batch* learning, each time an input is presented to the network, that input is propagated forward until the activation states of the output layer are obtained. Then, the error is backpropagated, obtaining the δ values for each neuron in each layer. But now the Δw values are accumulated and this process is repeated for all the inputs in the dataset. At the end of the epoch, the accumulated values are applied to the weights. In this case the order is not as relevant (though it is still better to sample randomly).

9.1.5 Multilayer Perceptron

Let us now go through the Multilayer Perceptron algorithm.

Algoritmo 6 Multilayer Perceptron Algorithm

Initialize $i = 0$
Initialize $w = initializeWeights()$
Initialize $error = 1$
Initialize $error_{min} = INTEGERMAX$
Initialize $w_{min} = 1$
Specify hyperparameters $BOUND$, $EPSILON$.
while $error_{min} > EPSILON \wedge i < BOUND$ **do**
    $i_x = ceil(rand(1,1) * p)$
    Get $\mu = rand(0, p)$
    $O^{\mu} = forwardPropagation(\mu)$
    $\Delta w = backPropagation()$
    $w = w + \Delta w$
    $error = computeError(data, w)$;
    if $error < error_{min}$ **then**
        $error_{min} = error$
        $w_{min} = w$
    end if
    $i = i++$
end while
w Result of the trained weights (if possible) for dataset x.

With the definition of the two functions for computing forward and backward propagation.

Algoritmo 7 forwardPropagation()

1: $V_k^0 = x_k^\mu$
2: $V_i^m = g(h_i^m) = g(\sum_j w_{ij}^m V_j^{m-1})$
3: Returns $O^\mu = V_i^m$ when m is the last layer of the network.

Algoritmo 8 backPropagation()

1: $\delta_i^M = g'(h_i^M)(\zeta_i^\mu - V_i^M)$ for the last layer.
2: $\delta_i^{m-1} = g'(h_i^{m-1}) \sum_j w_{ji}^m \delta_j^m$ for all m between M and 2.
3: Returns $\Delta_{ij}^m f'(h_i^{m-1}) \sum_j w_{ji}^m \delta_j^m$

The Multilayer Perceptron, with the backpropagation algorithm, presents an efficient bio-inspired method of machine learning 11.1.1 — of adjusting free parameters to optimize an error function — which implies the ability to supervisedly find the weights that, for a given set of inputs, produce the expected outputs. This is very powerful.

9.2 Architecture

From the fact that neurons can be agglomerated, the idea and concept of neural network architecture arises: what is the most appropriate architecture for solving a particular problem, or a family of problems? This is more of an art than a science[1] with no clear recipe.

In MLPs, the architecture is usually represented in terms of the number of layers and the number of neurons in each layer. A good tip is to always consider as a layer any place in the network where weights can be placed and that share inputs/outputs. This means that a network with a single hidden layer has two layers. The first corresponds to the hidden layer that receives the inputs directly, and the second takes the outputs of that intermediate layer, passes them through the weights of a second layer to generate a uni- or multi-dimensional output. However, the dimension of the input data must be added to this scheme to complete the architecture. All the weights can then be summed to determine the number of free parameters of the network, as induced by the architecture.

The architecture becomes an additional hyperparameter to define. It is therefore a problem in itself, for which a countless number of solutions have been tried. The network architecture is a key component because selecting the right one increases the chances of being able to converge the weights and achieve better generalization performance. But finding that architecture is not a simple or well-formalized process.

This makes the importance of ablation studies evident [53]. From that architecture hodgepodge one arrives at — along with other hyperparameters — which components can be eliminated or reduced while maintaining the same level of network efficiency. In the midst of the current hype in the field, this is a critical issue because it allows eliminating the extra complexity added solely out of a need to keep up with that hype: why sell a large network with a large architecture when a simple multilayer perceptron would suffice to solve the problem?

[1] This is a popular *cliché* euphemism also common in other disciplines, used to avoid saying that we actually have no idea how it is done.

In the end, it is trial and error, and adaptability to each particular problem in which the network is used.

9.3 Optimization in Neural Networks

The optimization process in the network depends on the problem[2] and the scheme proposed in the vanilla version of the Multilayer Perceptron may not be sufficient to achieve convergence. Returning then to what was seen in Chapter 7, it is possible to expand the methods presented there with the specific goal of application to neural networks.

9.3.1 Momentum

This variant of the weight update rule, which as mentioned in Chapter 7 is a variant of the gradient descent method, mitigates the drawback of oscillation around the minimum or divergence that tends to occur when W is near a local or global minimum.

$$\Delta w_{ij}(t+1) = -\eta \frac{\partial E}{\partial w_{ij}} + \alpha \Delta w_{ij}(t) \tag{9.12}$$

with $\alpha = 0.8$ or $\alpha = 0.9$. If the function $E(\cdot)$ is in a flat region, $\Delta w_{ij}(t)$ will be similar to $\frac{\partial E}{\partial w_{ij}}$ and therefore $\Delta w_{ij}(t+1)$ will increase in magnitude. Conversely, if the function $E(\cdot)$ is in a valley that would lead to more oscillatory behavior, $\Delta w_{ij}(t+1)$ will tend to cancel out in order to avoid oscillations.

9.3.2 Adaptive η

In the gradient descent algorithm, the value of η determines the step size, the jump from one value to the next. It is not ideal for this value to be constant, because if $E(\cdot)$ decreases at some point during learning (for example, over k consecutive iterations), it might be interesting to increase it so the algorithm moves faster. On the other hand, if $E(\cdot)$ starts to increase at some point during learning, it might be beneficial to decrease η to avoid divergence:

$$\Delta\eta = \begin{cases} +a & \text{if } \Delta E < 0 \text{ consistently,} \\ -b\eta & \text{if } \Delta E > \text{ starts to be consistent} \\ 0 & \text{in any other case} \end{cases} \tag{9.13}$$

It is also possible to update η per individual neuron (for the weights of the connections arriving at that neuron) or per connection (one η for each w_{ij}).

9.3.3 RMSProp

This algorithm was proposed by Hinton in 2012 [54]. It is essential for understanding Adam, since Adam is the union of RMSProp with the addition of momentum. The intuition of the algorithm is to adjust the learning rates for each of the weights, according to what is most suitable to avoid successive oscillations while simultaneously accelerating convergence. To this end, Equations 9.14 and 9.15 are defined:

[2] Here everything always depends on the problem.

$$g_t = \frac{\partial E}{\partial w_{ij}} \tag{9.14}$$

$$S_t = \gamma S_{t-1} + (1-\gamma) g_t^2 \tag{9.15}$$

The term g_t^2 in equation 9.3.4 corresponds to the element-wise squaring operation applied to the vector g_t.

$$\Delta w_{ij} = -\frac{\eta}{\sqrt{S_t + \varepsilon}} \times g_t \tag{9.16}$$

The parameter ε is added to avoid division by zero. The idea of the algorithm is to adjust the learning rate according to the root mean square (RMS). Bearing in mind that this is an optimization process, the learning rate is the step size, the jump that must be taken. In this case, by averaging the learning rate, one is trying to find a critical step size that serves the entire process and balances performance versus convergence. For example, a low learning rate dampens changes better and reduces oscillations, while a high learning rate learns faster.

The RMS value is given by the iterative computation of the weighted average of previous g_t values, according to:

$$\begin{aligned}
S_t &= (1-\gamma) g_t^2 + \gamma S_{t-1} \\
&= (1-\gamma) g_t^2 + \gamma((1-\gamma) g_{t-1}^2 + \gamma S_{t-2}) \\
&= (1-\gamma) g_t^2 + (1-\gamma)\gamma g_{t-1}^2 + \gamma^2 S_{t-2} \\
&= (1-\gamma) g_t^2 + (1-\gamma)\gamma g_{t-1}^2 + \gamma^2((1-\gamma) g_{t-2}^2 + \gamma S_{t-3}) \\
&= (1-\gamma) g_t^2 + (1-\gamma)\gamma g_{t-1}^2 + \gamma^2(1-\gamma) g_{t-2}^2 + \gamma^3 S_{t-3} \\
&= (1-\gamma)(g_t^2 + \gamma\, g_{t-1}^2 + \gamma^2 g_{t-2} + \ldots).
\end{aligned}$$

9.3.4 Adam

In 2014, an update to the RMSProp algorithm was proposed that precisely combined the characteristics of this approach while including momentum. This algorithm, named Adam [55], is currently the most widely used in Deep Learning because it precisely balances efficiency and convergence.[3]

The core of the algorithm is given by the steps established in Algorithm 9:

[3]The Kingma paper has nearly 200,000 citations, an exorbitant number that demonstrates the effectiveness of this algorithm.

$$m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t$$
$$v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$
$$\hat{m}_t \leftarrow \frac{m_t}{1-\beta_1^t}$$
$$\hat{v}_t \leftarrow \frac{v}{1-\beta_2^t}$$
$$\theta_t \leftarrow \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

The bias correction is performed to compensate for the computations carried out in the first iterations, which arise from the estimation of the moment estimators. This algorithm has a somewhat arbitrary but empirically validated assignment of hyperparameters — established in the paper itself — that are known to work well for implementations where the objective function is an error function. These are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\varepsilon = 10^{-8}$.

Algoritmo 9 Adam Optimization Algorithm

1: Initialize parameters: θ, learning rate α, decay rates β_1, β_2, and the constant ε
2: Initialize first and second moments: $m_0 = 0$, $v_0 = 0$
3: $f(\theta)$: Stochastic function of parameters θ
4: Initialize $t = 0$
5: **while** not converged **do**
6: $t \leftarrow t + 1$
7: Compute the gradient: $g_t \leftarrow \nabla_\theta f(\theta_{t-1})$
8: Compute the biased first moment estimator: $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t$
9: Compute the biased second moment estimator: $v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t^2$
10: Recompute the first moment: $\hat{m}_t \leftarrow \frac{m_t}{1-\beta_1^t}$
11: Recompute the second moment: $\hat{v}_t \leftarrow \frac{v}{1-\beta_2^t}$
12: Update the parameters: $\theta_t \leftarrow \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\varepsilon}$
13: **end while**
14: θ_t Result parameter

To summarize, this algorithm is found **within** the optimization and learning process itself (there is no internal *loop* inside Adam itself). Just as with RMSProp, Adam performs the update weight by weight (the operations are vectorial).

10. Evaluation Metrics

10.1 Identifying the Pattern Correctly

The MLP networks from Chapter 9 can act as a **Classifier**. That is, the network was exposed to a dataset that formed the training set, and was able at some point to identify an underlying pattern, which allows it, upon seeing new data from the same dataset, to correctly infer the corresponding label for each sample.

The question arises: what does it mean to classify correctly? And how many elements does it ultimately classify correctly?

From this arises the need to establish standard methods for evaluating the performance of a classifier. This allows comparing different classifiers, comparing the same classifier on different datasets, and evaluating changes in hyperparameters in relation to performance[1].

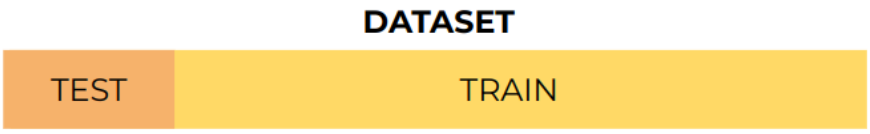


Figura 10.1: The dataset, the matrix X, can be split into at least two parts: training and test.

10.2 Standard Metrics

What is sought at a minimum is that the classifier **generalizes** to the dataset. For this, the dataset (Fig. 10.1) is divided in two: one part is used to **converge** the parameters with the training set (*train set*),

[1] An excellent introductory exposition can be found in Chapter 5 of [56]

and then the remaining group (*test set*) is used to evaluate how well the classifier was able to capture the pattern and obtain good recognition results.[2]

This raises two problems. The first is how to evaluate whether this split is appropriate or not. The second problem is how to evaluate the generalization capacity of the network.

10.2.1 Confusion Matrix

The confusion matrix serves to establish a performance evaluation metric for a binary/multiclass classifier. It is constructed by listing the predicted classes in the columns and the true classes in the rows. The elements of the **test** dataset are then enumerated according to their true label and the label predicted by the classifier.

Predictions	**Positive?**	**Negative?**
Positive	TP	FN
Negative	FP	TN

Cuadro 10.1: Confusion Matrix

This terminology comes from the medical field, which is why reference is made to positive and negative as the result of a biochemical test. TP, True Positives, are those examples that the predictor marked as POSITIVE and that turned out to be truly positive. That is, the cases where it correctly detected, let us call it, the pathology. In this case, we want this value to be as high as possible, so that the predictor does not miss any positives — so that it identifies at least all of those that exist. The second value we want to keep high is TN, True Negatives: the results that were truly negative and that the predictor also identified as such. This value represents the predictor's ability to separate the wheat from the chaff. The other two values, FP and FN, are false positives and false negatives — those POSITIVES and NEGATIVES that the predictor got WRONG. Of course, we want these to be as low as possible.

From this arises the most popular measure: Accuracy, or Performance.

Definición 10.2.1 — Accuracy. The performance of a classifier can be obtained by

$$Acc = \frac{TP+TN}{TP+TN+FP+FN} \tag{10.1}$$

In the case where the confusion matrix is diagonal, the classifier is perfect and achieves a performance of 1. It is important to keep in mind that the worst performance, when the classifier is binary, is not 0 but $\frac{1}{2}$, which corresponds to a classifier that simply flips a coin to determine the class.

The confusion matrix is commonly generalized to multiclass classification, where heatmaps are typically used:

10.2.2 Metrics

In addition to Accuracy, other standardized metrics provide information about the performance of a classifier from different perspectives.

[2] If the dataset is large enough, it can be divided into three parts, adding a *validation set* to evaluate the model's performance under different hyperparameter configurations before its final evaluation on the *test set*.

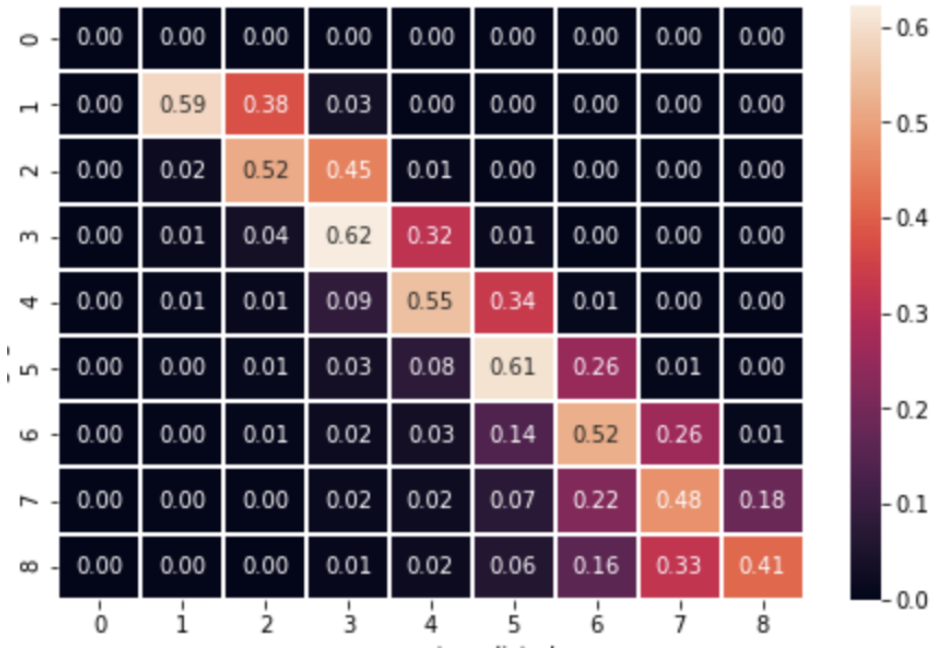


Figura 10.2: Example of a multiclass confusion matrix.

Definición 10.2.2 — Precision. Represents how well the predictor got right those it identified as positive relative to the problem.

$$Precision = \frac{TP}{TP+FP} \tag{10.2}$$

Definición 10.2.3 — Recall. How well the predictor captured all those that were ultimately positive.

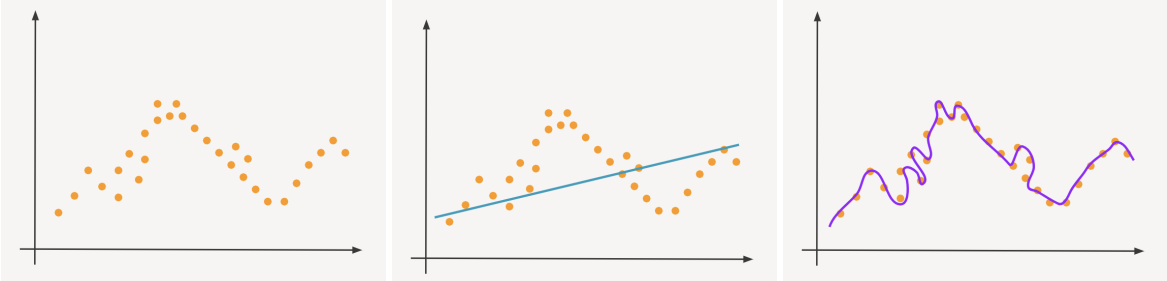
$$Recall = \frac{TP}{TP+FN} \tag{10.3}$$

Definición 10.2.4 — F1-Score. The performance of a classifier can be obtained by

$$F1\text{-}Score = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{10.4}$$

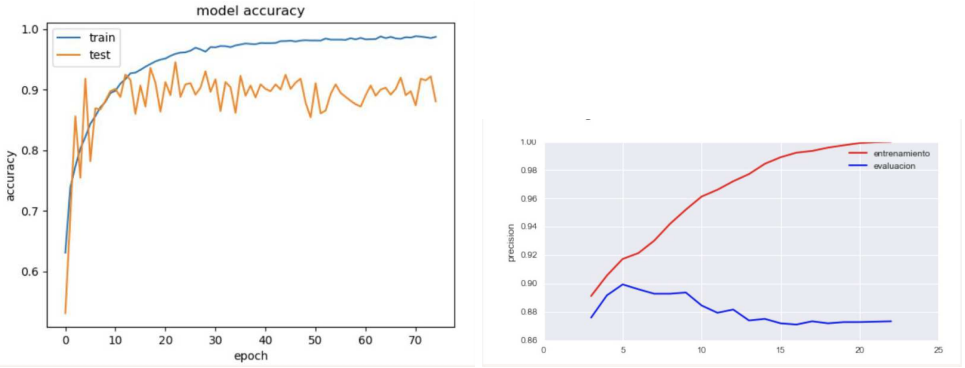
10.2.3 Fitting, Underfitting, and Overfitting

When training the classifier, the goal is to **converge** — that is, to adequately use the input data to **fit** the free parameters so that the cost/optimization function reaches its minimum (Fig. 10.3). When this does not happen — that is, when the solution is stuck at some local minimum, or not even that — the classifier is said to be **underfitted**. On the contrary, when the classifier is overtrained, the situation of **overfitting** arises, where the minimum found follows the noise curve in the data, causing the model to fit excessively to the variations in the training set and making it unable to truly identify the underlying



(a) Points to Fit (b) Underfitting (c) Overfitting

Figura 10.3: Levels of fitting on a set of points. Underfitting occurs when the fitting curve does not capture the inherent pattern in the points (in this case a curve). Conversely, when the fit is too precise, the curve captures the natural noise in the data and loses generalization power.



(a) Undertraining (b) Overtraining

Figura 10.4: Performance curves per epoch for MLPs that incur in undertraining or overtraining.

pattern: it fails to **generalize**[3].

Normally in an MLP (or similar) acting as a classifier, the underfitting situation is identified because the network's performance is high on the training set and there is a trend during training where, as training increases, performance on the test data also increases, as seen in Figure 10.4a. On the other hand, when an overfitting situation occurs, the opposite happens and with more training the performance on the test data decreases until all predictive power is lost. Overfitting is a major **boogeyman** because it can arise from problems in the data, such as class imbalance, data with excessive noise, or even an insufficient amount of data to discern the underlying pattern.

10.2.4 Cross-Validation

The suggested partition implies the assumption that the data are highly homogeneous — that is, that the choice of how to establish the partition will not impact the results. This is rarely the case, which is why there exist validation methods aimed at eliminating the bias introduced by the arbitrary choice of how to split the data into training and test sets.

The most widely used method for this is **k-Fold Cross Validation**.

[3]We will adopt this definition, even though the concept of overfitting is very loosely used in machine learning jargon, as it tends to be a post-hoc consideration when the method fails to generalize [57].

k-Fold Cross Validation

This method is very simple and proceeds as follows:

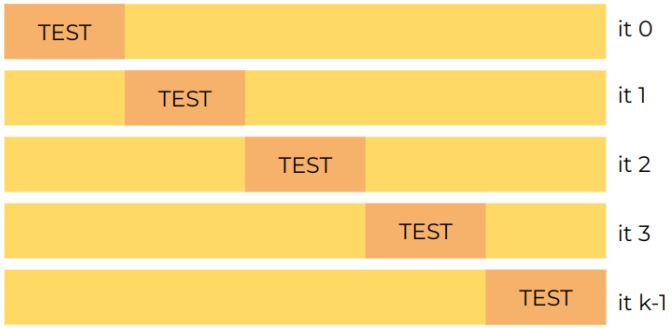


Figura 10.5: In this case, the dataset is divided into 5 parts, with the role of the test set alternating across each one.

1. Iterate T times (normally $T = k$)
2. Divide the dataset into k equal parts
3. Take $k - 1$ parts and use them to train the classifier
4. Use the remaining part that was **not** used for training as the test set
5. Evaluate performance on that part
6. Iterate

Finally, performance parameters are obtained from the average of the T iterations. This method helps avoid any bias that may arise in the dataset split. Additionally, if the deviation of the metrics is evaluated, a measure of the homogeneity of the data itself can be generated.

V Automating Learning

11 Machine Learning 105
11.1 Redefining Learning
11.2 Learning Methods
11.3 Machine Learning Pipeline

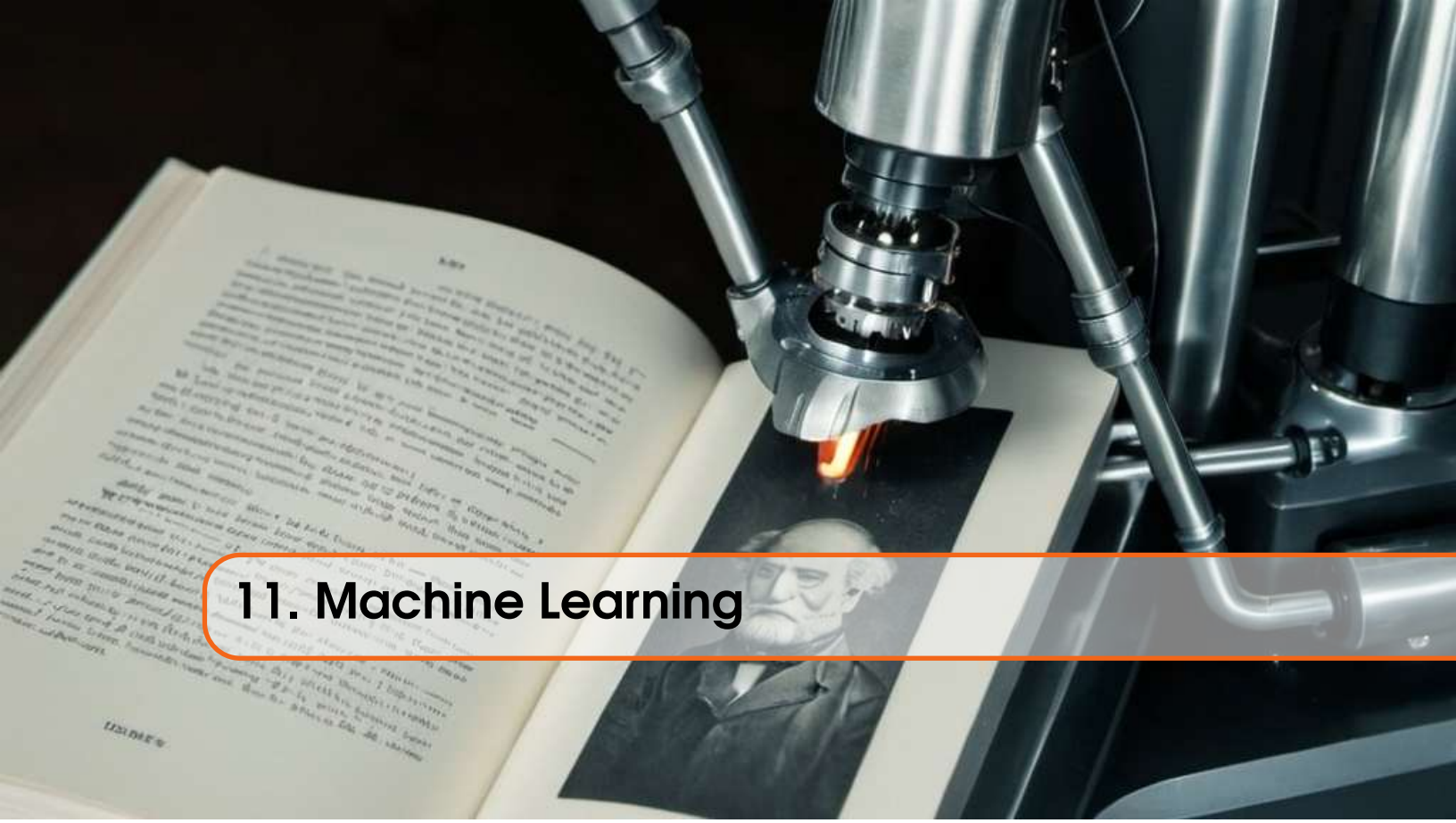
12 Data Transformations 111
12.1 Internal Structure of the Data

13 Principal Component Analysis 115
13.1 Preliminaries

14 Kohonen Network 121
14.1 Self-Organizing Map

15 Hopfield Network 125
15.1 Hopfield 1982

16 Oja and Sanger Model 131
16.1 The Search for the Power of Synthesis
16.2 Oja Network
16.3 Sanger Network



11. Machine Learning

11.1 Redefining Learning

The field of neural networks falls within the boundary of what is called **Machine Learning**. It can be defined as:

> **Definición 11.1.1 — Machine Learning.** Techniques for identifying relationships between data (X) or mappings ($X \to Y$) based on model-free algorithms that, given a set of free parameters structuring those relationships or mappings, can be adjusted through some mathematical optimization process [58].

To frame this definition, it is important to highlight the **model-free** aspect. This means that the algorithmic scheme to be implemented does not assume any specific guidelines regarding the existence and interaction of the components of the problem. That is, no model is established — generally quantitative and mathematical — about how the different elements of the system interact with each other.

This, which may seem like a trivial point, is actually a bombshell, and is truly a radical shift as a general methodology for tackling real-world problems. For example, consider a cannon with only one free parameter: the firing angle. Different angles will produce different distances where the projectile will land. And of course, what we want to do is predict where the projectile will land given a particular angle. Basic Newtonian physics gives us a tool — a model of the mechanics and dynamics of that motion, the projectile motion equation — that we can use to quantitatively solve the problem: we set an angle, do the calculation, and it tells us where it will land. However, the machine learning approach throws that model out the window, and instead simply fires the cannon many times, measures the distance, repeats that process many times, builds a dataset, and uses it to train an ML model[1] that will

[1] Note that we use the same word for two different things: a model of the world that mechanistically simplifies its behavior, and also a model to describe a scheme for adjusting free parameters by optimizing a cost function based on a dataset. Someone

allow us to solve the same problem: to predict.

Definition 11.1.1 is the most encompassing and the one we will adopt in this text. From it arises the concept of **Learning** (what we mean when we say a computer learns): it learns when what is established in definition 11.1.1 is executed.

11.2 Learning Methods

Being cruel, banal, and unforgiving toward the field, a pragmatic way to describe it is as a bag of cats of methods. But they all fall under definition 11.1.1 — that is, they all have some kind of free knob that can be adjusted to optimize some functional [56]. Figure 11.2 shows the different main branches into which the methods are grouped.



Figura 11.1: A bag of cats of varied methods, where all of them have free parameters to be adjusted.

The first division relates to bio-inspired/biomimetic methods and those that are not. Today (circa 2024) this division coincides with methods based on deep multilayer neural networks and those that are not, which by juxtaposition can be called *Shallow*. Among the latter — the traditional machine learning methods — four main categories can be included: frequentist, Bayesian, `if++`, geometric, and transformational. Frequentist methods are based on mathematical statistics and are the heirs of statistical analysis: linear regression, logistic regression (logit), LDA **L**inear **D**iscriminant **A**nalysis. Then, the sworn enemies of the first group, the Bayesian methods, are based on Bayes' formula and the possibility of performing causal inference using the ability to iterate this equation: from **Naive Bayes** to **EM**. Next come those methods that use conditionals with free thresholds acting as parameters, such as **Decision Trees** and **Random Forest**. These are followed by methods based on concepts from linear algebra and its geometry, such as k-NN **K**-**N**earest **N**eighbors, the great SVM **S**upport **V**ector **M**achine, **DBSCAN**, **k-means** and its variants. This humble taxonomic description culminates with the transformational methods, which seek to generate a transformation of the data by optimizing some cost function aimed at some reorganization and grouping of the data, such as PCA (Chapter 13), SVD (Chapter 19), and other dimensionality reduction methods such as t-SNE.

11.3 Machine Learning Pipeline

From this arises the model in Figure 11.3. This model is pervasive to any implementation of a system that falls within the category of machine learning. First, from the real world, from reality, a measurement is taken. This measurement may involve a manual process or even an automatic one (Automation). Then there is a signal acquisition process, which also involves the digitization of the information itself — converting phenomenological information into digital data in a computer. From that point begins what is called the machine learning **pipeline**. The first step is a preprocessing task: adjusting the information, correcting it, recovering lost or missing data, and above all preparing the data for what comes next. The second step of the pipeline is focused on assembling a synthesis — a compression of the data that numerically captures the pattern in a vector that one later wants to detect and generalize. This aspect is central in machine learning and is where what is known as **feature engineering** is concentrated. The final step relates to the ultimate goal of the entire model, which can be one of four objectives: classification, regression, optimization, and generalization. After the pipeline,

should invent a new word.

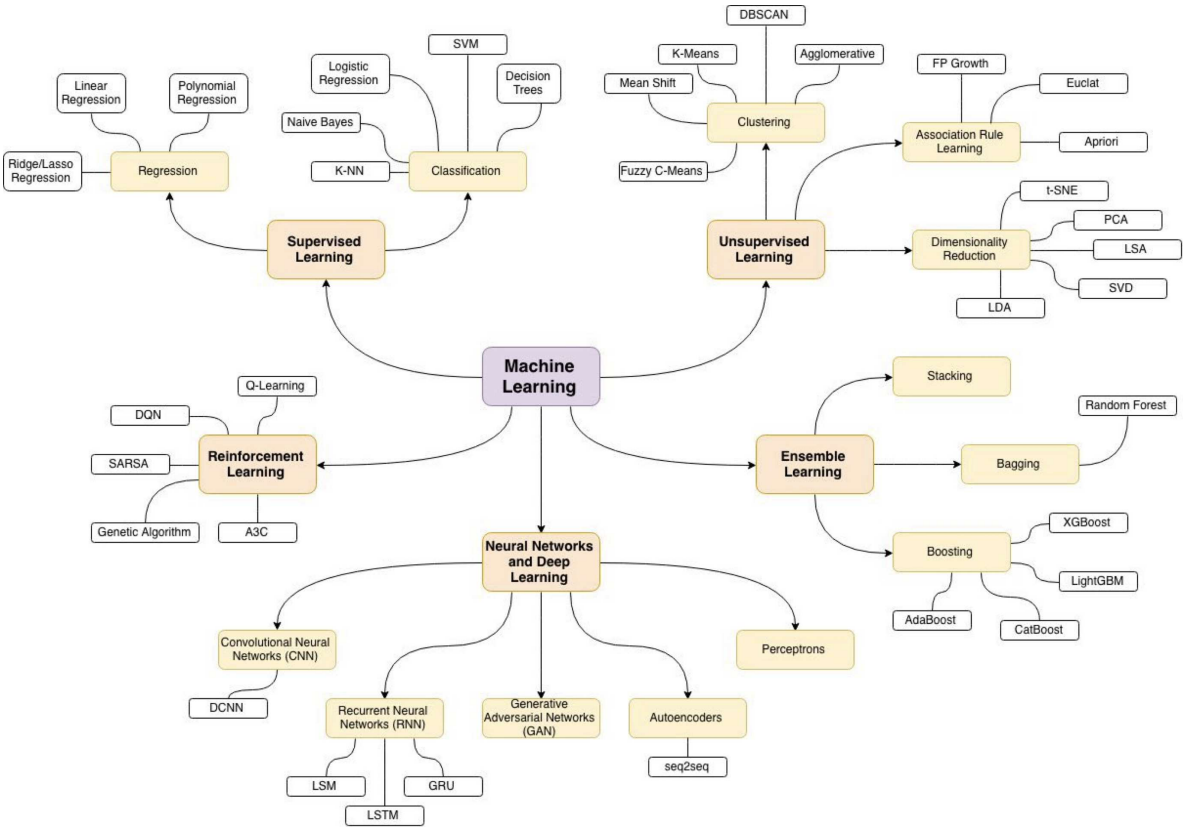


Figura 11.2: Machine Learning and associated areas.

the insight generated by the process is used by an application system, which can affect that same study environment in a closed loop.

11.3.1 The Inverse Problem

Figure 11.4a represents the structure of the inverse engineering problem, and is a critical point to keep in mind in any implementation of machine learning or artificial intelligence. The natural phenomenon we want to study has an intrinsic representation in L continuous analog signals. This can be anything, but they are the sources we are interested in understanding in order to identify some pattern and later generalize. This phenomenon is captured by N transducers that receive a representation of the sources that is not exactly the same information, but rather transformed by the vector function $x^N(t) = f(s^L(t))$. This process also has an inexorable transduction error, so the signal that is actually generated is $\hat{x}^N(t) = f(s^L(t)) + \xi$.

These signals are digitized by an ADC, forming the data matrix $x^N[n]$ which is truly what enters the model. From this point, the model derives from that information an output $y^K[t]$ representing the output, potentially forming a MIMO system, Multiple Input Multiple Output.

In parallel, one could attempt to recover an estimate of the original sources of the physical phenomenon itself — used to derive the entire process — the $\hat{s}^M[n]$. There exists a set of transformational techniques known as source separation, or **B**lind **S**ource **S**eparation, which fall under the umbrella of machine learning [59]. This is always an estimate because the information of the original sources, with absolute precision, is always inaccessible. This is independent of the model's own capacity — be it

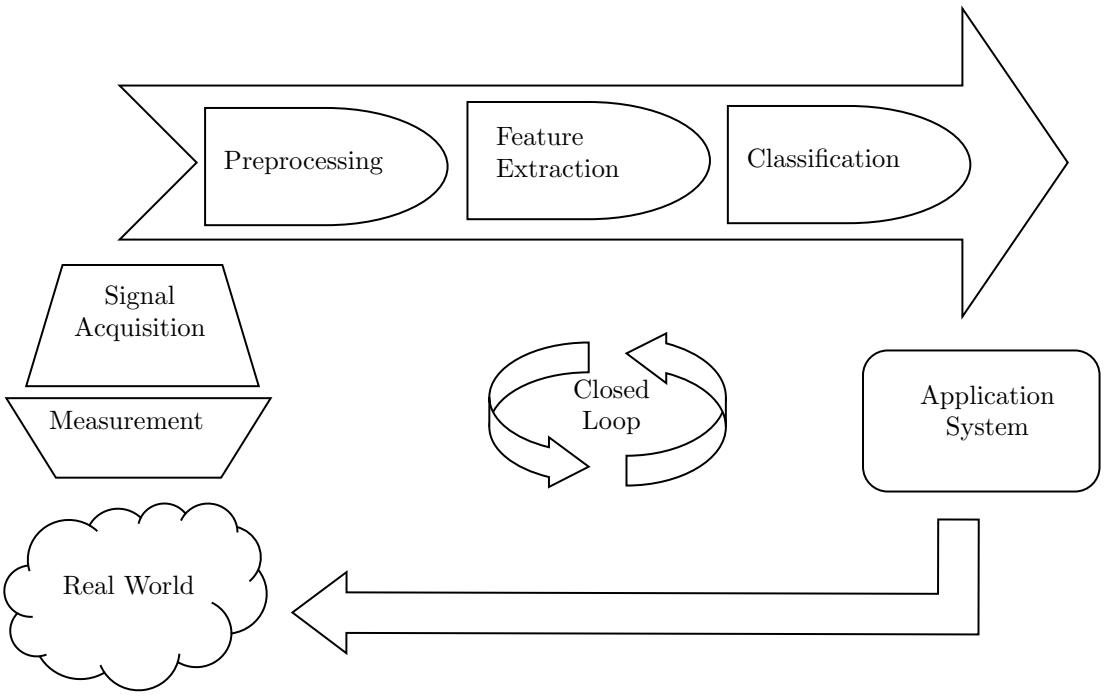
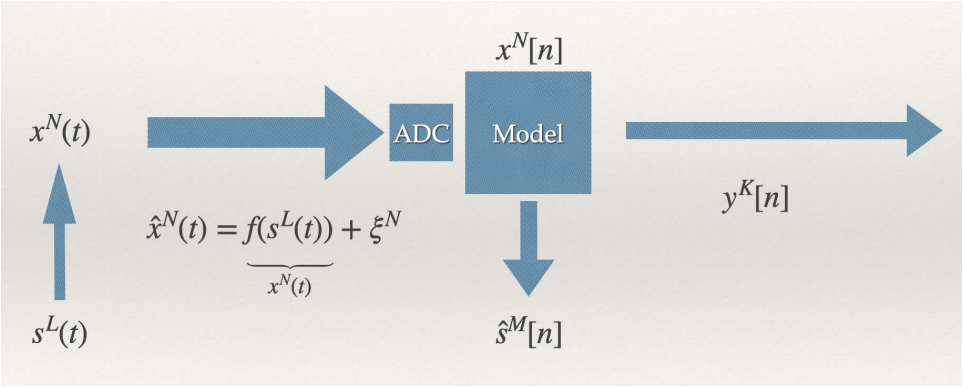
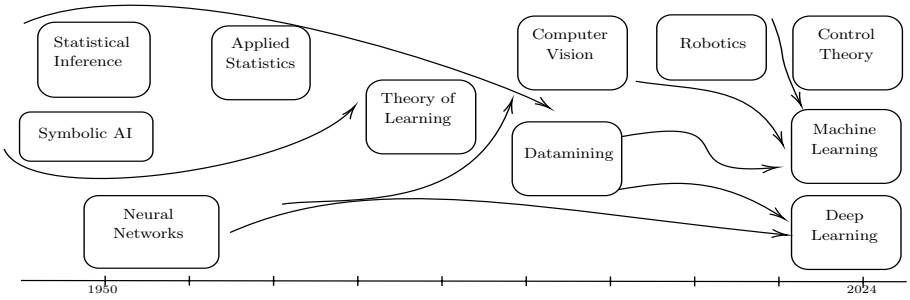


Figura 11.3: General Architecture of a Machine Learning System

AGI, superintelligent, or whatever.

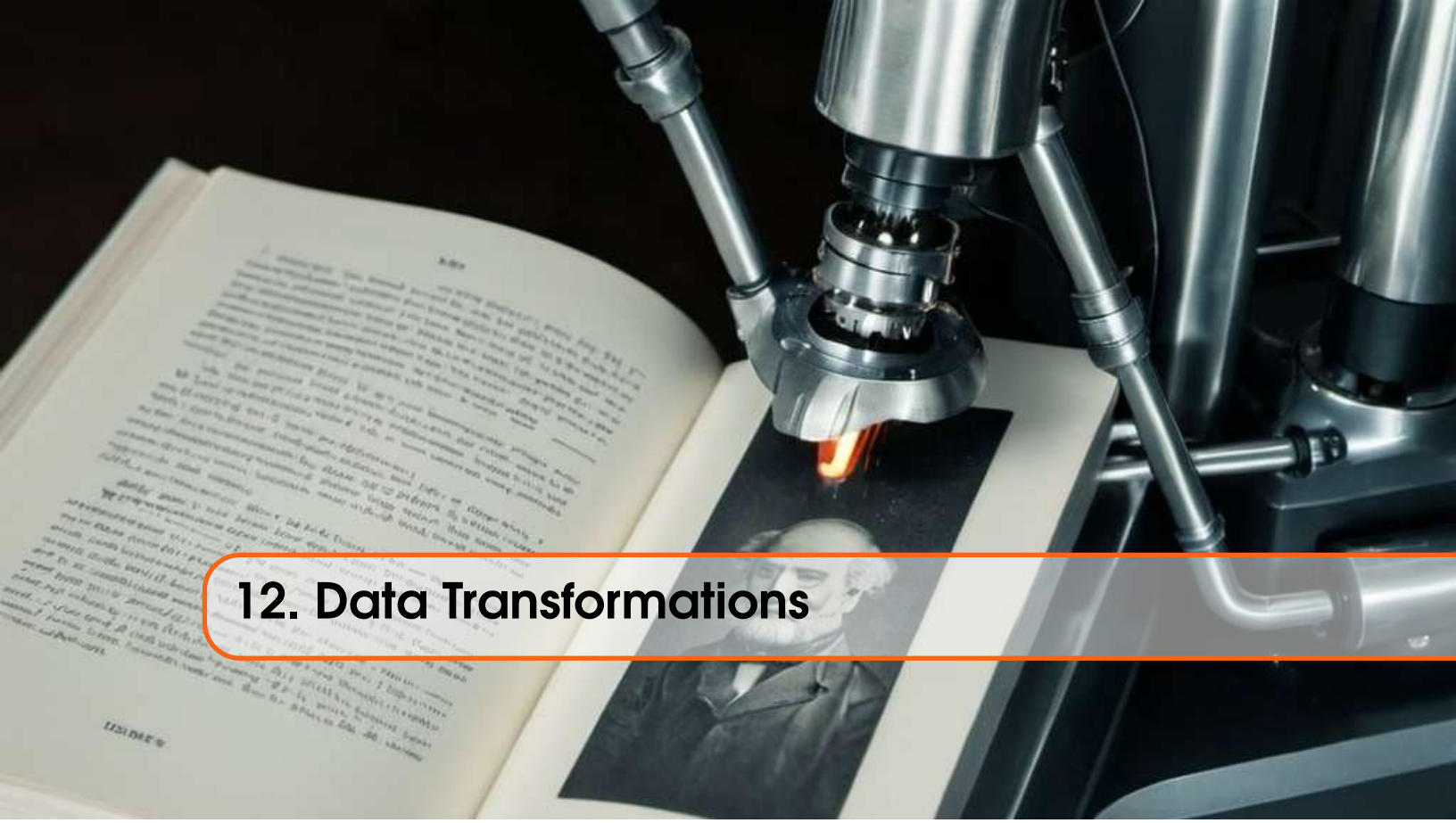


(a)



(b)

Figura 11.4: Inverse problem and historical evolution of machine learning techniques.



12. Data Transformations

Unsupervised learning focuses on learning patterns and relationships that exist among the samples themselves and their variables, rather than on learning a specific mapping between the independent variables and the response variable, the *label*. These methods are strongly related to the idea of transforming data, of seeking alternative representations that help precisely to capture that information. These algorithms are often used independently, but also as part of the processing pipeline itself, since generally that new transformation can be useful in the supervised setting as well.

12.1 Internal Structure of the Data

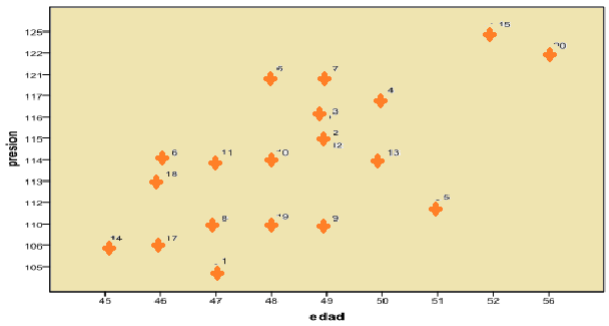
What is sought is the identification of the structure of the information itself. The goal is to understand how the information in the variables interacts with one another and how that can be used to identify underlying patterns.

■ **Ejemplo 12.1** For example, for a study, 20 hypertensive patients are randomly selected, for whom the following variables were measured:

- X_1: Mean arterial pressure (mm Hg)
- X_2: Age (years)
- X_3: Weight (Kg).
- X_4: Body surface area (m^2)
- X_5: Duration of Hypertension (years)
- X_6: Pulse (beats/minute)
- X_7: Stress measure.

This implies that the dimension of the sample matrix is 7. We can attempt to reduce it to two variables, arbitrarily: Pressure vs. Age.

Figura 12.1: Pressure vs. Age



From the graph it follows that two individuals that are *close* to each other could share a similar characteristic of these two combined variables. That is, two individuals who are close in this graph should represent similar real-world situations. This is the underlying idea in unsupervised learning methods.

■

To identify this internal structure, the following approaches are used:

1. Grouping or Clustering.
2. Associations.
3. Dimensionality reduction.

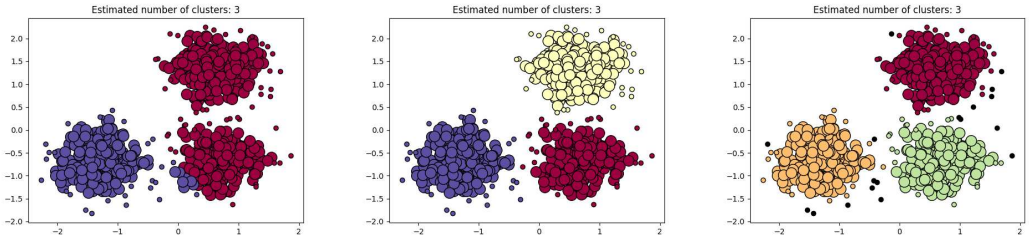
Unsupervised learning methods make use of the geometric structures of the spaces in which the samples X live, and establish similarity measures on them. When an element x_i is similar to another x_j. For this, ideas from linear algebra are predominantly used, but the approach expands rapidly into a broader range of techniques.

Particularly in this text, we will focus on the methods that implement these associations based on biomimetic neural network structures. However, a brief introduction is provided that will serve to understand the general concepts.

12.1.1 Clustering and Grouping

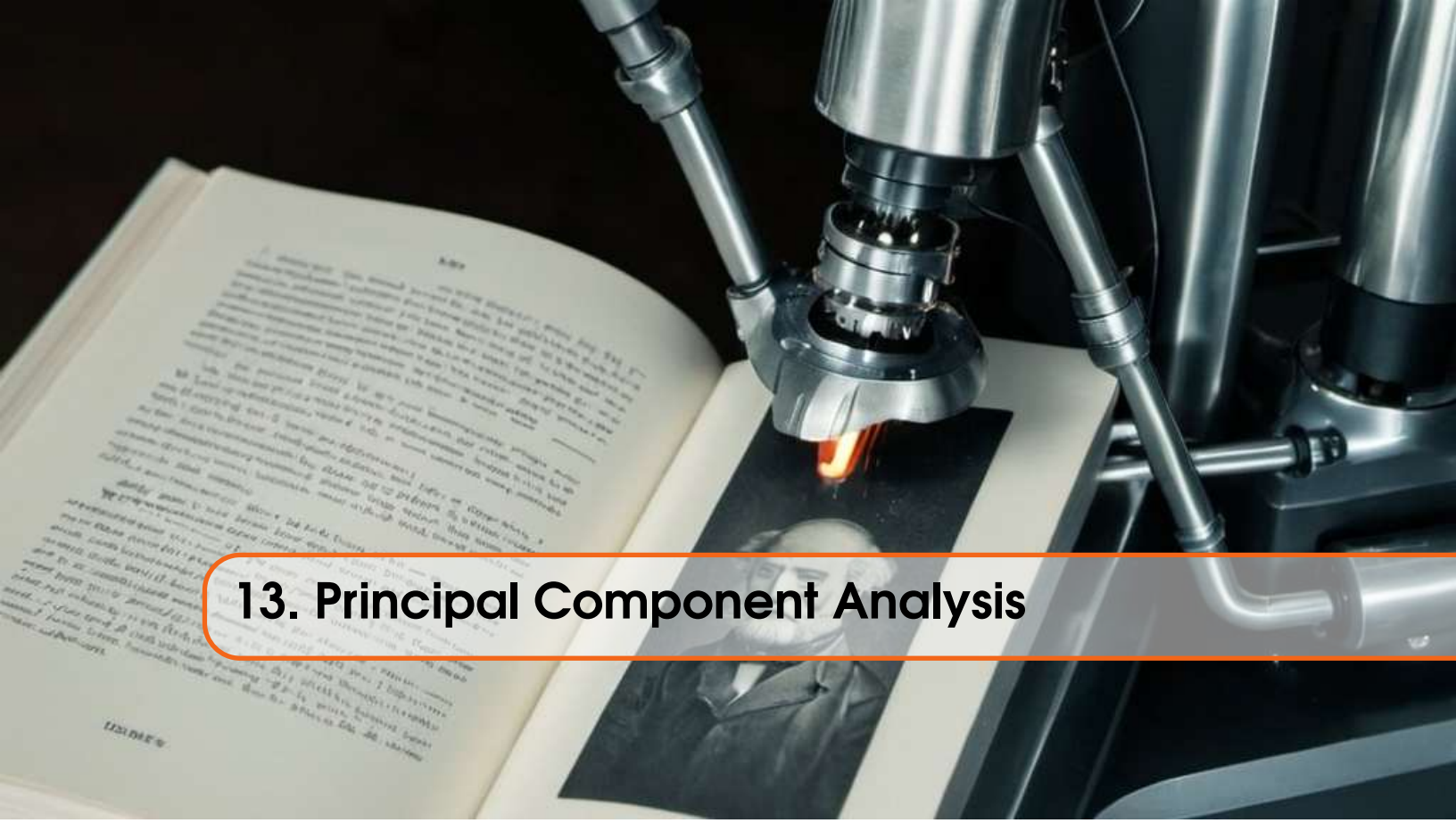
Based on establishing those similarity measures between the data to verify whether they belong to clearly identifiable internal subgroups: searching for clusters.

Figure 12.2 shows, for example, three variants that reflect the most common problems in clustering. The first is the grouping produced by *k-means* [5]. This simple algorithm identifies clusters based on a hyperparameter that specifies the number of clusters to find, and then assigns each element to the cluster already assigned for the k nearest neighbors. For Figure 12.2a, the choice of $k = 2$ would not seem to be the best, since graphically it is clear that there are 3 clusters, which is shown in the second graph 12.2b. On the other hand, the third graph 12.2c uses a different algorithm, *dbscan* [60], which takes into account the density of elements and allows identifying some elements as not belonging to any of the three clusters in particular (shown in black in the graph).



(a) k-Means with $k = 2$ (b) k-Means with $k = 3$ (c) DBSCAN with free k

Figura 12.2: Clustering Alternatives



13. Principal Component Analysis

We make here an interlude to present a data transformation method that emphasizes an aspect of intelligence: the capacity to summarize information, to transform it into a format that captures a certain essence of the information. One of these methods is Principal Component Analysis.

13.1 Preliminaries

- Sample Mean: Given a dataset of a variable $\mathbf{X} = \{\mathbf{x_1}, \mathbf{x_2}, \ldots, \mathbf{x_n}\}$

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

- Sample Variance:

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

  is a measure of dispersion.
- Sample Standard Deviation: Another measure of dispersion

$$s_{dv} = \sqrt{s^2}$$

- Sample Covariance: a measure of linear association between the data of two variables. Given the dataset:

$$\left[\begin{array}{c|c|c|c} \mathbf{X_1} & \mathbf{X_2} & \ldots & \mathbf{X_n} \\ \hline x_{11} & x_{12} & \ldots & x_{1n} \\ x_{21} & x_{22} & \ldots & x_{2n} \\ \vdots & & & \\ x_{m1} & x_{m2} & \ldots & x_{mn} \end{array}\right]$$

$$s_{ik} = \frac{1}{n}\sum_{j=1}^{m}(x_{ji} - \bar{x_i}) * (x_{jk} - \bar{x_k})$$

Covariance Matrix: The covariances form a symmetric positive definite matrix

$$S = \begin{bmatrix} s_{11} & s_{12} & \dots & s_{1n} \\ s_{21} & s_{22} & \dots & s_{2n} \\ \vdots & & & \\ s_{m1} & s_{m2} & \dots & s_{mn} \end{bmatrix}$$

Interpretation of the Sample Covariance

- $s_{ik} > 0$ indicates a positive linear association between the data of the variables.
- $sik < 0$ indicates a negative linear association between the data of the variables.
- $s_{ik} = 0$ indicates that there is no linear association between the data of the variables.

R The sample variance is the sample covariance between the data of the i-th variable with itself; it is sometimes denoted as s_{ii}.

Standardized variables:

$$\tilde{\mathbf{X}}_\mathbf{1} = \frac{\mathbf{x_1} - \bar{\mathbf{x}}_\mathbf{1}}{\mathbf{s_1}}$$

From each variable we subtract its own mean and divide by its own standard deviation.
Sample Correlation: Computing the covariances with the standardized variables:
It is another measure of linear association. For the data of the i-th and k-th variable it is defined as:

$$r_{ik} = \frac{s_{ik}}{\sqrt{s_{ii}}\sqrt{s_{kk}}}$$

The correlation is bounded between -1 and 1.

13.1.1 PCA Transformation

If the variables are highly correlated, they contain redundant information. The idea of the method is precisely to eliminate this redundancy. The goal is to transform the original set of variables into another set of new variables that are linear combinations of the original ones but are uncorrelated with each other, called the **set of principal components**. The principal components technique is due to Hotelling (1933), although its origins lie in the orthogonal least-squares fitting introduced by K. Pearson (1901).

Given p original variables, we seek $q < p$ variables that are linear combinations of the p original ones, capturing most of the information or variability of the data.

If the original variables are uncorrelated, then there is no point in performing a principal component analysis.

The characteristic that maximizes variability

is a good factor for differentiating objects in a dataset. For example, a dataset with vehicle information:

- Feature 1: Number of wheels.
- Feature 2: Vehicle length. (Greater variability)

Using the second one, we would be able to tell whether a record corresponds to a car or a bus.

Suppose we have the values of p variables for n elements of a population arranged in a matrix X of dimensions $n \times p$, where the columns contain the variables and the rows contain the elements.

Variables:

$$\{x_1, \ldots, x_p\}$$

PCA performs a transformation of the dataset X consisting of a translation and a rotation, defined in such a way that the variance of the new set of variables is maximized.

The First Component

$$y_1 = \sum_{j=1}^{p} a_{1j}(x_j - \bar{x}_j) = a_{11}(x_1 - \bar{x}_1) + \ldots + a_{1p}(x_p - \bar{x}_p)$$

with

$$\vec{a_1} = (a_{11}, a_{12}, \ldots, a_{1p}) \in \mathbb{R}^p$$

The set of principal components is a linear combination of the original variables.

We seek $\vec{a_1} / \|\vec{a_1}\| = 1$ such that $Var(y_1)$ is maximized among all possible linear combinations of the original variables.

Loadings:

The coefficients a_{ji}, $i = 1, \ldots, p, j = 1, \ldots, p$ are called loadings.

How to find the values of the loadings?

They are the eigenvectors of the covariance matrix.

This problem is solved by finding the $v_1, \ldots, v_n$ and the $\lambda_1, \ldots, \lambda_n$ such that

$$det(S_X - \lambda_i I) = 0$$

and the v_i such that

$$S_X v_i = \lambda_i v_i$$

Then

$$y_1 = v_{11}x_1 + \cdots + v_{n1}x_n$$

$$y_2 = v_{12}x_1 + \cdots + v_{n2}x_n$$

and so on for all the others.

Furthermore, the eigenvalue λ_i is the variance of component i.

By sorting the eigenvalues of S_X from largest to smallest, $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_n$, we can reduce the dimensionality by taking the eigenvectors corresponding to the first q eigenvalues, which are the ones that provide the most information (in terms of variability).

The percentage of variability captured by the first q components is

$$Variability = \frac{\sum_{i=1}^{q} \lambda_i}{\sum_{i=1}^{n} \lambda_i}$$

Therefore, to reduce the dimensionality of a dataset while preserving its most important characteristics, we should require that the projection covers, for example, 95 % of the variance, that is

$$Variability \geq 0.95$$

1. Step 1: Take a dataset X. The variables must be in the columns.
2. Step 2: Subtract the mean of each variable. Compute $X - \bar{X}$. Obtain a zero-mean dataset.
3. Step 3: Compute the Covariance Matrix.
4. Step 4: Compute the eigenvalues and eigenvectors of the covariance matrix and sort the eigenvalues from largest to smallest.
5. Step 5: Form the matrix E by taking the eigenvectors corresponding to the largest eigenvalues.
6. Step 6: Compute the new variables $Y = E(X - \bar{X})$

Covariance Matrix vs. Correlation Matrix: If one of the variables — for example the first one — has larger values than the rest, the way to increase the variance is to make the coordinate associated with this variable as large as possible. For example: if we switch from measuring in km to measuring in meters, the weight of that variable in the analysis will increase.

Therefore, when the measurement scales of the variables are very different, the maximization of variance will depend decisively on these scales and the variables with larger values will carry more weight in the analysis.

If we want to avoid this problem, it is advisable to use the standardized variables to compute the principal components, so that the magnitudes of the numerical values of the original variables are similar. This is equivalent to applying principal component analysis using the correlation matrix instead of the covariance matrix.

When the variables have the same units, both alternatives are possible.

Example:

Let the covariance matrix be

$$S = \begin{pmatrix} 3 & 1 & 1 \\ 1 & 3 & 1 \\ 1 & 1 & 5 \end{pmatrix}$$

corresponding to the random variables (X_1, X_2, X_3) with zero mean.

1. Write the principal component variables (Y_1, Y_2, Y_3) and compute what proportion of the total variance each component explains.

Eigenvalues: $\lambda_1 = 6\ , \lambda_2 = 3\ , \lambda_3 = 2$

Eigenvectors:

$$E = \left(\begin{array}{ccc} V_1 & V_2 & V_3 \\ \hline -0.40 & -0.57 & 0.70 \\ -0.40 & -0.57 & -0.70 \\ -0.81 & 0.57 & 0 \end{array} \right)$$

$$\begin{aligned} Y_1 &= -0.40X_1 + (-0.40)X_2 + (-0.81)X_3 \\ Y_2 &= -0.57X_1 + (-0.57)X_2 + 0.57X_3 \\ Y_3 &= 0.70X_1 + (-0.70)X_2 \end{aligned}$$

Proportion of variance of each component: Eigenvalues: $\lambda_1 = 6\ , \lambda_2 = 3\ , \lambda_3 = 2$

of $Y_1 = \frac{6}{11}$
of $Y_2 = \frac{3}{11}$
of $Y_3 = \frac{2}{11}$

What information do the loadings provide?

- If the loading (coefficient) of a variable in the principal component is positive, it means that the variable and the component have a positive correlation.
- If, on the contrary, the loading is negative, this indicates that the variable is negatively correlated with the first component.

Thus the first component represents an index (or a characteristic) by which the records can be ordered.



14. Kohonen Network

The Kohonen network, also called a SOM, Self-Organizing Map, is a bio-inspired network that allows detecting regularities in the input data. Its author was Teuvo Kohonen, a Finnish researcher who published his idea for the first time in 1982, and it was one of the reasons for the second wave of artificial intelligence [61, 62].



Figura 14.1: Teuvo Kohonen, pleased with his great work.

14.1 Self-Organizing Map

The network architecture consists of two layers, where the first corresponds to the inputs, and the second is a layer where neurons are connected to themselves positively and have inhibitory connections with neighboring neurons. Over time, this system causes some of the units to take on a higher level of activation while the activation level of the remaining units is driven to zero.

As observed in Figure 14.2, the central layer is a grid or map that can in turn have different topological layouts.

14.1.1 Competitive Learning

Kohonen's idea was to implement a biomimetic mechanism of competitive learning. Given an input unit x, the neuron whose weight vector w is the most similar — under some similarity measure — will be the winner. This is a clustering scheme, because the result it produces is that similar inputs will activate the same neuron in the grid, grouping together. It is competitive because the neurons end up competing with each other with the objective that ultimately only one of the output neurons activates and the rest are forced to minimum response values. This mechanism is called **winner take all**.

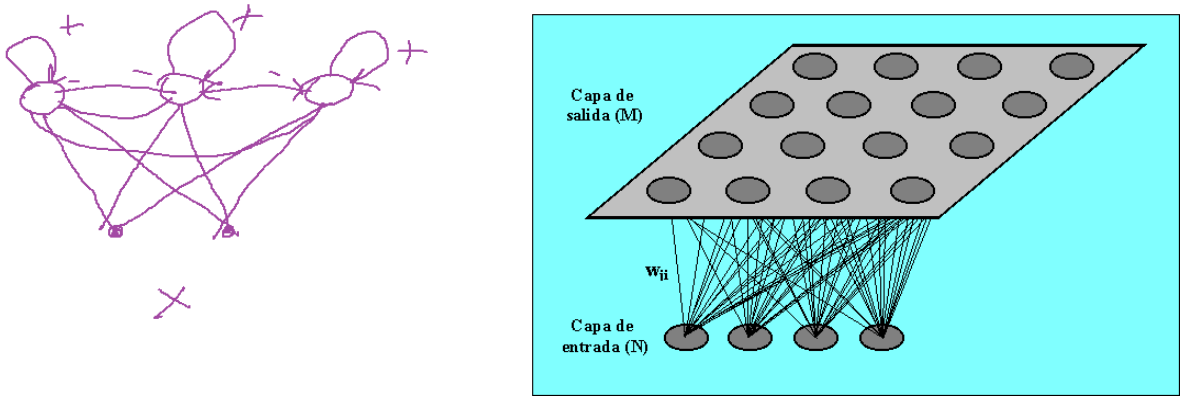


Figura 14.2: Self-organizing map, network connection structure and output grid.

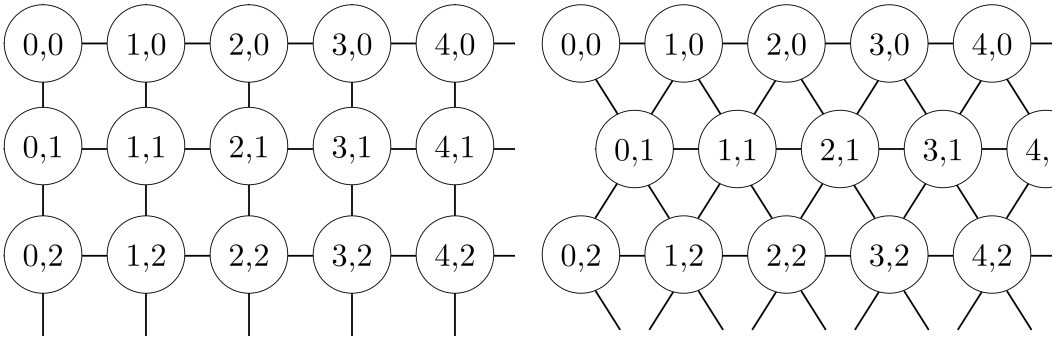


Figura 14.3: Possible structures for the network grid.

14.1.2 Architecture

The Kohonen network has a single layer of neurons (plus an input layer) that forms a two-dimensional grid ($k \times k$) in which each neuron is connected to all components of an n-dimensional input vector. The multidimensional input data are thus projected onto a two-dimensional space. The grid can be rectangular or hexagonal.

A **neighborhood** of 4-neighbors with radius $R = 1$ or 8-neighbors with radius $R = \sqrt{2}$ is defined over the grid. Each time a vector is placed at the input, a winning neuron is selected and an update rule is applied to that neuron and its neighborhood.

The characteristics of the network are:

- A particular grid size of $k \times k$ is established, with a topology — rectangular or hexagonal.
- Each output neuron $j \in \{1, ..., k^2\}$ has an associated weight vector $W_j = (w_{j1}, ..., w_{jn})$.
- The weights W_j of each output neuron have the same dimension as the input data.

14.1.3 Algorithm

The algorithm is quite straightforward and is based on an iterative update of the network weights as the examples from the training set are processed.

Algoritmo 10 Kohonen Network

1: $X^p = \{x_1^p, \ldots, x_n^p\},\ p = 1, \ldots, P$ are the input records.
2: Define the number of output neurons: $k \times k$.
3: Initialize the weights $W_j,\ j = 1, \ldots, k^2$, each $W_j = (w_{j1}, \ldots, w_{jn})$:
4: With random values drawn from a uniform distribution.
5: With random samples from the training set.
6: Select an initial neighborhood size $R(0)$.
7: Select an initial learning rate $\eta(0) < 1$.
8: **while** iteration t **do**
9: Select an input record X^p.
10: Find the winning $\hat{k}$ with weight vector $W_{\hat{k}}$ closest to X^p:
11: Define a similarity measure d and then compute:

$$W_{\hat{k}} = \arg \min_{1 \leq j \leq N} \{d(X^p - W_j)\}$$

12: Update the weights of the neighboring neurons according to the Kohonen rule.
13: **end while**

Kohonen Rule

Starting from the activation (the selection) of neuron $\hat{k}$ — the winning neuron — and using the definition of the neighborhood radius $R(t)$, the neighborhood set is:

$$N_{\hat{k}}(t) = \{neu / ||neu - neu_{\hat{k}}|| < R(t)\}$$

where $R(0)$ is an input parameter and $R(t) \to 1$ as $t \to \infty$. This implies a radius update scheme, although the radius can also remain constant throughout the process.

Once the set of neurons to be activated is selected, the weights of all these neighboring neurons of $\hat{k}$ are updated using the Kohonen rule:

- If $j \in N_{\hat{k}}(t) \to W_j^{t+1} = W_j^t + \eta(t) * (X^p - W_j^t)$
- If $j \notin N_{\hat{k}}(t) \to W_j^{t+1} = W_j^t$

The value of $\eta(t)$ acts as a discount factor that decreases at each iteration, $\eta(t) \to 0$. For example, one can use $\eta(t) = \frac{1}{t}$.

This factor is key to the convergence of the Kohonen network: so that the network weights come to resemble the input data and capture its similarity structure.

Demostración. We assume one iteration from step t to step $t+1$:
$W_{\hat{k}}^{t+1} - X^p = W_{\hat{k}}^t + \eta(t)(X^p - W_{\hat{k}}^t) - X^p = (1 - \eta(t))(W_{\hat{k}}^t - X^p)$
Therefore...

$$||W_{\hat{k}}^{t+1} - X^p|| \leq ||W_{\hat{k}}^{t} - X^p||$$

so the difference between the weight and the input keeps decreasing. This shows that the new weight vector is closer to pattern X than the previous one. ■

Similarity Measures (or Propagation Functions)

- Euclidean Distance:

$$W_{\hat{k}} = \arg \min_{1 \leq j \leq N} \{\left\|X^p - W_j\right\|\}$$

- Exponential Distance:

$$W_{\hat{k}} = \arg \max_{1 \leq j \leq N} \{e^{-\left\|X^p - W_j\right\|^2}\}$$

  (normalizing all vectors)

Initialization

- Weights can be initialized with random values: This may have the problem that some units end up far from the initial values and therefore never win. These are called dead units.
- To avoid this, the weights can be initialized with samples from the input data.
- The total number of iterations should be chosen based on the number of input neurons, for example $500 * n$.
- The value of $R(0)$ can be the total size of the network and decreases until it reaches $R_f = 1$, where only the immediately adjacent neighboring neurons are updated.

Data Preprocessing

The network requires standardizing (see 24.2.1) the variables of the training and test sets:

$$\tilde{X}_i = \frac{X_i - \bar{X}_i}{s}$$

where $\bar{X}_i$ is the mean and s_i is the standard deviation of X_i.

Dimensionality Reduction

The Kohonen network is a bio-inspired MDS, Multidimensional Scaling [63], algorithm — simple and direct — for establishing two-dimensional clusters. By using two-dimensional schemes, it is also possible to visualize the result of the network training and appreciate how the output neurons form a matrix, and verify at which coordinates the neuron associated with each training example is located.

- The Kohonen network can be faster than the Multilayer Perceptron.
- Being unsupervised, it can be applied in cases where the dataset has only unlabeled input values.
- If the set of variables is very large, it may be difficult to map it onto a two-dimensional space.
- It can only be used with numerical variables.
- The grid size must be decided from the start and there is no proven criterion for doing so.



15. Hopfield Network

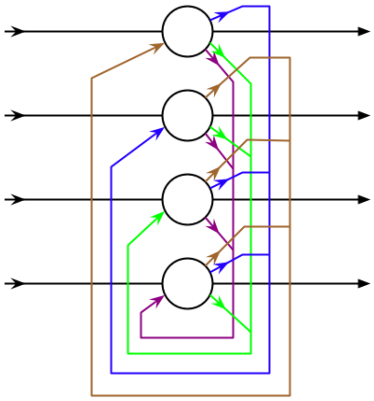
The Hopfield network is a neural network model inspired by computational physics models that has a biomimetic property of associative memory. Given a set of p stored patterns, when a new pattern A is presented to the network, it responds with one of the stored patterns B, such that it is the most similar to A. Pattern A is called the **stimulus** while B is called the **response**. This interesting behavior, with many parallels to biological systems, is achieved by establishing a recurrent network where temporality is present and which manifests the structure of a dynamical system — one that evolves over time and has an energy function associated with it. Hopfield formulated his proposal from the perspective of computational physics, drawing inspiration from the Ising model and Simulated Annealing [4]. This was very cleverly manipulated to generate the effect of an associative memory that associates input stimuli with memorized responses. The Hopfield network was instrumental in driving the second era of artificial intelligence, based precisely on the connectionist model, which was recognized by the award of nothing less than the 2024 Nobel Prize in Physics [64].



Figura 15.1: John Hopfield, very pleased after creating his eponymous network.

15.1 Hopfield 1982

The 1982 Hopfield network is a discrete Recurrent Network. Recurrent networks form dynamical systems where the outputs of each neuron correspond to the inputs at a subsequent time step. The 1982 version refers to the discrete version, while the 1984 version corresponds to the continuous version.



The Hopfield network has the following characteristics:

- All neurons are connected to each other. All-to-all in a star configuration.
- No neuron is connected to itself.
- The permitted set of input and output values is $\{-1,1\}$ — a binary set. In this way all neurons in a Hopfield Network are binary, taking only one of two possible states: active or inactive.
- All neurons are in a single input and output layer.
- The Hopfield model can represent a physical system where the state is given by the output of all neurons simultaneously, and the cost function of the system reflects the energy of the system, where the minima of that function correspond to the stable states where the stored patterns reside.

Hopfield networks act as associative memories. At time $t = 0$, a pattern is placed at all the inputs of the network (which correspond to the inputs of all neurons). Then all outputs are computed, which will serve as the inputs to the system at time $t + 1$. The system is allowed to evolve in this way until a stable point is reached where no further changes occur. The output values at that moment **will be** precisely one of the patterns stored in the network.

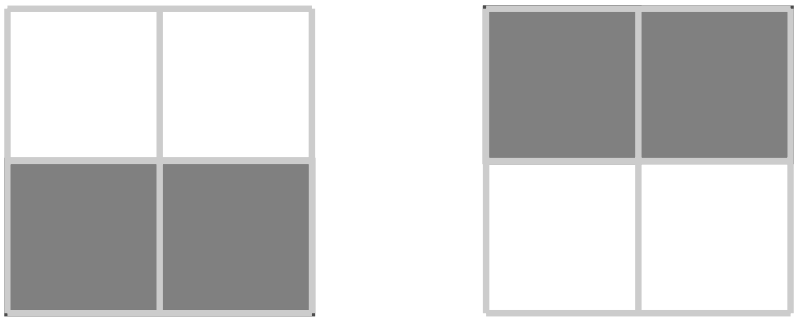


Figura 15.2: Binary image patterns. Assuming white=1 and grey=-1, the first pattern is $\xi_1 = (1,1,-1,-1)$ while the second is $(-1,-1,1,1)$.

These patterns can be images (Figure 15.2), forming a visual input pattern. Each neuron then has 2

possible states:

- $S_i = +1$
- $S_i = -1$

For a network of N neurons, the state is represented by the state vector $S = [S_1, S_2, ..., S_N]^T$.

15.1.1 Associative Memory

Suppose we have a set of stored patterns $\xi^\mu, \mu = 1, ..., p$, already learned, in a Hopfield network. We present a new pattern ζ with the objective of finding the stored pattern most similar to ζ. For this we need the network to store synaptic weights w_{ij}, $i = 1, \ldots, N$, $j = 1, \ldots, N$ such that the network returns the pattern ξ^μ closest to ζ.

The neurons in the network have the following characteristics:

- Each neuron i is a simple Perceptron with the step activation function (1, -1).
- Each pair of neurons (i, j) are connected by the synaptic weight w_{ij}.

We begin with a configuration $S_i = \zeta_i$, $i = 1, \ldots, n$, and want to see if there exists a set of weights w_{ij} that drives the network to reach the state $S_i = \xi_i^{\mu_0}$, where ξ^{μ_0} is a stored pattern, while $\xi_i^{\mu_0}$, $i = 1, \ldots, n$ is the state of the stored pattern most similar to ζ_i, $i = 1, \ldots, n$. Neuron i modifies its state S_i according to the rule:

$$h_i = \sum_{j=1}^{N} w_{ij} S_j, \; i \neq j$$

$$S_i = \begin{cases} +1 & \text{if } h_i > 0 \\ -1 & \text{if } h_i < 0 \end{cases}$$

In the case where $h_i = 0$, neuron i remains in the previous state. This is of course equivalent to:

$$S_i = sgn(h_i)$$

This previous step represents the update of the system state, where all output values are updated one by one, given the complete state of the system at the previous time step. The network thus evolves until S_i no longer changes $\forall i = 1, \ldots n$.

If we were to take only a single pattern in the set ($\xi_i = 1$ or $\xi_i = -1$), $i = 1, \ldots N$, assuming we have N nodes in the network, we can choose the synaptic weights in the following way:

$$w_{ij} = \frac{1}{N} \xi_i \xi_j, \; i, j = 1, \ldots N$$

and thus evolve the network by computing

$$S_i = sgn\left(\sum_{j=1}^{N} w_{ij} \xi_j \right) = sgn(\xi_i) = \xi_i \qquad \forall i.$$

This is because $\xi_j^2 = 1 \qquad \forall j = 1, \ldots N$ and therefore convergence occurs: the stored pattern is reached.

This extends to the case of many patterns, making use of Cooper's rule [65] for the closed-form computation of the weights:

$$w_{ij} = \frac{1}{N} \sum_{\mu=1}^{p} \xi_i^\mu \xi_j^\mu, \; i \neq j$$

where p is the total number of stored patterns. This equation assigns a high value to the weights between two nodes i and j when both are activated or both are deactivated.

Since the network architecture does not admit direct self-connections and the connections are symmetric, we also take $w_{ii} = 0$ and $w_{ij} = w_{ji}$. In matrix form this becomes:

$$W = \tfrac{1}{N} K \, K^T$$
$$S(t+1) = sign(W \, S(t))$$

.

Algorithm 11 shows the step-by-step procedure of the complete algorithm.

Algoritmo 11 Hopfield Model

1: Storage: We have p binary patterns $\xi^1, \xi^2, ..., \xi^p$ of dimension N.
2: The synaptic weights w_{ij} connect neuron i to neuron j.
3:
$$w_{ij} = \begin{cases} \frac{1}{N} \sum_{\mu=1}^{p} \xi_i^\mu \xi_j^\mu & j \neq i \\ 0 & j = i \end{cases}$$
4: Let ζ be an unknown N-dimensional query vector presented to the network.
5: $S_i(0) = \zeta_i$ is the state of neuron i at time $t = 0$.
6: **while** S does NOT remain stable **do**
7: $\quad S_i(t+1) = sgn\left(\sum_{j=1}^{N} w_{ij} S_j(t)\right), j \neq i$
8: $\quad t++$
9: **end while**
10: Return $S(t_{final})$

We note that the synaptic weights are precomputed only once at the start (that computation constitutes the learning step, which in this case is direct and instantaneous, as it has a closed-form formula). In the Multilayer Perceptron or any other network, this would only be possible if we could directly solve the minimization process, which would then require no iteration.

15.1.2 Stability

Let ξ_i^ν be the i-th element of pattern ν. The stability condition is:

$$h_i^\nu = \sum_j w_{ij} \xi_j^\nu = \frac{1}{N} \sum_j \sum_\mu \xi_i^\mu \xi_j^\mu \xi_j^\nu \tag{15.1}$$

which represents

$$h_i^\nu = \frac{1}{N}\sum_j \sum_\mu \xi_i^\mu \xi_j^\mu \xi_j^\nu$$
$$h_i^\nu = \frac{1}{N}\sum_j \sum_{\mu\neq\nu} \xi_i^\mu \xi_j^\mu \xi_j^\nu + \frac{1}{N}\sum_j \xi_i^\nu \xi_j^\nu \xi_j^\nu$$
$$h_i^\nu = \xi_i^\nu + \frac{1}{N}\sum_j \sum_{\mu\neq\nu} \xi_i^\mu \xi_j^\mu \xi_j^\nu .$$

The system is therefore stable if: (1) the second term, the **crosstalk**, is zero, or (2) when the patterns are orthogonal — this is one of the limitations of this network.

15.1.3 Convergence

Hopfield demonstrated that the network is associated with an Energy function, given in its Lyapunov form [4]:

$$H(w) = -\frac{1}{2}\sum_{i,j} w_{ij} S_i S_j \tag{15.2}$$

and that the local minima of this function are precisely the locations where the patterns reside.

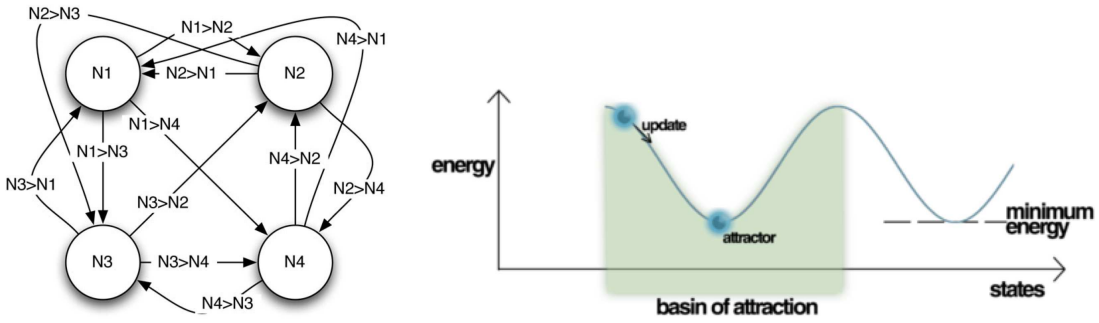


Figura 15.3: Hopfield network where all outputs are the inputs of the other neurons. The state of the network can be captured with a cost/energy function that seeks to minimize itself upon any perturbation (the input), with the learned patterns residing in the valleys.

Energy functions always decrease or remain constant as the system evolves. Since the weights are symmetric and $w_{ii} = 0$, we can sum only the upper part of the matrix and multiply by 2.

Likewise, let S_i' be the new value of $S_i \rightarrow S_i' = sgn(\sum_{j\neq i} w_{ij} S_j)$:

- If $S_i' = S_i$ then the energy does not change.
- If $S_i' = -S_i$ then

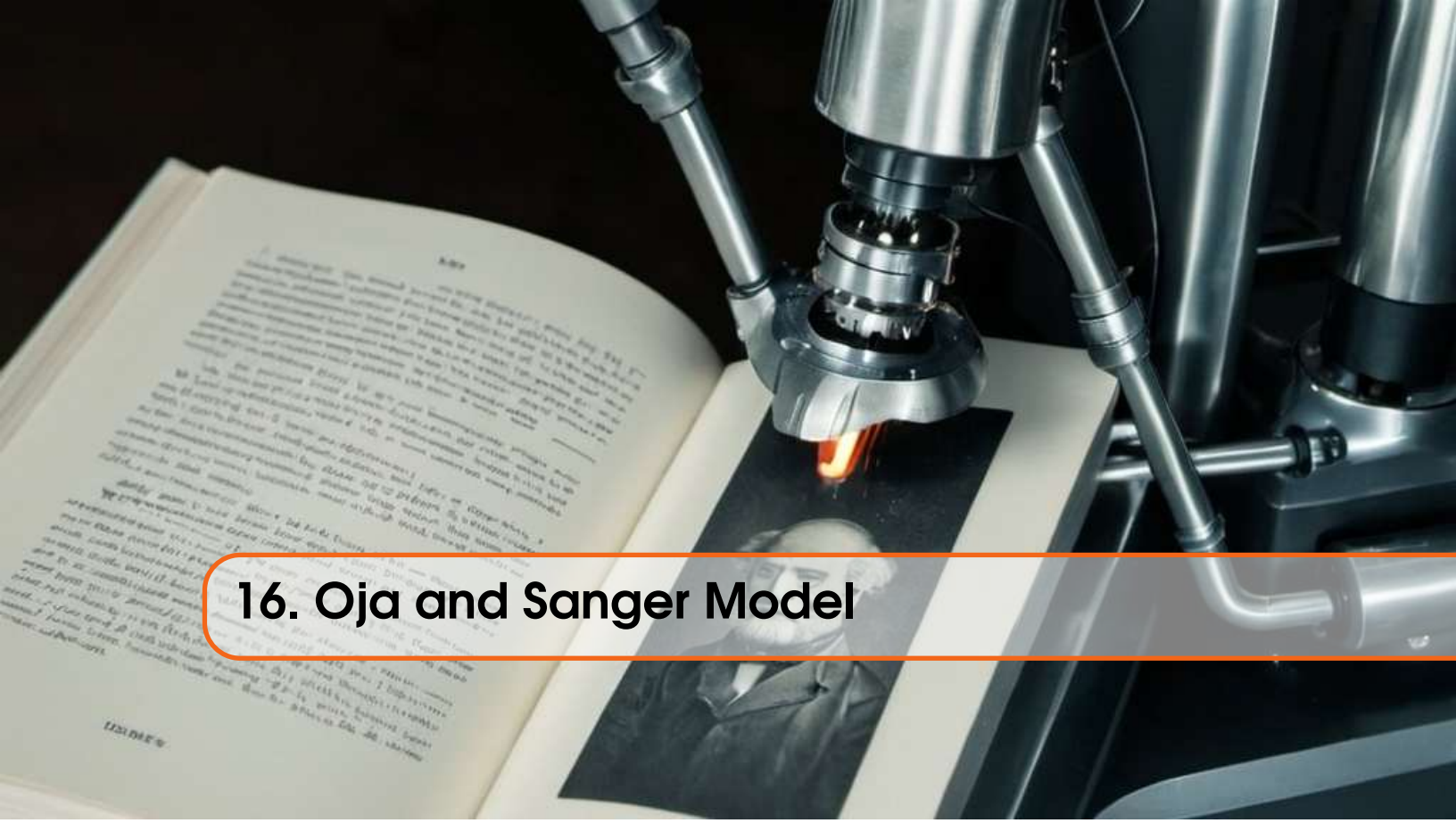
$$
\begin{aligned}
\Delta H &= H' - H \\
&= -\sum_{j>i} w_{ij} S'_i S_j + \sum_{j>i} w_{ij} S_i S_j \\
&= 2S_i \sum_{j>i} w_{ij} S_j \\
&= 2S_i \underbrace{\sum_{j} w_i j S_j}_{S'_i} < 0
\end{aligned}
$$

because we are considering S'_i with the opposite sign to S_i, and therefore the energy decreases every time a S_i changes.

15.1.4 Limitations

- The maximum number of patterns that can be stored is equal to 15 % of the number of neurons in the network. That is, $p \leq 0.15 * N$, where N is the dimension of the patterns.
- The patterns must be more or less orthogonal (in a vector sense), or the Hamming distance can be used as an evaluator of the similarity/orthogonality of the patterns.

The second limitation is the existence of **Spurious States**: since the patterns are attractors, the energy function H can have other local minima that are not the stored patterns. These minima are precisely stable spurious states that do not correspond in particular to any of the stored patterns [3].



16. Oja and Sanger Model

16.1 The Search for the Power of Synthesis



Figura 16.1: Erkki Oja, pleased to be at Aalto.

Is intelligence the capacity to synthesize, to compress [66] information? The transformation of data by the principal component analysis method in Chapter 13 can be viewed from an information theory perspective. From that angle, the intrinsic variability of the data can represent the medium through which information is transmitted. More variation, more information, more importance — and from that perspective, understanding the contribution of each datum in relation to how much variability it provides.

Toward the end of the 20th century, several researchers — among them Erkki Oja and Terence Sanger — found that certain simple neural network schemes, with biomimetic weight update rules similar to the Hebbian update rule, converged to a representation similar to that obtained in principal component decomposition. That is, they could find a new representation of the data where the axes were determined according to how much inherent variability existed along them.

Could this be a way of encoding information that is appropriate for capturing its essence, generating an abstraction, and from there a mechanism of intelligence?

16.2 Oja Network

Returning to the simple linear Perceptron equation:

$$O^{\mu} = \sum_{i=1}^{n} x_i^{\mu} w_i \tag{16.1}$$

The update of the synaptic weights follows from

$$\Delta w = \eta(\zeta^{\mu} - O^{\mu})x^{\mu} \tag{16.2}$$

with η as the learning rate, ζ the desired output, O the actual output obtained, and the iterative weight update rule given by

$$w^{n+1} = w^{n} + \Delta w. \tag{16.3}$$

This update rule is supervised. Is there a way to make it unsupervised with some optimality criterion? The answer is yes, and that is precisely what Oja proposes.

Teorema 16.2.1 — Hebbian Learning Rule for Unsupervised Learning. Given m input data points $x^1, ..., x^m$, with $x^{\mu} \in \mathbb{R}^N \forall \mu$, a simple linear Perceptron computes the forward output as:

$$O^{\mu} = \sum_{i=1}^{N} w_i x_i^{\mu} \tag{16.4}$$

where $w \in \mathbb{R}^N \; \forall \; \mu$. The unsupervised update of the synaptic weights can be performed as

$$\Delta w_j = \eta O^{\mu} x_j^{\mu} \tag{16.5}$$

$$w_j^{n+1} = w_j^{n} + \Delta w_j. \tag{16.6}$$

Oja [67] demonstrated that if a linear Perceptron with this update function converges, then the resulting weight vector $w(n_{final})$ is a projection onto the direction of maximum variation of the data — that is, onto the axis of the **first principal component**.

However, this Hebbian update rule diverges. For this reason, Oja proposes an update that keeps the weights in check by normalizing each update relative to the norm of the previous step:

$$w_j^{n+1} = \frac{w_j^{n} + \eta O^{n} x^{n}}{||w^{n}||} \tag{16.7}$$

bearing in mind that pattern μ is presented at step n, and with

$$||w^{n}|| = \sum_{j=1}^{N} (w_j^{n} + \eta O^{\mu} x_j^{\mu})^2)^{\frac{1}{2}}. \tag{16.8}$$

Oja estimates this update via a Taylor polynomial expansion in η

$$w_j^{n+1} = \frac{w_j^{n}}{||\mathbf{w}||} + \eta \left(\frac{y(\mathbf{x^n})\mathbf{x_j^n}}{||\mathbf{w}||} - \frac{w_j \sum_{j=1}^{N} y(\mathbf{x^n})\mathbf{x_j^n}\mathbf{w_j}}{||\mathbf{w}||^{\mathbf{3}}} \right)$$

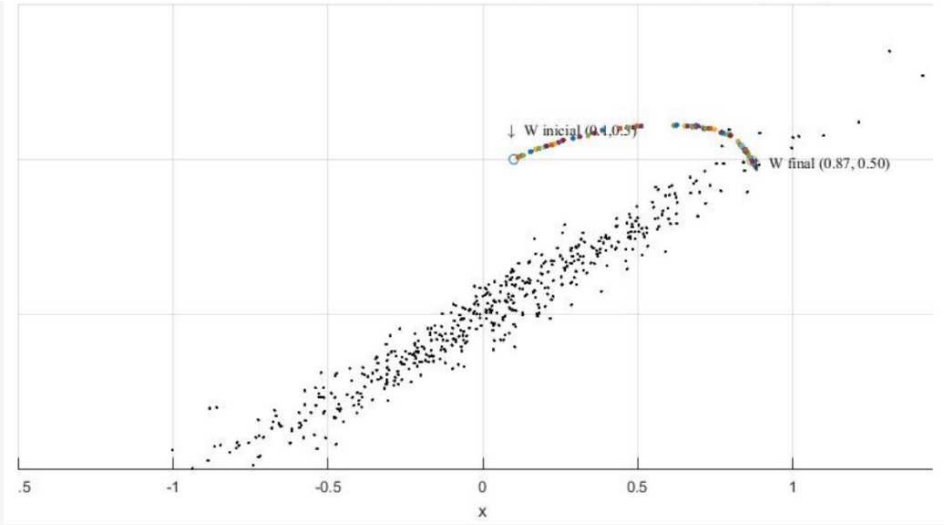


Figura 16.2: Simulation of a linear neuron with 2 inputs, updating w according to Oja's rule. The weight can be seen shifting toward the axis corresponding to the greatest variability in the data.

$$\frac{w_j^n}{||\mathbf{w}||} + \eta \left(\frac{y(\mathbf{x^n})\mathbf{x_j}}{||\mathbf{w}||} - \frac{w_j^n y(\mathbf{x^n}) \sum_{\mathbf{j=1}}^{\mathbf{N}} \mathbf{x_j^n} \mathbf{w_j^n}}{||\mathbf{w}||^{\mathbf{3}}} \right)$$

converging to:

$$\Delta w_j^{n+1} = \eta (O\, x_j^n - O^2\, w_j^n) \tag{16.9}$$

With this rule, the weight values remain bounded and do not diverge, and in fact converge after several iterations to the eigenvector corresponding to the largest eigenvalue of the correlation matrix of the input data — that is, to the first principal component of the PCA transformation.

Oja also demonstrated that to ensure convergence, $\eta = \frac{1}{1.2\lambda_1}$ is required, where λ_1 is the first principal eigenvalue. Since this value is not known, it can be approximated starting with $\eta_0 \leq 0.5$, and then, to facilitate convergence, this value can be gradually reduced by dividing it by the iteration step of the algorithm (though it is also not a good idea to reduce it drastically; the sum of all the η values used across all steps must diverge).

16.2.1 Implementation

The architecture is a simple Perceptron with a single output layer. The weight update follows Oja's own rule and the weights are initialized with uniform values between 0 and 1. Suitable hyperparameters for this network are $\eta(0) = 0.5$ and

Algoritmo 12 Oja Update Rule

Input: N data points with mean 0, η and w.
while epoch in epochs **do**
    while i iterating from 1 to N (the dimensions) **do**
        $s = \tilde{x} \cdot \tilde{w}$
        $w_i += \eta . s . (x_i - s . w)$
    end while
end while
w Result

Thus, this algorithm finds the weights that represent the first principal component based on a biomimetic update rule.

16.3 Sanger Network

Terence Sanger proposed an extension to Oja's rule, forming the Sanger Hebbian Network [68], which through Hebbian learning converges the weights to the full matrix of eigenvectors of the covariance matrix of the data, thereby enabling the recovery of the complete principal component decomposition. This algorithm is called the *Generalized Hebbian Algorithm*:

Teorema 16.3.1 — GHA, Generalized Hebbian Algorithm. Assume a vector output of the neural network with

$$O_j^{\mu} = \sum_{i=1}^{N} w_{ji} x_i^{\mu} . \tag{16.10}$$

Sanger's generalized Hebbian rule extends Oja's rule with

$$w_{il}^{n+1} = w_{il}^{n} + \eta O_i^n (x_l - \sum_{j=1}^{i} O_j^n w_{jl}^n) \tag{16.11}$$

with $1 \leq i \leq k$, $1 \leq k \leq N$, $1 \leq l \leq N$, $\eta \geq 0$.

What the rule does is keep the weight values in check by subtracting all the components found in previous steps, across all coordinates. This update rule can be applied directly in Algorithm 12 and allows finding the complete matrix of principal eigenvectors, which enables finding all the loadings for all axes of the PCA transformation [68, 65].

VI

Deep Learning

17 Deep Brain 137
17.1 Neural Biomimesis
17.2 The Upgraded Neuron

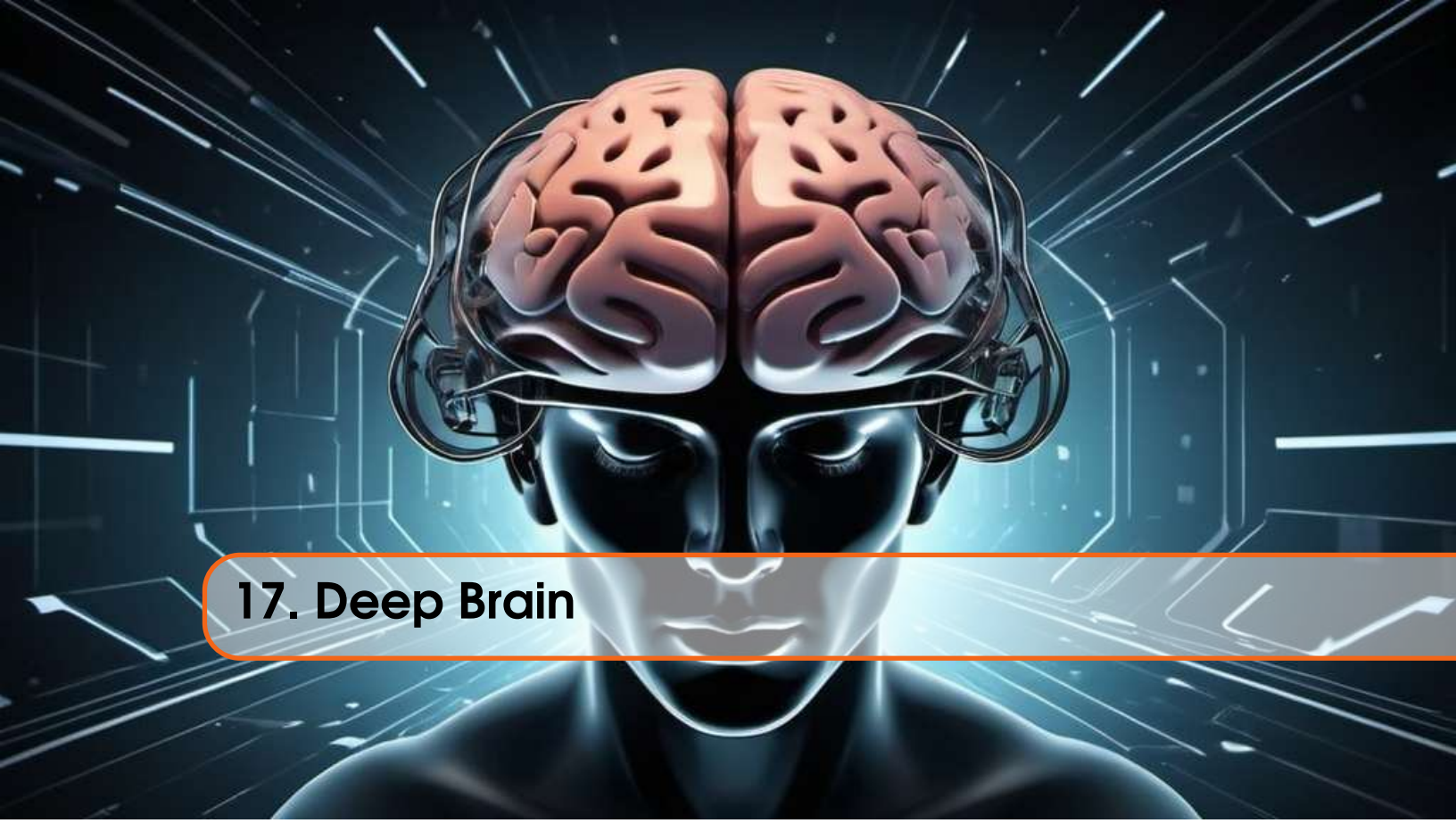
18 Deep Learning 145
18.1 A Guide for the Uncle and Aunt Who Ask
18.2 Depth
18.3 Representation Learning
18.4 Datasets
18.5 GPUs

19 Autoencoders 153
19.1 Autoassociators
19.2 Generative Autoencoder
19.3 VAE - Variational Autoencoder
19.4 Appendix

20 Convolutional Neural Networks 173
20.1 The Convolution Operation
20.2 Convolution on Images
20.3 Neocognitron
20.4 The Convolutional Layer
20.5 The Pooling Layer
20.6 Full Layer and Softmax
20.7 Backpropagation
20.8 Why Do They Work?
20.9 Intelligibility
20.10 Reusable Architectures
20.11 Beyond Images
20.12 I Want More

21 Generative Adversarial Networks 187
21.1 Generating Kittens
21.2 The MinMax Game
21.3 Mode Collapse
21.4 The Great Simulator

22 Transformers 193
22.1 Recurrent Networks
22.2 Transformer Architecture
22.3 Attention is All You Need
22.4 Embedding
22.5 Training



17. Deep Brain

17.1 Neural Biomimesis

As mentioned in Chapter 6 on genetic algorithms, nature implements a powerful optimization algorithm applied to life on Earth, separating what is useful for surviving and perpetuating itself from what is not, for each particular era. At some point, that algorithm converged elitistically to preserve and promote organisms that were developing the capacity to transmit and process information in increasingly sophisticated ways through nervous systems, thereby driving the development of what we call intelligence.

Vertebrate primate organisms, such as humans, have a nervous system divided into two parts. The CNS, Central Nervous System, and the PNS, Peripheral Nervous System. The CNS is located inside the skull while the peripheral system extends through the spinal column and covers the entire body, culminating in nerve endings responsible for executing action through the muscles as well as sensing what is happening around us, forming our senses — with the addition of identifying postural aspects through the identification of the elongation and relative position of our body via proprioception.

The resulting system is thus divided into two autonomic subsystems that *keep life going*: the sympathetic and parasympathetic. The first autonomizes the *fight or flight* response, while the second regulates homeostasis and metabolism, summarized as *rest and digest*.

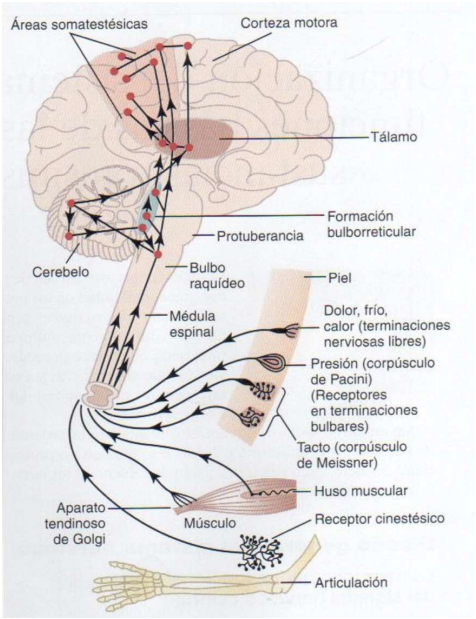


Figura 17.1: The Central Nervous System and the Peripheral Nervous System.

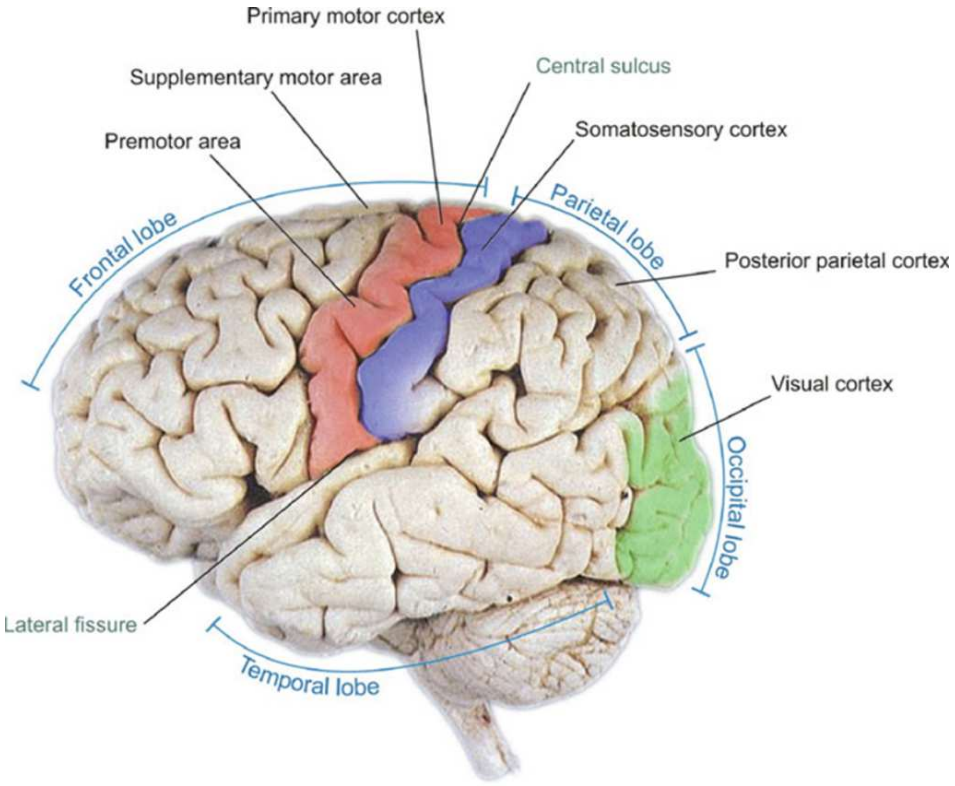


Figura 17.2: Different anatomical and functional regions of the brain.

17.1.1 Intelligent Fat

The brain is physically a collection of tissue with a high content of fatty acids, composed of grey matter in the cortex and white matter corresponding to neuronal axons. It is embedded in cerebrospinal fluid, contained within the skull. This fluid is generated in three ventricles located at the center of the brain and is reabsorbed in the meninges — the outer layer of the brain, in contact with the cortex — and in the dura mater, just at the inner surface of the skull.

The brain is in turn divided into different areas. On the outer layer is the cortex, the newest area of the brain, comparatively more developed in humans. Here lies the basis of abstract behavior. Figure 17.2 shows the different divisions of the cortex, many of them associated with the functional traits of each area.

Different fissures traverse the anatomy of the brain, most notably the central fissure that divides the brain into two hemispheres. The corpus callosum is literally a bundle of fibers connecting both parts, and its thickness is a distinctive feature of humans, not as present or pronounced even in other primates.

From the division into hemispheres and into different functional sections arises the principle of the rigid connectionist brain, where each part performs a specific functional task. Damage to a part implies the loss or impairment of its functional role. This idea dominated neuroscience for many years, heavily influenced by the fact that the field's own study and discoveries arose from concrete cases where a person had a part of the brain compromised or damaged — by disease or trauma — affecting some behavioral aspect. That correlation suggested that this zone was responsible for the compromised

aspect. Clear examples of this are the famous Phineas Gage and Louis Victor Leborgne [69].

17.1.2 A Neuro-Computer

We can draw a metaphorical analogy between the brain and a modern computer [70] and describe its different parts from that perspective.

Firmware

Internal structures in the brain — more primitive and histologically distinct — are formed by the cerebellum and the pons (bridge of Varoli). These are the regions where the autonomic systems are primarily regulated. This is the brain's **firmware**.

GPUs, Graphical Processor Units

The occipital cortex, located just above the nape of the neck, is responsible for processing visual information, much like a graphics card in a computer. It is connected to the eyes by the optic nerve, which along the way passes through subcortical areas such as the visual thalamus, where mechanisms of the parasympathetic system act, triggering for example defensive reactions (this is why, without thinking, we blink or make an evasive gesture when someone claps their hands near our face — a reflex).

The occipital area is highly relevant from a biomimetic standpoint because its study gave rise to many of the ideas that currently form the basis of various mechanisms applied in neural networks, typically convolutional networks detailed in Chapter 20.

NLP, Natural Language Processing

One of the greatest impulses that modern neuroscience received was with Paul Broca's discovery of the linguistic cortex. This arose from observations of how lesions in that area particularly affected normal functioning in language processing and generation [69]. Thus, *Broca's area* as well as *Wernicke's area* are related to the human capacity for processing and generating natural language. They are precisely for this reason the centers where human NLP is localized[1].

CPU, Central Processor Unit

The prefrontal cortex is also an evolutionarily characteristic and comparatively highly developed area in humans, and concentrates executive and social control — the center of decision-making. It is where the **self** is deposited. Conscious control of thought and action in accordance with an internal conscious goal takes place here.

The Motherboard

The areas responsible for sensing touch in our extremities — the somatosensory area — and the regions responsible for coordinating our movements — the premotor and motor areas — are located side by side in the temporal and central zones.

From the central area, within the fissure itself, the different zones in the cortex of these three areas map the different parts of the body in relation to what they sense and what they control. If we represent, on the human body, the percentage of these zones dedicated to mapping each part of the body, we can visualize the famous *homunculus* (Figure 17.3), which shows how much of each of these areas is devoted to each part. For example, the hands are large because a greater proportion of regions in these areas is responsible for perceiving and controlling the hands.

[1] Although computers currently have no physical NLP component, in the coming years we will surely see NLP components embedded directly in hardware, with some built-in internal language engine — the natural extension of TPUs.

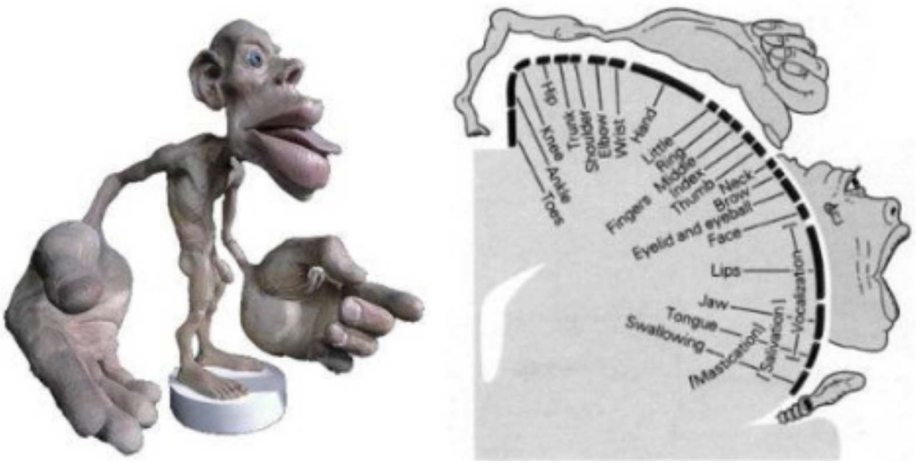


Figura 17.3: Homunculus

RAM

The hippocampus, central in the brain, is a key area in sleep and in the storage and generation of memory. It is where neurogenesis [71] takes place — the birth and specialization of new neurons.

Sensory Fusion — Temporo-Parietal Junction

The integration of sensory information is coordinated at the temporo-parietal junction, the TPJ, which coordinates information received through different modalities: proprioceptive, visual, vestibular — and combines them to give a single, integrated picture of reality. This is equivalent to the sensor fusion techniques [72] commonly sought in robotics.

17.2 The Upgraded Neuron

Returning to the presentation of the neuron model from Chapters 8 and 9, this section offers more details about its complexity and structure, showing that it is not as simple as presented there.

17.2.1 How Does a Neuron Fire?

The firing of a neuron is far more complex than what has been discussed so far. One of the most well-established models that attempts to capture this complexity was that postulated by Hodgkin and Huxley [73]. They studied how neuronal firing behaved in the giant squid axon and established a dynamical model that explained the behavior as an action potential of unstable equilibrium, actively coordinated by ionic currents.

$$I = -(C_m \frac{dV}{dt}) + \frac{Ie}{A}$$
$$I = C_m \frac{dV_m}{dt} + g_K(V_m - V_K) + g_{Na}(V_m - V_{Na}) + g_l(V_m - V_l)$$

What this differential equation establishes is that the current flowing across the cell membrane — which ultimately determines the appearance of the sudden action potential — is proportional to the rate of change of voltage over time, and to the voltage difference between the current voltage and

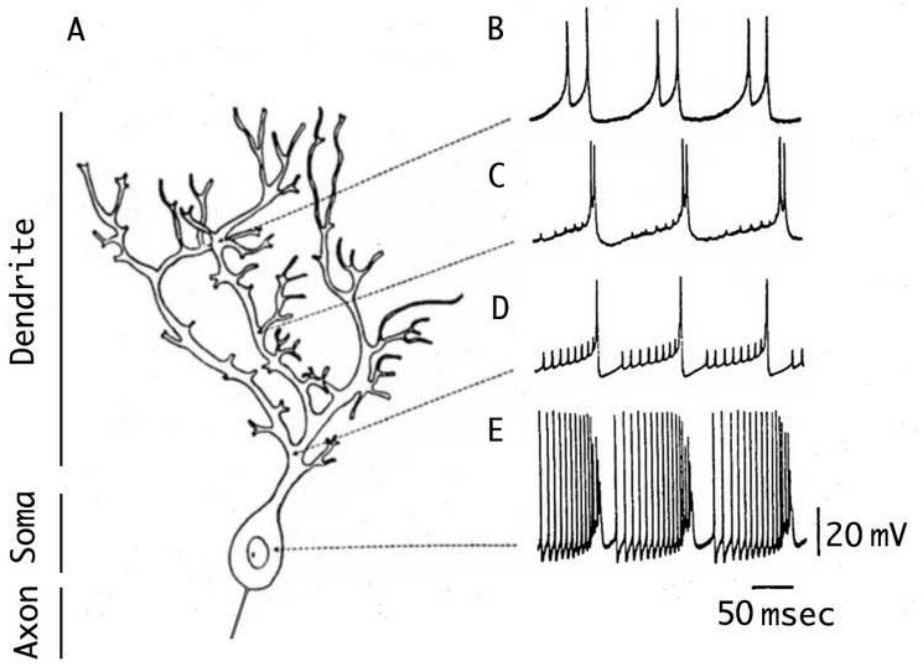


Figura 17.4: A gunslinger neuron, firing away.

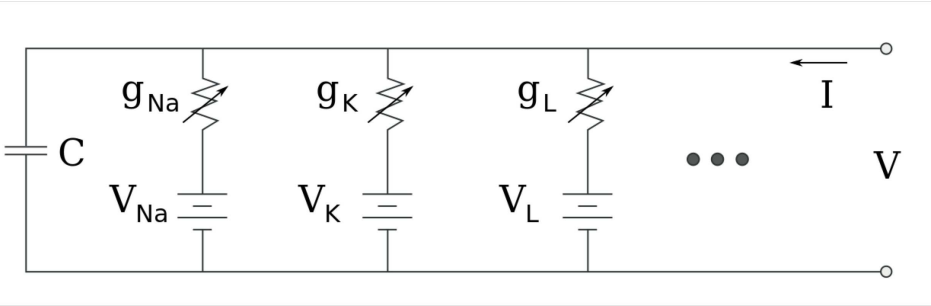
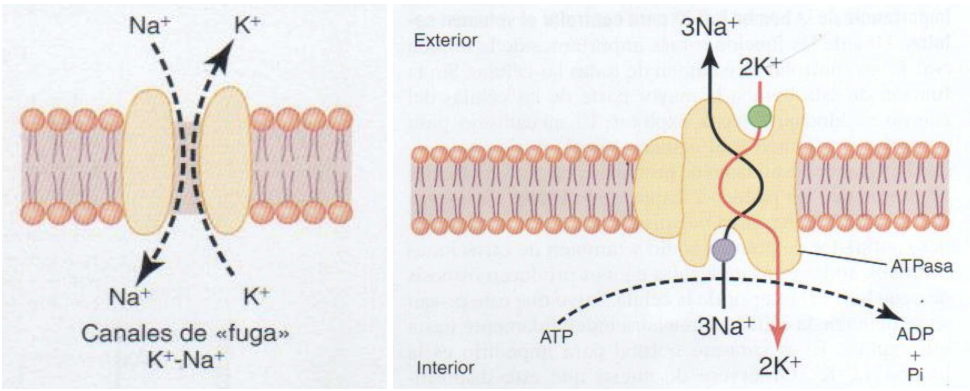


Figura 17.5: Kirchhoff diagram representing the Hodgkin-Huxley model.



(a) Leakage channels (b) Sodium-potassium pump

Figura 17.6: Membrane proteins that passively and actively maintain the membrane potential.

the reference voltage of three different references, each mediated by conductances, the reciprocal of electrical resistance. Experimentally, H & H found that each of these was mediated by concentrations of potassium ions (K), sodium ions (Na), and leakage ions, also called *leakage*. They discovered that the dynamics of action potential firing were regulated by these ions — electrically charged atoms due to an electron imbalance — that moved transversely to the direction of propagation of the potential, and that their model explained the all-or-nothing behavior, generated by the very sudden dynamics of the ion exchange.

17.2.2 A Nanotechnological Computer

The neuron passively and actively maintains a potential difference between the interior and exterior of the cell of around -90 mV. This difference arises precisely from the selective filtering performed by the plasma membrane, through which the different ions cannot pass directly. There are channels that are membrane proteins (Fig. 17.6) acting as mediators to maintain this potential. Leakage channels allow sodium+ to enter the cell and potassium+ to exit to the exterior, acting passively simply by electrochemical affinity. On the other hand, the sodium-potassium pump positivizes the interior of the cell by pumping out 3 sodium+ and bringing in only 2 potassium+, but does so actively by consuming ATP, the biological energy *currency*. These membrane proteins have the important characteristic that their electrochemical sensitivity to ions is voltage-dependent. This is what allows a small change in the electrical potential to alter the conformation of these proteins, changing the conductance, which in turn generates a rapid flow of ions — inward and outward through the neuron — which is what propagates the action potential, the spike. This is visualized in Figure 17.7, where the timeline runs from left to right, showing the successive behavior of the two channels. Initially, a small change in the *resting* equilibrium action potential (-90 mV) is sufficient to open a gate in the channel, allowing sodium cations to flood in and positivize the interior of the neuron, driving the potential toward zero. This rapid flow causes *depolarization*, where an overstimulation occurs that drives the membrane potential to $+40$ mV, corresponding to the peak of the spike. In parallel, in the leakage channels, the change in voltage alters the channel conformation, opening a gate for potassium cations to begin exiting and negativizing the interior — but they do so with a delay, being slower. This delayed action is what allows the action potential to begin returning to its equilibrium value of -90 mV, ultimately manifested by the reclosure of the gates in the channels, returning the entire situation to the equilibrium state where *repolarization*

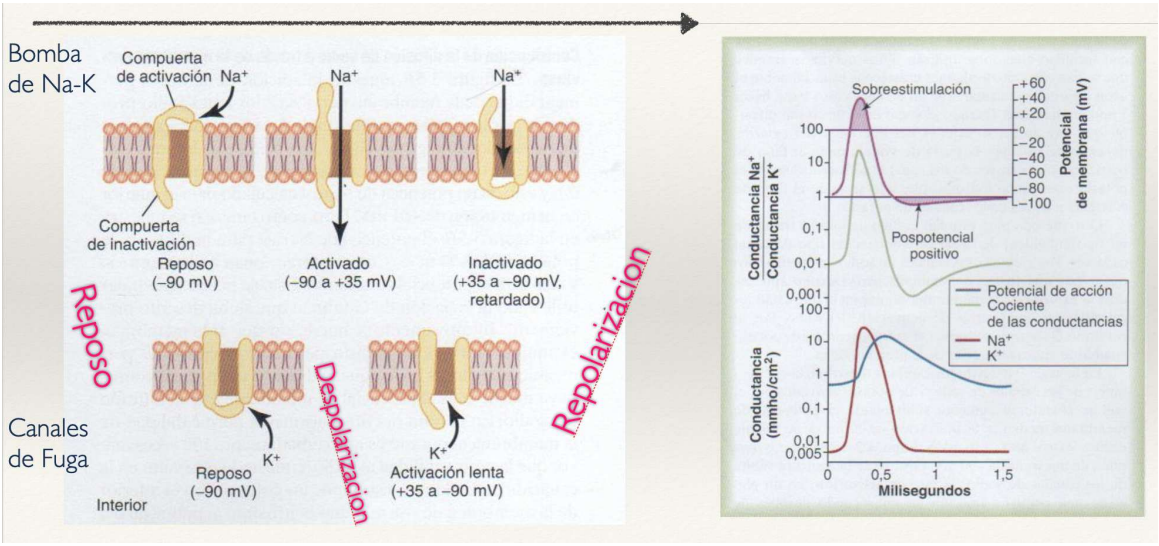
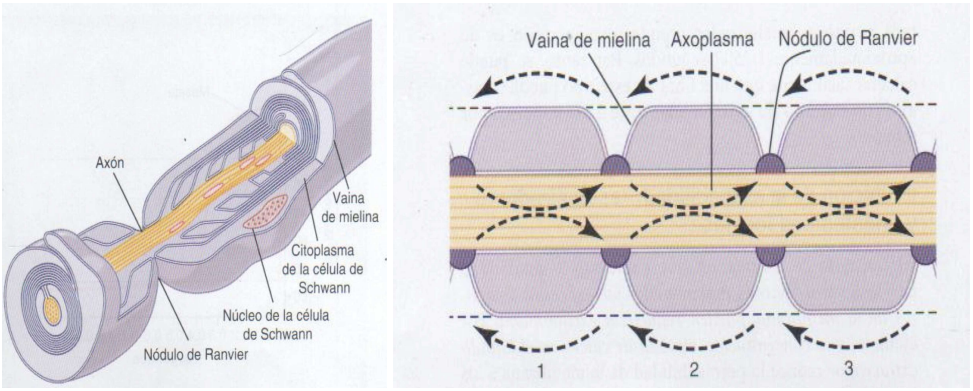


Figura 17.7: Sequence of ion exchange through membrane proteins that generates the spike of neuronal firing.



(a) Schwann cells (b) Nodes of Ranvier

Figura 17.8: Saltatory conduction mediated by the insulation of myelin sheaths and by the *repeaters* at the Nodes of Ranvier.

occurs.

17.2.3 Saltatory Conduction

The size of a cell, considering the soma and dendrites, is on the order of $20\mu m$. However, axons can be long — very, very long — spanning several meters. For example, they leave the brain where the neuron lives, travel down the spinal cord, connect to a second neuron, and reach a nerve ending. It is a great distance over which the neuronal firing spike must travel. However, the action potential propagating along the plasma membrane has a low propagation velocity of 0.25 $\frac{m}{s}$ and also suffers significant loss. For the axons of neurons to propagate over the large distances of the human body, a mechanism is therefore required to allow the spikes to be transmitted. This is achieved through a

group of cells — Schwann cells, or myelin sheaths — that wrap around the axon and promote its insulation, in the same way that in engineering we develop plastic insulators for cables. Additionally, these cells arrange themselves along the axon in such a way as to leave room for some fairly evenly spaced structures called Nodes of Ranvier, which act as repeaters to regenerate the spike with greater amplitude, thereby allowing it to propagate over greater distances.

17.2.4 Synapse

Neurons seek to connect with other neurons at all times. A neuron that connects with no other tends to die. The point of that connection is the neuronal synapse (Fig. 17.9), where a presynaptic neuron connects to a postsynaptic neuron. The action potential travels along the axon to the presynaptic terminal, where there are transmitter vesicles containing a neurotransmitter (a protein). When the potential reaches the presynaptic cleft, membrane proteins disassemble the vesicles, releasing the neurotransmitter into the synaptic cleft (200 to 300 angstroms wide). There are more than 40 different types of neurotransmitters, which, floating freely through the synaptic cleft, bind by electrochemical affinity to ligand-gated channels located on the membrane of the postsynaptic neuron. Upon binding with the neurotransmitter, these channels open, generate an ion exchange, and fire a new action potential that now travels through the postsynaptic neuron, propagating it — or they force a closure of the ionic channels, producing an inhibitory effect. Neurotransmitters, with their three-dimensional conformation, make up a large part of the pharmacological arsenal of psychiatry, which focuses on producing them or producing similar drugs that can also operate at the synapse through electrochemical affinity — like a game of Tetris — with the ligand-gated channels. Thus at the synapse, the Hebbian learning mechanism manifests mechanistically at many levels. At its core, the coordinated firing of the presynaptic and postsynaptic neuron reinforces the synapse, through the recruitment of more vesicles, greater mitochondrial activity (and hence greater energy production to sustain the action potential), and also more ligand-gated channels in the postsynaptic neuron. But it is not limited to these direct changes alone — it also extends to what are called second messenger activations: enzymatic changes, activation and transcription of new genes, or even structural changes. Attempting to capture and reduce all of this marvelous complexity into a single number is, at the very least, a bold assumption.

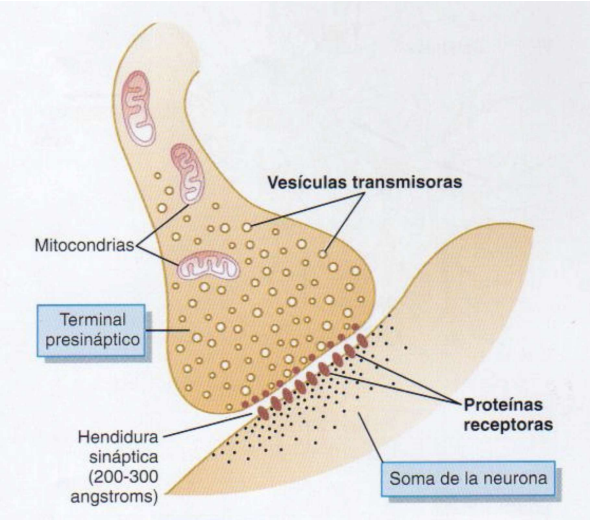
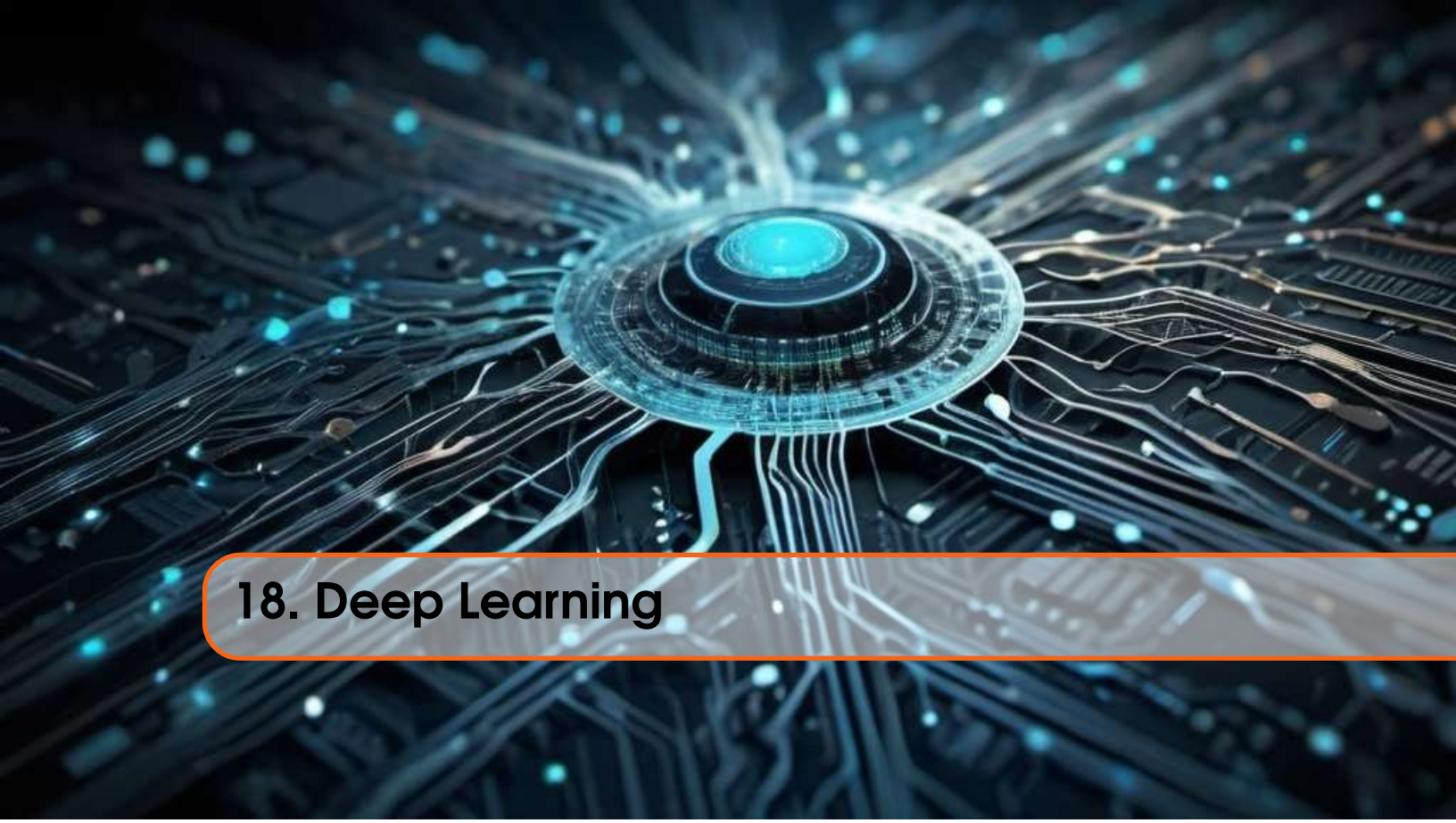


Figura 17.9: The neuronal synapse — where things can go wrong in the end.



18. Deep Learning

Why Deep? Why Profound? The real reason is marketing. As has happened, happens, and will happen, technology is a cornerstone of today's world and needs commercialization strategies. Often, old technologies receive new names — *buzzwords* or neologisms — to identify them, distinguish them from others, structure a message around that word, make them fashionable, and ultimately sell more [74]. Neural networks, a very academic discipline, mired in a certain ostracism at the start of the 21st century[1], experienced exactly that and received a new, highly marketable name that combined a real characteristic of artificial neural networks — the hierarchical agglomerative scheme — with a strong meaning riding on the back of a commercial strategy.

18.1 A Guide for the Uncle and Aunt Who Ask

What is Deep Learning? It is a development in computer science, but at the same time highly interdisciplinary, with biomimetic foundations, that uses a hierarchical agglomerative structure.

The following are myths — literal excerpts taken from various media, and even books — of mistaken concepts relating to what Artificial Intelligence, and specifically Deep Learning, actually is:

AI algorithms are not programmed: a myth probably influenced by Arthur Samuel's 1959 phrase, "[Machine learning is the] field of study that gives computers the ability to learn without being explicitly programmed" [75]. That is, the step-by-step procedure for solving the task is not programmed; instead, a generic parameterized algorithm is implemented, which in turn incorporates a procedure for adjusting those parameters. All of that, of course, must be programmed.

[1] The 2019 Turing Award winners, Yann LeCun, Geoffrey Hinton, and Yoshua Bengio, mention that they made efforts to publish their neural network work disguised in Computer Vision journals — the star of that era — precisely in order to access the higher impact factors associated with the scientific journals publishing those topics.

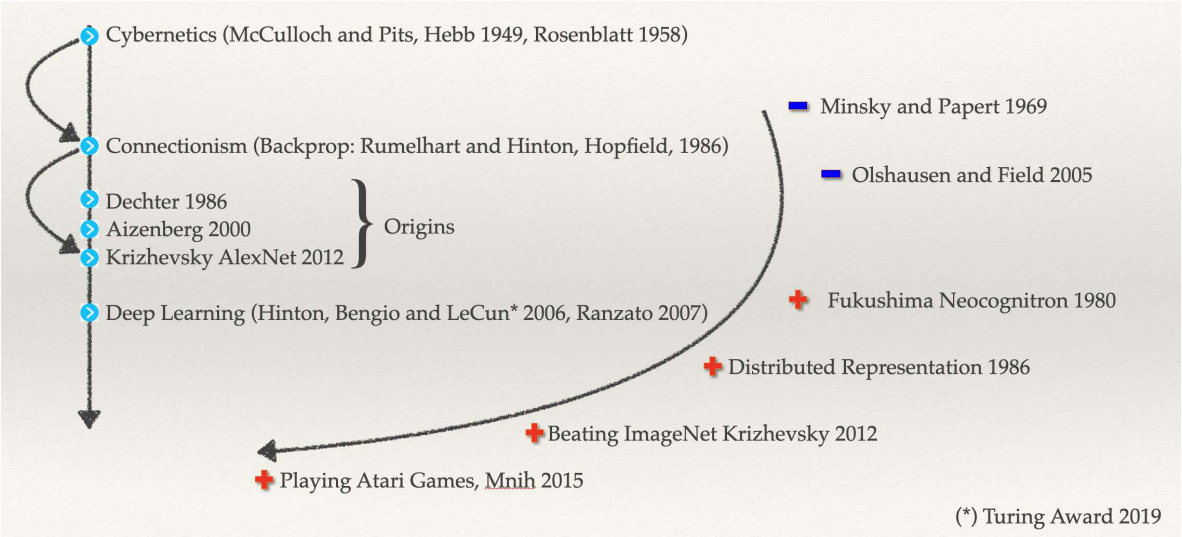


Figura 18.1: The works that determined the different summers of artificial intelligence (+) and those that cemented the two previous AI winters (-). The first era was cybernetics, the second was connectionist, and we can affirm that this third is the era of deep networks.

Robots are coming to replace us: a myth for two reasons. The first is that current advances in AI do not automatically imply similar advances in robotics; confronting the natural world, the physical world, is extremely complex. The second is that the idea of replacement is a more philosophical-sociological concept, not a technological concept per se (this vision that a particular technology will replace human beings has occurred in the past and will continue to occur).

Technology has already started to replace us, and we had better make the most of it rather than dwell on the wound it inflicts: in line with the above, this is the same replacement myth, but adds a sense of guilt placed on human beings, which makes no sense whatsoever.

Algorithms also have the advantage that they do not get tired and can interact with context at supersonic speeds compared to people: this myth, in addition to the inaccuracies it carries, is a myth because it pointlessly attempts to compare a person — a human being — with an algorithm. This is a nonsense with a high degree of triviality and leads to a misguided debate.

Is this technology the advent of intelligent machines, or of robots that control everything, or worse, robots that trigger the robot apocalypse? No, we are still very far from that situation [76]. Generating intelligence like that which we identify in other beings or in human beings is exceedingly difficult [77].

Concretely, it is a revolutionary technology that today stands at the very core and frontier of the digital world, and which is characterized by having achieved advances in the following areas [78]:

- Computer Vision:
- Natural Language Processing:
- Speech Recognition:

The tremendous advances in this area are dominating the current scene, and everything we today call AI focuses exclusively on applications in these areas, for example:

- Magnetic control of the plasma in a tokamak fusion reactor using deep reinforcement learning [79].
- Discovery of the remains of an ancient sunken ship that could not previously be found [80].
- Development of new, more efficient forms of matrix multiplication [81].
- AlphaFold: the deep learning network that allows predicting the three-dimensional structure — and therefore the functionality — of a protein based on its amino acid sequence [82].

Focusing in, then, the current field of Deep Learning is characterized by four pillars, four clear components that truly define it. These four pillars of Deep Learning are:

- Deep, Depth: Multiple Levels of Composition
- Feature Learning: Representation Learning
- Massive and standardized datasets
- GPUs, Graphic Processing Units: specialized hardware for the parallel processing of matrix multiplication.

18.2 Depth

18.2.1 The Problems of Shallow Learning

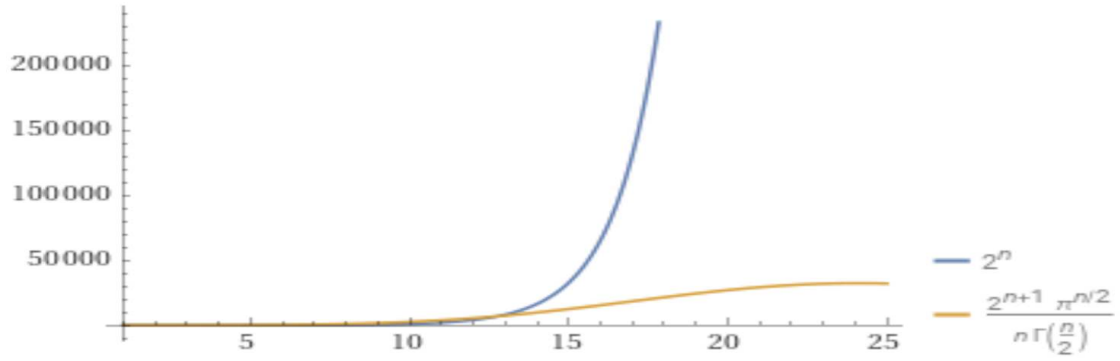
During the 1990s and into the 2000s, the focus of artificial intelligence was centered on Computer Vision and on data-driven algorithms heavily grounded in statistics. This is the traditional domain of **Machine Learning**.

The stars of those years were complex algorithms that attempted to understand and decode what image processing and computer vision were about. Very sophisticated schemes were developed that were highly successful for small, controlled problems but failed when the problem scaled up or the noise level increased [83]. The same occurred initially in solving the game of chess, or speech processing. The goal was to understand the particularities of each case, attempting to algorithmically encode how human beings believe we understand these problems. However, this approach was only valid for bounded demonstrations and failed when scaled or faced with more realistic problems.

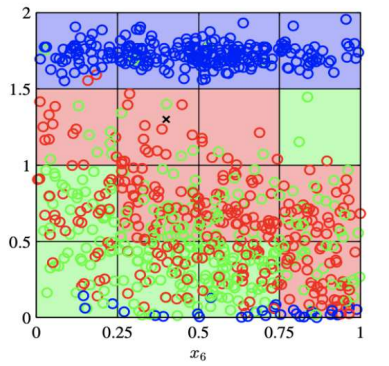
One of the reasons why *these methods had limited success* is related to the problem of high dimensionality. These methods are typically based on feature compositions (see 3.1) and these features have very high dimensions. And this **hacks** our intuition.

The Curse of High Dimensionality

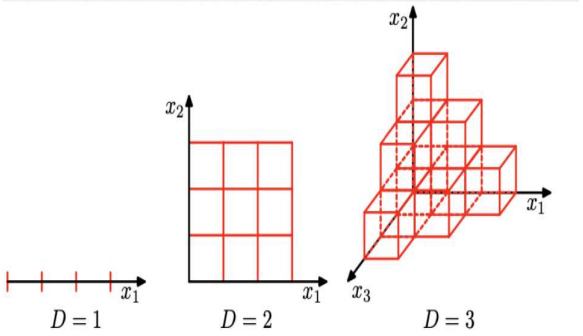
Suppose we have a circle inscribed in a square. The diameter of the circle coincides with the side of the square. If we measure the area covered by each, **intuitively**, we will perceive them as practically equal, with only the small corners outside the circle but covered by the square. If we extend this to a sphere inscribed in a cube, this perception holds. If we continue extending this to more dimensions, we find that the trend is actually quite different. This can be seen in Figure 18.2a, which plots the hypervolume covered by the hypercube — clearly exponential — and the hypervolume covered by the hypersphere — hypergeometric — both as a function of the number of dimensions. Not only does the hypervolume of the hypercube grow **exponentially** in absolute terms and relative to the hypersphere, but the hypersphere actually begins to **shrink!** around dimension 21. This is a affront to common sense, and shows that the inscribed hypersphere in high dimensions occupies an infinitesimal fraction of the available volume [84].



(a) Hypersphere



(b) K-NN in 2D



(c) K-NN in multiple dimensions

Figura 18.2: The curse of high dimensionality.

Assuming that local consistency holds throughout the available space may not be true in high dimensions, since the existing data are sparse, may be distributed across an entirely empty hyperspace, and generate little structure to hold on to. This is well illustrated in Figure 18.2b: this is the typical k-NN algorithm [85]. Each cell in the graph corresponds to a color given by the majority vote of the elements already placed within it. However, when this idea is extended to more dimensions (Fig 18.2c), hypercubes begin to be constructed that generate more and more available space, and at the extreme there may be only one element per cube or even none at all. This also clearly emphasizes that the amount of data needed to generate the structure that reveals the inherent patterns grows exponentially with the dimensionality of the feature, imposing an enormous constraint on the scalability of these methods.

18.2.2 The Fundamental Problem of Neural Networks

One of the most fundamental results in neural networks is given in Theorem 18.2.1. This theorem, proved by Cybenko in 1989 [86], establishes that:

Teorema 18.2.1 — UAT - Universal Approximation Theorem. An MLP with a single hidden layer containing a finite number of neurons can approximate any continuous function on a compact subset

of $\mathbb{R}$ given appropriate activation functions.

Let $C(X,\mathbb{R})$ be a set of continuous functions from $\mathbb{R}^n$ to $\mathbb{R}^m$. Let $\sigma \in C(\mathbb{R},\mathbb{R})$, with $(\sigma \circ x)_i = \sigma(x_i)$ representing the activation function.

Then, for each $n \in \mathbb{N}, m \in \mathbb{N}$, a compact set $K \subseteq \mathbb{R}^n, f \in C(K,\mathbb{R}^m), \varepsilon > 0$, there exists $k \in \mathbb{N}, A \in \mathbb{R}^{kxn}, b \in \mathbb{R}^k, C \in \mathbb{R}^{mxk}$ such that

$$\sup_{x \in K} ||f(x) - g(x)|| < \varepsilon \tag{18.1}$$

with $g(x) = C.(\sigma \circ (A.x + b))$

This result implies that a network with a single hidden layer is eventually sufficient to represent any type of computable function. It is a universal approximator. This has the advantage that it does not require applying any type of intuition about the particulars of the problem and sidesteps, to some extent, the issue of high-dimensional features. Simply, with sufficient training, it should be capable of finding the appropriate weight values and approximating anything.

But if a neural network with a single layer can approximate anything, what do we gain by adding many layers? The advantage of depth comes from the fact that the compositional, hierarchically agglomerative structure offers two benefits: translation invariance and locality at different scales. This is visualized very clearly with convolutional networks. Translation invariance means that regardless of where in the input an important feature exists in the data, the compositional and agglomerative nature of a network with many layers will allow that feature to be captured somewhere in the hierarchy. The second point, locality at different scales, allows indistinguishably perceiving events that occur at different scales and capturing them regardless of scale: they are scale-invariant. This is a **perfect fit** for many natural phenomena, particularly in images:



Many natural questions on images correspond to algorithms that are compositional

That is, many problems in nature present the need for invariance to displacement (in time or space) and invariance to scale. An example of the latter are coastline maps, where regardless of the scale, coastlines look very similar.

Additionally, there is a more practical perspective: deeper neural networks are in general easier to train effectively, more efficient at finding optima (easier to optimize), and generalize the problem better. These are very pragmatic reasons [87].

But this is not new. It was known that neural networks had to be suitable for solving many of the traditional problems of the natural world, but something did not quite close. Specifically, there existed what became known as **The Fundamental Problem of Neural Networks**: when the backpropagation algorithm is applied across many layers, the errors that propagate can become so small that they fall below the computational resolution, and this eventually causes the gradients to vanish or explode. That is, a numerical problem arises [88]. A series of works addressed this issue, among them the idea of Unsupervised Pretraining [89], which is where the name **Deep** was coined — jumping onto the hype train and completely rebranding the field.

However, all these advances had behind them a larger problem: the hardware needed to perform scalable and effective implementations of deep neural networks simply did not exist [90].

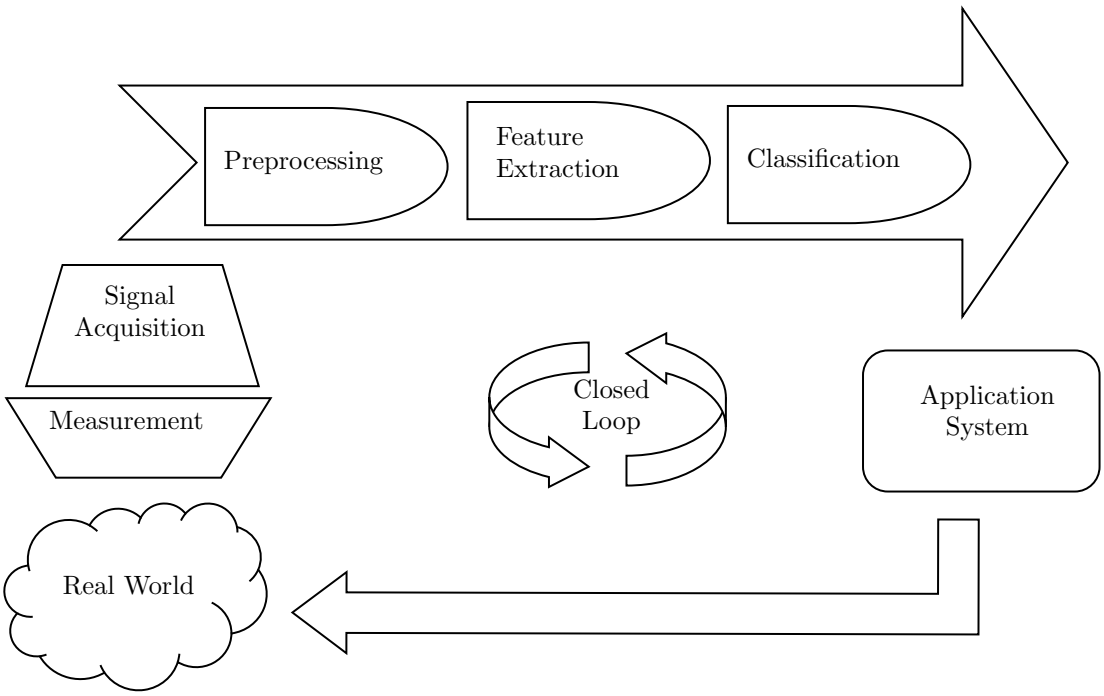


Figura 18.3: Architecture of a processing model based on machine learning

18.3 Representation Learning

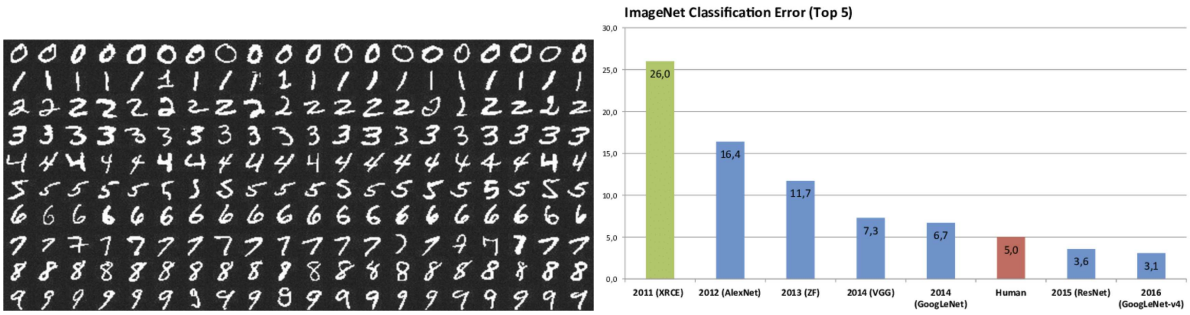
This second pillar of Deep Learning becomes visible when we consider the architectural scheme of Figure 18.3, which details how machine learning algorithms operate.

An important part of this process is feature engineering. This is the design of the vector that encodes and synthesizes the most important information needed to capture the pattern. Deep learning networks have the characteristic of delegating the identification of what that feature should look like directly to the learning process itself, forcing it to learn synaptic weights that are precisely what determine which feature to use and what it looks like. This means that deep learning networks receive as input directly the raw information — with some preprocessing but mostly in its original form — with the same variables and dimensions it naturally has. The network assumes, within the entire learning process, the task of somehow synthesizing the information guided by the objective encoded in the cost function. This scheme is clearly visualized in convolutional networks (Chapter 20).

18.4 Datasets

The third pillar of Deep Learning relates to the massive accumulation of data brought about by the digital transformation of all of society. This was initiated with personal computers, the dot-com booms, the rise of mobile phones, tablets, apps, and now AI. We have massive data that practically encompasses all human societies, and potentially every person in the world — along with a great many processes in the world — is a generator of data.

Additionally, the existence of standardized datasets helped and promoted collaboration in the field,



(a) MNIST Dataset (b) Success rates in the ImageNet challenge

Figura 18.4: The MNIST and ImageNet datasets were used for many years to verify and test the quality of classification algorithms.

and enabled working by comparing different methods against a common metric defined by the data itself.

18.5 GPUs

Current processors — CPUs, Central Processor Units — are based on many years of development and advances, governed by Moore's Law. This self-fulfilling prophecy held that every two years, the number of transistors in an integrated circuit would double, yielding incremental gains in processor performance and capacity. That growth came to an end because the physical limits of miniaturization were reached. In parallel, the very lucrative video game industry began to need massive parallel computing, for which GPUs, Graphic Processor Units, were developed with a different, simpler architecture but emphasizing massive parallelism (Figure 18.5). This enabled the graphical processing computations — which require parallel matrix multiplication to move and rotate polygons, as well as operations of projective geometry (more matrix multiplication) — to be performed.

In 2012, Hinton realized that the operations of neural networks could leverage the GPU hardware that was essentially designed for the same type of computational operation — parallel matrix multiplication — and decided, together with his doctoral student Alex Krizhevsky, to use a PlayStation to implement a convolutional neural network, AlexNet, and attempt to win the ImageNet challenge [91]. Their success was so resounding that the following year all participants in the challenge submitted proposals based on the same scheme.

Goodfellow describes it and the text is excellent on this point [92]:

> **The video game industry financed the hardware necessary for Deep Learning to become today the star and engine of the artificial intelligence revolution.**

This is because GPUs were optimized to perform matrix multiplication in parallel, and this is exactly what neural networks ultimately do. That is, today's neural networks won the hardware lottery (or perhaps simply waited long enough) [90].

The flip side of this is that one must not minimize the fact that this current revolution exists because hardware is being thrown at the problem. For example, to train GPT-3, with 175 billion parameters, OpenAI generated 10^{23} flops on 1024 GPUs for an entire month, at a cost of several million dollars.

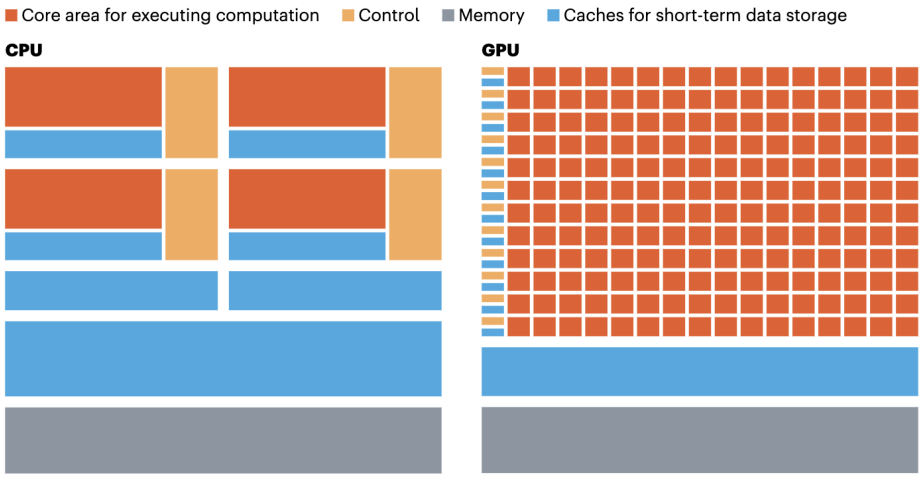


Figura 18.5: CPU architectures — more complex and specialized — versus GPU architectures — simpler and more parallel [91]

19. Autoencoders

The first implementations of autoencoders emerged under the name of **Autoassociators**[93] during the 1990s. Autoencoders have an unsupervised neural network architecture that can be used for dimensionality reduction of data, perform compression, and above all form the foundation of some generative neural network models. Together with GANs, they are the first networks to start playing with the concept of generation: how to **generate** new samples from a dataset X.

They arise from the following premise: what happens if the final output of an MLP is the input to another, inverted MLP?

19.1 Autoassociators

19.1.1 Architecture

The base architecture consists of two multilayer perceptron artificial neural networks, where the output of the first network is connected to the input of the second. The second network has an inverted distribution of neurons across layers and its output has the same dimension as the input of the first network. What the autoencoder seeks is that, given a particular input presented to the first network, the correct combination of weights is established so that the output obtained from the second network is exactly equal to the input.

That is:

$$Z = f(X) = h(XW + b)$$
$$X' = g(Z) = h(ZV + p)$$

and this must be satisfied such that $L(X, X') = ||X - X'||^2$.

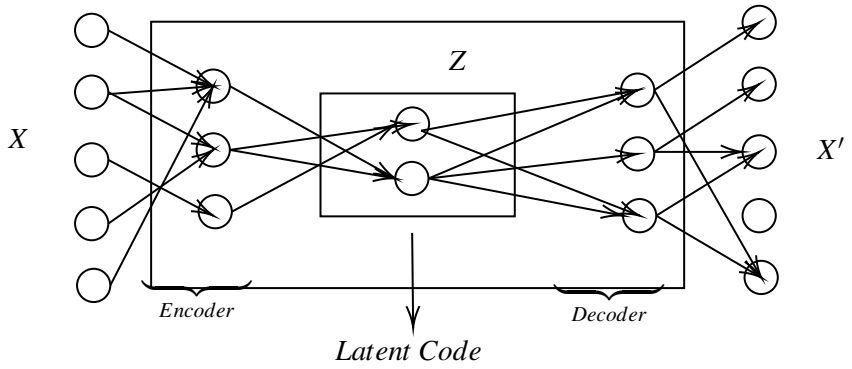


Figura 19.1: Different parts of an Autoencoder.

Training is then given by any optimization method that minimizes the difference between the output X' and X, and that converges to the synaptic weights that achieve that mapping.

Figure 19.1 shows the representation of these two networks. The first part is called the **Encoder**, while the second is the **Decoder**. The middle point, where the output of the first network meets the input of the second, is called the **Latent Code**. Once trained, an input datum X to the first network — the encoder — is transformed into a new code in the latent space (of whatever dimension it may be), after which that value enters the second network, the decoder, and becomes again the value presented at the input. That is, a new code is generated for all the data in the set X, captured in the latent space.

19.1.2 Linear Autoencoder

First, it is worth reviewing two important definitions.

Definición 19.1.1 — Spectral Decomposition. Let X be an invertible matrix with real eigenvalues; the matrix can then be diagonalized and factored as:

$$X = ELE^T \tag{19.1}$$

where L is the diagonal matrix formed by the eigenvalues and E is the matrix whose rows are the corresponding eigenvectors.

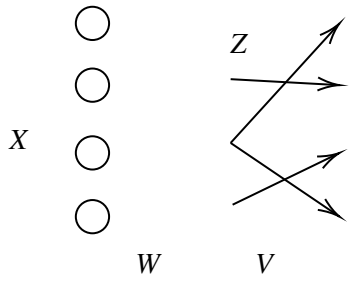
Definición 19.1.2 — Singular Value Decomposition. For the case of non-square matrices, there exists a similar but more general operation: the singular value decomposition, SVD. Given a matrix X of size $n \times d$,

$$SVD(X) = \tilde{Z}\Sigma V^T \tag{19.2}$$

where $\tilde{Z}$ are the left singular column vectors, Σ is a positive diagonal matrix, and V contains the right singular vectors. The SVD is computed by minimizing $||X - \tilde{Z}\Sigma V^T||$.

We can take the definition of an Autoencoder but apply only identity activation functions (Fig. **??**). From this, the minimization operation of the autoencoder reduces to that expressed in Equation **??**.

Once trained, the autoencoder minimizes



$$J = ||X - ZV^T|| \tag{19.3}$$

$$X \approx ZV^T \tag{19.4}$$

where,

- X: The data matrix of size $n \times d$. There are n data points of dimension d.
- Z: The output of the inner layer of the autoencoder, the code of size $n \times k$.
- V: The synaptic weight matrix associated with the Decoder, of size $k \times d$.

This then allows us to arrive at the following theorem[93].

Teorema 19.1.1 — Linear Autoencoder Converges to PCA. The internal code output Z of the linear autoencoder corresponds to the projections of the data onto the principal components.

$$T_{PCA}(X) = X\,E = Z \tag{19.5}$$

Proof

If V^T, which holds the decoder weights of the Autoencoder, is orthonormal, then $X \approx ZV^T$ from the Autoencoder can be seen as the singular value decomposition $X \approx SVD(X) = \tilde{Z}\,\Sigma\,V^T$ with $\tilde{Z}\Sigma = Z$, since the SVD decomposition arises from solving the minimization of Equation 19.3. In turn, by spectral decomposition, using the eigenvectors of the covariance matrix $\frac{X^TX}{n-1}$, we can obtain $\frac{X^TX}{n-1} = E\,L\,E^T$ where E is the PCA transformation matrix with the eigenvectors.

Then, substituting X as the factorization $X \approx SVD(X) = \tilde{Z}\,\Sigma\,V^T$:

$$\frac{X^TX}{n-1} = (\tilde{Z}\,\Sigma\,V^T)^T(\tilde{Z}\,\Sigma\,V^T)\frac{1}{n-1}$$
$$\frac{X^TX}{n-1} = (V^T)^T\Sigma^T\tilde{Z}^T\tilde{Z}\,\Sigma V^T\,\frac{1}{n-1}$$
$$\frac{X^TX}{n-1} = V\Sigma^T\Sigma V^T\frac{1}{n-1} = V\Sigma^2V^T\,\frac{1}{n-1}$$

where X is the covariance matrix.

$$\frac{X^TX}{n-1} = V\Sigma^2V^T\,\frac{1}{n-1} = V\left(\frac{\Sigma^2}{n-1}\right)V^T$$

But by spectral decomposition, this is also equal to $E\ (L)\ E^T = \frac{X^T X}{n-1} = V\left(\frac{\Sigma^2}{n-1}\right) V^T$, so if X has zero mean:

- $\lambda_i = S_i^2/n-1$, with S_i the singular values of Σ.
- $V = E$

Then transforming the data with PCA gives $T_{PCA}(X) = X\ E = (\tilde{Z}\ \Sigma\ \underbrace{V^T)\ E}_{I} = \tilde{Z}\ \Sigma = Z$, which corresponds to the output of the latent space.

This is not a very efficient way to obtain the projections onto the principal components, but what it allows is to think of the autoencoder as a nonlinear extension of principal component decomposition (when nonlinear activation functions are used). It is a strong assumption. The other point is that it shows that principal component decomposition is apparently a **canonical** form of synthesizing information, where structures that seek to synthesize information converge to the same solution (see 16).

19.1.3 Outlier Detection

Autoencoders generate a new encoding for the dataset — a more efficient encoding that attempts to capture the essence of the internal structure of the patterns contained in it. For this reason they are effective at detecting outliers, particular elements in the dataset that deviate from the norm.

Suppose we have an autoencoder trained on a set X that has already been learned. If we submit as input to that autoencoder new elements x that did not belong to the initial set, we obtain an output x', and the difference between the two $||X - X'||$ should be minimal if x has a similar structure to the input data, and should grow as that new value of x differs from the rest of the elements in the set. Thus, by evaluating the difference between input and output, it is possible to obtain a metric representing how much of an *outlier* each of the new elements x is.



19.1.4 SAE - Sparse Autoencoders

SAEs, on the other hand, use an architecture where the latent space has more dimensions than the input. To impose constraints that allow capturing information about the data structure, they introduce a regularization term that encourages sparsity — that is, a number of latent space neurons are set to zero and deactivated.

SAEs can be used for **Feature Learning**, that is, to identify characterizations or transformations of the data that are useful for discriminating between them.

Sparsity is enforced through a thresholded constraint that, if not exceeded, deactivates all neurons involved.

For example, SAEs can be used to unambiguously identify how many distinct encodings are being captured by an autoencoder with binary codes that only activate a single neuron (this is a technique currently used, for instance, to understand the encoding performed inside ChatGPT).

19.2 Generative Autoencoder

Given a dataset, two types of models can be established, depending on what one wants to do with the data.

- **Discriminative Model**: it operates directly on the data. For example, an ANN seeks the hyperplane that separates the data into two classes without caring about how those data are generated (or how they could be generated).
- **Generative Model**: it establishes a certain causal relationship. The model hypothesizes how the data themselves are generated. It seeks to have a kind of generation engine for those data.

The discriminative model is the one we have mostly used up to now. The generative model, on the other hand, poses the question: given a set of existing samples, how can I generate new samples that are indistinguishable from the existing ones?

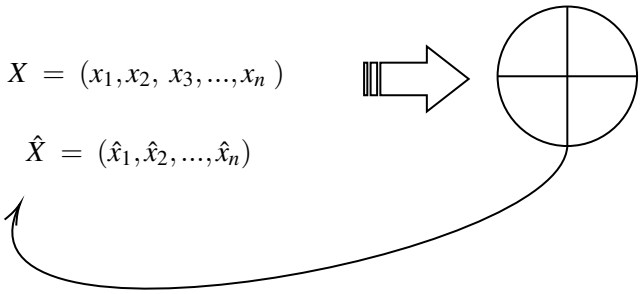


Figura 19.2: The generative model seeks to understand the process of how to generate new samples, indistinguishable from the originals.

The question that then arises is whether it is possible, using a neural network, to implement a causal generative mechanism that allows obtaining samples sharing representative characteristics of a dataset. The answer — or one answer — is the Autoencoder. For this purpose, we can make use of the new encoding established in the latent space Z and the decoder, the second part of the autoencoder.

As seen in Figure 19.3, two images of dogs are encoded by an already-trained autoencoder, generating two points — two codes — in the latent space Z.

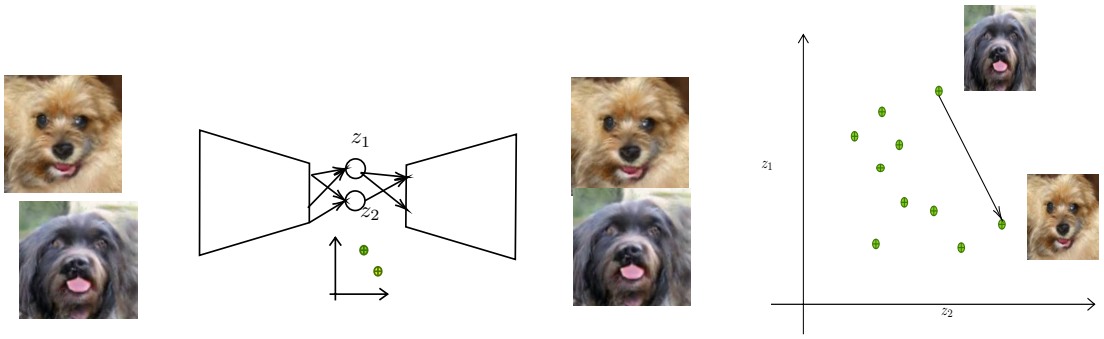
The idea is precisely that by exploiting the encoding constructed in the latent space itself, it is possible to obtain new representations of the elements X through the selection of Z values that were not directly mapped from any X in the training set.

The procedure to implement a generative autoencoder is therefore:

- We use an Autoencoder to encode all patterns in the Latent Space.
- We then set aside the Encoder and navigate within a representation vector in the latent space of Z.
- We generate successive samples by specifying directly, for example in two dimensions, the values of z_1 and z_2.
- For each vector of z_i values we obtain a new sample generated by the Decoder.

19.2.1 The Structure of the Latent Space

If the generative autoencoder is fully deterministic, it is only able to strictly map all elements of the dataset X, and in general, without some additional process that allows exploiting the encodings of the latent space, the latter will have no structure, and intermediate representations within it will not correspond to anything concrete in the set X. For this reason, the latent space is required to have



(a) Latent Space (b) Concept Vector

Figura 19.3: Each point represents a correspondence in the latent space with a concrete datum from the dataset X. By traveling from one point to another through the intermediate values, new intermediate codes Z can be generated, and for each of them image representations X' can be obtained that may share the structure of the originals. We would be generating little puppy faces.

some structure. Figure 19.4 shows two latent space scenarios. The first is a sparse one where many intermediate values have no representation. The one on the right, on the other hand, shows a more cohesive structure, indicating that intermediate values have a representation in the data.

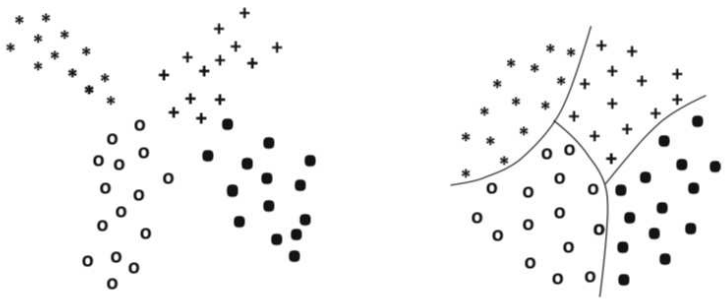


Figura 19.4: On the left, a latent space with no structure at all, where intermediate points have no encoding representation. On the right, a more cohesive, compact latent space where most points have a continuous representation.

We therefore need something more that allows us to **capture** what happens in the latent space, in order to generate new **valid** samples as we move through it.

One way to solve this is to give the latent space a statistical structure. That is, instead of looking at the output obtained for a single point Z, we look at the outputs obtained for a set of points sampled around each of the representations of the values of X in the latent space.

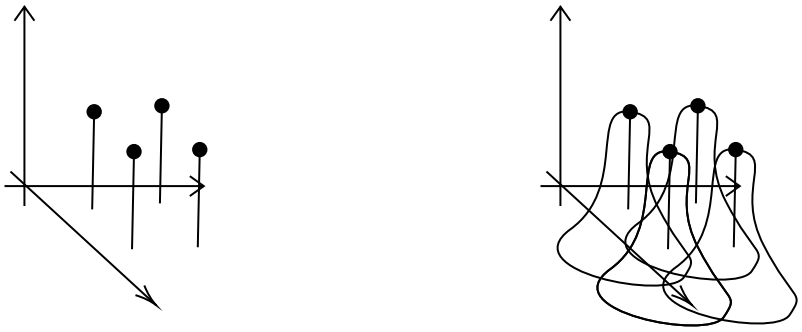


Figura 19.5: In the generative autoencoder (left), each value of X has a unique representation in the latent space. In the variational autoencoder, values are successively sampled around each of the points, combining them with each other, allowing any point Z to have some possible representation of the values of X.

19.3 VAE - Variational Autoencoder

VAEs, variational autoencoders [94], are a convergence of two ideas, where once again it is verified that the key to innovation lies in connecting facts that only appear to be disconnected:

- A proposal for a stochastic neural network — a stochastic multilayer Perceptron divided into an Encoder and a Decoder — forming an Autoencoder.
- A variational estimation of a generative model[95, 96, 97, 98].

From a more practical, programmatic point of view, what we are going to do with the VAE is give the possibility that when we encounter an X used as input, it maps stochastically to different values of Z, and in this way force the Decoder to generate a decoding in X' for that Z that resembles X as closely as possible. That is, we will be mapping the X values, which have a given cardinality, to a complete space of Z of a much larger cardinality.

19.3.1 Stochastic Neural Networks

A single hidden layer network is a universal approximator[99] that can represent any computable function (18.2.1). Among other things, it can represent a stochastic function governed by a p.d.f.

Definición 19.3.1 — Stochastic Functions. It is a function where the assignment of x to y is stochastic and governed by a conditional p.d.f. [100].

$$f(x) = L(x) + \varepsilon(x)$$

$y = f(x)$ such that, given x, $y = f(x)$ according to $p(y/x)$.

Stochastic functions make non-deterministic mappings, as can be seen in Figure 19.6. A neural network that allows representing a stochastic function is called a **Stochastic Neural Network** [99]. To implement them, a node is added — a layer where a random number is generated, but in a way that allows the network to continue backpropagating the error — and for this the reparameterization trick is

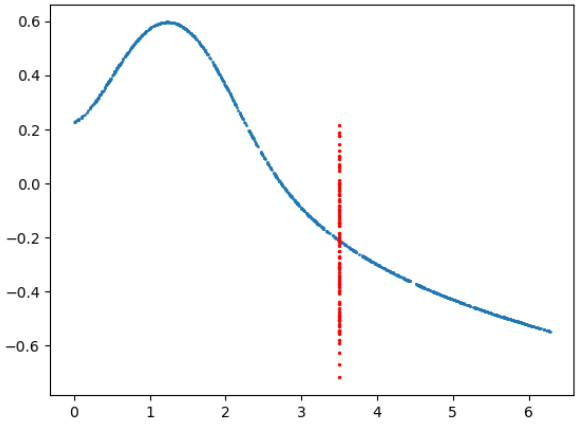


Figura 19.6: A stochastic $\sin(\cdot)$ function. In blue the average of the values generated by the function is shown. At 3.5, in red, all the particular instances generated for that point are shown.

used.

The Reparameterization Trick

The key lies in the **reparameterization trick**, making the value of z a function of $\bar{\mu}$ and $\bar{\Sigma}$.

$$z = h(\bar{X}) = \varepsilon \odot \bar{\Sigma}(\bar{X}) + \bar{\mu}(\bar{X}) \tag{19.6}$$

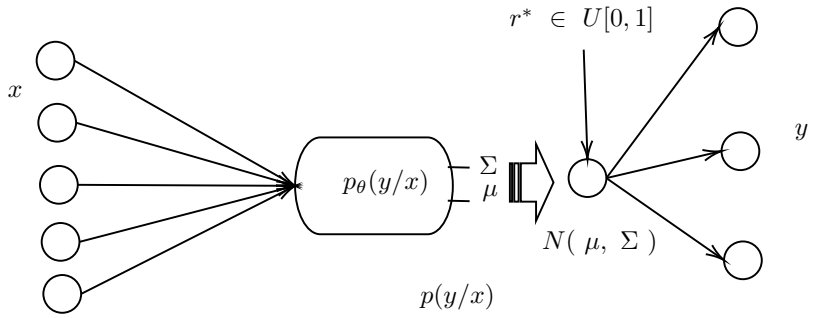


Figura 19.7: A stochastic $\sin(\cdot)$ function

The stochastic node then receives the parameters of a probability distribution — for example, the mean and standard deviation for a normal distribution — and then with those parameters samples a value r which is used with the reparameterization trick to sample from that distribution with those parameters. The final effect is that the network becomes a stochastic mapping between x and y (Fig 19.7).

19.3.2 Variational Inference

The second idea converging in the variational autoencoder is variational inference. Variational theory is used to solve problems of complex systems in physics, attempting to determine a probability

distribution function that normally has no closed-form definition or is not computationally tractable. It is usually solved via sampling (Monte Carlo), or by exploring data from the phenomenon.

The connection is that inferring a p.d.f. bears a certain resemblance to the machine learning process (Peterson, Anderson 1987 with Mean Field Theory [101], Michael Jordan[1] in 1999 [102], and Hinton, Van Camp in 1993 [103]). That is, there is a connection between learning a probability distribution parametrically — which has free parameters — and the idea of adjusting those parameters based on data to identify patterns.

Thus in 2014 Kingma et al. [94] proposed the variational autoencoder, reviving the autoassociators and proposing for the first time (as far as we know) the **generative** idea. The central tool is given in Definition 19.3.2.

Definición 19.3.2 — Variational Inference. Variational inference attempts to solve the problem of estimating a probability density function (p.d.f.) by approximating it with known functions, and optimizing the parameters of that function through the solution of an optimization problem.

Precisely behind variational inference there is an optimization process, and that can be solved through neural network convergence.

19.3.3 Optimizing the Variational Autoencoder

We can now upgrade the deterministic Autoencoder using the stochastic network, where the Encoder will be used to represent a stochastic function $p(z/x)$, such that given an input x, the function generates **some** z in the latent space following that p.d.f. On the other hand, for the Decoder, we will take **some** z from the latent space and send it probabilistically back to the world of x with the (conjugate prior) function $p(x/z)$.

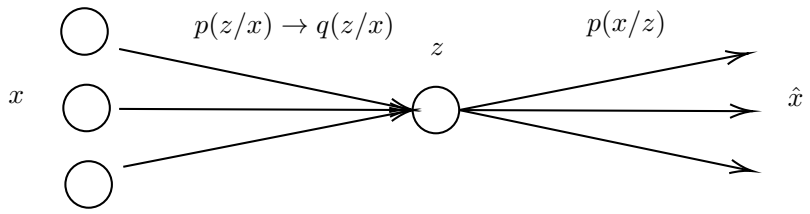


Figura 19.8: The variational autoencoder and the p.d.f.'s it maps.

What is the form of $p(x/z)$ or $p(z/x)$? We do not know, nor can we ever know, because they depend on the intrinsic structure of the data x. But now, with variational inference in hand, we can use the autoencoder to approximate them — through optimization — by distribution functions that we do know, that we specify ourselves, parametrically.

The ingredients we have are:

- $p(z/x)$: the p.d.f. of the data that we do not know, which we will approximate with $q(z/x)$.
- $q(z/x)$: the p.d.f. that we will **propose** (at will) to approximate $p(z/x)$.
- $q(z)$: we will assume that the data have a normal distribution around each z.

[1] Not the basketball player, the other one.

- $p(z)$: we will assume that probabilistically each x maps multidimensionally normally to z. Being **normal** is a restriction on $p(\cdot)$. It could be something else.

It is necessary to determine what the optimization function will be. For this, we can start with the variational problem objective: estimate $p(z/x)$ with $q(z)$ using the KL divergence:

$$\text{mín}\, KL(s||t) = -\sum t(x) \log \frac{s(x)}{t(x)} \tag{19.7}$$

$$\begin{aligned} KL(q(z)||p(z/x)) &= -\sum q(z) \log \frac{p(z/x)}{q(z)} \\ &= -\sum q(z) \log \frac{\frac{p(z,x)}{p(x)}}{\frac{q(z)}{1}} \\ &= -\sum q(z) \log \frac{p(z,x)}{q(z)} \frac{1}{p(x)} \\ &= -\sum q(z) \{\log \frac{p(z,x)}{q(z)} - \log p(x)\} \end{aligned}$$

$$\begin{aligned} &KL(q(z)||p(z/x)) \\ &\qquad = -\sum q(z) \{\log \frac{p(z,x)}{q(z)} - \log p(x)\} \\ &\qquad = -\sum_z q(z) \log \frac{p(x,z)}{q(z)} + \sum_z q(z) \log p(x) \\ &\qquad = -\sum_z q(z) \log \frac{p(x,z)}{q(z)} + \log p(x) \underbrace{\sum q(z)}_{1} \end{aligned}$$

With this we arrive at:

$$KL(q(z)||p(z/x)) = -\sum_z q(z) \log \frac{p(x,z)}{q(z)} + \log p(x) \tag{19.8}$$

But these terms can be rearranged by moving the sum to the left-hand side and mirroring the equality.

$$\log p(x) = KL(q(z)||p(z/x)) + \sum_z q(z) \log \frac{p(x,z)}{q(z)} \tag{19.9}$$

In this equation, the value $\log p(x)$ is fixed because it depends only on the data and therefore cannot be adjusted. Since we want the divergence to be as small as possible, this is equivalent to maximizing the second term.

$$\underbrace{\log p(x)}_{\text{fixed for x}} = \underbrace{KL(q(z)||p(z/x))}_{\text{as small as possible}} + \underbrace{\sum_z q(z)\log\frac{p(x,z)}{q(z)}}_{\mathscr{L}\text{, as large as possible}} \tag{19.10}$$

$$\mathscr{L} \leq \log p(x) \tag{19.11}$$

That term $\mathscr{L}$ is called the *Variational Lower Bound*, and must be as large as possible; when $KL \to 0$ it is exactly $\log p(x)$. Therefore, originally minimizing 19.8 is equivalent to maximizing $\mathscr{L}$.

$$\begin{aligned}
\mathscr{L} &= \sum q(z)\log\frac{p(x,z)}{q(z)} \\
&= \sum q(z)\log\frac{p(x/z)p(z)}{q(z)} \\
&= \sum q(z)\{\log p(x/z) + \log\frac{p(z)}{q(z)}\} \\
&= \sum q(z)\log p(x/z) + \sum q(z)\log\frac{p(z)}{q(z)}
\end{aligned}$$

$$\begin{aligned}
\mathscr{L} &= \sum q(z)\log p(x/z) && + \sum q(z)\log\frac{p(z)}{q(z)} \\
\mathscr{L} &= \mathbb{E}_{q(z)}\log p(x/z) && -KL(q(z)||p(z))
\end{aligned}$$

This is the final expression we arrive at as the optimization function that will allow us to solve the optimization problem, and it conveniently splits into two terms: the base term plus the regularization term.

Ingredients: $p_\theta(z)$, $q_\phi(z)$

$$\theta^*, \phi^* = \arg\max \mathscr{L}(\theta, \phi, x)$$

We can assume a simple distribution for $p_\theta(z)$ and choose another distribution for $q_\phi(z)$ that will determine the structure of the latent space. With this we approximate $p_\theta(z/x)$ with $q_\theta(z/x)$. By optimizing $\mathscr{L}$, the values for the parameters of those distributions are found: ϕ^*, θ^*.

We assume:

- $p_\theta(z) = \mathscr{N}(0, \mathscr{I})$
- $q_\phi(z) = \mathscr{N}(\mu(x), \Sigma(x))$, with $\Sigma(x)$ diagonal.

The first part, which is the Reconstruction Error, is equivalent to:

$$p_\theta(x/z) = \frac{1}{\sqrt{(2\pi)^k}\sqrt{\Sigma(z)}} exp\{(x-\mu(z))^T\Sigma(z)^{-1}(x-\mu(z))\}$$

The log of this is proportional to the difference between the input X and the output $\hat{X}$.

$$\log p_\theta(x/z) \;\alpha\; (x-\underbrace{\mu(z)}_{\hat{x}})^T\Sigma(z)^{-1}(x-\underbrace{\mu(z)}_{\hat{x}})$$

And regarding the regularization term with $p_\theta(z)=\mathcal{N}(0,\mathcal{I})$,

$$KL(q_\phi(z))=\mathcal{N}(\mu(x),\Sigma(x))||\mathcal{N}(0,\mathcal{I}),$$

this, after several steps[2], leads to the following expression:

$$\begin{aligned}
KL &= \frac{1}{2}\{trace(\Sigma(x))+\mu^T\mu(x)-k-\log(|\Sigma(x)|)\}\\
&= \frac{1}{2}\{\sum_k\Sigma(x)+\sum_k(\mu(x))^2-\sum_k 1-\log(\Pi_k\Sigma(x))\}\\
&= \frac{1}{2}\{\sum_k\Sigma(x)+\sum_k(\mu(x))^2-\sum_k 1-\sum_k(\log\Sigma(x))\}\\
&= \frac{1}{2}\sum_k(\Sigma(x)+(\mu(x))^2-1-\log\Sigma(x)).
\end{aligned}$$

where for numerical reasons $\Sigma(x)=\exp(\Sigma(x))$ is substituted, giving

$$KL=\frac{1}{2}\sum_k(\exp\Sigma(x)+(\mu(x))^2-1-\Sigma(x)).$$

Thus the final formula for the function to optimize is

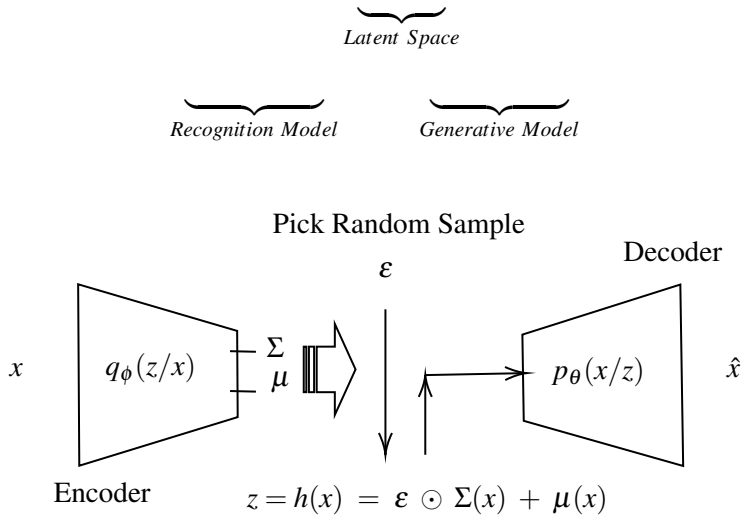
$$J=L+\lambda R \tag{19.12}$$

$$\begin{aligned}
&\text{máx}\,\mathcal{L}=\mathbb{E}_{q(z)}\log p(x/z)-KL(q(z)||p(z))\\
&\text{mín}\,\mathcal{L}=-\mathbb{E}_{q(z)}\log p(x/z)+KL(q(z)||p(z))\\
&\text{mín}\,\mathcal{L}=||\bar{X}-\bar{X}'||+\frac{1}{2}\sum_k(\exp\Sigma(x)+(\mu(x))^2-1-\Sigma(x))\\
&\text{mín}\,\mathcal{L}=||\bar{X}-\bar{X}'||-\frac{1}{2}\sum_k(1+\Sigma(x)-(\mu(x))^2-\exp\Sigma(x))
\end{aligned}$$

To summarize, the variational autoencoder algorithm is:

We take an x and place it at the network input. With that x we see what values of Σ and μ we obtain at the Encoder output. We then sample a z from $p(z)=\mathcal{N}(0,\mathcal{I})$ (this is equivalent to obtaining an ε, multiplying it element-wise by the variance, and adding the mean).

[2]The step-by-step derivation is given in the appendix 19.4 of this chapter.



With that z we pass it through the Decoder and obtain a $\hat{x}$. We train the Autoencoder to minimize $\mathscr{L}$ by first updating the Decoder weights, then passing through the function $h(x)$, and from there backpropagating the error obtained in μ and Σ toward the Encoder weights.

The *Reparameterization Trick* of the stochastic layer of the network allows backpropagating the error through that layer, which has an extra input that is a random variable [104]:

Note that the regularization term does NOT depend on the output of the decoder ($\bar{X}$). It depends on internal variables (Σ and μ) that arrive at the latent layer.

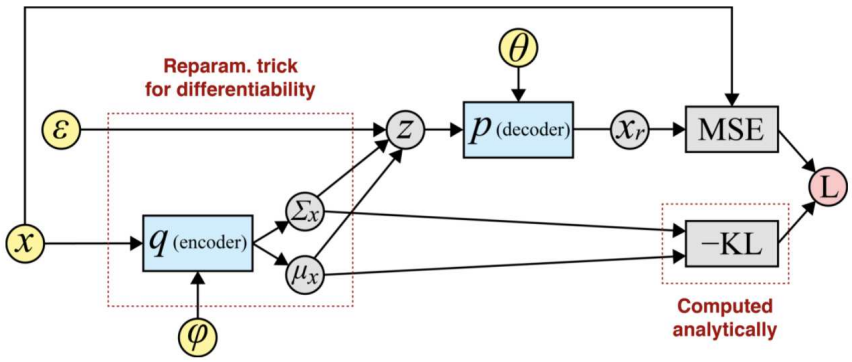
How to backpropagate[105] the error through the stochastic node?

Suppose we want to update a particular encoder weight ω_e based on the cost function:

$$J = L + \lambda R$$

What we want to compute is $\frac{\partial J}{\partial \omega_e}$ in order to use it in

$$\omega_e^{t+1} = \omega_e^t - \eta \frac{\partial J}{\partial \omega_e}$$



$$\frac{\partial J}{\partial \omega_e} = +(\frac{\partial L}{\partial z}\frac{\partial z}{\partial \mu}\frac{\partial \mu}{\partial \omega_e} + \lambda \frac{\partial R}{\partial \mu}\frac{\partial \mu}{\partial \omega_e})...$$
$$= +(\frac{\partial L}{\partial z}\frac{\partial z}{\partial \sigma}\frac{\partial \sigma}{\partial \omega_e} + \lambda \frac{\partial R}{\partial \sigma}\frac{\partial \sigma}{\partial \omega_e})$$

The term $\frac{\partial L}{\partial z}$ corresponds to the backpropagation of the error in the decoder, while $\frac{\partial z}{\partial \mu}$ is 1, and $\frac{\partial \mu}{\partial \omega_e}$ is the backpropagation toward the encoder. $\frac{\partial R}{\partial \mu}$ corresponds to the derivative of Eq. 19.12.

On the other hand, the second term corresponds to the changes on the μ side, where $\frac{\partial z}{\partial \sigma}$ is ε, $\frac{\partial R}{\partial \sigma}$ corresponds to the derivative of Eq. 19.12 (now with respect to σ), and $\frac{\partial \sigma}{\partial \omega_e}$ is the backpropagation toward the encoder.

Variational autoencoders are today central to many algorithms and solutions that seek to generate new data, produce new samples, and are used as a **data augmentation**[106] tool. More information can be found on Variational Inference [107], Variational Methods [94], VAE [108], and Autoencoders [109, 110, 111].

19.4 Appendix

Probability and Statistics

$$N(x/\mu,\sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} exp(-\frac{(x-\mu)^2}{2\sigma^2}) \tag{19.13}$$

$$N(\mathbf{x}/\vec{\mu},\Sigma) = \frac{1}{\sqrt{|2\pi\Sigma|}} exp(-\frac{1}{2}(\mathbf{x}-\vec{\mu})^T\Sigma^{-1}(\mathbf{x}-\vec{\mu})) \tag{19.14}$$

Information Theory

A brief, no-frills review of the most important definitions from information theory.

Random variable: $X = x_i$

Information: $I = -\log p(x_i)$

Entropy: Can be interpreted as a measure of the amount of information contained in all possible states that x can take. It is a weighted average.

$H = \mathbb{E}_{p(x_i)} I(x_i) = -\sum p(x_i)\log p(x_i)$

Mutual Information: $I(x,y) = H(y) - H(y/x) = \sum_x \sum_y p(x,y) log \frac{p(x,y)}{p(x)p(y)}$

$H(x) = I(x,x)$, and it holds that,

- $I = 0$ implies $p(x) = 1$ — Certainty!
- $I > 0$ — some information.
- $I(x_k) > I(x_i) \Rightarrow P_k < P_i$ — the rarest (least probable) events carry more information.

Entropy is at its minimum, zero, in a situation of total certainty (noting that $\lim_{p\to 0^+} p\log p = 0$), and at its maximum when all events are equally probable (uniform probability). The values it can take are therefore $0 <= H(x) <= \log(2k+1)$.

Conditional entropy: $H(x/y) = H(x,y) - H(y)$

Joint entropy: $H(x,y) = \sum_{i=1}^{m}\sum_{j=1}^{m} p(x_i,y_i)\log\frac{1}{p(x_i,y_i)}$

Definición 19.4.1 — Cross Entropy.

$$H_p(q) = -\sum_{k=1} q(y_k) \log p(y_k) \tag{19.15}$$

Definition 19.4.1 is used when two distributions are at play and the one used to determine the individual information contributed by each event differs from the one used for weighting. This establishes a comparison from the perspective of data encoding. This definition can be simplified for the case where there are two possible events.

Definición 19.4.2 — Binary Cross Entropy.

$$H_p(q) = -(y_i) \log p(y_i) + (1 - y_i) \log(1 - p(y_i)) \tag{19.16}$$

In the context of machine learning, the average binary cross entropy can be used as a cost function for a binary supervised classification problem. Here precisely the two distributions appear: the real one coming from the data — the label representing the true class — and the one produced by the classifier.

Definición 19.4.3 — Average Binary Cross Entropy.

$$H_p(q) = -\frac{1}{N} \sum_{i=1}^{N} \{(y_i) \log p(y_i) + (1 - y_i) \log(1 - p(y_i))\} \tag{19.17}$$

Here, $q(y_i) = y_i = 0|1$, y represents the label, $p(y_i), 1..N$ is the probability assigned to that label for sample k. *Binary Cross Entropy*[3] serves as a function for estimating the effectiveness of a classifier that assigns label y_k to each element $1..N$ with probability $p(y_k)$. A perfect classifier that estimates y_k with probability 1 yields a binary cross entropy value of zero.

Definición 19.4.4 KL: Kullback-Leibler Divergence or Relative Entropy It is a norm (strictly speaking, not quite) that measures the distance between probability distributions: if it tends to zero, the distributions are similar. A measure of similarity.

$$KL(q||p) = -\sum_{x_i} q(x) log \frac{q(x)}{p(x)} \tag{19.18}$$

where $p(x)$ is the reference to which $q(x)$ is compared.

Properties:

- $KL(q||p) \neq KL(p||q)$
- $KL(q||q) = 0$
- $KL(q||p) \geq 0$
- $KL(q||p) = H_p(q) - H(q) \geq 0$

Maximum Likelihood Estimator

MLE, Maximum Likelihood Estimator. Given samples $(x_1, x_2, ..., x_n)$ and a p.d.f. $p(x)$. The question being addressed is: given the samples x_i, for what value of ϕ is the observed sample x most likely to have occurred?

[3] Cross Entropy Loss can also be called *Logistic Loss* or *Multinomial Logistic Loss*.

Then $L(x_i;\phi) = p(X = x_1)p(X = x_2)...p(X = x_n)$. The goal is to maximize $L(\phi)$ via:

$$\frac{dL}{d\phi} = 0 \text{ o } \frac{dlnL}{d\phi} \tag{19.19}$$

Teorema 19.4.1 — Maximum of a Monotonic Function.

$$\hat{\phi} \text{ is MLE of } \phi \Rightarrow f(\hat{\phi}) \text{ is MLE of } f(\phi) \tag{19.20}$$

Regularization

Regularization reduces generalization errors. It is a way of adding constraints to an optimization functional to force the learning of something useful. It translates as a penalty term that is kept small while minimizing a cost function. It helps to avoid considering complex or extreme solutions in the optimization problem itself.

This method comes from optimization theory and is known as *Tikhonov Regularization*. In ML, it can be L^2-regularized or L^1-regularized. For example, in least squares,

$$\min_{\phi} \frac{1}{N}||y - x\phi||(+\lambda||\phi^2||) \tag{19.21}$$

The MLE method transforms into MAP when an additional regularization is considered. Early Stopping, Dropout, or Weight Decay ($w_{t+1} = w_t - \alpha\Delta_w - \lambda w_t$) are methods that intrinsically involve regularization.

Derivation of the Least Squares Method from Graphical Models

Y depends on X and W.

The objective is to minimize $L(w) = \sum_{i=1}^{N} (W^T x_i - y_i)^2$ to find $\hat{w} = \arg\min_w L(w)$.

$$P(y,w,x) = P(y/w,x)P(w)P(x) \tag{19.22}$$

For this, the idea would be to find the values of w that maximize the joint probability of having the y values we have, given the x values we observe — so we must condition on those x values (the values x for which we want to find the least squares solution).

$$\begin{aligned} P(y,w,x) &= P(y/w,x)P(w)P(x) \\ P(y,w,x/x) &= P(y/w,x,x)P(w/x)P(x/x) \\ P(y,w/x) &= P(y/w,x)P(w)1 \end{aligned} \tag{19.23}$$

Noting that $P(x/x)$ is 1. To specify this model, we can assume two things: first, that the values of y given x and w are all distributed around a normal with mean approximately $w^T x$ and variance $\sigma^2 I$ (i.e., only the diagonal of the covariance matrix).

We can also assume that the values of w have zero mean and also a diagonal standard deviation $\omega^2 I$.

That is,

$$\begin{aligned} P(y/w,x) &= N(y/w^T x, \sigma^2 I) \\ P(w) &= N(w/0, \gamma^2 I) \end{aligned} \tag{19.24}$$

Assuming this, $P(y,w/x)$ becomes

$$\begin{aligned} P(y,w/x) &= P(y/w,x)P(w) \\ &= log(P(y/w,x)P(w)) \\ &= log(P(y/w,x)) + log(P(w)) \\ &= log(exp(-\frac{1}{2}(y-w^T x)^T (\sigma^2)^{-1}(y-w^T x))) + log(exp(-\frac{1}{2}w^T(\gamma^2)^{-1}w^T)) \\ &= -\frac{1}{2\sigma^2}(y-w^T x)^T (y-w^T x) - \frac{1}{2\gamma^2}w^T w \\ &= (-1)[\alpha||y-w^T x||^2 + \delta||w||^2] \end{aligned} \tag{19.25}$$

Maximizing this is the same as directly minimizing $||y-w^T x||^2 + \delta||w||^2$. The first term is thus the expected result corresponding to the least squares solution for finding a linear approximation of the relationship between x and y. But in this case, additionally, it gives us a regularization term, which determines that of all possible solutions, the one where w is as small as possible appears to be the one generating the most probable solution (given the distributions we assumed).

Inverse Sampling Theorem

When only a pseudorandom number generator with a uniform distribution $U[0,1]$ is available — the typical **rand** — Theorem 19.4.2 can be used to generate values from any arbitrary distribution.

Teorema 19.4.2 Choose a value $Y \sim U[0,1]$ and check what $X = F^{-1}(Y)$ gives. The distribution of those X values will correspond to the p.d.f. in question.

$$\begin{aligned} Y \sim U[0,1], F \text{ invertible and } F = CDF(pdf()), \\ X = F^{-1}(Y) \text{ has } CDF(X) = F \end{aligned}$$

$$Y \sim U[0,1], F = CDF(pdf()), X = \inf_x \{x : F(x) \geq Y\}$$

Tied Weights

a) For a fixed value of V, the optimal matrix W satisfies $D^T D(W^T V^T V - V) = 0$.

$$\begin{aligned} DW^T V^T - D &= 0 \\ D(W^T V^T - I) &= 0 \\ D^T D(W^T V^T - I) &= D^T 0 \\ D^T D(W^T V^T - I)V &= D^T 0V \\ D^T D(W^T V^T V - V) &= 0 \end{aligned} \tag{19.26}$$

b) If D has rank d, then $D^T D$ is invertible and cannot be zero. Therefore, in 19.26, there is no choice but $W^T V^T V - V = 0$, which implies $W^T V^T V = V$.

c) Show that if $W^T V^T V = V$ then $W = (V^T V)^{-1} V^T$, provided $V^T V$ is invertible.

$$\begin{aligned}
W^T V^T V &= V \\
W^T (V^T V) &= V \\
W^T (V^T V)(V^T V)^{-1} &= V(V^T V)^{-1} \text{ invertibility of } V^T V \\
W^T &= V(V^T V)^{-1} \\
(W^T)^T &= (V(V^T V)^{-1})^T \\
W &= (V(V^T V)^{-1})^T \\
W &= ((V^T V)^{-1})^T V^T \\
W &= ((V^T V)^T)^{-1} V^T \text{ The transpose of the inverse is the inverse of the transpose} \\
W &= (V^T (V^T)^T)^{-1} V^T \\
W &= (V^T V)^{-1} V^T \textbf{ Done}
\end{aligned} \tag{19.27}$$

d) Now what happens if the autoencoder weights are the same (a, b, and c). This occurs when $W = V^T$ where the columns of V are orthonormal.

a)

$$||DW^T V^T - D||||D(W^T W) - D|| \tag{19.28}$$

from which it follows that

$$\begin{aligned}
D^T D(W^T W - D) &= 0 \\
W^T W &= I
\end{aligned} \tag{19.29}$$

b)

$$\begin{aligned}
W^T V^T V &= V \\
W^T W V &= V \\
V &= V
\end{aligned} \tag{19.30}$$

and c)

$$\begin{aligned}
W^T V^T V &= V \\
W^T (W) V &= V \\
W^T W V &= V \\
V &= V
\end{aligned} \tag{19.31}$$

If the w_i are orthonormal,

$$w_i^T w_j = \begin{cases} 0 & j \leq i \\ 1 & j \neq i \end{cases} \tag{19.32}$$

Derivation of the VAE Regularization Term

Demostración. We will go from

$$KL(q_\phi(z)) = \mathcal{N}(\mu(x), \Sigma(x)) || \mathcal{N}(0, \mathcal{I}),$$

to

$$KL = \frac{1}{2}\{trace(\Sigma(x)) + \mu^T \mu(x) - k - \log(|\Sigma(x)|)\}$$

Let $P(x) \sim \mathcal{N}(\mu_1, \Sigma_1)$ and $Q(x) \sim \mathcal{N}(\mu_2, \Sigma_2)$ with $\mu_1, \mu_2 \in \mathbb{R}^k$ and $\Sigma_1, \Sigma_2 \in \mathbb{R}^{k \times k}$.

■



20. Convolutional Neural Networks

Convolutional networks have their origin in the 1962 work of Hubel and Wiesel [52], which earned them the Nobel Prize in Medicine in 1981. They discovered that in the visual cortex of cats, certain layers showed a firing rate among their neurons that corresponded to the inclination of a bar shown to the cat.

They detected a **neural coding** — a layer that was sensitive to the inclination of the bar in the cat's visual field. This was one of the first works in which information of this kind could be obtained: neural firings that corresponded to perceptual information in a simple way.

This work also served as a tremendous inspiration for the field of Computer Vision. If the brain uses a mechanism that allows detecting inclination in objects in the visual field, perhaps that is a good way to process visual information and derive content from it. This is what Fukushima captured with his Neocognitron proposal — heavily inspired by this work — and which LeCun, independently, formalized very well from the signal processing side with his proposal for convolutional networks.

CNNs, Convolutional Neural Networks, or **ConvNets** from a more formal standpoint, are based on the convolution operation. They have been tremendously successful and are one of the engines behind the current rise of Deep Learning. In fact, they have been so successful that we are starting to forget they are even there — as happens with all very good technologies.

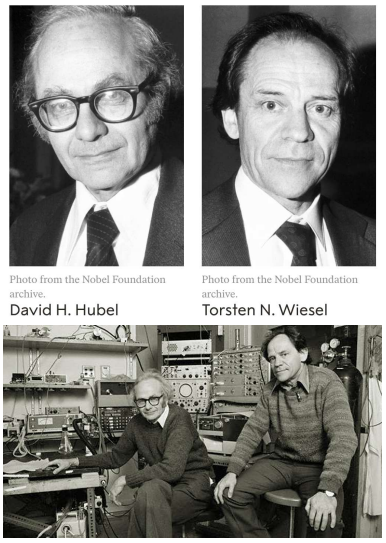


Figura 20.1: Hubel and Wiesel, Nobel Prize in Medicine 1968

20.1 The Convolution Operation

To understand convolutional networks it is first important to define and understand the convolution operation, which arises as Definition 20.1.1.

Definición 20.1.1 — Convolution. The convolution operation is a linear operation applied to a signal — that is, a one-dimensional function in time.

- Assuming $x(t)$ is a one-dimensional signal as a function of time and $w(t)$ is a convolution **kernel**,
- $s(t) = \int x(k)w(t-k)dk$
- $s(t) = (x*w)(t) =< x(k), w^*(t-k) >$, where $*$ denotes the conjugate.
- (Correlation: $s(t) = \int x(k)w(t+k)dk$ (+ positive))
- Discrete: $s[n] = \sum_k x[k]w[n-k]$

Mathematical convolution plays a primary role in digital signal processing. For example, any linear digital filter can be represented by an equation based on discrete convolutions.

Definición 20.1.2 CCDE - Constant Coefficient Difference Equation

- $\sum_{k=0}^{N-1} w_1[k]y[n-k] = \sum_{k=0}^{M-1} w_2[k]x[n-k]$
- $Y(z) = H(z)X(z)$
- w_1, w_2 are convolution kernels, x, y are discrete signals.
- X, Y are the system states and H is the transfer function.

20.1.1 Moving Average

An interesting way to understand convolution is to see it as an extension of the weighted average over a signal. Imagine there is a signal — a function where the X axis is time t. Discretely, for each point in t there is a value in the signal, such as the price of Bitcoin. If we want to see the trend in the information without getting lost in daily variations, what we can do is replace, for each day, the specific price data with the average of the 5 previous days, today, and the 4 following days. This will smooth the price signal and allow us to see the trend without worrying too much about daily fluctuations.

The following code, for example, is an implementation of this operation — which is nothing more or less than a convolution of a signal with a convolution kernel composed of values corresponding to a uniform distribution to produce the averaging effect.

```
windowlength = 10
avgeeg = np.convolve(signal,
     np.ones((windowlength,))/windowlength,
           mode='same')
```

The convolution kernel can have any length, and it precisely represents how many values around the central value will be used to compute the average (in this case). Of course, other kernel values can be used to produce an entirely different effect.

This can be used, for example, to process a signal of another type, such as sensor data, as seen in Figure 20.2, where the application of the moving average produces a smoothing of the signal that allows better visualization of the most significant events while hiding the noise.

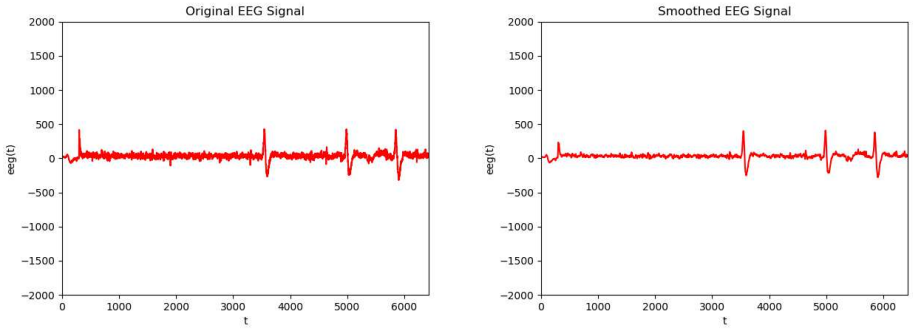


Figura 20.2: On the left, a signal from an electrophysiology sensor that is smoothed with a moving average convolutional filter.

20.1.2 Padding

The convolution process requires special care at the edges. Because the support of the signal is finite and must be aligned with the kernel, it is necessary to handle what happens at the boundaries in terms of how the signal and kernel are aligned. There are three ways to do this. The first is **valid**, which means aligning the kernel values with the existing signal values. Since we are computing a weighted average, this means that the result of this operation has fewer elements (the support size is reduced in the output). The second is **same**, which corresponds to the situation where the original signal is **padded** — zeros are added, one on each side — allowing the output signal to have exactly the same length as the input, preserving the size (hence *same*). Finally, the third option **full** involves adding two zeros of padding on each side so that the generated signal increases in size.

For example, with a discrete signal [1,2,-5,4,2,-1] and a kernel [-1,2,-1], the following results are obtained with each padding scheme:

- Valid: [8, -16, 11, 1]
- Same: [0, 8, -16, 11, 1, -4]
- Full: [-1,0, 8, -16, 11, 1, -4,1]

The padding scheme thus determines how the operation reacts to boundary conditions, and the size of the output signal.

20.2 Convolution on Images

The basic filtering operation in Computer Vision is the application of a two-dimensional convolution operation with different convolution kernels. In the case of an image there are two indices over which the operation moves — height and width — but intrinsically it is the same operation.

$$\begin{aligned} S(i,j) = (I*k)(i,j) &= \sum_m \sum_n I(m,n)K(i-m,j-n) \\ &= \sum_m \sum_n I(i-m,j-n)K(m,n) \end{aligned}$$

In Computer Vision or Image Analysis and Processing, it is necessary to apply filters to images to highlight certain features. These filters are based on the idea of composing, by hand via **feature engineering**, a convolution kernel that generates what one wants to highlight — and with that, one

can identify information in the image. For example, to detect horizontal or vertical edges, as shown in Figure 20.3, filters that emphasize sudden changes in pixel values in the horizontal or vertical direction are used.

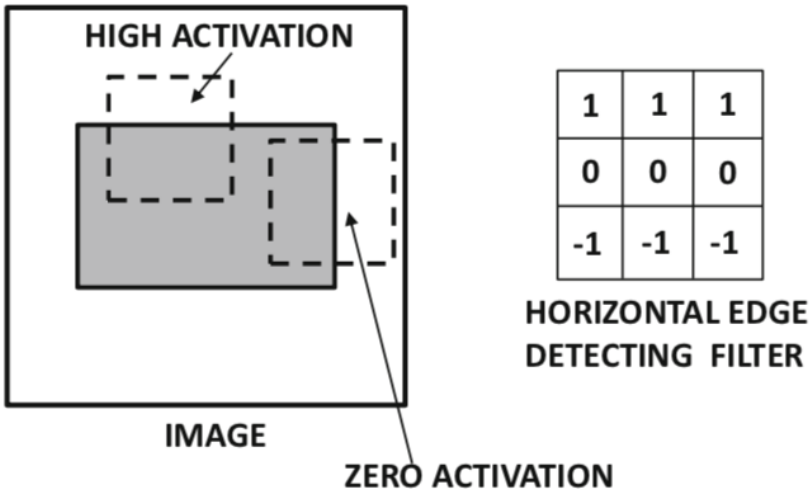


Figura 20.3: Convolutional filters for edge detection.

The 3×3 convolution filters that allow detecting these edges are matrices such as the following:

$$S_x = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}, S_y = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{pmatrix}$$

Applying the convolution operation with this kernel to each pixel of an image (followed by a thresholding operation) produces output images where edges in one direction, another, or both are emphasized.

For several years, the field of Computer Vision relied on algorithms that, with extensive knowledge of the dynamics of visual image generation, proposed different hand-crafted convolution kernels that produced specific outputs useful for identifying different patterns in images.

20.3 Neocognitron

Biological nervous systems are excellent at rapidly and in real time identifying visual patterns.



Figura 20.5: Kunihiko Fukushima, the creator of the Neocognitron.

In the work of Wiesel and Hubel, they discovered that the firing rate of neurons in different layers of the visual cortex of cats was correlated with orientation — with the angle at which they tilted a bar presented in the cat's visual field. For angles near 45 degrees, the rate was higher. Understanding that the visual cortex, responsible for this incredible feat of biological engineering, is composed of hierarchical layers sensitive to orientations in the dominant images of the visual field, these ideas laid the groundwork for finding a computational

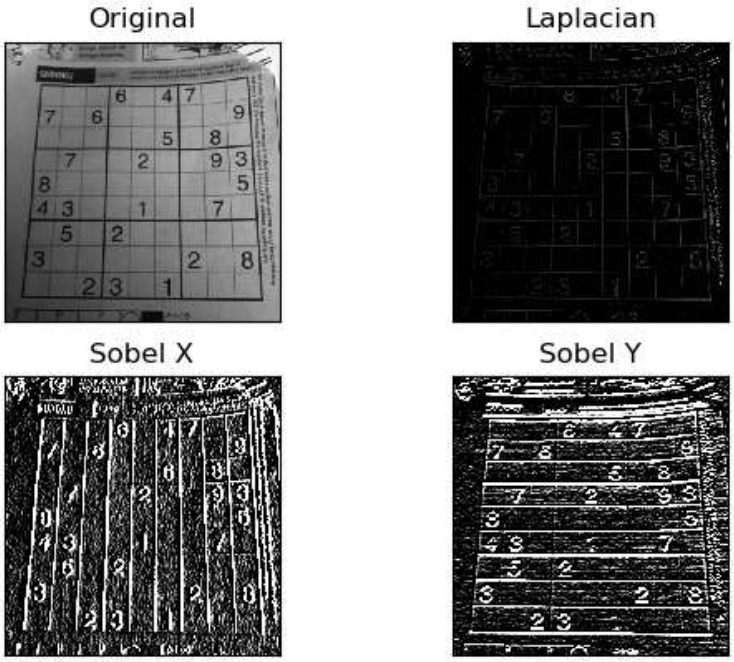


Figura 20.4: Result of applying the convolution operation with different filters that cause horizontal or vertical edges to be highlighted. The Laplacian is the result of summing the second derivatives of the image values, thereby identifying both horizontal and vertical edges simultaneously.

structure that could artificially replicate it. In 1989, Kunihiko Fukushima, who had joined an interdisciplinary research group at NHK[1][2], learned of the advances in neuroscience from the work of Wiesel and Hubel and materialized the idea of the Neocognitron, with the concrete practical goal of identifying characters in images — an OCR system.

In the proposals of the Neocognitron, and in LeCun's independent mathematical formalization, the foundation of what modern convolutional networks are was established. The crux of the matter is to modify an MLP so that it performs a convolution operation from one layer to the next, instead of a weighted sum of all inputs with different weights, in such a way that the operation also acts as a filter. This allows imitating the very same idea observed in the visual cortex, through the use of convolution as an operation.

Figure 20.7 shows the general structure of a convolutional network, similar to the one proposed by Fukushima/LeCun. The input is processed by convolutional layers where said operation is applied, then the pooling operation is applied, and finally it is processed normally by a multilayer Perceptron. The neural network view is that the traditional filters used in computer vision are shallow

[1]Japan's BBC.[2]

[2]The British public television broadcaster.

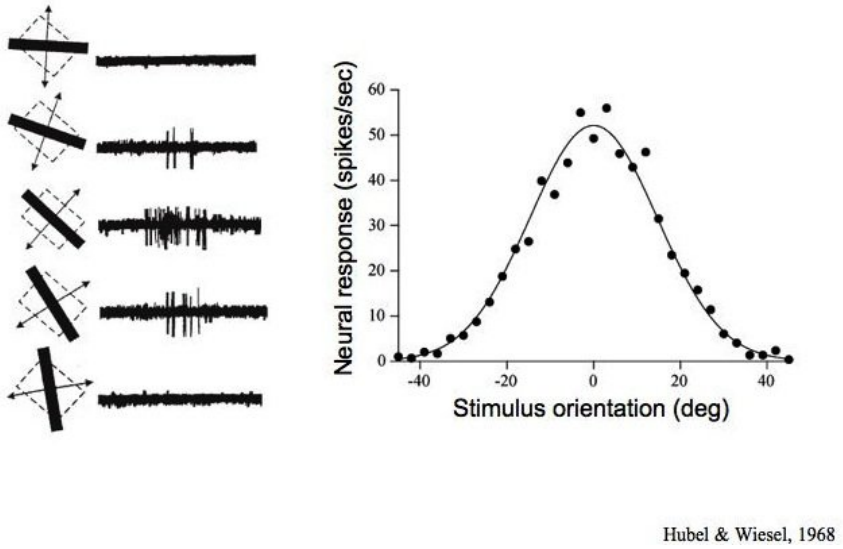


Figura 20.6: In the Hubel and Wiesel experiment, the neural firing rate in different layers of the cat's visual cortex correlates with the inclination of the bar shown to the feline.

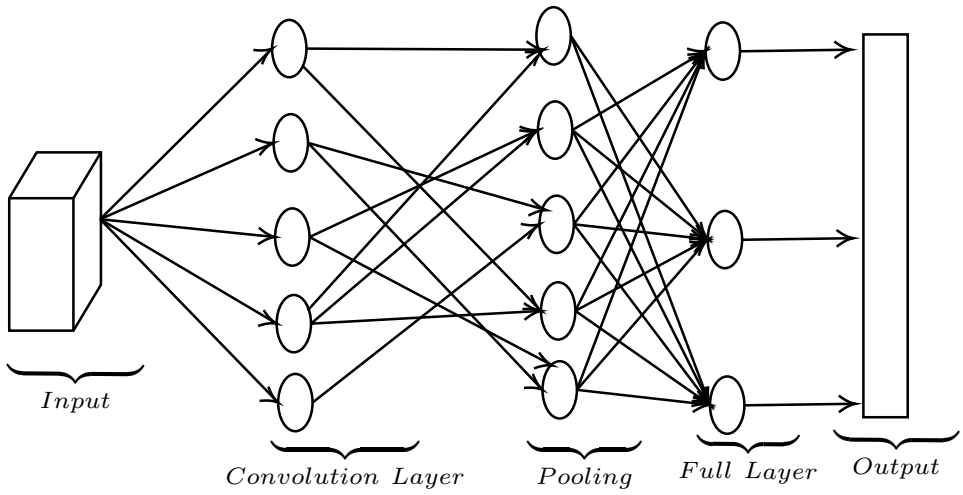


Figura 20.7: General structure of a convolutional network with its different layers.

versions of convolutional neural networks, and depth is precisely one of the keys to the success of this approach[112].

20.4 The Convolutional Layer

The convolutional layer is central, and it is useful to understand it as an operation similar to that of the Perceptron, but where the weights used are shared **spatially**. The different components can be

visualized in Fig. 20.8 and Equation 20.1. The operation in question goes from layer q to layer $q+1$.

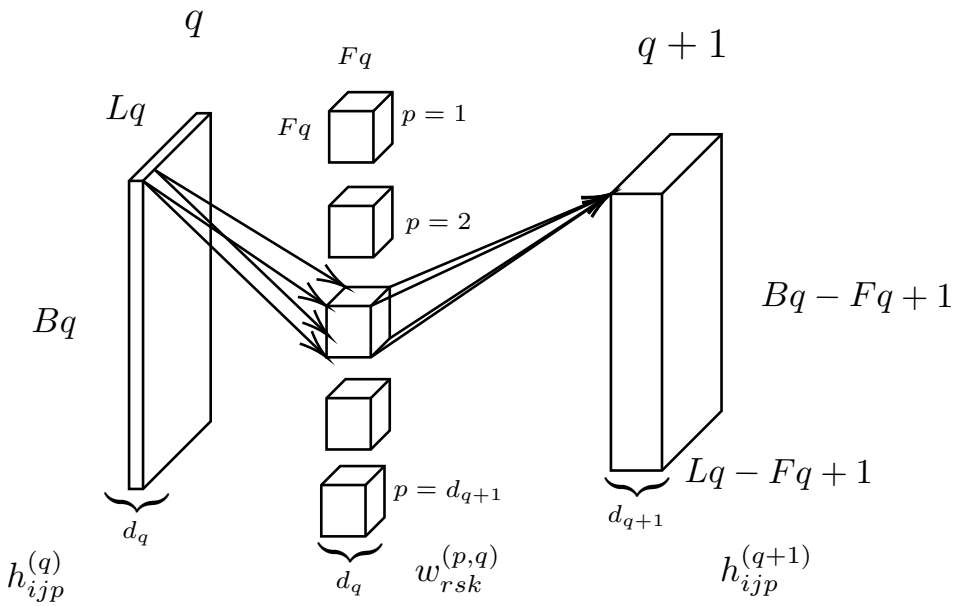


Figura 20.8: Components of the convolutional layer

$$W^{p,q} = [w_{ijk}^{(p,q)}], h_{ijp}^{(q+1)} = \sum_{r=1}^{Fq} \sum_{s=1}^{Fq} \sum_{k=1}^{dq} w_{rsk}^{(p,q)} h_{i+r-1,j+s-1,k}^{q} \tag{20.1}$$

i : height, $\forall i \in \{1, ..., Lq - Fq + 1\}$
q : Layer, j : width, $\forall j \in \{1, ..., Bq - Fq + 1\}$
k : depth, p : Filter, $\forall p \in \{1, ..., d_{q+1}\}$

The input image is $h_{ijp}^{(q)}$, of dimensions $Lq \times Bq$ in height and width, composed of d_q color channels[3]. From layer q to $q+1$, for example p convolutional filters (kernels) are positioned, with dimensions $Fq \times Fq$ — for example 3×3 — but they must have exactly the same number of color channels as the input image, in order to assign to each pixel of each channel each of the kernel weights. The kernels hold the weights — the free parameters of the convolutional network $w_{rsk}^{(p,q)}$. The output of the convolutional layer is a new, smaller image (assuming valid padding), of size $Lq - Fq + 1 \times Bq - Fq + 1$, with a number of *channels* equal to the number of filters in this convolutional layer d_{q+1}. In this way each filter operates in parallel, generating one additional channel in the output image.

[3]The colors in traditional images — **R**ed, **G**reen, and **B**lue — are those normally used for the spectrum of images visible to humans, but can perfectly be extended to many more; the distinction is entirely arbitrary. Many images are **hyperspectral**, indicating precisely that they have many channels.

Nonlinearity, as in the multilayer Perceptron, is provided by activation functions that take the output of $h_{ijp}^{(q+1)}$ as input. Particularly in convolutional networks designed for images where values can be very large, activation functions such as ReLU are used; their advantage is that they do not saturate as quickly and allow the gradient to have a much larger range of possible values, providing more information — finer information — for weight adjustment (whereas sigmoidal functions such as the hyperbolic tangent would quickly saturate, driving everything to the same value).

The ReLU definition is very simple:

$$f(x) = x^{+} = max(0,x) \tag{20.2}$$

$$f(x) = \begin{cases} 0 & x \leq 0 \\ x & x > 0 \end{cases}$$

Being a convolution operation, two new hyperparameters appear: the **stride**, which is the step of the spatial jump made when moving from one pixel to the next (and computing the weighted average of all surrounding values); and the second is, of course, the **boundary effects** and how to handle padding.

20.5 The Pooling Layer

The Pooling layer is **hardwired** — that is, it has no trainable free parameters. It serves to capture translational invariance and operates in a manner similar to a thresholding operation. Different regions of the image pass through convolutional layers, which act as filters and emphasize particular features of the image. The pooling layer ensures that regardless of where that emphasized point is located, it is taken into account in successive layers and has relevance in affecting the network's output. The result of this layer is similar to the effect obtained by **downsampling** or by reducing the resolution of an image. The pooling operation generally captures a region of the image and synthesizes it into a single pixel, normally by selecting the maximum value — forming **MaxPooling** — as seen in Figure 20.9. This translational invariance can be extended to make other properties invariant, depending on how the filters and subsequent pooling layers are grouped.

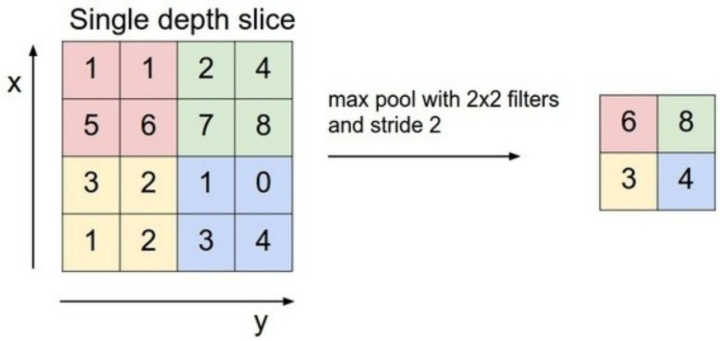


Figura 20.9: The pooling layer summarizes information obtained in previous layers so as to provide translational invariance.

20.6 Full Layer and Softmax

At the end of the Network (Fig. 20.7) there is one (or more) layer(s) responsible for making the final classification (if that is the objective). This sub-network is traditionally a multilayer perceptron trained to perform multi-class classification of the input image. In convolutional networks it is very common to use the Softmax activation function defined in Eq. 20.3.

$$\sigma(\vec{V})_i = \frac{e^{V_i}}{\sum_{j=1}^{K} e^{V_i}} \tag{20.3}$$

with V_i as the linear output of the neuron in the final layer. This function also acts as a normalizer of the output, emphasizing the differences to yield a single winning output — the one with the highest output value given by Eq. 20.3. Being a normalization, the output of all values sums to 1, so it can be interpreted as a probability function.

20.7 Backpropagation

How does the change in network structure affect the convolutional layers when applying the Backpropagation algorithm? The goal is always to maintain the identification of each weight's influence at the moment of backpropagating the errors. That is why in convolutional layers, since weights are shared, it is necessary to accumulate all the variations when performing the weight update. There are several ways to handle this. The first is the trivial approach of simply sliding the kernel spatially and accumulating the required weight modifications. This is the same as what happens in each epoch with respect to the impact of each input, but now it must be done across the spatial distribution.

1. Suppose a network to perform a three-class classification.
2. The network is initialized randomly, as always.
3. Forward values are computed by applying the convolution operation in the corresponding layers until reaching the output values, layer by layer.
4. In the *Fully Connected* layers it is exactly the same as in the multilayer Perceptron.
5. In the Pooling layers, only the weight contribution of the winning output is updated (assuming an identity function).
6. In the convolutional layers, the filter weights must be kept in the correct spatial relationship.

Backpropagation via sparse matrix representation

The convolution operation can be viewed as an efficient way of performing matrix multiplication by reusing weights. From this perspective, it is possible to represent the convolution operation

as a sparse matrix, as detailed in Figure 20.10. To do this, the kernel weights are represented in one-dimensional vector form of length equal to all elements of the input matrix of the operation (in the figure, 9). Each weight is placed in the same position that corresponds to how the weight aligns with the input value. Each row of this new matrix represents one alignment of the weights over the input. This generates a sparse matrix C where the majority of entries are zeros. The input can then be directly multiplied — taken as a **flattened** vector of 9 elements — with matrix C. The result of this corresponds to what would be the output matrix of the complete convolution operation. In it, the weights remain as parameters that can be used in this way — as a linear multiplication layer — for which applying backpropagation via the chain rule is trivial.

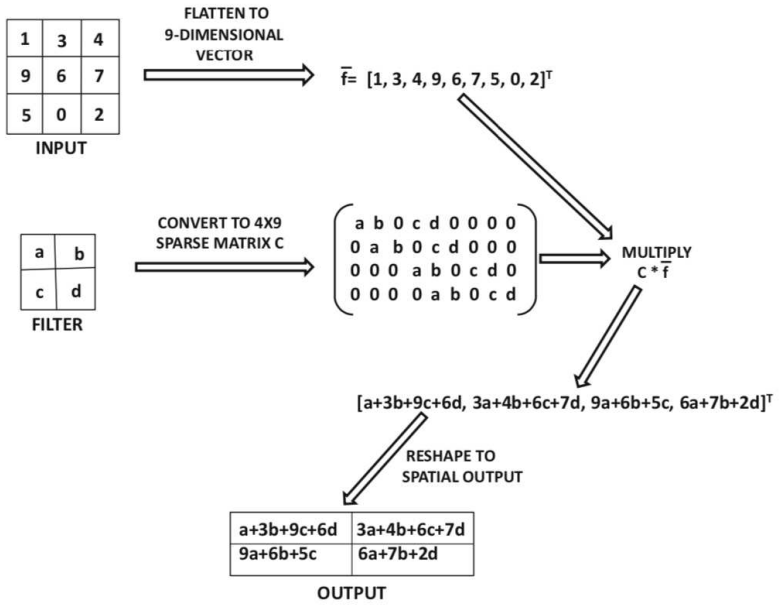


Figura 20.10: The sparse representation of the convolution operation allows operating in the same way as in a multilayer perceptron.

20.8 Why Do They Work?

Convolutional networks work very well for capturing structured behavior in hierarchies that is invariant to translations — very common in visual images. The reasons are:

- Sparse matrices are much more efficient: fewer computations.
- Free parameters are shared. This demands more of them — they must capture information better. This is similar to the compression mechanism established in an autoencoder at the latent layer, or to adding a regularization term in the cost function.
- *Equivariant*: $g(f(x)) = f(g(x))$, where f is translation. This means it makes no difference whether one first translates and then tries to identify, or identifies first and then translates.
- Spatial Hierarchies: Hierarchical Feature Engineering. There is a hierarchical composition of structures that are similar regardless of scale.

20.9 Intelligibility

Intelligibility and Explainability

The definition given in 11.1.1 structures algorithms that learn relationships among existing data but do not assume any mechanistic model in their operation. This implies that by their very structure, it is difficult to understand why they arrive at any particular conclusion when classifying or performing regression. This is the black-box characteristic of all these algorithms [113], which was wonderfully anticipated by Turing himself in 1950 as a natural consequence of creating an automaton with the capacity to learn[a]. In contrast, what is required of these algorithms when the decisions made are consequential are two related but distinct principles. The first is the **Intelligible Property**, which

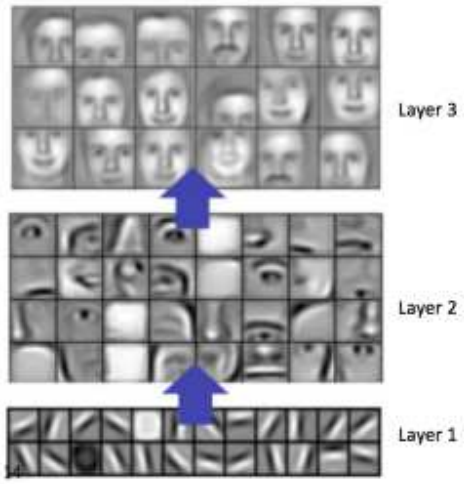


Figura 20.11: The different layers of a convolutional network capture increasingly abstract features of the information contained in the image, which also help explain the final result the network produces.

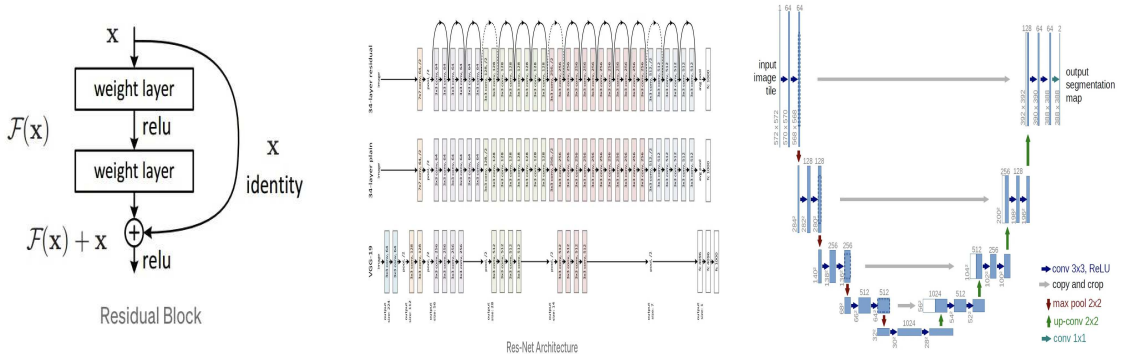
refers to the capacity of the scheme to offer an intermediate representation that allows a human being to understand what characteristic the algorithm identified to make a decision — for example, highlighting the zones characterizing pneumonia in an X-ray. On the other hand, the second property is **Explainability**, which is the capacity of the system to explain the step-by-step chain of reasoning that leads to a particular decision, directly modeled on how human beings can follow a chain of reasoning and give an explanation. These principles are often grouped under what is called **xAI**, Explainable Artificial Intelligence. Given the scope and social impact these technologies are having — as these models act as decision agents — it is necessary to take these points into consideration, and even go a step further and consider schemes that are also **Responsible AI**: that is, designing these systems while offering guarantees and operational bounds, aligning them with ethical and regulatory standards.

[a] "An important feature of a learning machine is that its teacher will often be very largely ignorant of quite what is going on inside, although he may still be able to some extent to predict his pupil's behaviour [15]".

The possibility of applying convolutional filters — weights to which a particular meaning can be assigned as filter coefficients — allows the intermediate layers of the network to capture behavior that helps both offer intelligibility and eventually generate a chain of explainability. This is well represented in what are called **Saliency Maps**, which capture which elements of the intermediate convolutional layers, if slightly varied, produce a drastic change in the network's output. Being represented in images, they are presented as information that can be directly understood by human beings.

20.10 Reusable Architectures

An excellent aspect of convolutional networks is that the different combinations of convolutional, ReLU, pooling, and fully connected layers allow building deep architectures that can be reused as modular components acting as building blocks of more complex networks. This makes them a very versatile technology and explains why they are at the heart of everything. For example, the weights of the convolutional layer are assumed to generate suitable filters for processing visual images; they are pre-trained and made available — as in the case of *Pretrained FC7 Features* — or as in the well-known,



(a) Residual connections (b) Structure of a ResNet (c) A U-shaped ResNet

Figura 20.12: Extensions of convolutional networks

downloadable, and reusable networks such as:

- LeNet
- AlexNet
- VGG

As always, the choice of the correct architecture does not escape the generalities of neural networks: a little art, a little educated guessing, and trial-and-error.

20.10.1 ResNet

Since images tend to be enormous, convolutional networks are truly deep and the VGP(?) becomes very important. One alternative to reduce its effects is to use what are called residual connections. That is, when passing through a layer, a direct residual connection is maintained — acting as a linear perceptron — which is added to the output of the convolutional layer itself. This allows the network to capture the change but minimizes the possibility of the gradient going to zero, since it keeps the values close to those taken at the input.

20.10.2 UNet

The first use given to convolutional networks was for classification, but they can be very useful for segmentation, or even for altering an image or completing it, as in **inpainting** situations. In this case, successive layers are required to obtain good filters over the image, but then additional layers are needed to recover that information in order to regenerate an image with characteristics similar to the original. Here, the ResNet idea is extended in UNets: the different convolutional layers capture important information from an image, but then the inverse operation — deconvolution — is applied, which successively generates images attempting to reverse the deconvolution process while using the filter outputs. To make this coherent, residual connections are added that come from the original images at each convolutional layer and produce the effect of recovering the information that existed in the image at that level. Hence the name of this network: what it produces is a U-shaped diagram where the outputs of each network layer are used to deepen the application of the filters, but are also used as input to regenerate the image in the deconvolution **up-scaling** on the other side of the U's valley.

20.11 Beyond Images

Convolutional networks are very versatile in terms of the type of data and problems they can be applied to, depending on how the dimensions are assigned. Table 20.11 shows some examples.

Dimensions	Single channel	Multi-channel
1-D	Audio	Electroencephalography Motion Capture Markers
2-D	Spectrograms	Color Images
3-D	Computed Tomography	Video

20.12 I Want More

- Fukushima Neocognitron [114].
- LeCun 1989, LeNet Architecture [115].
- Aggarwal's book Neural Networks and Deep Learning [116].
- Guide to understanding the arithmetic of convolutions [117, 118]
- Excellent CNN overview from a very broad perspective [119]
- AlexNet 2012 [120].
- https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/

21. Generative Adversarial Networks

GAN networks — Generative Adversarial Networks — emerged at approximately the same time as Variational Autoencoders, addressing the same problem from a different perspective: how to **generate** new samples from a set X. They were proposed by a group of researchers from the core of Deep Learning, including Ian Goodfellow and Yoshua Bengio. The story goes that Ian had been thinking about a **Discriminator** network that could evaluate whether different speech synthesis mechanisms were good or not, and during a get-together at a bar with some nerdy friends, discussing the complexities of how to efficiently generate images, the idea of combining a generator with a discriminator crossed his mind — using the information from the latter to precisely train and adjust the former. The kicker of the story is that that very same night he started coding up the GAN idea [121].

21.1 Generating Kittens

Typically, given for example a set of cat images — similar to what happened in Chapter 19 with Autoencoders — we want to identify the probability distribution that characterizes images of cats. That is, we want to find a generative model.

Definición 21.1.1 — Generative Model. A generative model describes the generation structure of a dataset X in terms of its probabilistic structure. By sampling from this model, it is possible to generate new samples X_j that will be probabilistically indistinguishable from the original samples X_i.

The problem is of course estimating that distribution. GAN networks [122] seek to solve this by positing an interaction between two networks. A first network is responsible for discriminating between real samples belonging to the set X — the real dataset — versus artificial samples. This discriminator is thus subjected to receiving input samples and, in a supervised manner, learning to decide whether each sample produces the output label 1 or -1, indicating whether it belongs or not to the real sample set or

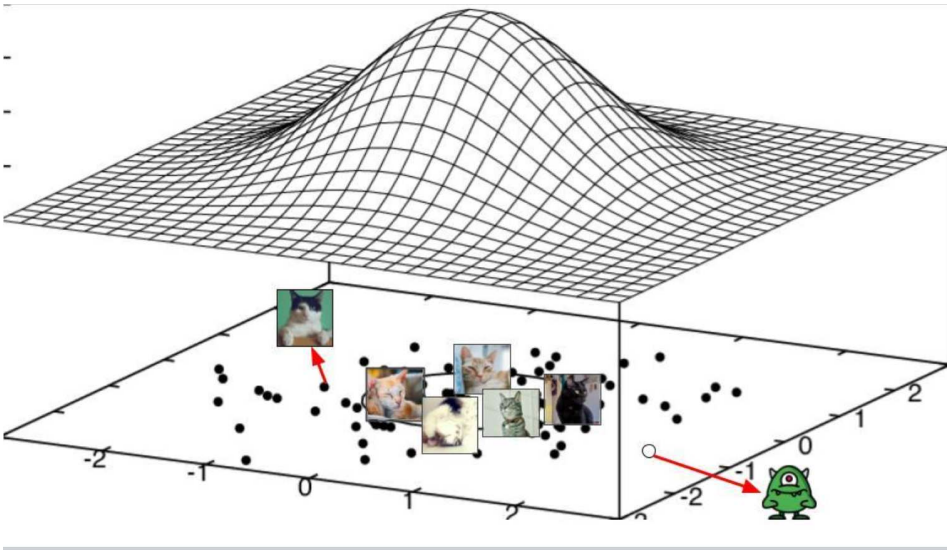


Figura 21.1: Generative model of kitten images

is artificial. We call this network the **Discriminator** D.

On the other hand, a second network that we will call the **Generator** will have the objective of producing the fake samples. This network operates in a manner very similar to the decoder of the Autoencoder. From a latent space, it obtains a random sample to which noise is added, and this passes through a network with free weights that will generate a new sample to be submitted to the discriminator for judgment.

The algorithm operates as follows:

1. Given a dataset X of valid real samples.
2. The Discriminator and Generator networks are initialized.
3. The Generator network is used to augment the dataset X with a set of artificial samples.
4. The Discriminator network is trained in a supervised manner to try to adjust the weights to predict whether the sample is real or artificial, until convergence.
5. With the Discriminator weights fixed, the Generator network is trained to produce samples such that the discriminator network accepts them as real. For this step, the Generator network + the Discriminator network are treated as a single network.

The training process continues alternating between training one network and the other, and of course the important thing is to define a good cost function that captures the underlying objective of this training.

21.2 The MinMax Game

There is a conflict at play here. One network wants one thing, and the other network wants exactly the opposite. The Discriminator wants to clearly identify which samples belong to the original dataset, while the Generator wants to produce a sample that the Discriminator cannot, precisely, discriminate. This conflict is mediated by a cost function that captures the full picture and finds a balance.

The elements of the equation are:

- $D =$ Discriminator
- $G =$ Generator

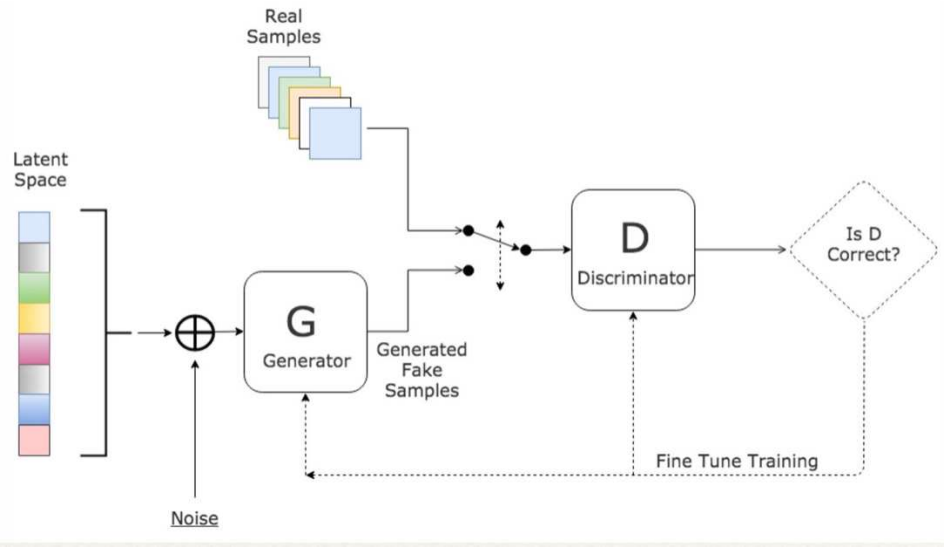


Figura 21.2: Architecture of a GAN network, composed of a Generator network and a Discriminator network that play an adversarial game.

- ϕ_d = Discriminator parameters
- ϕ_g = Generator parameters
- $P_z(z)$ = Distribution of the input noise to the generator.
- $P_{data}(x)$ = Distribution of the original data.
- $P_g(x)$ = Distribution of the generated data.

First the equation from the Discriminator's perspective:

$$L_D = H(y, D(x)) = \frac{1}{N}\{\sum_{i=1}^{N}(y_i)log(D(x_i)) + (1-y_i)log(1-D(x_i))\} \tag{21.1}$$

bearing in mind that y are the real labels while $D(x)$ are the labels assigned by the discriminator — the predictions.

What the discriminator wants is that when the sample is real, the first term equals 1 and the second term vanishes. This means the discriminator gets it right, and $D(x_i) = 1$. Conversely, when the sample is fake, the discriminator wants the first term to vanish and $1 - D(x_i) = 1$.

Equation 21.1 can be rewritten as:

$$\max_D H(y, D(x)) = \max_D \ \mathbb{E}_{x\sim P_{data}}\{log(D(x)\} + \mathbb{E}_{z\sim P_z}\{log(1-D(G(z)))\} \tag{21.2}$$

where what the Discriminator wants is to maximize that function in order to get it right when it should.

On the other hand, what the Generator wants, from its own perspective, is precisely to minimize Equation 21.2: to cause the Discriminator to fail to identify when samples have been generated by the Generator.

This is, simultaneously, optimizing:

$$\min_G \max_D V(D,G) = \min_G \max_D \Big\{\mathbb{E}_{x\sim P_{data}}\{log(D(x)\} + \mathbb{E}_{z\sim P_z}\{log(1-D(G(z)))\}\Big\} \tag{21.3}$$

In the paper [122], it is shown that the minimum of this equation occurs at a Nash equilibrium [37], where precisely what was established at the outset is achieved: the discriminator has no information to discriminate a sample from X from those generated by the generator $G(z)$ — it can only flip a coin — while the Generator produces new samples that have the same statistical distribution as the original data, that is, $P_z \rightarrow P_{data}$.

21.3 Mode Collapse

GAN networks have two major drawbacks. The first is that the order in which the alternating training of the Discriminator and the Generator is carried out is very important from a pragmatic standpoint for reaching the equilibrium point between the two networks, and consequently of the full GAN network. The second problem, which is related to the first, is that a situation can occur called *mode collapse*: the network finds an alternative solution point where different points in the latent space generate a single point — a single sample — instead of generating a full space or capturing the true distribution [123].

21.4 The Great Simulator

GANs present an interesting phenomenon. They begin to generate images that truly capture the essence present in the data, and allow us human beings to verify that with our own eyes. This can be seen in the work of several digital artists [124] as shown in Figure 21.3.

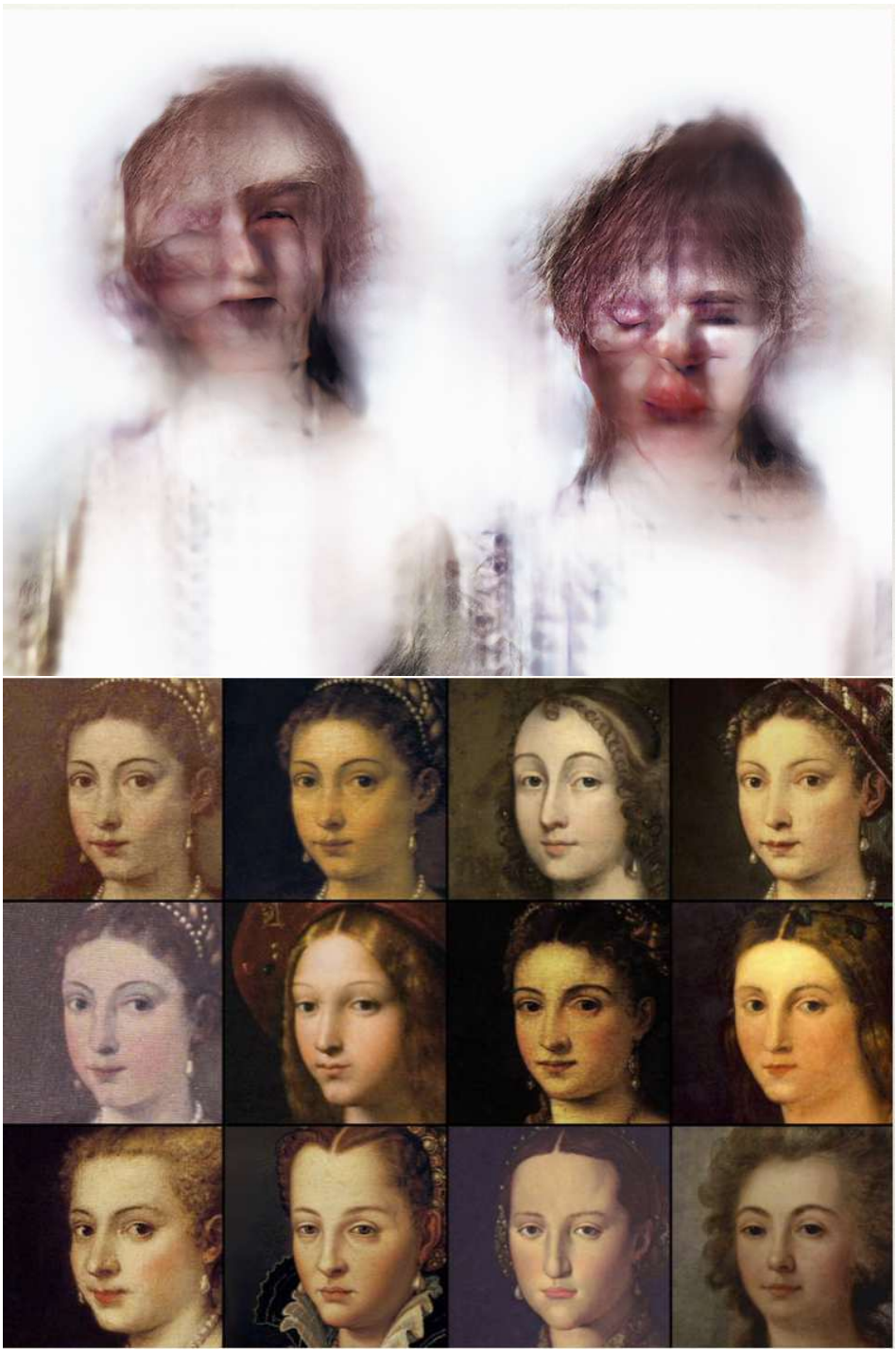


Figura 21.3: Images generated with GAN networks. In the image on the right it is difficult to identify which of those images is taken from a real frame and which were generated artificially.

22. Transformers

Transformers were proposed by [125] in the famous paper **Attention is all you need**. Their proposal was, at that moment, the culmination of many years of work and attempts to implement an adequate mechanism for Natural Language Processing (NLP) — the processing of text tokens in a sequence — while solving the vanishing/exploding gradient problem (VGP) of recurrent networks and the limitations of LSTM networks. A curious aspect, one that cannot escape the very hype that affects this entire field, is how opaque the literature is regarding many of these topics. That is why we are going to try to put things in black and white in terms of identifying where the names come from and what precisely they refer to.

22.1 Recurrent Networks

Recurrent neural networks have strong connections to complex, dynamic physical systems (see Chapter 15). The model of a recurrent neuron can be visualized in Figure 22.1 [126].

Recurrent networks are characterized by having an input x_t and an output y_t; as in the traditional

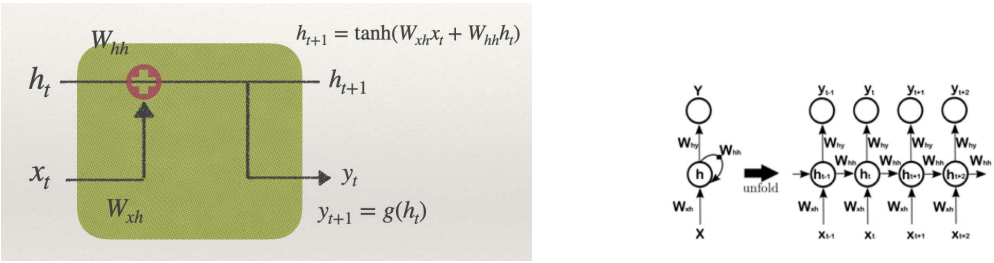


Figura 22.1: RNN, Recurrent Neural Networks. The network operates as a model with delay, with an additional output representing a state that serves as input for the next step.

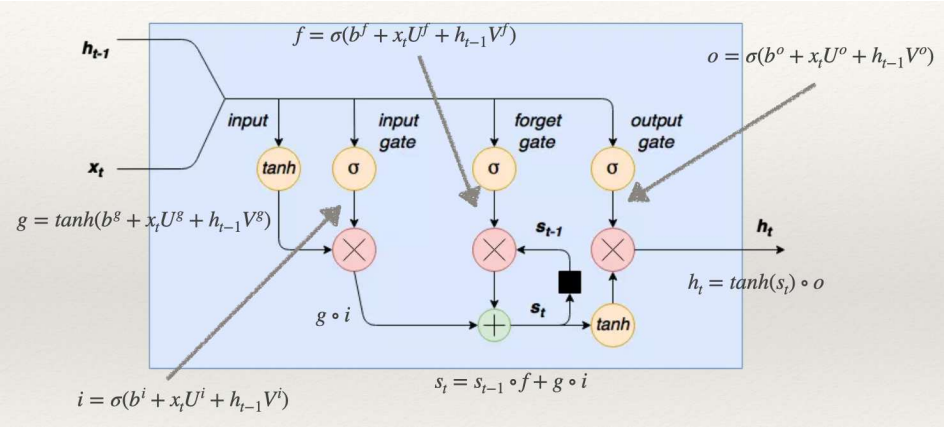


Figura 22.2: LSTM networks — Long Short Term Memory — were designed specifically to overcome the vanishing/exploding gradient problem through an activation mechanism for an internal memory, enabling causal inference chains that accommodate many temporal layers.

neuron model, the input is typically a vector while the output is a scalar. But additionally, recurrent neurons have an extra output representing an internal state that, after a delay, becomes an additional input to the next processing step. The dynamic scheme this sets up is discretized by the delay, receiving h_t as input and generating an output h_{t+1}. It may occur, as in Hopfield networks, that the output and the state are the same and not differentiated. Figure 22.1 shows the general scheme, where the input and the previous state are multiplied by weight matrices and the next state and the output are computed after applying a nonlinear function.

Recurrent networks are the embodiment of the vanishing/exploding gradient problem. In Figure 22.1, on the right side, the unfolding process is shown. Recurrent networks, by their dynamic nature, were conceived to process information in sequences — from time series or stochastic processes — for example, written language. Thus the first token of a sentence enters the network as input, affects the output in state h_{t+1}, which in turn propagates to the next state, where it receives a new input that again alters the state, and so on. This rapidly grows in the number of steps and naturally creates a numerical constraint on how to backpropagate the error far into the past over many, many iterations[1].

22.1.1 Long Short Term Memory

In 1997 Hochreiter and Schmidhuber [127] proposed a solution to this problem of recurrent networks, putting forward an alternative network that made intelligent use of neural activation gates, which allowed the network to store, retrieve, and discard relevant information in a memory — an internal state of the network — where the network, by automatically adjusting the weights, learned when to store, retrieve, or discard it; and where the storage in the internal state resolved the problem of having to backpropagate errors through many layers.

LSTM networks were very successful and became the de facto standard for predictive keyboards in so-called **feature phones** — GSM mobile phones with small physical keyboards designed for sending text messages. The limitation they encountered was that network training was complex and its generalization very restricted, since the internal memory mechanism found it difficult to achieve a

[1] In fact, the concept of *Deep* in Deep Learning refers both to the spatial depth of networks and to the temporal depth generated in recurrent networks, which can quickly grow very abruptly and exponentially with the problem.

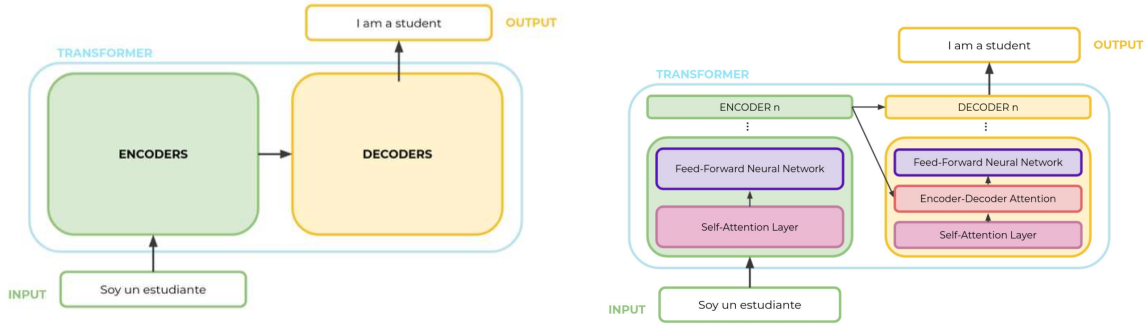


Figura 22.3: Basic component diagram of a Transformer.

broad range of coverage and became an excessive bottleneck.

22.2 Transformer Architecture

Transformers were designed to interpret sequences of tokens in written text. They have a general architecture determined by an **Encoder** and a **Decoder** — the nomenclature perhaps influenced by the Autoencoder architecture, though without any direct relationship. The Encoder's objective is to process the inputs, while the Decoder receives information from the input but also from the output generated so far.

Thinking of the Transformer as an architecture for solving the translation of text from one language to another reveals the justification for its different components.

The network first has a stack of Encoder and Decoder blocks. In the paper, 6 Encoder and 6 Decoder blocks are used.

Encoder: The Encoder receives the information from the input sequence. It has two components: the first corresponds to a layer that implements the **Attention** mechanism — specifically the **Self-Attention** mechanism — which will relate the tokens of the input to one another. The second layer is an MLP. The output passes to the subsequent Encoder in the stack.

Decoder: The final output of the last Encoder is the input to each of the Decoders, and they receive an additional input that comes from the output generated so far (for the first Decoder) and from the output of the previous Decoder (for all others). The Decoder has a Self-Attention layer, then an Encoder-Decoder-Attention layer (which takes the output of the previous layer against the inputs received from the Encoder), and finally an MLP. The final output of the last Decoder generates the current output — in this case, the current translation token.

The Encoders have the objective of finding the relationships between the tokens of the input. The first layer attends to token-against-token relationships. The second layer takes the relationships between pairs of tokens. And so on.

The Decoders then relate the inputs to the elements of the output generated so far (so the current output is the main input to the first Decoder in the Stack). The subsequent Decoders follow the same logic as the Encoders for processing n-tuples.

At the end of the Decoder chain, a linear layer is placed, followed by a SoftMax function, and finally the (numerical) tokens are re-decoded back into tokens by reversing the Embedding operation. The SoftMax layer determines what the output is for each of the possible tokens, and the idea is to

choose the one with the highest output value[2].

22.3 Attention is All You Need

The attention mechanism is so central to Transformers that it is almost synonymous with them, representing primarily the most fundamental detail of the architecture. It is an internal layer present in all Encoders and Decoders, whose function is to take into account other words in the sequence while a particular one is being analyzed. In the case of the Encoders, this layer allows focusing on any other word in the input, while the Decoders can only analyze previous words in the output sequence.

The attention layer allows relating the tokens of the language to one another. For example, in the sequence "I am a student", the word *student* is referencing the pronoun *I*. Therefore the weights — the free parameters in the attention mechanism — should be stronger between these two words.

There are three ways to view the attention mechanism.

$$SoftMax\left(\frac{QK'}{\sqrt{d_k}}\right) * V \tag{22.1}$$

with the definition of the SoftMax activation function as $\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_i}}$.

22.3.1 A Continuous Indexed Lookup Table

The nomenclature of the variables used in describing the attention mechanism derives from this approach. Suppose we have an indexed lookup table — a *hash table*. We need to retrieve a person's data based on their ID number. The *query* Q represents the person's ID number. We use this query Q to access an index and obtain the index value — the key K. Finally, with K we can access the actual table and ultimately retrieve V, the value we need.

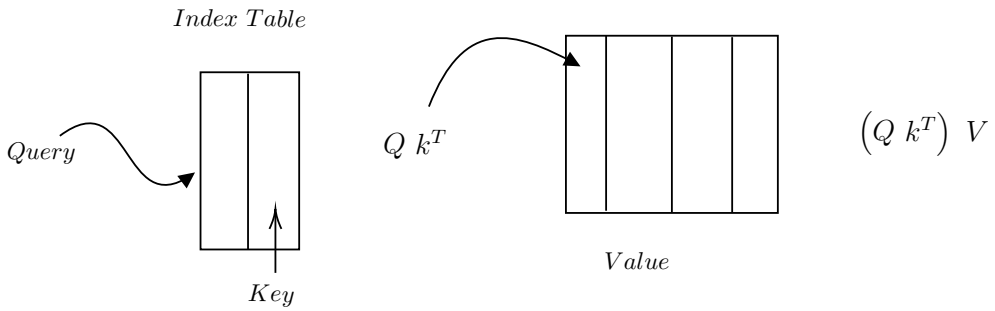


Figura 22.4: Continuous indexed access to a table.

Now we can think of this scheme in a continuous variant, where we do not access the exact entries in the table but instead establish a measure of similarity and look for the most similar ones. With this, for the query Q we need to find the most similar index in the table. Let us use the inner product as a similarity measure (others are possible). So to see which of all the indices k is the most similar, we

[2]Softmax can be viewed as a probability since the outputs sum to one, but it does not truly correspond to output probabilities since it is not calibrated.

compute $Q \cdot k^T$. Then with this result, we access the actual table to find the actual value that most closely matches this measure, and again we use the inner product, mediated by an activation function that rescales the values to identify the most similar among all: $(Q \cdot k^T) \cdot V$.

Finally, if we consider a scaling of the data and that they form a convex sum — to distribute the weight among all — we arrive again at:

$$\sigma\left(\frac{Q.k^T}{\sqrt{d_k}}\right) * V \tag{22.2}$$

What we obtain with this is a way to first identify the key — the index most similar to the query (the datum we have in focus). We then need to see what value, the Value, most closely matches the combination of the query with the key. The advantage of this is that we can add weights — free parameters — that will serve to mediate these calculations and thus establish learning mechanisms for these relationships.

$$\sigma\left(\frac{QW_q.(kW_k)^T}{\sqrt{d_k}}\right) * VW_v \tag{22.3}$$

where

- Q : Query, the input datum currently in focus.
- k : Key, the indices — the tokens that must be attended to.
- V : Value, the output values to be generated, weighted by the kernel.
- W_q, W_k, W_v: weights acting as free parameters.

22.3.2 Kernel Smoothing

Suppose we have a dataset X and its assumed desired outputs Y, the former being the input token sequence and the latter the output sequence. Given a target input x_o that we want to translate, a basic approach is to use **kNN** with $k = 1$: identify the nearest neighbor x_i and its corresponding y_i. If we also want information from other nearby neighbors, we can take a larger k and average the y_i values corresponding to several of those neighbors:

$$\hat{y_o} = \sum_{i=1}^{k} \frac{1}{k} y_i \tag{22.4}$$

The problem is that this does not take into account how close each neighbor is to the *query* x_o. What we can do instead is compute a weighted average, giving more weight to those neighboring points that are more similar to x_o. For this we add a **kernel** $K(x_i, x_o)$ that tells us how similar one is to the other. With this weighted average, the previous equation becomes:

$$\hat{y_o} = \sum_{i=1}^{k} \frac{K(x_i, x_o)}{\sum_{j=1}^{k} K(x_j, x_o)} y_i \tag{22.5}$$

This technique is called Kernel Smoothing [128]. Since the values x are vectors, any vector similarity function can be used as the kernel function — such as the inner product, or variants like $K(x_i, x_o) =$

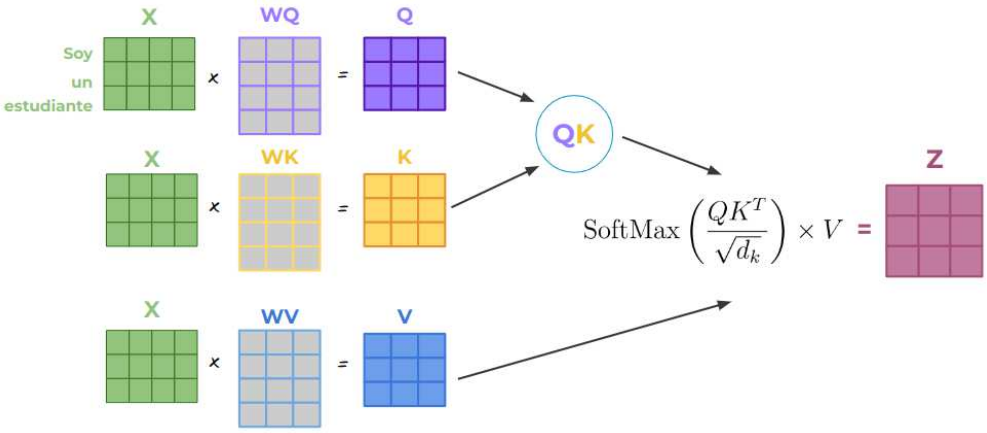


Figura 22.5: Basic Attention mechanism and its components.

$exp(x_i \cdot x_o)$, or another variant where the inner product can be defined via a matrix: $K(x_i, x_o) = exp(x_i \cdot w_1^T w_2 \cdot x_o)$. It can eventually be normalized by the vector dimension using $\sqrt{d}$.

If we substitute $x_o \rightarrow Q$, $x_i \rightarrow k^t$ and $y_i \rightarrow V$, we arrive at the attention mechanism equation, which from this perspective can be seen as a weighted average by similarity kernels between the key k^T and the query Q of the values V.

$$\hat{y_o} = \sum_{i=1}^{k} \frac{K(x_i, x_o)}{\sum_{j=1}^{k} K(x_j, x_o)} y_i \tag{22.6}$$

and to this the weights can be added as free parameters:

$$\left(\frac{exp(QW_q.(kW_k)^T.\sqrt{d_k}^{-1})}{\sum_{i=1}^{N} exp(QW_q.(kW_k)^T.\sqrt{d_k}^{-1}))} \right) * VW_v \tag{22.7}$$

22.3.3 Cognitive Attention

This last aspect refers to a biomimesis and parallel between the Transformer Attention mechanism and the human cognitive mechanism of *Cognitive Attention*: this refers to the process by which organisms select a subset of available information to concentrate, integrate, and process it more precisely. It has three distinctive aspects: orientation, filtering, and search. Attention is related to consciousness [129] and, unlike the **Attention** mechanism of Transformers, depends on an internal state.

All of this is very elegant, but we need to convert token sequences into numbers, and for that we need an encoding scheme.

22.4 Embedding

Embeddings are the different encoding schemes for tokens. The first process consists of dividing the input into components. In languages such as English or Spanish the division is more straightforward,

since for example the input could be divided into words. This process is called **tokenization**. The words (or *tokens*) are transformed into numbers based on a global encoding dictionary. Then each token is represented as a fixed-dimension numerical vector, capturing semantic and syntactic relationships based on its context in the language. For this, an embedding algorithm is applied. Several alternatives exist, such as *Word2Vec*, *CBOW*, *Bag of Words*, or even SVD.[3]

This scheme has the problem that the order is permutation-invariant, which we know is not a characteristic of language. Therefore the encoding must also take into account the position of the token in the sequence.

Positional Encoding: One possible solution is to add information to each word about its position in the sentence. This information is called Positional Encoding — a set of p_i values that are added to each embedding. While there are various ways to address this issue, in the paper by [125] the following sinusoidal functions are chosen to obtain the position vector of token i:

$$PE(pos, 2i) = \sin\left(\frac{pos}{1000^{\frac{2i}{d}}}\right) \tag{22.8}$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{1000^{\frac{2i}{d}}}\right) \tag{22.9}$$

where

- pos is the position of the token in the sequence.
- i is the index of the position vector.
- d is the dimension of an embedding, which will be the same as the dimension of a position vector p_i.

22.5 Training

For training, a vocabulary must be defined, including the *EOF* token. The vocabulary in the target language (e.g., English) is also needed. The Transformer structure is very versatile and can be adapted to many types of similar problems.

Weights are initialized randomly. The untrained model returns a probability distribution with entirely arbitrary values. It is therefore necessary to continue iterating in a supervised manner so that the output produces the desired word, thereby forcing the entire structure to learn [130].

[3] Note that we are discussing an example where a text is taken as input, but a token does not necessarily have to be a word — one can even work with images or audio signals.

VII Professional Pragmatism

23 The Science of Data 203
23.1 Quantitative Analysis
23.2 EDAs and ETLs
23.3 Data Science Fallacies
23.4 Artificial Intelligence and the Digital Frontier

24 Guide for the Impatient 209
24.1 Presentations
24.2 Ingredients of a Neural Network
24.3 Software Infrastructure
24.4 I Want More

Index 227

23. The Science of Data

23.1 Quantitative Analysis

During the first two decades of the 21st century, the digital revolution led corporations around the world — of every kind — to establish digital processes to carry out their enterprises. This generated a mountain of data.

Thus emerged in the IT world what are known as the areas of Data Mining, Data Warehousing, and the general field of Business Intelligence — that is, how to store and exploit this data to improve corporate decision-making and internal processes. In these developments, a new approach gained momentum, one in which the *intelligent* use of this information became central. To that end, certain practices and concepts for dealing with data — very typical of academia and the scientific method (at least some of them) — were adopted.

This happened at the confluence with a second line of development, now revitalized and generalized: the actuarial disciplines — a field with decades of applied mathematics, which developed models to study uncertainty, life expectancy, and credit risk. From this emerged the field of **Data Science**, along with the roles of data engineer, data analyst, and data scientist.

Figure 23.2 summarizes the different current areas of an organization from the perspective of this approach. First, the Data Science (DS) area centers on data and on offering tools to make decisions and derive processes from them. In today's organizations, data becomes central. The techniques for processing them today focus on Machine Learning (ML), which in turn divides into Deep Learning (DL) — the methods heavily based on bioinspired schemes of biological mechanisms, today strongly determined by hierarchical agglomerative neural networks — and the machine learning methods that are not deep learning (sometimes called *shallow learning*). This is the terrain where **data scientists** live. The central point, where truly the bulk of activity is concentrated, is **MLOps**: a digital organizational process centered on the entire path that goes from the actual generation of data, its storage, its processing, model maintenance and *governance*, and its execution. This is the primary terrain of the **data engineer**.

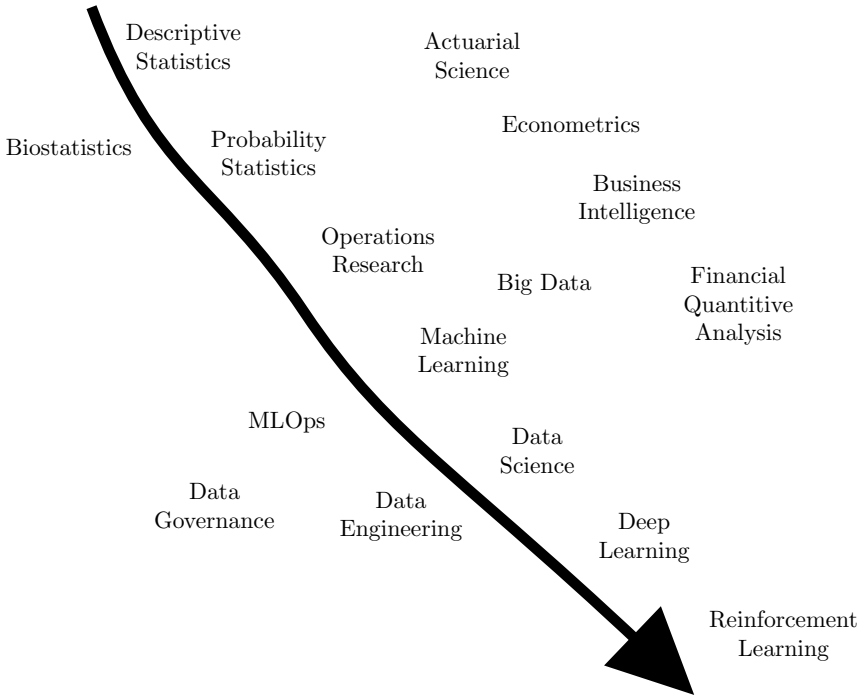


Figura 23.1: Map of the different areas linked to Data Science

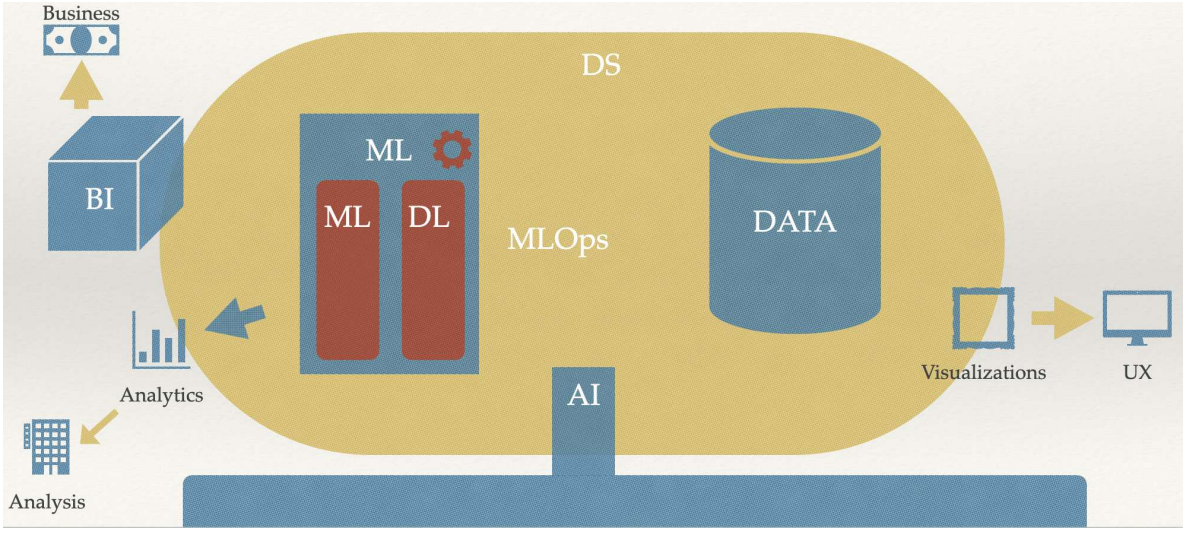


Figura 23.2: Data science as the nucleus of an organization.

Finally, the product of data analysis generates **insights** — this is where data returns to the real world, and there they generate real ideas that are analyzed precisely by **data analysts**: these are domain experts in the problem that the data represents, who, much more quantitatively than before, analyze the information and use it to make data-driven decisions: **analytics**.

This area, from a corporate perspective, requires a connection to design concepts through data **visualizations**, because it is precisely powerful visual abstractions that are needed to convey what the cold, raw data suggests. Finally, the field of **AI** makes its mark on this process, being responsible for pushing the digital frontier — capturing fresher data and executing decisions in the real world directly. The advances in Artificial Intelligence, especially in NLP, expanded the Data Science field and its roles to bring about the role of **AI Engineer**, who specializes precisely in applying these technologies for the digitalization of corporate processes.

23.2 EDAs and ETLs

The work and process of Data Science involves two very concrete and relevant stages that stand out from the rest. The first concerns ETL, *Extract, Transform and Load*, and relates to the activities of obtaining data from diverse and varied sources, and aggregating them into centralized and standardized databases. The ETL process also involves the complex and laborious activity of data curation[1]. It can involve developing elaborate SQL queries on databases, as well as sophisticated **web crawlers** or **scrapers** that navigate web pages extracting information from them.

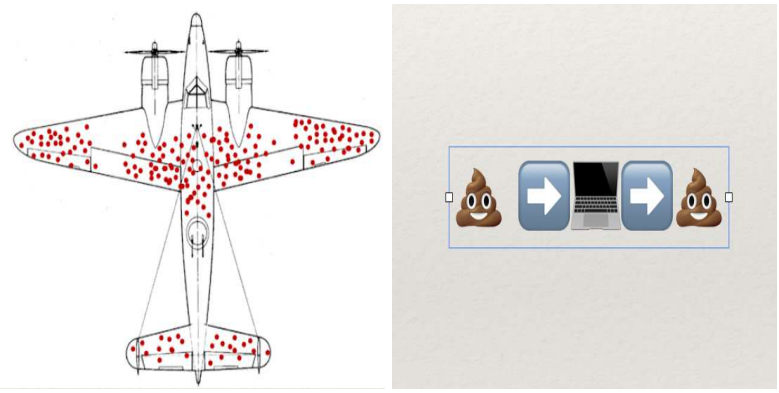
On the other hand, once the data are more standardized and curated, data analysts can embark on the activity known as EDA, *Exploratory Data Analysis*. This means understanding the inherent structure in the data through a set of tools that seeks transformations representing the information in various ways simultaneously, as well as visualizations that allow understanding both the statistical structure of the information and — with the specific context of what the data represents — help identify the possible patterns that need to be found.

23.3 Data Science Fallacies

In Data Science there are many fallacies and pitfalls inherited directly from the scientific world, where it is well recognized how easy it is to arrive at conclusions because **we deceive ourselves**. This is where the need for rigor appears — beginning with being aware of what kinds of fallacies we can fall into.

- **Biased Cherry Picking**: refers to selectively identifying results to support what we believe or want to say.
- **Data Dredging**: occurs when we assign greater importance than it truly deserves to a correlation that is due to chance.
- **Survivorship Bias**: conclusions are drawn based on the existence of samples that, by definition, exclude a large group simply because those are the only ones available. Example of the airplane and of those who respond to surveys. This is shown in Figure 23.3a.
- **Perverse Incentive**: while not specific to data analysis, it is specific to data-driven decisions. The fallacy manifests when one seeks to maximize a KPI, establishes a policy aimed at that, and precisely by maximizing the KPI produces the opposite effect to what was originally sought in the real world. Example of the cobras in India.

[1] The best word in local slang for this is **data cooking**.



(a) Survivorship Bias (b) Garbage In Garbage Out

Figura 23.3: (a) The red dots represent the bullet holes found on planes that returned to base, from which one might erroneously infer that those are the spots that need the most protection. (b) Any model can at best do what the input data allows it to.

- **False Causality**: when two things occur at the same time and a causal relationship is erroneously assigned to them.
- **Garbage In, Garbage Out**: this is the neural networks mantra. It points out that any data-based implementation will only ever be as good as the quality of the input data allows.
- **Sampling Bias**: techniques such as cross-validation or bootstrapping attempt to reduce the incidence of this fallacy.
- **Gambler's Fallacy**: assuming that the probability of occurrence in independent events increases (or varies) because they actually occurred (it is a fallacy because the conditional probability equals the unconditional probability). If I bring a bomb onto a plane, I reduce the chance of someone else bringing one.
- **Unbalanced Datasets**: very important in healthcare.
- **Simpson's Paradox**: a trend appears when the data are disaggregated but disappears entirely when they are grouped.
- **Clever Hans Effect**: very common in artificial neural networks and MLPs, when the network learns hidden patterns in the data that are not due to the phenomenon under study but to something indirectly associated with the outcome. There is a hidden information leakage in the data. For example, X-rays that reveal the patient's disease label because they are predominantly taken when the person is lying down [131].

This last fallacy — the most controversial of all — is saved for the end.

The Great Data Science Fallacy
Decisions are Data-Driven.

The reason this is a fallacy is that data without context, without the mantle of interpretability, say nothing. **You can torture your data until they tell you whatever you want to hear**. That is, data inevitably require interpretation, with a component of subjectivity that can and should be reduced, but which is not entirely eliminable. There is always some residual remainder. A similar situation arises

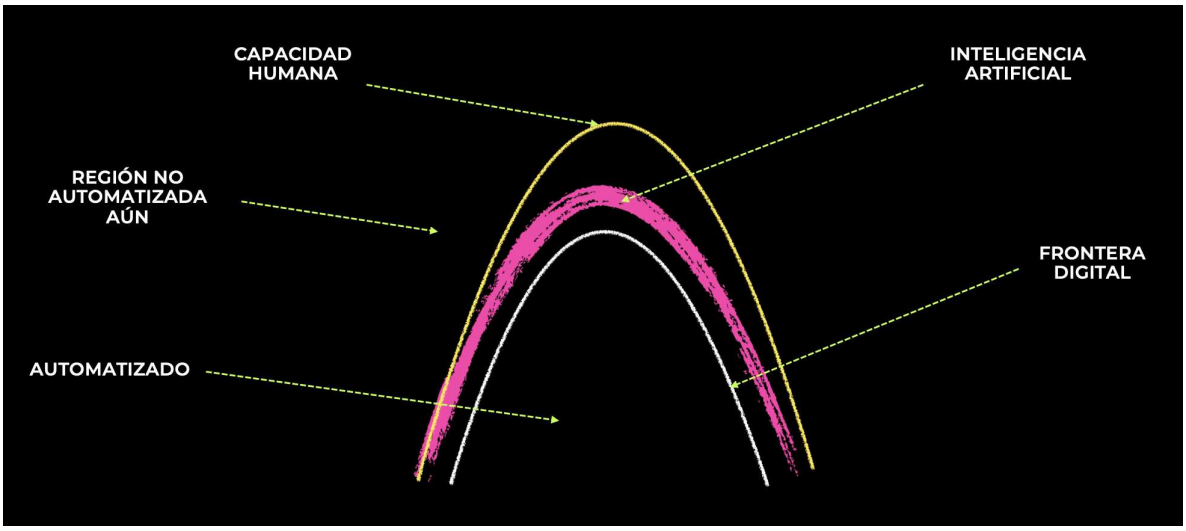


Figura 23.4: Artificial Intelligence understood as the digital frontier that marks the boundary between what is and is not automated.

with statistical hypothesis testing [132].

23.4 Artificial Intelligence and the Digital Frontier

Artificial Intelligence is, in addition to being a scientific discipline, a technological and product development concept. From this perspective, it is possible to invoke Figure 23.4. The inner region of that curve represents the activities for which human beings have managed to find a mechanism of automation — that is, we know how to implement them autonomously, like a row of dominoes that requires moving one initial piece and the rest is simply a sequence of fully automatic steps. We will call that frontier the *Digital Frontier*, because it is today through digitalization and computers that we are driving the development of technology that enables these automatisms.

The outer region is where we have not yet managed to find a way to automate — but not only that: it is the region for which there exist problems that human beings do not yet know how to solve, or that lie directly beyond our direct capacity to solve (or beyond the tools we have at our disposal so far). A new frontier then appears, generated by *human capability* — what we human beings are capable of (on average). From this perspective, artificial intelligence is the region immediately beyond the digital frontier. It is everything we do not yet know how to automate, but that sits immediately beyond what we do know, and for that reason we place it in our focus. This is consistent with the idea that artificial intelligence is a *moving goalpost* that has changed over time and has referred to different topics across the 7 decades of its existence [78]. For example, the area of Automata and Compilers used to fall within the field of artificial intelligence — until the problem became technically mature, an effective way of addressing it was found, it acquired a body of its own, moved the digital frontier, and ceased to belong to this field.

This perspective also allows us to understand that there may come a moment when the digital frontier line surpasses human capability, giving us what are called *superhuman* capabilities: automatic cognitive mechanisms that perform better than the average human being, or perhaps better than any human being. Consider, for example, a device designed to take an image of a stadium stand with

thousands of people and recognize every visible face within one minute. Human beings are good at recognizing faces, but not so good at achieving something like that.

At the digital frontier, and at the limit of what can be automated, there always lie the zones where innovation is being pushed: these are the areas where technical advancement allows generating a direct impact on society, creating products or services that can be channeled as commercial developments. Today (circa 2024) the topics occupying these spaces are precisely the areas of NLP, Computer Vision, Data Science, generative algorithms, and so on.

In line with this and associated with job market offerings, the Data Science field generated a subvariant called "AI Engineering." This area focuses primarily on implementations that explore the latest technical developments, especially in NLP and Computer Vision (on the generative networks side). This is understandable from a technology development perspective since the largest amount of information available in industries and companies is generally text [133].

24. Guide for the Impatient

24.1 Presentations

- The presentation should be approximately 20 minutes long, plus time for individual questions.
- Oriented toward results (which in turn are oriented toward data (which in turn are oriented toward experiments (which in turn are oriented toward implementations (which in turn are oriented toward understanding the topics (which in turn are oriented toward attending the lectures or the time invested in watching the videos))))))
- Avoid showing code (and even more so, screenshots of code).
- Evaluation of parameters.
- Representative graphs of the results.
- Since these are stochastic algorithms, remember to average across multiple runs and visualize the variance.
- Watch the scales on graphs. Keep them consistent when making comparisons.
- Fix a particular seed to reproduce results.
- Conclusions are very important and must not be missing.
- Submission: include the repository, with ppt, readme.md and config.json.
- AI Coding Literacy: rules and recommendations for getting the most out of coding assistants and agents `https://medium.com/@rramele/ai-coding-literacy-ce650eeb9ade`.
- More information [134].
- Videos of all classes: `https://www.youtube.com/playlist?list=PLJHMIS4ekxNRJlqsVzpmh6pWqBsI2FOGd`.

24.2 Ingredients of a Neural Network

When using a neural network to solve a problem, the following components must be decided.

Architecture: you need to decide the network architecture — the number of layers and the number of neurons per layer — as well as the different components and how they are chained together.

Activation Function: depending on the nature of the problem, understand which activation function should be the most appropriate. And above all, what the input and output value ranges of those functions will be.

Cost Function: choose the best cost function based on the objective given to the network: classify, regress, optimize, or generate. For example, the traditional error function 24.1 is the one used in MLPs but it is not the only one.

24.2.1 Data Transformation

There are different ways to preprocess and transform data, and the nomenclature is not very clear as to what each case involves. Here we offer a clarification by way of a summary and an arbitrary selection of the nomenclature used throughout this text.

- Normalize
- Min-Max Scaling
- Feature Scaling
- Standardization Z-Score
- Box-Cox.

Definición 24.2.1 — Normalization.

$$X_{new} = \frac{X}{||X||} \tag{24.1}$$

Definición 24.2.2 — Min-Max Scaling.

$$X_{new} = \frac{X_{old} - X_{min}}{X_{max} - X_{min}}(b-a) + a \tag{24.2}$$

Definición 24.2.3 — Feature Scaling.

$$X_{new} = \frac{X_{old}}{X_{max}}, X_{min} = 0, b = 1, a = 0 \tag{24.3}$$

Definición 24.2.4 — Z-Score, Standardization = Statistical Standardization, Standardize.

$$X_{new} = \frac{X_{old} - \bar{X}}{X_{std}} \tag{24.4}$$

where $\bar{X}$ is the mean and X_{std} the standard deviation.

Definición 24.2.5 — Box-Cox Normalization. Transforms the data through an optimization so that they look normal (i.e., drawn from a distribution $\mathcal{N}()$).

24.2.2 Class Balancing

Both when using supervised and unsupervised learning, data balancing — or the inherent bias — are important to consider in order to clearly evaluate results [135]. This is a well-known key aspect that is

nevertheless very often overlooked entirely. For example, when using a neural network with supervised learning to implement a binary classifier: if the data are extremely imbalanced — a population of hundreds of data points versus millions between one class and the other — a classifier that always assigns the label of the class with more samples will have extremely high efficiency that is completely useless.

24.2.3 Cost Functions

There are different cost functions that can be used in neural networks [136]. Table 24.1 shows some of the most commonly used ones covered in this text.

Name	Expression
SSE Adaline	$\frac{1}{2}\sum_{\mu=1}^{p}(\zeta^{\mu}-O^{\mu})$
RMS	$\sum_{i,\mu}\left\{\frac{1}{2}(1+\zeta_i^{\mu})\,log\frac{1+\zeta_i^{\mu}}{1+O_i^{\mu}}+\frac{1}{2}(1-\zeta_i^{\mu})\,log\frac{1-\zeta_i^{\mu}}{1-O_i^{\mu}}\right\}$
MAE	$\frac{1}{p}\sum_{\mu=1}^{p}\lvert\zeta^{\mu}-O^{\mu}\rvert$
Binary Cross Entropy	$-\frac{1}{p}\sum_{i=1}^{p}\{(y_i)\log p(y_i)+(1-y_i)\log(1-p(y_i))\}$
Huber	$\begin{cases}\frac{1}{p}\sum_{i=1}^{p}\frac{1}{2}(\zeta^{\mu}-O^{\mu})^2 & \lvert\zeta^{\mu}-O^{\mu}\rvert <= \delta \\ \frac{1}{p}\sum_{i=1}^{p}\delta(\lvert\zeta^{\mu}-O^{\mu}\rvert-\frac{1}{2}\delta) & \lvert\zeta^{\mu}-O^{\mu}\rvert > \delta\end{cases}$

Cuadro 24.1: Different error functions

24.2.4 Activation Functions

There are also various activation functions used in neural networks, which are primarily responsible for introducing nonlinearity. From the simplest and most traditional to the most complex, used in very specific scenarios — such as Softplus and Mish, which are smoothed versions of ReLU. Table 24.2 details those used in this text. The derivative here is expressed in terms of the inputs, not the outputs (that is, it is expressed in terms of the input to the activation function, not its output).

24.3 Software Infrastructure

As discussed in this document, the development of this discipline is built on the hardware that enabled the timely realization of many of the ideas and techniques also presented here. But it is equally important to highlight the software infrastructure assembled for this task, which allows structuring a processing *stack* upon which to build upward. The block diagram in Figure 24.1 shows the stack of platforms and libraries most widely used for developing Artificial Intelligence models and solutions.

CPU, GPU, and FPGA

On top of the traditional hardware substrate of CPUs, plus the current addition of GPUs, the possibility of programming on other devices such as FPGAs is also available. **F**ield **P**rogrammable **G**ate **A**rrays are general-purpose electronic devices that can be electronically modified to restructure

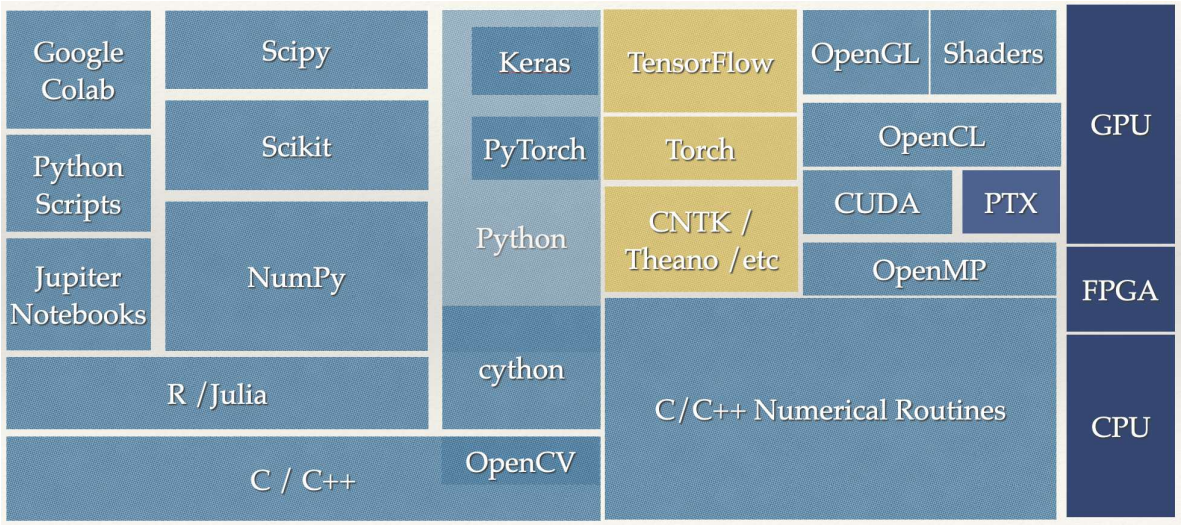


Figura 24.1: An overview of the software and hardware infrastructure that underpins the stack used for developing data analysis and artificial intelligence applications and systems. The majority are open-source libraries.

their circuits in a specific, hardwired configuration, maximizing efficiency. ASICs, **A**pplication **S**pecific **I**ntegrated **C**ircuits, are a variant that cannot be reprogrammed afterward. This level of specialization materializes as hardware specifically designed for the typical operations of artificial intelligence, in concrete hardware such as NPUs, **N**eural **P**rocessing **U**nits.

OpenGL and Shaders

OpenGL is the traditional open library for graphics programming. It is an old, highly specialized library oriented toward the creation of three-dimensional graphics. It encapsulates the complex logic required for 3D programming in a set of high-level instructions. The emergence of dedicated graphics hardware — the early days of GPUs — forced the creation of GLSL shaders, **G**raphics **L**ibrary **S**hading **L**anguage, which is code that compiles and runs directly on the GPU. This made it possible to resolve the dual processing logic where general game code runs primarily on a CPU, while graphics calculations — i.e., matrix multiplications — run directly on the GPU managed by GLSL, using standardized memory-copy interfaces between the two.

OpenMP and OpenCL

OpenMP is an API for parallel programming, designed precisely to allow simple porting of C++ programs to any type of platform. It is operating-system and hardware agnostic, and is also based on preprocessor directives, making it easy to modify existing programs. OpenCL, on the other hand, was conceived to run the same code on different types of processors — CPUs, GPUs, FPGAs, etc. This also makes it useful for building and developing programs that require high processing capacity.

CUDA and PTX

CUDA is one of the reasons for NVidia's dominance in the GPU market. It is a very complete software platform for programming directly using the parallelism capabilities of NVidia GPUs. This library allows programs to be structured to operate in a traditional manner on x86-64-like architectures, and from there derive code that operates on GPU memory, performing data exchanges between the

different memory spaces. PTX, **P**arallel **T**hread **E**xecution, is the assembly language for the virtual machine on which instructions run that are directly mapped to one or more NVidia GPUs. It is the intermediate language used by CUDA.

Python

This old scripting language has found its place and risen to the top podium as the lingua franca for data science and artificial intelligence activities. Beyond its simple syntax, one of the keys to this success is precisely the *cython* library, which enables interoperability between highly optimized numerical libraries built in Fortran or C++ [137].

NumPy

The most important of those libraries in the Python ecosystem is NumPy [137]. These are linear algebra routines for matrix multiplication that perform very efficient memory management. They are routines optimized to run on CPUs.

SciPy

This library is part of the combo, providing many practical routines for handling scientific data and information — particularly image and signal processing. It is a key library that encapsulates a great deal of functionality associated with the preprocessing and curation of scientific information.

Scikit-Learn

The base library of machine learning routines for Python. If a machine learning routine exists for Python, it is in this library.

Pandas

Pandas is a practical library that facilitates data curation and data engineering. Modeled and structured around the R language and platform — particularly the concept of the *dataframe* — this eases the use of Python for those already familiar with R, which is strongly oriented toward actuarial and statistical analysis. An important aspect is that this library places emphasis on the data and its associated metadata.

Matplotlib, Plotly, Seaborn, Altair

Python is a sea of libraries for doing practically everything. A basic task that arises in data processing and data-oriented AI is precisely the need to plot information. **Matplotlib** is the traditional library for doing so; although its function interface is quite unforgiving, you can do just about anything with it. **Plotly**, on the other hand, is much simpler to use but more limited in the functionality it implements. **Seaborn** is far more refined in terms of usability and the graphical design of visual elements, and at the same time works hand-in-hand with **Pandas**, making it very useful for highlighting information while also considering the associated metadata. However, a key aspect to consider is good design practices for scientific graphs, and from that perspective **Altair** is the clear winner, as it precisely respects different aesthetic and communicational efficiency guidelines [138].

TensorFlow and Keras

TensorFlow is one of the main libraries enabling the programming of neural network models and their execution on CPUs or GPUs, facilitating the use of low-level libraries such as **CUDA**, **OpenML**, and **OpenCL**, and establishing the abstractions of layers, activation functions, cost functions, etc., needed to build neural networks. TensorFlow also has a great secret: **AutoDiff** [139]. These libraries, in their various variants across different platforms, build the computational graph that determines how to

calculate the gradient based on the operations performed in the different functions of a program, and carry out the symbolic and numerical calculations to determine the gradient of the operation. This is of course key for building deep learning libraries. TensorFlow is a lower-level library for C++, and its Python port is Keras [53].

Torch and PyTorch

Torch is another popular library that provides the foundations and abstractions for working with deep neural networks. Its Python version, PyTorch, is used in many closed neural network implementations that are distributed as components — as is the case with convolutional neural networks.

OpenCV

OpenCV is an extraordinary, very comprehensive library that includes a large number of specific algorithms for image processing and computer vision. Among them, it includes many packages that predate the current deep learning libraries but share the same objectives. It is also a very useful tool for creating applications of all kinds.

R and Julia

R is a language designed and structured for statistical analysis, and for that reason it is also very popular in data science and artificial intelligence. *If a probability distribution does not appear in R, then it does not exist.* Julia, on the other hand, is a language specifically created for data processing and machine learning. Will it be the language and platform of the future for this field? Perhaps it will be — or perhaps it will be natural language itself — but for now the scene is dominated by old faithful Python.

24.4 I Want More

More? That's enough for today. Go outside, breathe some fresh air, and get a little sun.

Name	$g(h)$	$g'(h)$
Identity	h	1
Hyperbolic Tangent	$tanh(\beta h) = \frac{e^h - e^{-h}}{e^h + e^{-h}}$	$\beta(1 - g(h)^2)$
Sigmoid	$\frac{1}{1+exp^{2\beta x}}$	$2\,\beta g(h)(1 - g(h))$
Softsign	$\frac{1}{1+\lvert x\rvert}$	$\frac{1}{(1+\lvert x\rvert^2)}$
Step Function	$\begin{cases} 0 & h < 0 \\ 0.5 & h = 0 \\ 1 & h > 0 \end{cases}$	Dirac Delta $del(h)$
Sign Function	$\begin{cases} -1 & h < 0 \\ 0 & h = 0 \\ 1 & h > 0 \end{cases}$	$2\,del(h)$
Piecewise Linear	$\begin{cases} 1 & h >= \frac{1}{2} \\ h + \frac{1}{2} & -\frac{1}{2} < h < \frac{1}{2} \\ 0 & h <= -\frac{1}{2} \end{cases}$	$\begin{cases} 0 & h >= \frac{1}{2} \\ 1 & -\frac{1}{2} < h < \frac{1}{2} \\ 0 & h <= -\frac{1}{2} \end{cases}$
ReLU	$max(0,h)$	$\begin{cases} 0 & h < 0 \\ 1 & h >= 0 \end{cases}$
Softplus	$\frac{1}{\beta} ln(1 + e^{\beta\, h})$	$\frac{e^h}{1+e^h}$
Mish	$tanh(softplus(h))$	$\frac{e^h\,\omega}{\delta^2}, \omega = e^{3h} + e^{2h}(4) + e^h(6 + 4h) + (1+h), \delta = (e^h+1)^2 + 1$

Cuadro 24.2: Simple activation functions with their derivatives.

Name	$g(h)$	$g'(h)$
Softmax	$\sigma(\vec{V})_i = \frac{e^{V_i}}{\sum_{j=1}^{K} e^{V_i}}$	$\begin{cases} \sigma(\vec{V})_i(1-\sigma(\vec{V})_i) & i = k \\ -\sigma(\vec{V})_i(\sigma(\vec{V})_i) & i \neq k \end{cases}$

Cuadro 24.3: Simple activation functions with their derivatives (2/2).

Bibliografía

[1] Microsoft, *Neurips Conference Analytics*, 2024 (accessed Feb 27, 2024), https://www.microsoft.com/en-us/research/project/academic/articles/neurips-conference-analytics/.

[2] S. J. Russell and P. Norvig, *Artificial intelligence a modern approach*. London, 2010.

[3] J. A. Hertz, *Introduction to the theory of neural computation*. Crc Press, 2018.

[4] S. Haykin, *Neural networks: a comprehensive foundation*. Prentice Hall PTR, 1998.

[5] M. Mitchell, *An introduction to genetic algorithms*. MIT press, 1998.

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.

[7] C. C. Aggarwal *et al.*, "Neural networks and deep learning," *Springer*, vol. 10, no. 978, p. 3, 2018.

[8] S. Axler, *Linear algebra done right*. Springer Nature, 2023.

[9] J. Gallier and J. Quaintance, "Algebra, topology, differential calculus, and optimization theory for computer science and engineering," 2019.

[10] S. P. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[11] A. M. Turing *et al.*, "On computable numbers, with an application to the entscheidungsproblem," *J. of Math*, vol. 58, no. 345-363, p. 5, 1936.

[12] P. Stone, *The Artificial Intelligence Papers*. Sebtel Press, 2024.

[13] W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *The bulletin of mathematical biophysics*, vol. 5, pp. 115–133, 1943.

[14] R. E. Brown, "The life and work of donald olding hebb," *Acta Neurologica Taiwanica*, vol. 15, no. 2, p. 127, 2006.

[15] A. M. Turing, *Computing machinery and intelligence*. Springer, 1950.

[16] D. Taraborrelli and R. Gala, "Génesis y actualidad de la inteligencia artificial (ia) en las instituciones públicas de la argentina, una mirada desde los escyt," in *XI Jornadas de Sociología. Facultad de Ciencias Sociales, Universidad de Buenos Aires, Buenos Aires, 2015*. Acta Académica, 2015, p. 01.

[17] P. N. Johnson-Laird and M. Ragni, "What should replace the turing test?" *Intelligent Computing*, vol. 2, p. 0064, 2023.

[18] A. Newell and H. Simon, "The logic theory machine–a complex information processing system," *IRE Transactions on information theory*, vol. 2, no. 3, pp. 61–79, 1956.

[19] P. Stone, R. Brooks, E. Brynjolfsson, R. Calo, O. Etzioni, G. Hager, J. Hirschberg, S. Kalyanakrishnan, E. Kamar, S. Kraus *et al.*, "Artificial intelligence and life in 2030: the one hundred year study on artificial intelligence," 2016.

[20] J. E. Hopcroft and J. D. Ullman, *Formal languages and their relation to automata*. Addison-Wesley Longman Publishing Co., Inc., 1969.

[21] N. Wiener, "Cybernetics or control and communication," *The animal and the machine*, 1948.

[22] R. Solomonoff, "Dartmouth archives," https://raysolomonoff.com/dartmouth/, 2024 (accessed Jan 10, 2025).

[23] B. Bloomfield, "The question of artificial intelligence," https://www-formal.stanford.edu/jmc/reviews/bloomfield/bloomfield.html, 2024 (accessed Jan 10, 2025).

[24] N. Nillson, *The quest for Artificial Intelligence*. Standford University, 2010.

[25] S. L. Brunton and J. N. Kutz, *Data-driven science and engineering: Machine learning, dynamical systems, and control*. Cambridge University Press, 2022.

[26] R. McNaughton, "The theory of automata, a survey," in *Advances in Computers*, ser. Advances in Computers, F. L. Alt, Ed. Elsevier, 1961, vol. 2, pp. 379–421. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0065245808601448

[27] C. Bassett, "The computational therapeutic: exploring weizenbaum's eliza as a history of the present," *AI & SOCIETY*, vol. 34, no. 4, pp. 803–812, 2019.

[28] X. Yang and C. Zhu, "Industrial expert systems review: a comprehensive analysis of typical applications," *IEEE Access*, 2024.

[29] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *nature*, vol. 323, no. 6088, pp. 533–536, 1986.

[30] M. Minsky and S. Papert, "An introduction to computational geometry," *Cambridge tiass., HIT*, vol. 479, no. 480, p. 104, 1969.

[31] R. Kurzweil, "The singularity is near," in *Ethics and emerging technologies*. Springer, 2005, pp. 393–406.

[32] E. Strickland, "The turbulent past and uncertain future of ai: Is there a way out of ai's boom-and-bust cycle?" *IEEE Spectrum*, vol. 58, no. 10, pp. 26–31, 2021.

[33] T. Urban, *Wait but Why: AI Superintelligence*, 2022 (accessed July 1, 2022), https://waitbutwhy.com/2015/01/artificial-intelligence-revolution-1.html.

[34] S. Legg, M. Hutter *et al.*, "A collection of definitions of intelligence," *Frontiers in Artificial Intelligence and applications*, vol. 157, p. 17, 2007.

[35] A. Ciaunica, A. Safron, and J. Delafield-Butt, "Back to square one: the bodily roots of conscious experiences in early life," *Neuroscience of Consciousness*, vol. 2021, no. 2, p. niab037, 2021.

[36] A. Clark, *Supersizing the mind: Embodiment, action, and cognitive extension*. OUP USA, 2008.

[37] S. Tadelis, *Game theory: an introduction*. Princeton university press, 2013.

[38] P. A. Tipler, *Physics for Scientists and Engineers: Regular Version, Ch. 1-35 and 39*. Macmillan, 1999.

[39] T. Virkkala, "Solving sokoban," Ph.D. dissertation, Master's thesis, University of Helsinki, 2011.

[40] J. H. Holland, "Genetic algorithms and the optimal allocation of trials," *SIAM journal on computing*, vol. 2, no. 2, pp. 88–105, 1973.

[41] N. Ashton and C. Stringer, "Did our ancestors nearly die out?" *Science*, vol. 381, no. 6661, pp. 947–948, 2023.

[42] D. L. Goodstein, *States of matter*. Courier Corporation, 2014.

[43] P. Pedregal, *Introduction to Optimization*. Springer, 2003.

[44] H. A. Taha, *Investigación de operaciones*. Pearson Educación, 2004.

[45] R. P. Brent, *Algorithms for minimization without derivatives*, Prentice-Hall, Ed. Englewood Cliffs, N.J., Prentice-Hall, 1973.

[46] J. A. Snyman, D. N. Wilke *et al.*, *Practical mathematical optimization*. Springer, 2005.

[47] P. Moritz, R. Nishihara, and M. Jordan, "A linearly-convergent stochastic l-bfgs algorithm," in *Artificial Intelligence and Statistics*. PMLR, 2016, pp. 249–258.

[48] L. Bottou, *Online Algorithms and Stochastic Approximations. Online Learning and Neural Networks.*, C. U. Press., Ed. Cambridge University Press., 1998.

[49] M. Lefkowitz, *Perceptron pave way to ai 60 years ago*, 2024 (accessed July 1, 2024), https://news.cornell.edu/stories/2019/09/professors-perceptron-paved-way-ai-60-years-too-soon.

[50] E. Agliari, M. Aquaro, A. Barra, A. Fachechi, and C. Marullo, "From pavlov conditioning to hebb learning," *Neural Computation*, vol. 35, no. 5, pp. 930–957, 2023.

[51] K. Shirriff, "The surprising story of the first microprocessors," *IEEE Spectrum*, vol. 53, no. 9, pp. 48–54, 2016.

[52] D. H. Hubel and T. N. Wiesel, "Receptive fields, binocular interaction and functional architecture in the cat's visual cortex," *The Journal of physiology*, vol. 160, no. 1, p. 106, 1962.

[53] F. Chollet, *Deep learning with Python*. Simon and Schuster, 2021.

[54] G. Hinton, N. Srivastava, and K. Swersky, "Neural networks for machine learning lecture 6a overview of mini-batch gradient descent," *Cited on*, vol. 14, no. 8, p. 2, 2012.

[55] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[56] T. Hastie, R. Tibshirani, J. H. Friedman, and J. H. Friedman, *The elements of statistical learning: data mining, inference, and prediction*. Springer, 2009, vol. 2.

[57] R. Roelofs, V. Shankar, B. Recht, S. Fridovich-Keil, M. Hardt, J. Miller, and L. Schmidt, "A meta-analysis of overfitting in machine learning," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[58] V. N. Vapnik, "Statistical learning theory," 1998.

[59] C. F. Caiafa, E. Salerno, and A. N. Proto, "Blind source separation applied to spectral unmixing: comparing different measures of nongaussianity," in *Knowledge-Based Intelligent Information and Engineering Systems: 11th International Conference, KES 2007, XVII Italian Workshop on Neural Networks, Vietri sul Mare, Italy, September 12-14, 2007, Proceedings, Part III 11*. Springer, 2007, pp. 1–8.

[60] E. Schubert, J. Sander, M. Ester, H. P. Kriegel, and X. Xu, "Dbscan revisited, revisited: why and how you should (still) use dbscan," *ACM Transactions on Database Systems (TODS)*, vol. 42, no. 3, pp. 1–21, 2017.

[61] T. Kohonen, "Self-organized formation of topologically correct feature maps," *Biological Cybernetics*, vol. 1, no. 43, pp. 59–69, 1982.

[62] ——, "The self-organizing map." *Neurocomputing*, pp. 1–6, 1998.

[63] B. Everitt, G. Dunn *et al.*, *Applied multivariate data analysis*. Wiley Online Library, 2001, vol. 2.

[64] R. S. A. of Sciences, *Nobel Prize in Physics 2024*, 2024 (accessed October 17, 2024), https://www.nobelprize.org/prizes/physics/2024/press-release/.

[65] A. K. John Hertz and R. Palmer, *Introduction to the Theory of Neural Computation*, Addison-Wesley, Ed. Addison-Wesley, 1991.

[66] M. Scilipoti, M. Fuster, and R. Ramele, "A strong inductive bias: Gzip for binary image classification," *arXiv preprint arXiv:2401.07392*, 2024.

[67] E. Oja, "Principal components, minor components, and linear neural networks," *Neural networks*, vol. 5, no. 6, pp. 927–935, 1992.

[68] T. D. Sanger, "Optimal unsupervised learning in a singlelayer linear feedforward neural network," *Neural Networks*, vol. 2, no. 6, pp. 459–473, 1989.

[69] O. Sacks, *The man who mistook his wife for a hat and other clinical tales*. Simon And Schuster, 1985.

[70] R. E. Bryant and D. R. O'Hallaron, *Computer systems: a programmer's perspective*. Prentice Hall, 2011.

[71] M. Krzisch, S. G. Temprana, L. A. Mongiat, J. Armida, V. Schmutz, M. A. Virtanen, J. Kocher-Braissant, R. Kraftsik, L. Vutskits, K.-K. Conzelmann *et al.*, "Pre-existing astrocytes form functional perisynaptic processes on neurons generated in the adult hippocampus," *Brain Structure and Function*, vol. 220, pp. 2027–2042, 2015.

[72] W. Elmenreich, "An introduction to sensor fusion," *Vienna University of Technology, Austria*, vol. 502, pp. 1–28, 2002.

[73] A. L. Hodgkin and A. F. Huxley, "A quantitative description of membrane current and its application to conduction and excitation in nerve," *The Journal of physiology*, vol. 117, no. 4, p. 500, 1952.

[74] R. Brooks, "An inconvenient truth about ai: Ai won't surpass human intelligence anytime soon," *IEEE Spectrum*, 2021.

[75] A. L. Samuel, "Some studies in machine learning using the game of checkers," *IBM Journal of research and development*, vol. 3, no. 3, pp. 210–229, 1959.

[76] L. Acion, L. Alemany Alonso, E. Ferrante, E. L. Holm, V. Martinez, D. H. Milone, R. Rodriguez, G. Simari, and S. Uchitel, "Desmistificando la inteligencia artificial," *Inteligencia Artificial, Una Mirada Interdisciplinaria X Encuentro Interacademico 2021*, vol. 1, no. 1, pp. 63–86, 2021.

[77] M. Mitchell, "Why AI is harder than we think," *CoRR*, vol. abs/2104.12871, 2021. [Online]. Available: https://arxiv.org/abs/2104.12871

[78] P. Stone, R. Brooks, E. Brynjolfsson, R. Calo, O. Etzioni, G. Hager, J. Hirschberg, S. Kalyanakrishnan, E. Kamar, S. Kraus *et al.*, "Artificial intelligence and life in 2030: the one hundred year study on artificial intelligence," *arXiv preprint arXiv:2211.06318*, 2022.

[79] J. Degrave, F. Felici, J. Buchli, M. Neunert, B. Tracey, F. Carpanese, T. Ewalds, R. Hafner, A. Abdolmaleki, D. de Las Casas *et al.*, "Magnetic control of tokamak plasmas through deep reinforcement learning," *Nature*, vol. 602, no. 7897, pp. 414–419, 2022.

[80] M. Simon, "How a plucky robot found the long-lost endurance shipwreck," https://www.wired.com/story/how-a-plucky-robot-found-the-long-lost-endurance-shipwreck, 2022 (accessed Jul 18, 2024).

[81] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F. J. R Ruiz, J. Schrittwieser, G. Swirszcz *et al.*, "Discovering faster matrix multiplication algorithms with reinforcement learning," *Nature*, vol. 610, no. 7930, pp. 47–53, 2022.

[82] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko *et al.*, "Highly accurate protein structure prediction with alphafold," *nature*, vol. 596, no. 7873, pp. 583–589, 2021.

[83] R. Sutton, "The bitter lesson," http://www.incompleteideas.net/IncIdeas/BitterLesson.html, 2019 (accessed May 29, 2024).

[84] P. Domingos, "A few useful things to know about machine learning," *Communications of the ACM*, vol. 55, no. 10, pp. 78–87, 2012.

[85] O. Boiman, E. Shechtman, and M. Irani, "In defense of nearest-neighbor based image classification," *26th IEEE Conference on Computer Vision and Pattern Recognition, CVPR*, 2008, this paper is the NBNN method which is similar to what I am doing with Sift descriptors.

[86] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of control, signals and systems*, vol. 2, no. 4, pp. 303–314, 1989.

[87] T. Poggio, H. Mhaskar, L. Rosasco, B. Miranda, and Q. Liao, "Why and when can deep-but not shallow-networks avoid the curse of dimensionality: A review," *International Journal of Automation and Computing*, vol. 14, no. 5, pp. 503–519, 2017. [Online]. Available: https://doi.org/10.1007/s11633-017-1054-2

[88] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural networks*, vol. 61, pp. 85–117, 2015.

[89] G. E. Hinton, S. Osindero, and Y.-W. Teh, "A fast learning algorithm for deep belief nets," *Neural computation*, vol. 18, no. 7, pp. 1527–1554, 2006.

[90] S. Hooker, "The hardware lottery," *Communications of the ACM*, vol. 64, no. 12, pp. 58–65, 2021.

[91] D. Garisto, "How cutting-edge computer chips are speeding up the ai revolution," *Nature*, vol. 630, no. 8017, pp. 544–546, 2024.

[92] M. Pandey, M. Fernandez, F. Gentile, O. Isayev, A. Tropsha, A. C. Stern, and A. Cherkasov, "The transformational role of gpu computing and deep learning in drug discovery," *Nature Machine Intelligence*, vol. 4, no. 3, pp. 211–221, 2022.

[93] M. A. Kramer, "Nonlinear principal component analysis using autoassociative neural networks," *AIChE journal*, vol. 37, no. 2, pp. 233–243, 1991.

[94] D. P. Kingma and M. Welling, "An introduction to variational autoencoders," *Foundations and Trends in Machine Learning*, vol. 12, no. 4, pp. 307–392, 2019. [Online]. Available: http://dx.doi.org/10.1561/2200000056

[95] K. P. Murphy, *Machine learning: a probabilistic perspective*. MIT press, 2012.

[96] jhu edu., *Variational Tutorial*, 2011 (accessed September 3, 2018), http://www.cs.jhu.edu/~jason/tutorials/variational.html.

[97] C. W. Fox and S. J. Roberts, "A tutorial on variational bayesian inference," *Artificial intelligence review*, vol. 38, no. 2, pp. 85–95, 2012.

[98] M. E. Khan and G. Bouchard, "Variational em algorithms for correlated topic models," Technical report, University of British Columbia, Tech. Rep., 2009.

[99] C. Tang and R. R. Salakhutdinov, "Learning stochastic feedforward neural networks," *Advances in Neural Information Processing Systems*, vol. 26, 2013.

[100] E. W. Weisstein, *Stochastic Function*, 2023 (accessed November 17, 2023), https://mathworld.wolfram.com/StochasticFunction.html.

[101] J. R. Anderson and C. Peterson, "A mean field theory learning algorithm for neural networks," *Complex Systems*, vol. 1, pp. 995–1019, 1987.

[102] M. J. Wainwright and M. I. Jordan, *Graphical models, exponential families, and variational inference*. Now Publishers Inc, 2008.

[103] G. E. Hinton and D. Van Camp, "Keeping the neural networks simple by minimizing the description length of the weights," in *Proceedings of the sixth annual conference on Computational learning theory*, 1993, pp. 5–13.

[104] G. Gundersen., *Reparamatrization Trick*, 2021 (accessed October 2, 2021), https://gregorygundersen.com/blog/2018/04/29/reparameterization/.

[105] B. Rocca, *Stack Exchange, Machine Learning*, 2021 (accessed October 2, 2021), https://stats.stackexchange.com/questions/420974/backpropagation-on-variational-autoencoders.

[106] E. S. Piñeiro, R. Ramele, and J. Gambini, "Variational autoencoder as a data augmentation tool for confocal microscopy images," in *2023 IEEE 36th International Symposium on Computer-Based Medical Systems (CBMS)*. IEEE, 2023, pp. 882–885.

[107] D. M. Blei, A. Kucukelbir, and J. D. McAuliffe, "Variational inference: A review for statisticians," *Journal of the American Statistical Association*, vol. 112, no. 518, pp. 859–877, Feb 2017. [Online]. Available: http://dx.doi.org/10.1080/01621459.2017.1285773

[108] C. Doersch, "Tutorial on variational autoencoders," 2021.

[109] C. C. Aggarwal, *Neural Networks and Deep Learning*. Cham: Springer, 2018.

[110] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2014.

[111] J. Jordan, *Autoencoders*, 2022 (accessed Novemenber 1, 2022), https://www.jeremyjordan.me/autoencoders/.

[112] G. W. Lindsay, "Convolutional neural networks as a model of the visual system: Past, present, and future," *Journal of Cognitive Neuroscience*, pp. 1–15, Feb 2020. [Online]. Available: http://dx.doi.org/10.1162/jocn_a_01544

[113] B. E. Ail, R. Ramele, J. Gambini, and J. M. Santos, "An intrinsically explainable method to decode p300 waveforms from eeg signal plots based on convolutional neural networks," *Brain Sciences*, vol. 14, no. 8, p. 836, 2024.

[114] K. Fukushima and N. Wake, "Handwritten alphanumeric character recognition by the neocognitron," *IEEE transactions on Neural Networks*, vol. 2, no. 3, pp. 355–365, 1991.

[115] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural computation*, vol. 1, no. 4, pp. 541–551, 1989.

[116] C. C. Aggarwal *et al.*, *Neural networks and deep learning*. Springer, 2018.

[117] V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning," mar 2016. [Online]. Available: http://arxiv.org/abs/1603.07285

[118] E. Shelhamer, J. Long, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 4, pp. 640–651, nov 2017. [Online]. Available: http://arxiv.org/abs/1411.4038

[119] S. Jiang and V. M. Zavala, "Convolutional Neural Nets: Foundations, Computations, and New Applications," jan 2021. [Online]. Available: http://arxiv.org/abs/2101.04869

[120] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012, pp. 1097–1105.

[121] D. Foster, *Generative deep learning*. O'Reilly Media, Inc., 2022.

[122] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," *Advances in neural information processing systems*, vol. 27, 2014.

[123] M. Kahng, N. Thorat, P. Chau, F. Viégas, and M. Wattenberg, *Gan Lab*, 2023 (accessed November 17, 2023), https://poloclub.github.io/ganlab/.

[124] M. Klingermann, "Quasimondo," https://quasimondo.com/, 2024 (accessed Jul 18, 2024).

[125] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[126] H. T. Siegelmann and E. D. Sontag, "On the computational power of neural nets," in *Proceedings of the fifth annual workshop on Computational learning theory*, 1992, pp. 440–449.

[127] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[128] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, "Transformer dissection: a unified understanding of transformer's attention via the lens of kernel," *arXiv preprint arXiv:1908.11775*, 2019.

[129] R. Wright and W. LM, *Orienting of Attention*, 2008.

[130] E. Fedorenko, S. T. Piantadosi, and E. A. Gibson, "Language is primarily a tool for communication rather than thought," *Nature*, vol. 630, no. 8017, pp. 575–586, 2024.

[131] J. Kauffmann, L. Ruff, G. Montavon, and K.-R. Müller, "The clever hans effect in anomaly detection," *arXiv preprint arXiv:2006.10609*, 2020.

[132] D. Huff, *How to lie with statistics*. Penguin UK, 2023.

[133] A. Said and V. Torra, *Data science in practice*. Springer, 2019.

[134] E. Piñeiro, "SIA tips," https://github.com/eugepineiro/sia-tips, 2022 (accessed September 3, 2023).

[135] A. Zhou, F. Tajwar, A. Robey, T. Knowles, G. J. Pappas, H. Hassani, and C. Finn, "Do deep networks transfer invariances across classes?" in *International Conference on Learning Representations*, 2021.

[136] M. Brooks, "Inside the maths that drives ai," *Nature*, vol. 631, no. 8019, pp. 244–246, 2024.

[137] C. R. Harris, K. J. Millman, S. J. Van Der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith *et al.*, "Array programming with numpy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[138] L. Wilkinson, "The grammar of graphics," in *Handbook of computational statistics: Concepts and methods*. Springer, 2011, pp. 375–414.

[139] N. Ketkar, J. Moolayil, N. Ketkar, and J. Moolayil, "Automatic differentiation in deep learning," *Deep Learning with python: learn best practices of deep learning models with PyTorch*, pp. 133–145, 2021.

Índice alfabético

A

A* search 34
Agent 31, 41
Allele 57
Assessment 11
Autoencoder 153

B

Backpropagation 86
Backtracking 46
BFS 46
Breadth-first search 33

C

Chromosome 56
Coca Cola 65
Communication 13
Convolution 174

D

Data Science 203
Deep Learning 145
Depth 147
Depth-first search 33
Depth-limited search 34
DFS 46

E

Environment 32

F

Fitness 57

G

GA 55
GAN 187
Generation 0 57
Generative Model 187
Genotype 56
GPUs 151
Graph-Search 45
Greedy best-first search 34

H

Heuristic 47
Heuristic design 35

Heuristic search 34
High Dimensionality 147
Hopfield 125

I

Intelligence 17
Inverse Sampling Theorem 169
Iterative deepening search 34

K

k-Fold Cross Validation 101
KL 167
Kohonen 121

L

Local search 35
Locus 56

M

Machine Learning 105
Massive Datasets 150
MLE 167
MLP 92
MLS 168
Momentum 70

O

Oja 131
Optimization algorithms 35

P

Passing Requirements 13
PCA 115
Phenotype 56
Practical Assignments 12
Problem 32
Proprioception 137

R

Regularization 168
Regulations 11
Relative Fitness 57

S

Sanger 134
Search 43
Simple Perceptron 79
Simplex 67
Softmax 181
Spectral Decomposition 154
Stochastic Functions 159
SVD 154

T

Tied Weights 169
Tips 209
Transformers 193
Travel Salesman Problem 50

U

UCS 46
Uncle and Aunt 145
Uniform-cost search 34
Uninformed search 33

V

Variational Inference 161